ASC - Subiecte date anii anteriori

1a.Organizarea datelor în structurile SIMD și MIMD

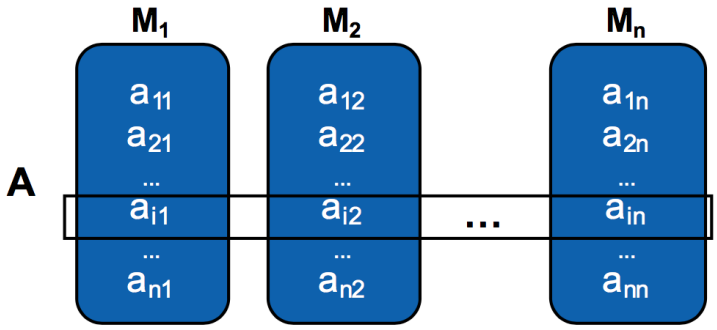
SIMD

Intr-o structura de tip SIMD se cauta ca vectorii sa poata sa fie cititi intr-o singura instructiune; intrucat la matrice accesul se poate face atat pe linii cat si pe coloane, se ia in calcul ideea de a fi necesara transpunerea sau deplasarea datelor.

Memoria poate face un acces intr-un ciclu de ceas, deci n operanzi din n module diferite pot fi accesati simultan. Din acest motiv, plasarea operanzilor in acelasi modul este defavorabila.

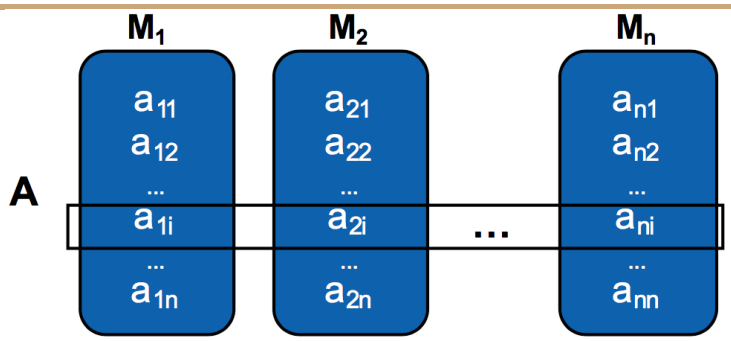
REPARTIZARE DIRECTA PE LINII:

- asigura acces paralel la linii si acces secvential pe coloane



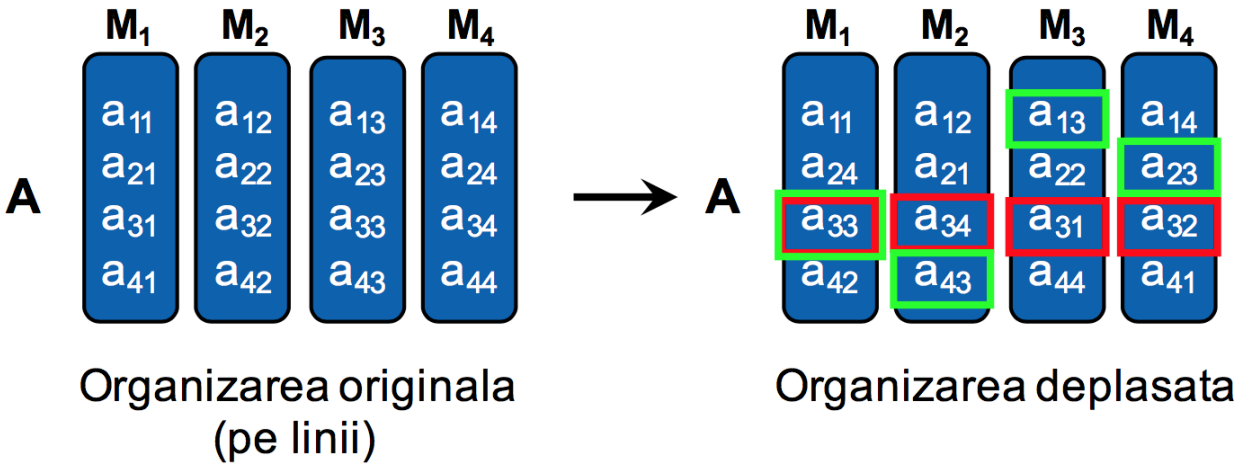
REPARTIZARE DIRECTA PE COLOANE:

- asigura acces paralel la coloane si acces secvential la linii



REPARTIZARE DEPLASATA CICLICA

- ofera acces paralel atat la liniile, cat si la coloanele matricei



Reprezentarea pe linii este folosite in limbaje cum ar fi C/ C++, Java, in timp ce reprezentarea pe coloane este utilizata de limbaje ca Fortran sau Octave.

MIMD

Ar fi practic modurile de organizare - pe linii, pe coloane, deplasată. Diferența la MIMD e că lucrează cu matrici, nu cu vectori, și aici poți să zici despre faza cu reprezentarea matricilor (care sunt 2D) în memorie (care sunt 1D) - row-major, column-major, în funcție de limbaj. (I doubt)

Pentru MIMD, toate procesoarele ar trebui sa aiba acces la intreaga memorie. Totusi, capacitatea de adresare a procesoarelor este mult mai mica decat dimensiunea memoriei. De asemenea, trebuie implementate mecanisme speciale de protectie a memoriei pentru a asigura integritatea datelor in sistem.

Sistemele MIMD pot avea:

- memorie partajata
- memorie distribuita

Sistemele multiprocesor au acelasi spatiu de adrese (comunica printr-o memorie partajata); pot avea memorie distribuita (scalabilitate) sau memorie centrala (nu exista scalabilitate - conexiunea se face prin retele de comutatie; conexiuni de tip magistrala multipla)

Sistemele multicalculator au spatii multiple de adrese si comunica prin transmiterea de mesaje.

MIMD -> memorii separate , UMA, NUMA,<CC-NUMA>, COMA

SIMD -> memorie comuna, procesare batch, < bus larg >

1b. Comparatie SIMD si MIMD dpdv al structurii si organizarii datelor

1c. Organizarea datelor la SIMD

1d. Comparatie SIMD MIMD la algoritmi

Comparam inmultirea matricilor.

SIMD: exista N procesoare, toate executa aceeasi instructiune odata, deci in fiecare ciclu se claculeaza cate o linie, nu doar un element. Se opereaza pe toti indicii k simultan (se opereaza pe linii)

pseudocod:

```
for i = 0 to n - 1
      c_ik = 0
      for j = 0 to n - 1
            c_ik += a_ij * b_jk
      end j loop
end i loop
```

MIMD:

O unitate de comanda / procesor preia functia de master: organizare si control + partajarea calculelor pe procesoare individuale.

Conway a propus o metoda folosind doua primitive, FORK si JOIN.

```
for i = 0 to n - 2
      FORK Adr
end i loop
Adr:
      for i = 0 to n - 1
            cik = 0
            for j = 0 to n - 1
                  cik += aij * b jk
            end j loop
      end i loop
      JOIN
```

1e. Comparatia generala intre SIMD si MIMD

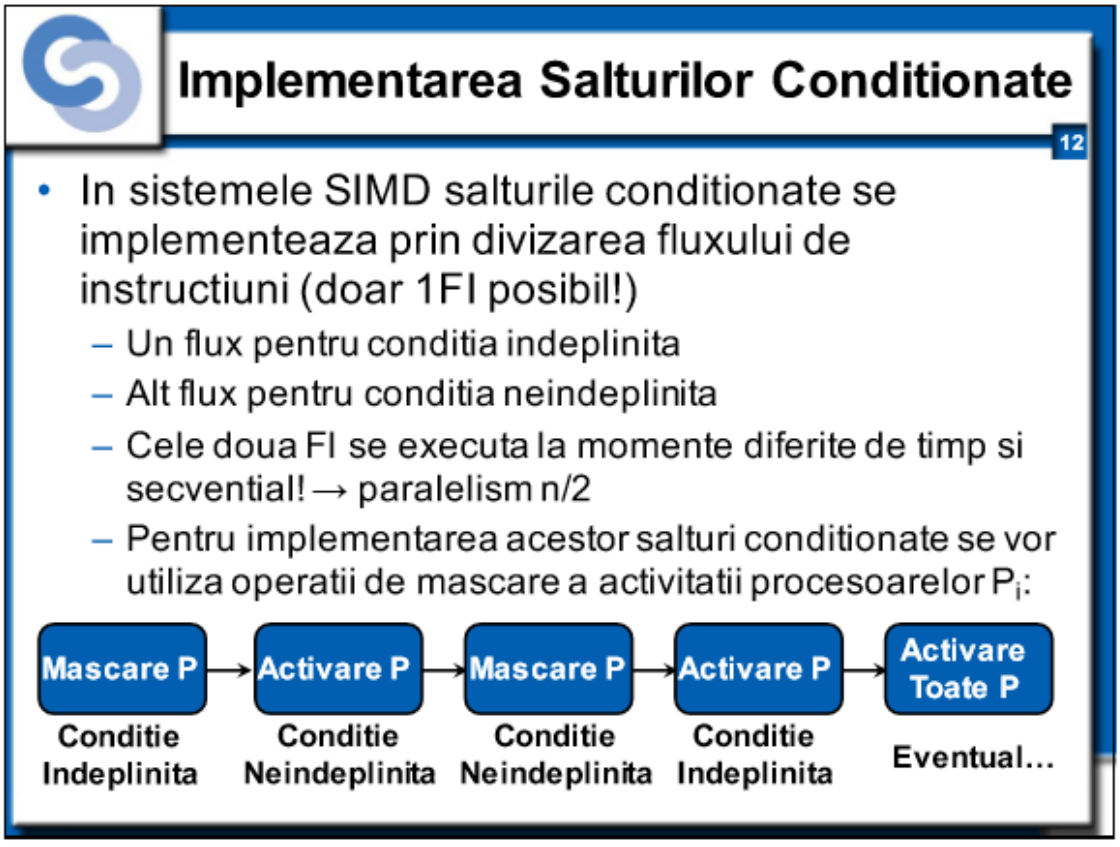
In SIMD, procesoarele se sincronizeaza instructiune cu instructiune.

In MIMD, nu exista neaparat sincronizare in fluxul de instructiuni ale procesoarelor din structura. In SIMD, exista un flux de instructiuni unic: spre exemplu, la inmultirea de matrici, se calculeaza elementele din C pe linie si pe coloana.

In schimb, la MIMD, fiecare procesor poate avea propriul sau fir de instructiuni: orice procesor poate calcula orice element din matricea C.

1f. Implementarea salturilor conditionate la SIMD si MIMD

1g.Comparatie salturi conditionate SIMD MIMD



SIMD - se considera cate un bit de mascare asociat fiecarui procesor, intr-un registru de masca.

- MASK[i] = 1 ⇔ procesorul Pi functioneaza
- MASK[i] = 0 ⇔ procesorul P1 este in asteptare si nu executa instructiunea curenta

Operatiile de testare a mastilor se fac in fiecare procesor.

MIMD - sunt procesoare independente, deci practic salturile MIMD ar trebui sa se execute normal, fiecare procesor ruleaza bucata lui de cod independent

E ca si cum ai rula mai multe thread-uri. Ideea e ca fiecare core are propriul program counter, flag-uri, etc. (o parte de registre proprii) asa ca salturi se petrec ca si cum ar fi un procesor separat fiecare core.

La **SIMD**, ai o instructiune, de genu' cum erau spu_add(), numai ca atunci cand "faci if" se activeaza un flag daca conditia a fost sau nu adevarat pt fiecare data. De exemplu poate exista inst m = spu_greater_or_equal(v, 0). Ei in urma executarii in m o sa existe pentru fiecare element din v, rezultatul operatiei, adica pt fiecare element, in m o sa fie o valoare diferita de zero pentru true, si 0 pentru false. Acum poti sa folosesti alta instructiune SIMD care il primeste pe m si executa ceva pe un core sau nu, daca elem specific din m e diferit de 0.

Modelul abstractizat la curs, e ca se activeaza niste flag-uri intr-un registru, si fiecare core primeste bit-ul sau, in loc de bytes intregi de date (Nu stiu ce zice omu' aici.)

2a. Să se scrie programul de împărțire a doi vectori de dimensiune n pentru o structură SIMD și MIMD. Comentați performanțele.

a[n], b[n]

SIMD:

```
k = 0 : n - 1 (mai multe module care executa aceeasi instructiune)

rez[k] = a[k] / b[k]; => O(1)
```

MIMD:

```
For k = 0 : n-2
      Fork Adr
End k loop
Adr:
      Rez[k] = a[k] / b[k];
      Join
```

O(1) + O(n) = O(n) (generare de fork-uri)

2b. Sa se scrie programul de insumare a elementelor unei matrice A(n, n) pentru o structura SIMD si pentru o structura MIMD. Comentati performantele. Obs. Trebuie specificata organizarea datelor.

SIMD:

```
Modulele sunt impartite pe coloane astfel incat sa poata accesa liniile pentru a face suma in O(1).

s[k] = 0 (0 <= k <= n - 1)

// calcul sume pe linii
for i = 0 to n - 1
        s[k] = s[k] + a[i][k] (0 <= k <= n - 1)
end i loop

// suma vector
for i = 1 to log2n
        s[i] = s[i] + s[i-1]
        invshuffle[s], MASK
        compute MASK
end i loop
```

```
for k = 0 to n - 2
        FORK Adr
end k loop

Adr:
        s[k] = 0 (0 <= k <= n - 1)

        // calcul sume pe linii
        for i = 0 to n - 1
                s[k] = s[k] + a[i][k] (0 <= k <= n - 1)
        end i loop
```

MIMD:

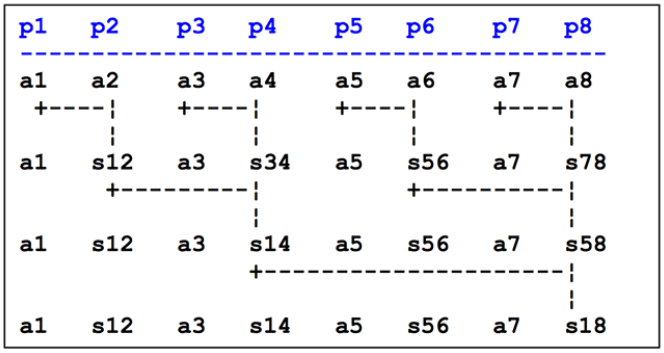


Figura 2.2

```
int a [n];
process suma [k=1 to n]{     // suma ~ procese pi din figura 2.2
      for [j = 1 to sup(log2 n)]{
            if (k mod 2 ^ j == 0) a[k] = a[k-2^(j-1)] + a[k];
      barrier;
      }
}
```

Trebuie imbinate cele doua poze. A doua face suma in logN dupa ce prima a calculat in O(N) toate sumele pe linii.

2c. Adunarea elementelor a doi vectori in SIMD si MIMD (cod + cum retii datele)

- SIMD:

  c[k] = a[k] + b[k] (0 <= k <= n - 1)

  fiecare modul de memorie retine un element din vectori astfel incat sa se poata accesa in paralel
- MIMD:

  for k = 1 to n - 2

  FORK addr

  end k loop

  addr:

  c[k] = a[k] + b[k] (0 <= k <= n - 1)

  JOIN

[RC] nu imi e foarte clar cum e organizarea memoriei la MIMD, dar din ce am inteles e mai flexibila si s-ar putea sa nu conteze asa mult. Eu as mentiona ca se poate si in cazul acesta face o organizare ca la SIMD

2d. Sa se scrie programul SIMD si MIMD care aduna 2 vectori (pe elemente). Sa se analizeze performantele. Specificati organizarea datelor.

- [RC] Exact ca mai sus

2e. Sa se scrie programul ce determina produsul elementelor unei matrice pe SIMD si MIMD. Sa se analizeze performanta. Reprezentarea datelor.

SIMD:

```
for i = 0 to n - 1
   c_ik = 0  (0 ≤ k ≤ n - 1)
   for j = 0 to n - 1
       c_ik = c_ik + a_ij * b_jk (0 ≤ k ≤ n - 1)
   end j loop
end i loop
```

- Complexitatea acestui algoritm este… $O(n^2)$! → considerabil mai bine ca in cazul SISD

MIMD:

```
for k = 0 to n - 2 (nu si pt el insusi)
   FORK Adr
end k loop
Adr:
   for i = 0 to n - 1
       c_ik = 0  (0 ≤ k ≤ n - 1) - pe coloane k fix
       for j = 0 to n - 1
             c_ik = c_ik + a_ij * b_jk (0 ≤ k ≤ n - 1)
       end j loop
   end i loop
   JOIN
```

Complexitatea este… $O(n^2)$!
→ la fel ca SIMD

2f. Inmultirea a doi vectori an si bn in SIMD si MIMD unde c[i]=a[i]*b[i]

// inmultirea ar trebui sa fie ca suma (2.c/2.d), nu? [RS]

// exact [RC]

2g. Se cere implementarea unui algoritm ce determina maximul de pe coloane pentru o matrice A[n x n] pe o masina SIMD

Modulele sunt impartite pe coloane.

S[k] = a[0][k] (0 <= k <= n - 1)

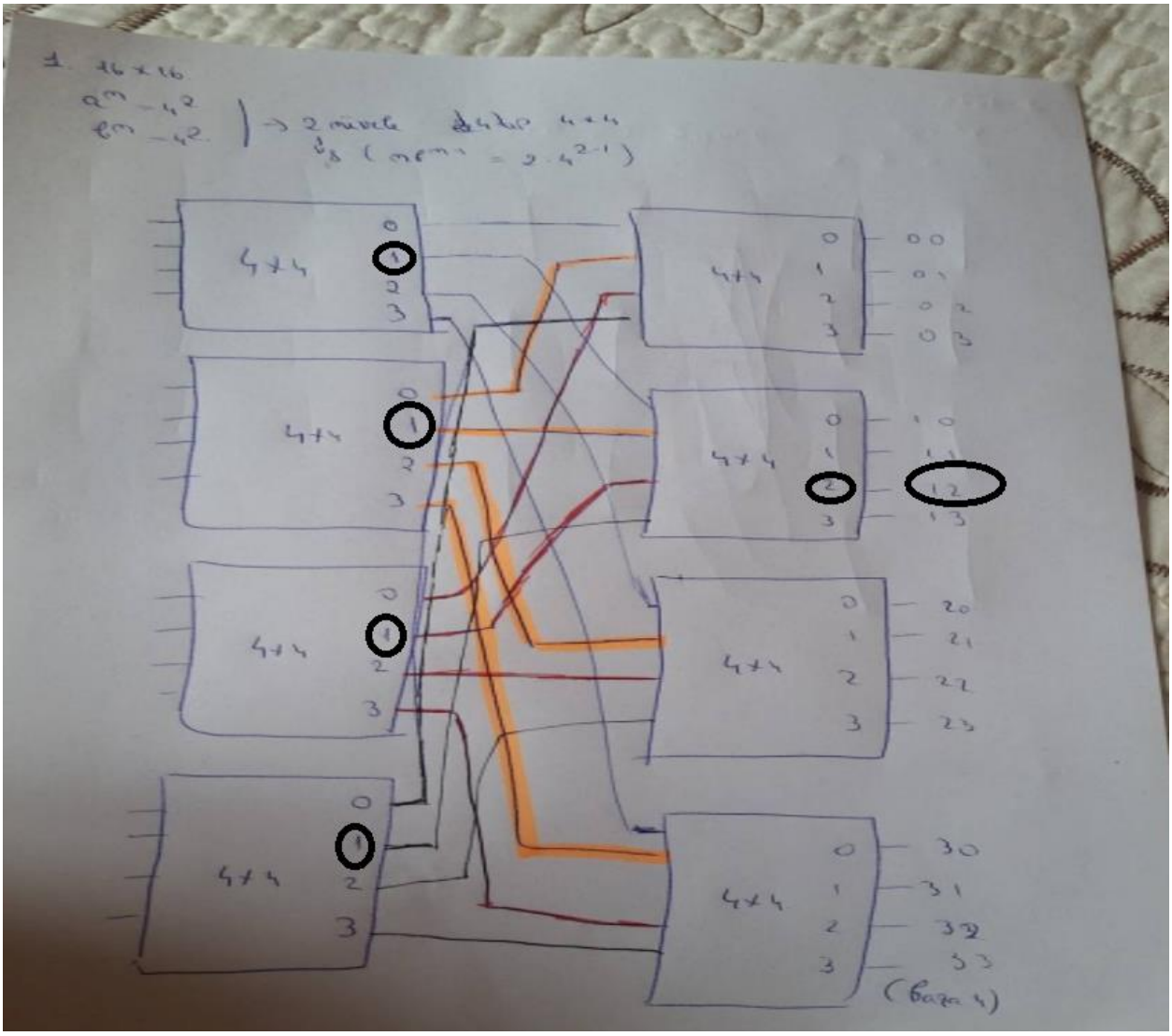
For i = 1 to n - 1

    S[k] = max(s[k], a[i][k]) (0 <= k <= n - 1)

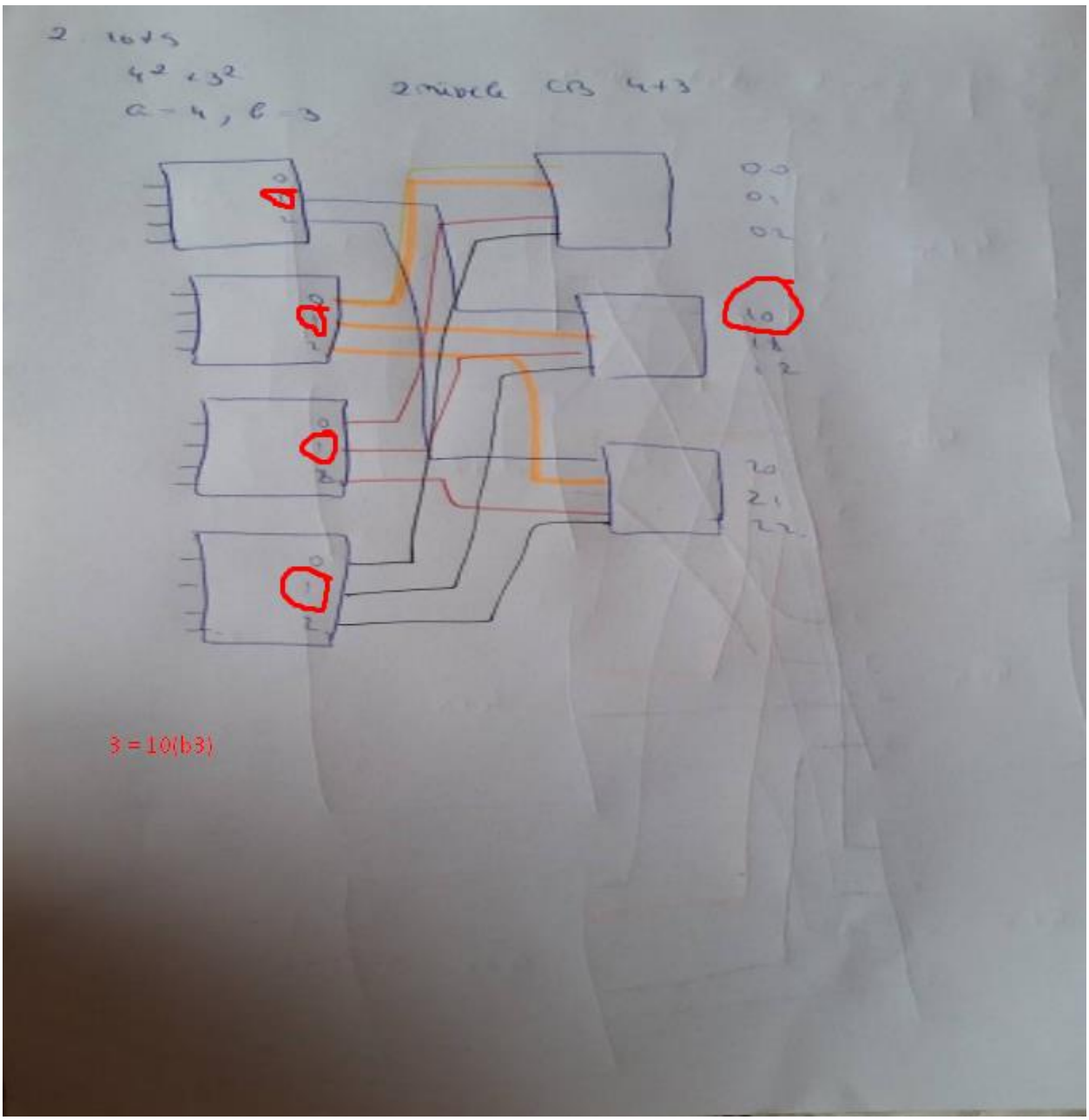
End i loop

3a. Să se cupleze 16 procesoare cu 16 memorii utilizând rețea de tip Delta; implicit procesorul i să fie conectat la memoria i. Indicați conexiunea de la procesorul 2 la memoria 6.

Aici nu garantez ca e bine, mai gasisem o varianta; mie asta mi s-a parut cea corecta.



3b. Sa se proiecteze o retea de comutare de tip delta intre 16 procesoare si 9 module de memorie (specificati calea intre procesorul 8 si memoria 3)



4. Considerând o structură multiprocesor pe o magistrală comună descrieți unitatea de acces la magistrala sistemului.

Kmap asigura controlul in cadrul unui cluster, prin excluderea mutuala a accesului la mapBus; comunicarea intre clustere se realizeaza prin InterclusterBus.

Functiile de baza ale Kmap-ului sunt maparea de adrese, comunicarea si sincronizarea.

Kbus = unitatea de comanda a MapBus ce controleaza toate tranzactiile de pe magistrala

Kbus - unitatea de comanda pentru accesul la magistrala

Kbus- reprezinta procesorul local in relatie cu magistrala externa

Starea A: adresa e locala?

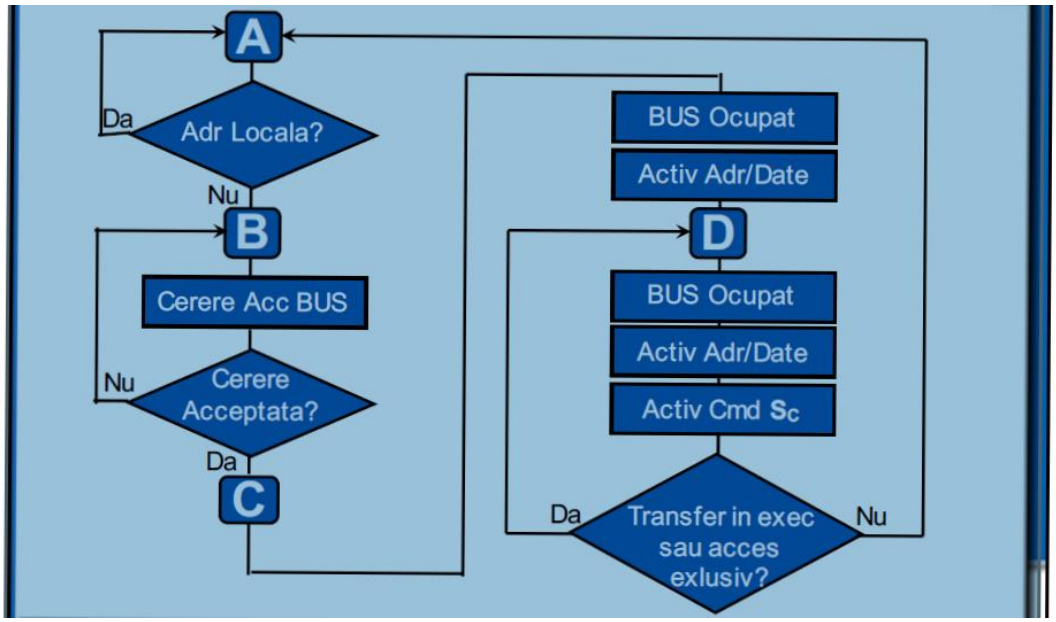
- da: se adreseaza resurse locale, nu trebuie iesit de pe mag
- nu: adr nu e locala, se trece in starea B

Starea B: cerere de acces la mag, daca cererea e acceptata-> starea C

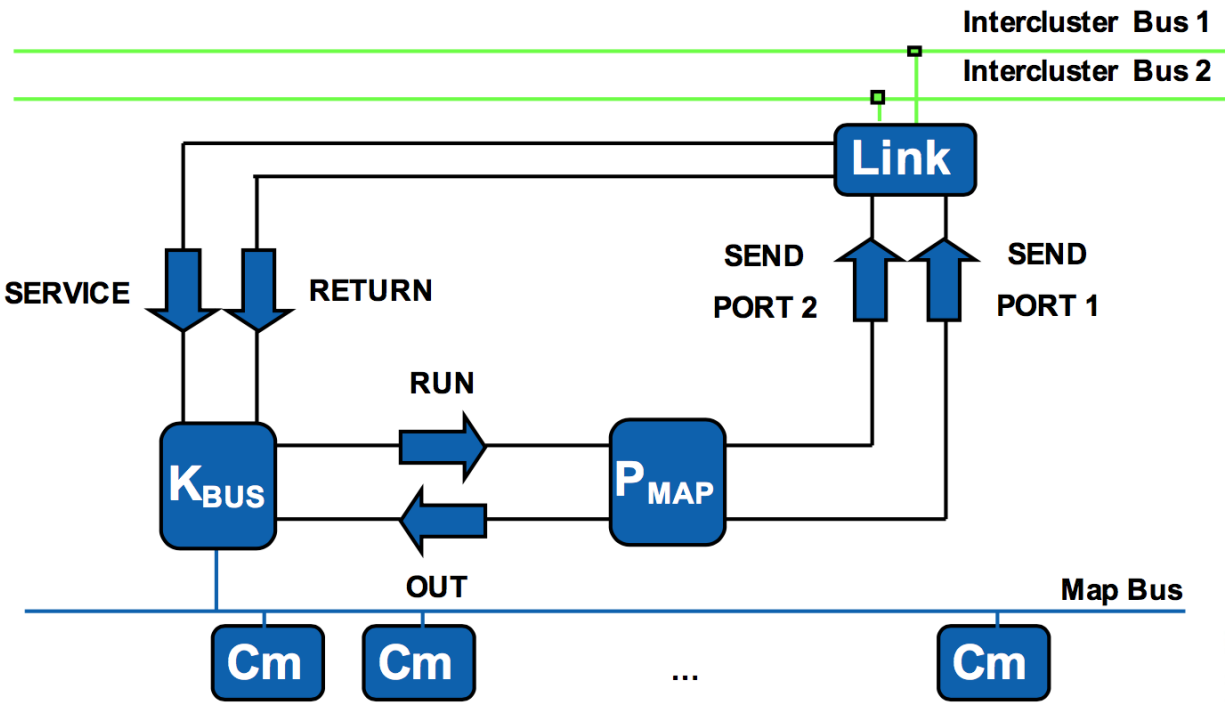
Starea C: emite bus busy, doar in C si D accesul pe mag e exclusiv si cu rafala(overwrite) raman in D pentru un nou ciclu daca mai este cazul, automatul preia functia procesorului. Dupa preluarea comenzii, procesorul face functia de secventiere a adr, date, cmd

Obs: Kbus trebuie sa dispuna de unitati logice combinationale ce identifica:

- daca adr mem/port I/O implicate in tf sunt locale
- semnalul de cerere acceptata e combinat din mai multe conditii: mag libera si modul SBC activ



5. Structura generală Kmap.



Elementele Kmap:

Kbus: unitate de comanda a Map Bus si va controla toate tranzactiile de pe aceasta mag

Pmap:

- procesor de mapare care face toate translatarile de adresa
- maxim 8 procese simultan

Link: gestioneaza si supravegheaza transmiterea mesajelor intercluster Send/Receive

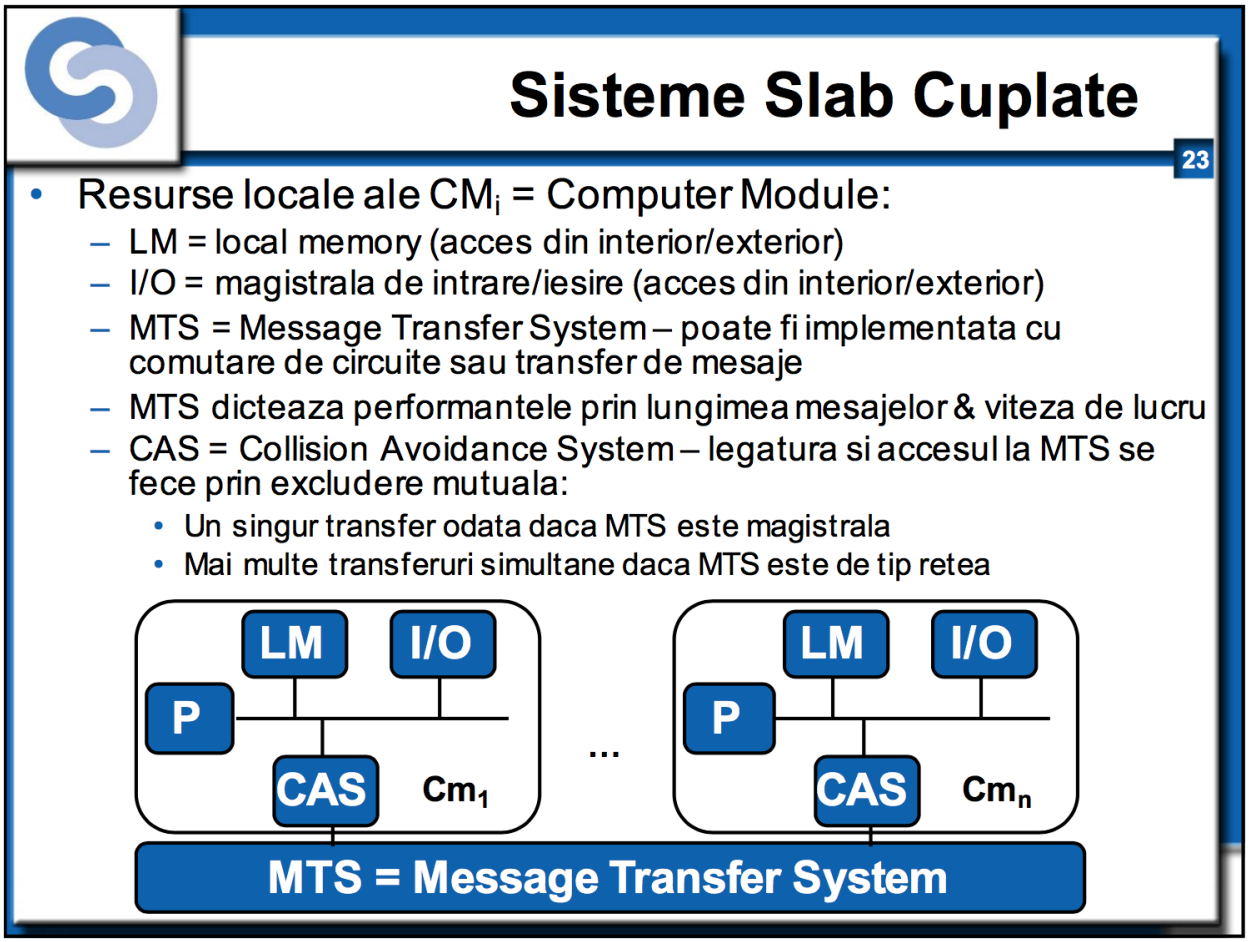
2 tipuri de comutatie:

- de circuite: in cadrul cluster-ului
- de mesaje: intre clustere

Kmap prelucreaza tranzactii concurente, mapare de adrese, comunicare, sincronizare. Controlul in cadrul clusterului se face prin excludere mutuala.

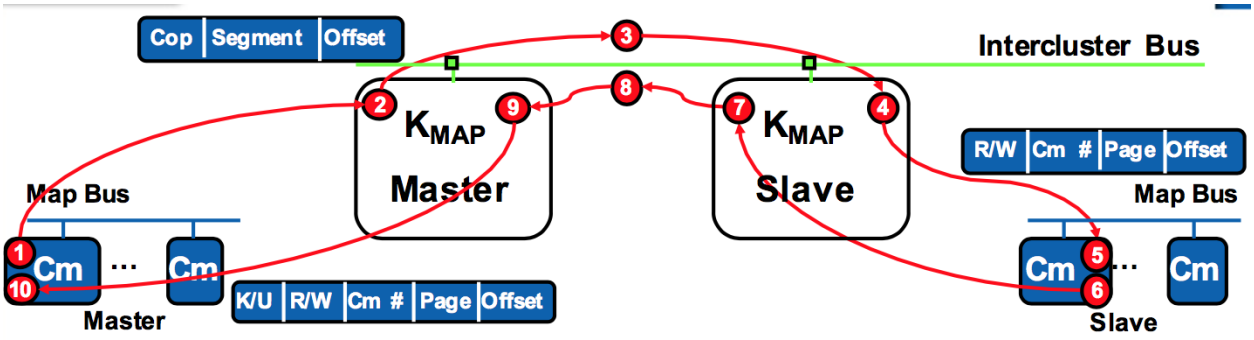
6a. Comunicația intercluster în structurile slab cuplate.

cred ca e perfect valabila aceeasi chestie descrisa la 6C



6b. Comunicatia intercluster in MIMD slab cuplate.

6c. Comunicatia intercluster

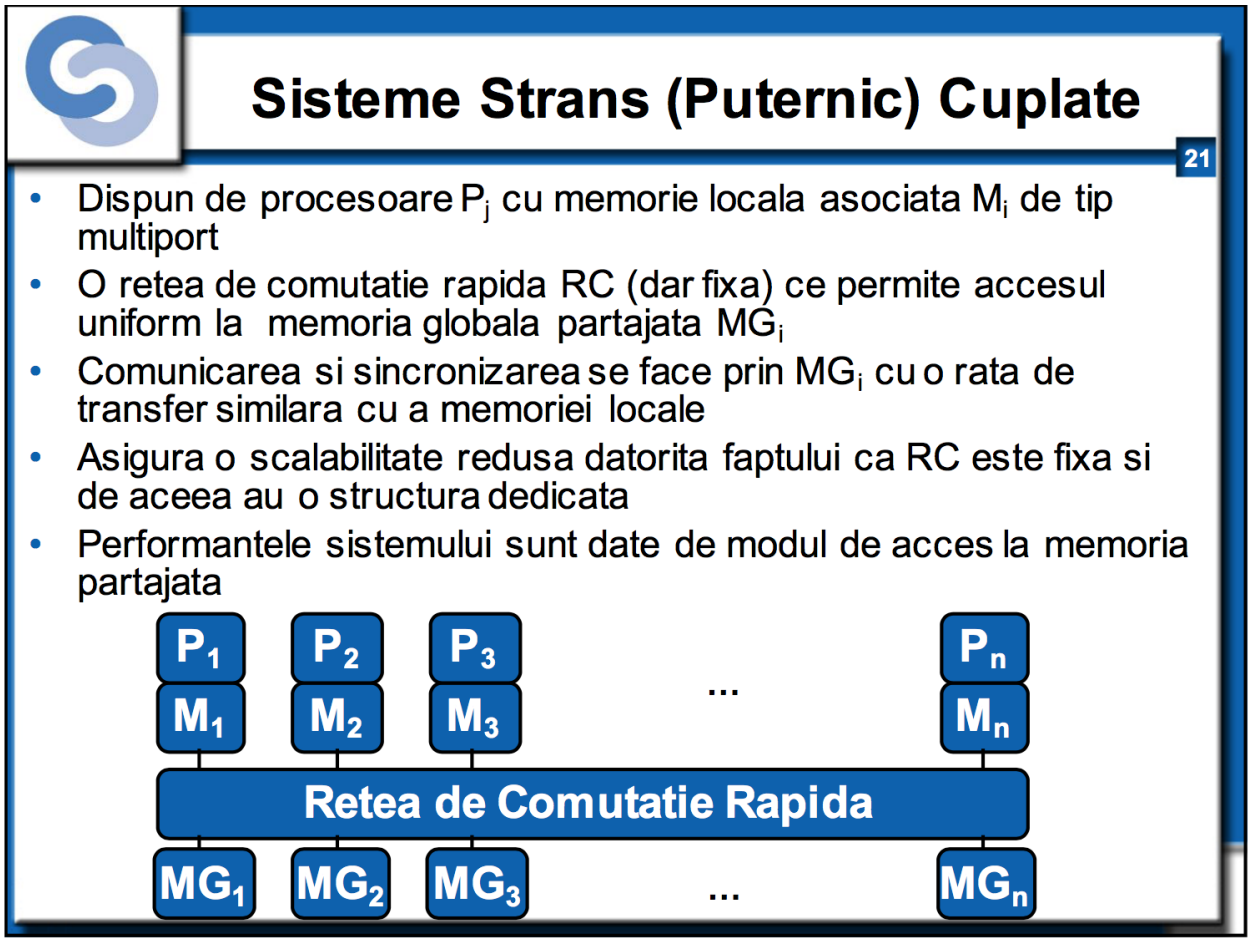


1. Cm master trimite cerere de transfer la Kmap master.
2. Kmap master pregateste mesajul/ pachetul si codifica cererea
3. Mesajul este transmis pe intercluster bus folosind algoritmi de rutare
4. Kmap slave decodifica cererea si o trimite catre clusterul sau local
5. Cererea este trasmisa la Cm slave
6. Cm slave transmite rezultatul la Kmap slave

7. Kmap slave pregateste mesajul intercluster (reactivarea contextului respectiv)
8. Kmap slave transmite rezultatul prin intercluster bus catre Kmap master
9. Kmap master receptioneaza mesajul primit
10. Rezultatul este transmis catre Cm master

11.

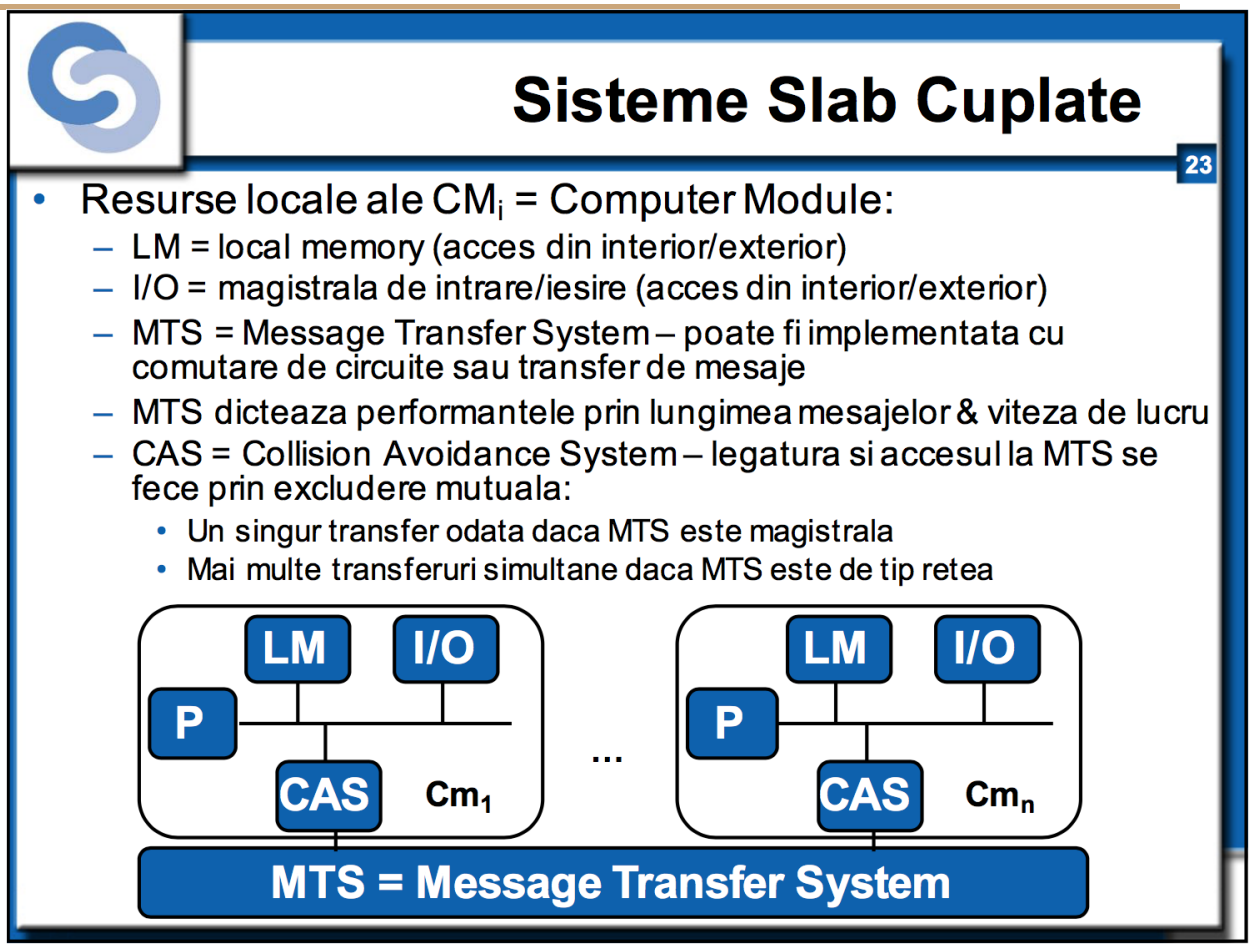
6d. Caracteristicile principale ale sistemelor strans cuplate si slab cuplate



Sisteme Slab Cuplate

22

- De tip cluster → calculatoarele au resurse locale necesare functionarii independente
- Procesele se executa pe procesoare care dispun de toate resursele pentru implementarea task-ului respectiv
- Comunicarea intre procesoare se face prin
  - Transfer de mesaje
  - Comutare de circuite
- Sistemele slab cuplate (distribuite) sunt eficiente atunci cand interactiunea intre procesoare este minima (granularitatea problemei este mare)


Sisteme Slab Cuplate
23
• Resurse locale ale CMi = Computer Module:
– LM = local memory (acces din interior/exterior)
– I/O = magistrala de intrare/iesire (acces din interior/exterior)
– MTS = Message Transfer System – poate fi implementata cu comutare de circuite sau transfer de mesaje
– MTS dicteaza performantele prin lungimea mesajelor & viteza de lucru
– CAS = Collision Avoidance System – legatura si accesul la MTS se fece prin excludere mutuala:
• Un singur transfer odata daca MTS este magistrala
• Mai multe transferuri simultane daca MTS este de tip retea
LM
I/O
P
CAS
Cm1
...
LM
I/O
P
CAS
Cmn
MTS = Message Transfer System

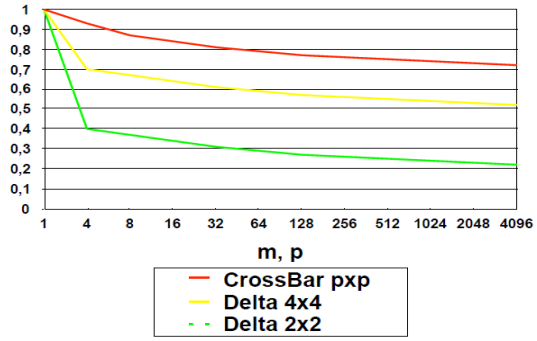
7. Comparație între crossbar și delta privind performanțele.

Analiza Comparativa Delta / CB

- Probabilitatea ca o cerere sa fie acceptata este:

$$P_A = \frac{b^n \cdot r_n}{a^n \cdot r}$$

- Ce influenta au m si p-ul unei retele?
  - CrossBar pxp cu r=1 rata de cereri a fiecarui procesor
  - Delta 4^nx4^n cu 4x4
  - Delta 2^nx2^n cu 2x2
- Pentru m mare, P_A-ul scade puternic
  - Multe rejectii si conflicte
  - Limita la 16^nx16^n este 0.6

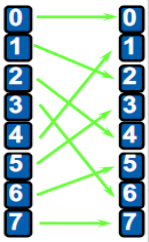


8a. Rețeaua de comutare de tip intercalare perfectă și comparație cu rețeaua butterfly.

2 – Shuffle Permutation

23

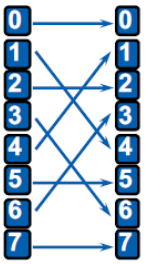
- Bitul cel mai semnificativ devine cel mai nesemnificativ:
  P_shuffle (a_n a_{n-1} … a_1 a_0) = (a_{n-1} … a_1 a_0 a_n)
- Pentru 8 resurse avem P_shuffle (a_2 a_1 a_0) = (a_1 a_0 a_2)
- Variante posibile sunt
  - Permutarea shuffle k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_shuffle_kinf (a_n … a_k a_{k-1} … a_1 a_0) = (a_n … a_k … a_1 a_0 a_{k-1})
    - Utilizare posibila: conexiunea memoriilor – adresa de baza a unui modul + adresa in cadrul acelui modul
  - Permutarea shuffle k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_shuffle_ksup (a_n … a_{n-k+1} a_{n-k} … a_0) = (a_{n-1} … a_{n-k+1} a_n a_{n-k} … a_0)
  - Permutarea shuffle q # c, intre q·c resurse (nu putere a lui 2):
    - P_shuffle_q#c (ADR) = q · V_{ADR} mod (q·c -1) pentru 0 ≤ V_{ADR} < q·c-1
    - P_shuffle_q#c (ADR) = V_{ADR} pentru V_{ADR} = q·c-1
    - V_{ADR} = 0, 1, 2, etc, n-1 succesiv



4 – Butterfly Permutation

25

- Se interschimba bitul cel mai semnificativ cu cel mai nesemnificativ:
  P_butterfly (a_n … a_1 a_0) = (a_0 a_{n-1} … a_1 a_n)
- P_butterfly (a_2 a_1 a_0) = (a_0 a_1 a_2)
- Butterfly este o permutare simetrica
- Variante posibile sunt
  - Permutarea butterfly k inferioara:
    - P_butterfly_kinf (a_n … a_k a_{k-1} … a_1 a_0) = (a_n … a_k a_0 … a_1 a_{k-1})
    - Ofera posibilitatea de conectare între resursele clasei
  - Permutarea butterfly k superioara:
    - P_butterfly_ksup (a_n … a_{n-k+1} a_{n-k} … a_1 a_0) = (a_{n-k+1} a_{n-1} … a_n a_{n-k} … a_0)
    - Ofera posibilitatea de conectare intre clase



La intercalarea perfecta bitul cel mai semnificativ devine cel mai nesemnificativ; la butterfly se schimba intre ele

Butterfly este o permutare simetrica

Intercalare perfecta

(an,.....a1,a0)->(an-1,...a1,a0,an)

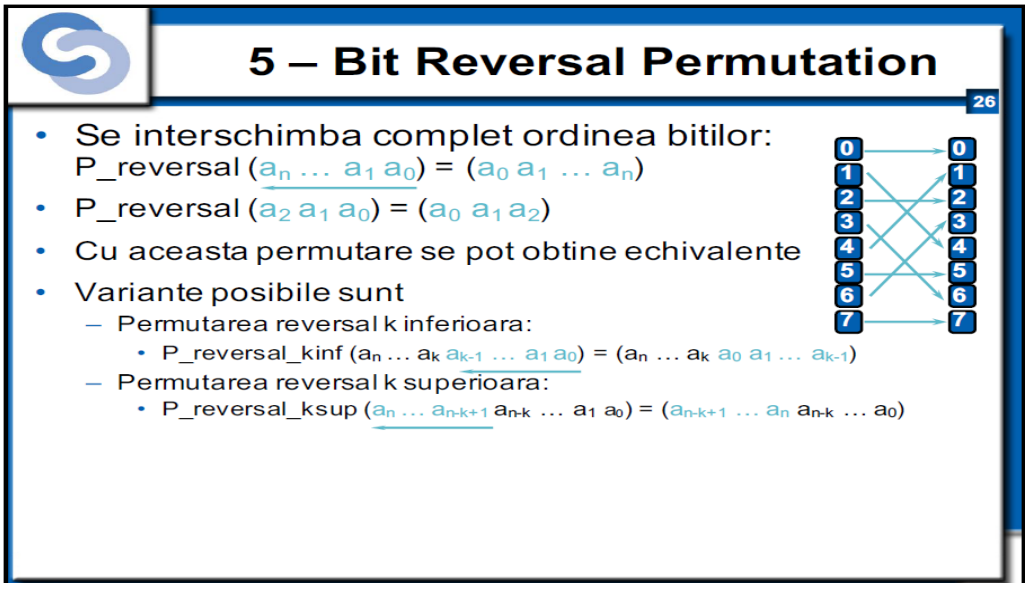
Butterfly

(an,...a1,a0) -> (a0,an-1,....a1,an)

b. Permutare fundamentala de tip intercalare perfecta

Vezi slide-ul de mai sus

8c. Retea de comutare de tip intercalare perfecta si comparatie cu reteaua cu ordine inversa.



La **reteaua cu ordine inversa** se interschimba complet bitii -> (a0,...an)-> (an,...a0)

La **intercalarea perfecta** bitul cel mai semnificativ devine cel mai nesemnificativ;

(an,...a1,a0) -> (an-1, …..a1, a0, an)

8d. Comparatie intercalare perfecta si permutarea de baza

Permutarea de baza este intercalarea perfecta inversa

Intercalarea perfecta: bitul cel mai semnificativ devine cel mai nesemnificativ

(an,...a1,a0)->(an-1, ..a1,a0,an)

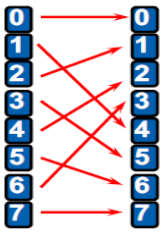
Permutarea de baza: bitul cel mai nesemnificativ devine cel mai semnificativ

(an,...a1,a0)->(a0,an ..a1)

1 – Base Line Permutation

22

- Bitul cel mai nesemnificativ devine cel mai semnificativ:
  P_base ($a_n\ a_{n-1} \dots a_1\ a_0$) = ($a_0\ a_n\ a_{n-1} \dots a_1$)
- Permutarea de baza este intercalarea perfecta inversa!
- Pentru 8 resurse avem P_base ($a_2\ a_1\ a_0$) = ($a_0\ a_2\ a_1$)
- Variante posibile sunt
  - Permutarea de baza k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_b_kinf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0\ a_{k-1} \dots a_1$)
    - Asigura conexiunea in submultimi de resurse
  - Permutarea de baza k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_b_ksup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_0$) = ($a_{n-k+1}\ a_n \dots a_{n-k+2}\ a_{n-k} \dots a_0$)
    - Asigura conexiunea intre submultimi de resurse
- TA: Variantele p_base pentru 8 resurse

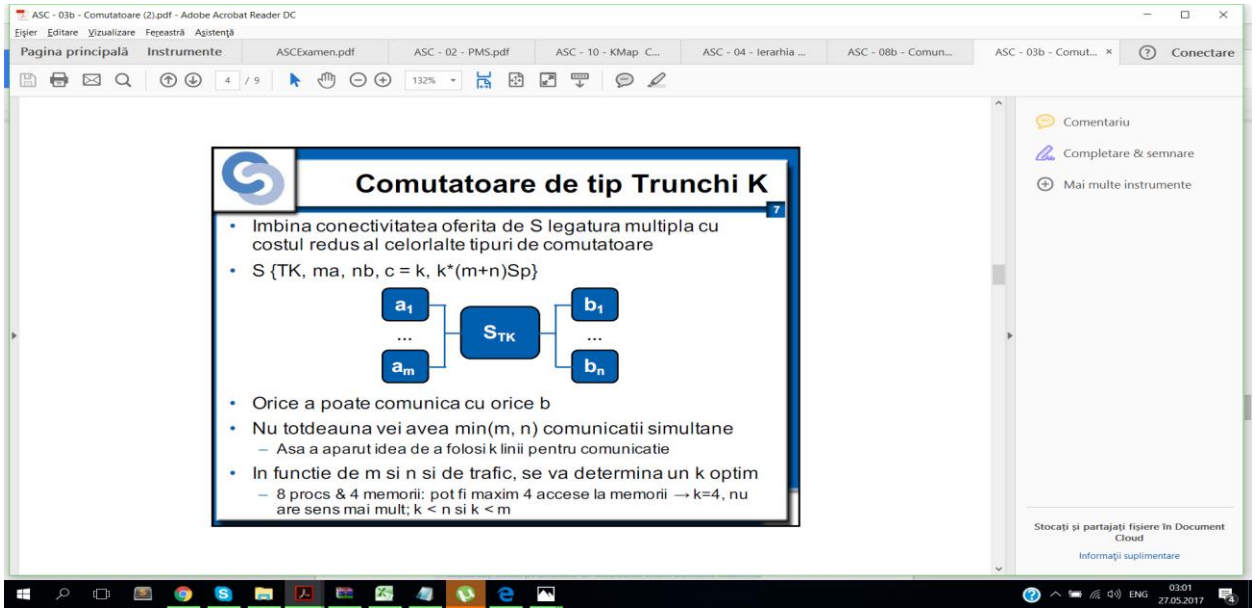


9. Subiect la alegere din materia de ASC.

10. Comutatorul de tip trunchi K ierarhic

Comutator ierarhic - conecteaza o componenta de tip a cu una de tip b

Comutator de tip trunchi K - Imbina conectivitatea oferita de S legatura multipla (asigura comunicatia intre m componente de tip a cu n de tip b) cu costul redus al celorlaltor tipuri



11a.Structura PMS

PMS - Processor Memory Switches

M-Memoria

- Rolul memoriei:
  - pastreaza informatia
  - actualizeaza informatia prin op de citire/scriere
  - necesita un sistem de adresare cu o corespondenta liniara intre adresa si continut
- Atribute:
  - Functie: primara | auxiliara
  - Tehnologie: bipolara | MOS | statica | dinamica
  - Operatii: citire | citire/scriere
  - Mod acces: aleator | secvential | FIFO | LIFO | asociativ
  - Lungime cuvant: 8+1 | 16+2 | 32+4 | 64+8
  - Capacitatea: 4Mb | 8Mb | 16Mb | …| 1Gb | 2Gb | 4Gb
  - Ciclu de lucru: 2:4ms | …| 400ns

P - procesorul central

- Rolul PC:
  - Citeste, interpreteaza si executa instructiuni masina
  - In acest proces are loc generarea comenzilor spre toate resursele sistemului + citirea starilor acestora
- Atribute:
  - Functie: universal | specializat (de semnale, etc)
  - Implementarea: conventionala | μprogramata |μprogramata specializata
  - Formatul instructiunilor: fix | variabil
  - Lungimea instructiunilor: 8 | 16 | 32 | 64 | 128 biti
  - Ciclu instructiune: ciclu fix | ciclu varibil (S cicli masina)
  - Tehnologia: LSI | VLSI |

S - Switch - Comutatorul

- Rolul S:
  - Asigura conexiuni intre componentele sistemului
  - Poate evolua
    - • de la structuri foarte simple = buffer 3state de acces pe magistrala
    - • la structuri complexe cu unitati de comanda proprii = comutatoare
    - cu discipline de servire a cererilor de stabilire a legaturilor
- • Atribute:
  - Structura: ierarhica | neierarhica
  - Tip: simplex | semiduplex | duplex (e vorba de sensuri)
  - Numar de legaturi realizate:
    - • m intrari | n iesiri din S
    - • n intrari | m iesiri din S

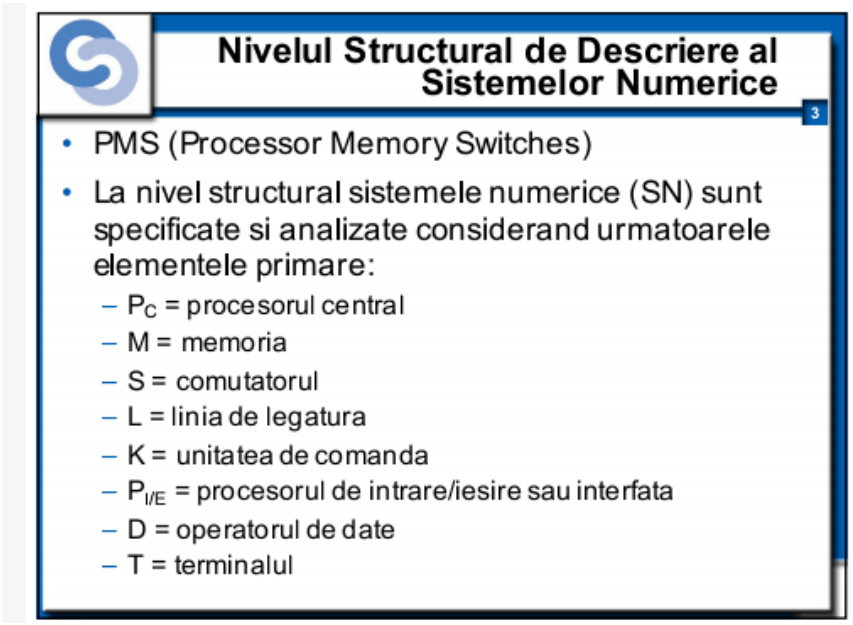
- Concurenta: c

11b. Descrieti folosind PMS structura cu care ati lucrat la laborator. [RS] ca la 11e

11c.Descrieti in metoda PMS resursele de la laborator. [RS] ca la 11e

11d. Structura PMS, descriere generala. Prezentare calculator propriu cu primitive pms [RS] ca la 11e

11e. Descrie calculatorul tau folosind primitive PMS



Pc = CPU

- 2 cores
- 4 threads
- Base freq: 2.60GHz
- Universal
- 64 bit

M = Cache + RAM + HDD

RAM

- Primara
- Dinamica
- citire/scriere
- Mod de acces aleator I guess
- Capacitate 4GB
- Duty cycle: 400ns

Cache

- Auxiliara
- 3MB

HDD

- Auxiliara
- citire/scriere

- 750GB

S

- Duplex
- Concurent

L = BUS

- Asincron
- 64-bit
- Un subsisetm care transfera date intre componentele computerului sau intre computere
- FSB = Front Side Bus
  - Asigura comunicarea dintre CPU si un controller de memorie central
- DMI = Direct Media Interface
  - Interconexiune punct la punct intre un controller de memorie si un controller I/O de pe placa de baza
- QPI = Quick Path Interconnect
  - Interconexiune punct la punct intre CPU si controller-ul memoriei integrate

D

- Intel 80287 (prelucrare cu virgula mobila) (pare cam vechi dar nu stiu ce foloseste laptopul meu)
- FPU integrat (floating point unit)
- Co-processor

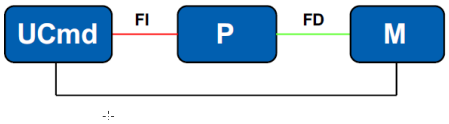
11f. Descrieti primitiva P la calculatorul personal. [RS] ca la 11e

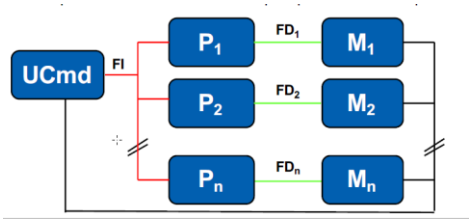
11g.Descrieti primitiva M (memorie) utillizand PMS [RS] ca la 11e

12. Clasificarea Flynn a sistemelor de calcul.

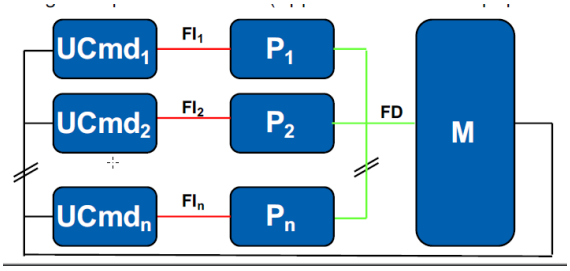
Taxonomia lui Flynn presupune impartirea sistemelor de calcul in functie de fluxul de instructiuni (secventa de instructiuni executate de procesor) si fluxul de date (secventa de operanzi manipulata de procesor).

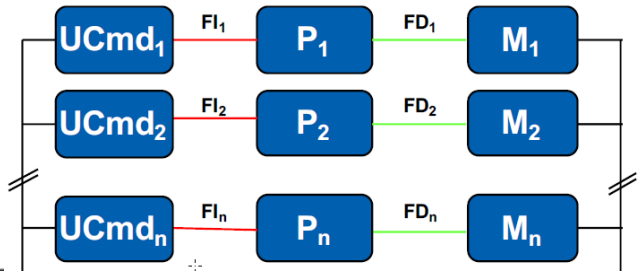
- SISD - Single Instruction Single Data Stream



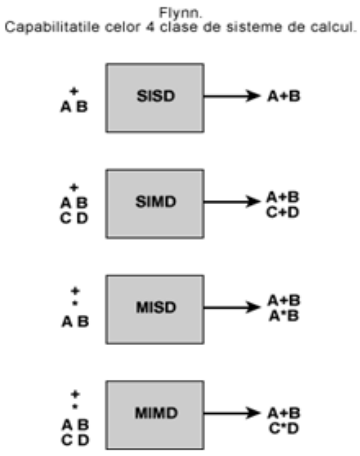


- SIMD - Single Instruction Multiple Data Stream
- MISD - Multiple Instruction Single Data Stream





- MIMD - Multiple Instruction Multiple Data Stream



13. Descrieti caracteristicile generale CELL

Combines multiple high performance processors in one chip

• 9 cores, 10 threads

• A 64--bit Power Architecture™ core (PPE)

• 8 Synergistic Processor Elements (SPEs) for data--intensive processing

Heterogeneous multi--core system architecture

- – Power Processor Element for control tasks
- – Synergistic Processor Elements for data--intensive processing

• Synergistic ProcessorElement (SPE) consists of

- – Synergistic Processor Unit (SPU)
- – Synergistic Memory Flow Control (SMF)
  - • Data movement and synchronization
  - • Interface to high--performance Element Interconnect Bus

PPE handles operating system and control tasks

- – 64--bit Power ArchitectureTM with VMX
- – In--order, 2--way hardware simultaneous multi--threading (SMT)
- – Coherent Load/Store with 32KB I & D L1 and 512KB L2

SPE provides computational performance

- – Dual issue, up to 16--way 128--bit SIMD
- – Dedicated resources: 128d128--bit RF, 256KB Local Store
- – Each can be dynamically configured to protect resources
- – Dedicated DMA engine: Up to 16 outstanding requests

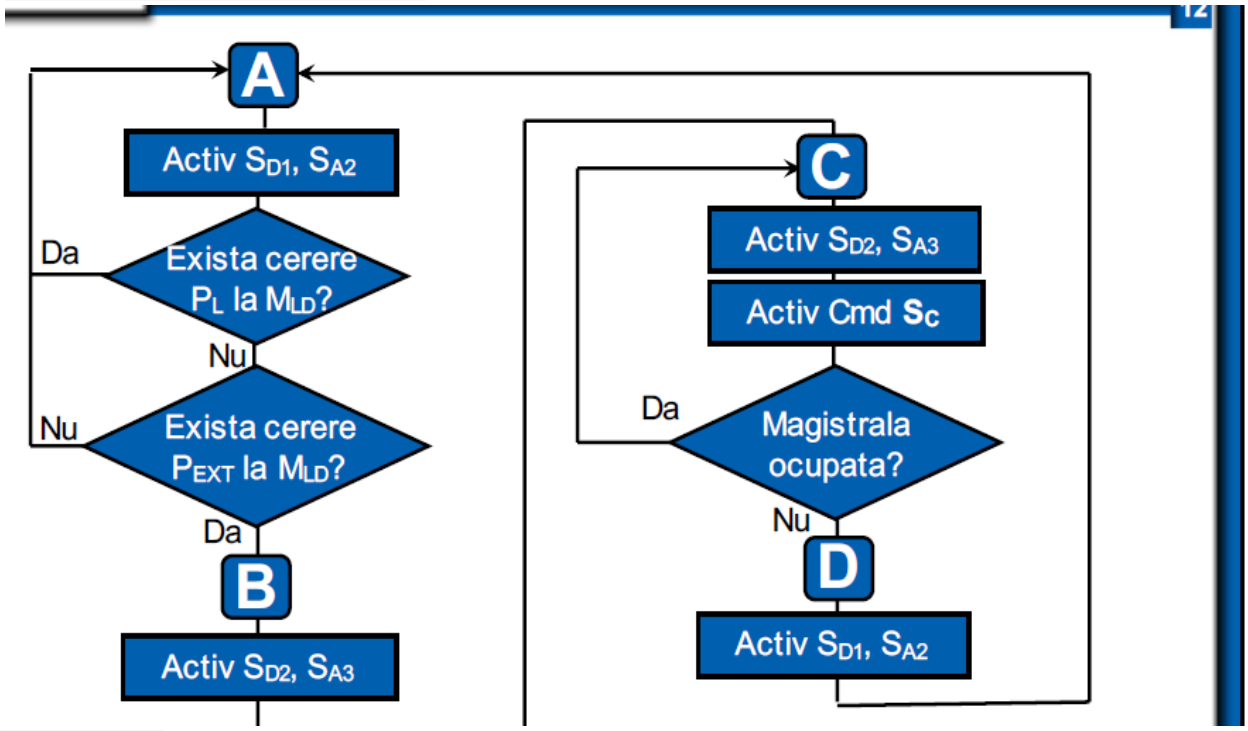
14. Schema logica Kml

Kml - unitate de control al accesului la memoria locala

KML = controleaza S--urile de adrese si de date

– Accesul local sau extern la memorie

– Daca deschide toate S--urile se ajunge la accesul direct al procesorului local in exterior



Descriere Kml

• KML – asigura accesul la Mem locala a PL prin intermediul SD1 si SA2 & a proc externe prin SD2 si SA3

• PL are prioritate fata de proc externe (PEXT) == daca vrea PL la MLD NU ma intereseaza ce vor celelalte PEXT

• Starea A: PL poate avea comenzi locale

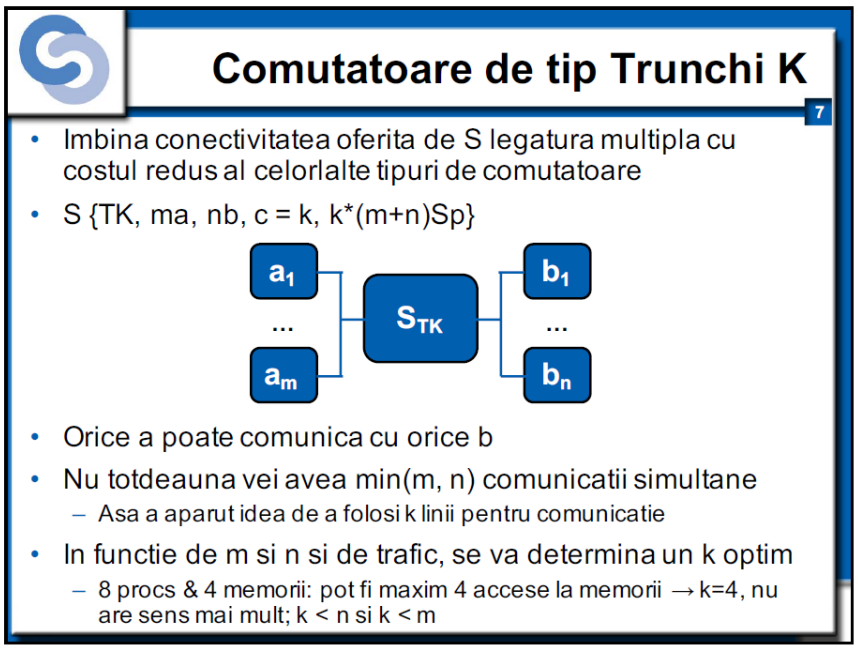
• Starea B: activez accesul ext la MLD – semnalele asociate starii

• Obs: Mem locala e vazuta de fiec proc intr--un ac spatiu de mem, iar de proc ext in spatiul complementar → trebuie sa existe o unitate de translatare a adreselor

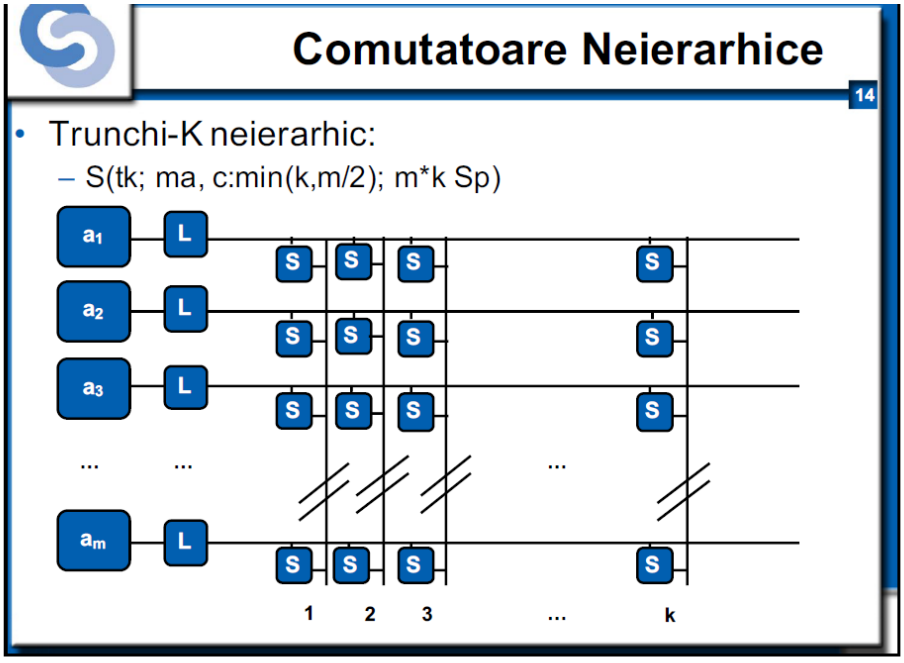
• Mag ocupata: nu stii ciclii masina ai unui proc extern. Numai cand magistrala e libera un PEXT a terminat ciclul masina curent → abordare top--down

15. Trunchi K ierarhic si neierarhic

Ierarhic



Neierarhic



16a. Probabilitatea ca o cerere sa fie acceptata la un comutator CB (formula-)

16b. Sa se calculeze probabilitatea de satisfacere a unei cereri intr-un switch cross-bar cu p procesoare si m memorii

(DN: acelasi raspuns: acceptat = cerere satisfacuta)

- Consideram un **CB pxm** (CrossBar cu p procesoare si m memorii)

- **r = probabilitatea ca un procesor sa genereze o cerere intr-un ciclu**

- **E(i) = numarul de cereri acceptate** de reteaua pxm intr-un anume ciclu; depinde de numarul i de cereri, de capabilitatile memoriei, de numarul de unitati de memorie

- **B(p,m) = rata de transfer cu memoria**; se calculeaza dupa formula

  B(p, m) = m - m * (1 - r / m) ^ p

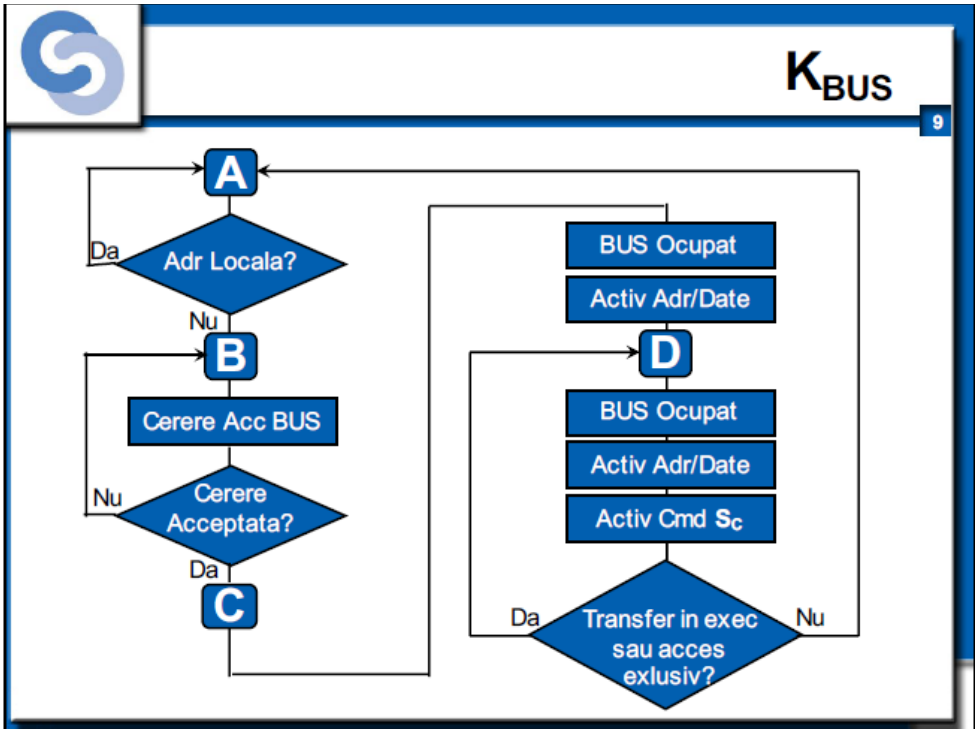
- **Probabilitatea de acceptare** a unei cereri oarecare PA este definită astfel
  Pa = B(p,m) / (r * p)

  În stabilirea relaţiei de mai sus s-a considerat că cererile sunt independente. În realitate o cerere rejectată la un anumit ciclu va fi transferată în ciclul următor mărind astfel rata de transfer.

  Apoi analizeaza **un caz particular**:
  Daca **m -> inf** (numar infinit de memorii) atunci
  lim (1 - r / m) ^ p = e ^ (- p * r/m)

  Reactualizeaza formula pentru **B(p, m) si Pa**
  Obs1. Aceste doua formule sunt valabile daca m si p sunt foarte mari.
  Obs2. Erori de 1-5% (destul de bune in practica)**.**

17. Organigrama Kbus; Descrierea schemei logice Kbus (2010, 2010, 2009, 2009)



KBUS – reprezinta procesorul local in relatie cu magistrala externa.

Starea A: adresa e locala?
– Da = se adreseaza resurse locale, nu trebuie iesit pe magistrala
– Nu = adr nu e locala → se trece in starea B

• Starea B: cerere de acces la magistrala;; daca cererea e acceptata se trece in starea C

• Starea C: se emite Bus Busy (mag ocupata) – doar in C & D;; accesul pe BUS e exclusiv & cu rafala (overwrite) raman in D pt un nou ciclu daca mai e cazul;; automatul preia functia procesorului;; Dupa preluarea comenzii, procesorul face functia de secventiere a Adr, Date, Cmd

18. Sa se aleaga un calculator din TOP 500 si sa vorbim despre el.

Despre Top:

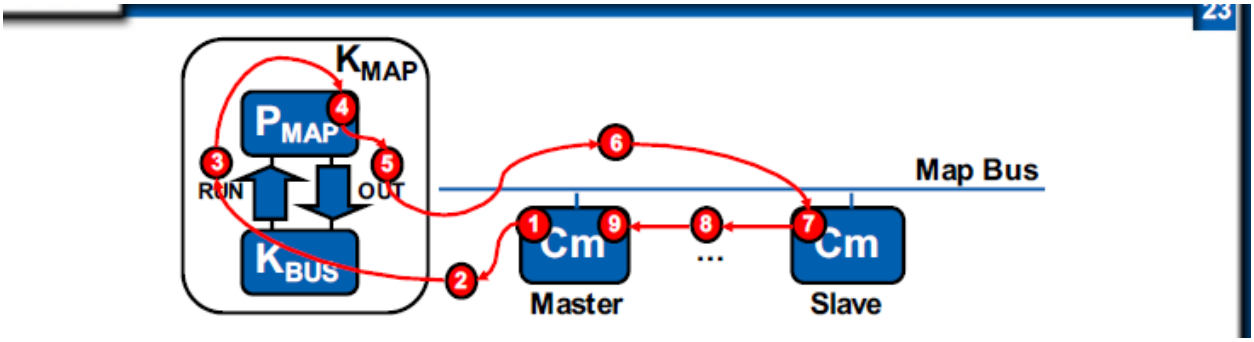
- **Proiect demarat in 1993**
- **actualizat de 2 ori pe an**
- **foarte des se utilizeaza The LINPACK Benchmark**

(am simplificat tot - fara nume pompoase de forma NVIDIA xzjczcjziq9-2130)

Titan Cray XK7 - Locul 2 in Top

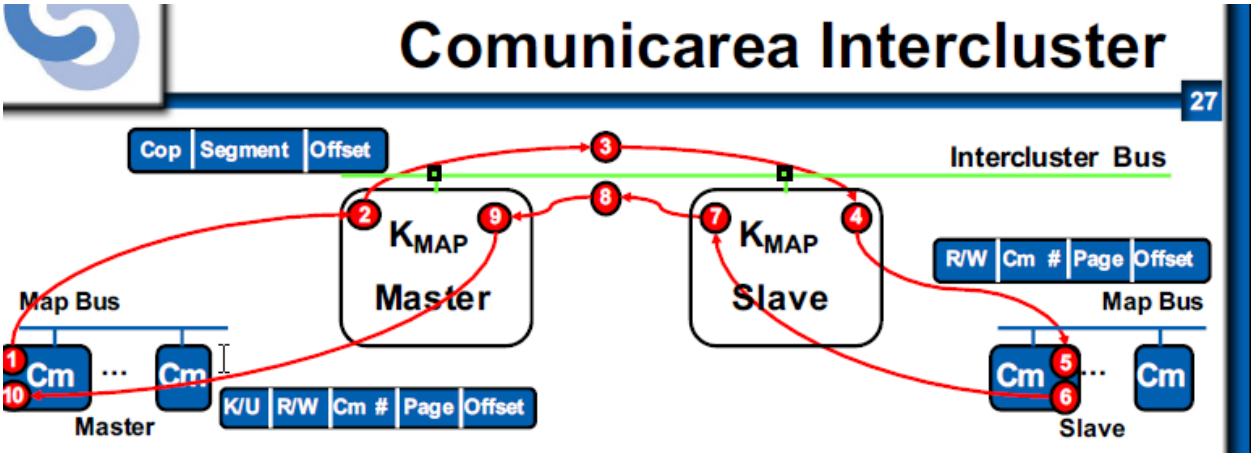
- **Familia de sisteme: Cray (XK7)**
- **Procesoare: Opteron, NVIDIA**
- **OS: Cray Linux**
- **Arhitectura: MPP (Massively Parallel Processing - un singur task executat cu ajutorul a mai multor procesoare). De remarcat: aproape toate din Top 10 au arhitectura MPP.**
- **Anul lansarii 2012**
- **~600 000 de core-uri**
- **Memorie: ~700TB**
- **De ce am ales model? => Foarte multe aplicatii in cercetare**
    - **simularea reactiilor chimice de obtinere a combustibilului**
    - **simularea schimbarilor climatice**
    - **experimente cu laser**
    - **schimbarea codului molecular**
    - **statistica**
- **Pe scurt: ofera procesare paralela masiva, putere mare de procesare, volum mare de stocare. Aplicatiile pot fi extinse (la orice supercalculator din top ..)**

19. Etapele de Acces in Cadrul unui Cluster (succesiunea din poza)



1. Cm Master - initiaza acces la o memorie nelocala
2. Cm Master - emite adresa virtuala catre KBUS
3. KBUS - activeaza un context ce se pune in coada RUN catre PMAP
4. PMAP - trateaza contextul si face translatarea adresei
5. PMAP - pune in coada OUT o cerere pentru ciclu de memorie pentru Cm Slave din cluster-ul curent
6. KBUS - trimite adresa fizica catre Cm Slave prin Map Bus
7. S-ul - acces la memoria locala prin furt de ciclu
8. KBUS - rezultatul operatiei de acces la memorie furnizat catre Cm Master
9. Cm Master - preia datele, termina operatia

21. Mecanisme de referinta intercluster (comunicarea intercluster).



1. Cm Master - trimite o cerere de transfer la KMAP Master
2. KMAP Master - pregateste mesaj si codifica cererea
3. Intercluster Bus - mesajul e transmis pe baza algoritmilor de rutare
4. KMAP Slave - decodifica si o trimite catre clusterul sau local
5. Cm Slave - primeste cererea de ciclu de memorie
6. Cm Slave - transmite rezultatul catre KMAP Slave
7. KMAP Slave - pregateste mesajul intercluster
8. KMAP Slave - transmite rezultatul catre KMAP Master
9. KMAP Master - receptioneaza si interpreteaza mesajul primit
10.Cm Master - receptioneaza mesajul

22. Implementarea JOIN si FORK la MIMD.

Exemplu de Utilizare – MIMD

25

- Aceeasi problema: A[n, n], B[n, n], C = A x B
- O UCmd/P trebuie sa preia functia de master: organizare si control + partajarea calculelor pe procesoare individuale
- Conway a propus o metoda cu 2 primitive: FORK & JOIN
  - FORK: desface un FI in n FI executabile simultan pe proc indep
  - JOIN: reuneste n FI intr-unul singur cand cele n FI s-au terminat

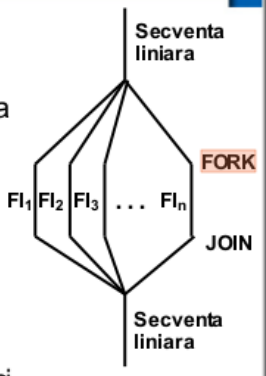
```
for k = 0 to n - 2 (nu si pt el insusi)
   FORK Adr
end k loop
Adr:
   for i = 0 to n - 1
       cik = 0 (0 ≤ k ≤ n - 1) - pe coloane k fix
       for j = 0 to n - 1
             cik = cik + aij * bjk (0 ≤ k ≤ n - 1)
       end j loop
   end i loop
   JOIN
```

Complexitatea este… $O(n^2)$!
→ la fel ca SIMD

Implementarea FORK & JOIN

27

- P_1 (master) pregateste taskurile intr-o coada de asteptare impreuna cu contextele asociate lor
- Celelalte procesoare P_2 … P_n inspecteaza coada pana gasesc un task ce asteapta & il executa
- Daca numarul de procesoare = numarul de procese → se executa simultan toate
- Daca numarul de procesoare << numarul de procese → dupa executia unui proces, un procesor preia din coada un nou proces pt exec
- Atentie: aici procesoarele sunt considerate omogene si procesele independente intre ele (caz f rar in realitate!)



- Instructiunea JOIN n:
  - Pentru fiecare FI exista un contor unic ce e initializat cu 0 la inceput
  - Pentru fiecare FI contorul este incrementat si comparat cu n
  - Trebuie ca toate procesele sa se fi terminat (pana la n) si abia apoi se continua cu urmatoarea secventa liniara pe P_1 (master)

23. Element de procesare Blitzen

Blitzen este un sistem computerizat care foloseste SIMD creat pentru NASA prin anii 80. El era format dintr-o unitate de control si un set de procesoare simple legate intr-o topologie de tip grid.

Aplicatii: procesare de imagini la viteze foarte mari. Fiecare procesor opereaza doar asupra unui pixel si comunica cu vecinii; aplica filtre pe imagini.

24. Comparatie intre comutatorul Cross-Bar si Trunchi K

Asemanari:

- Ambele asigura conexiunea intre oricare doua componente de tip diferit.

Deosebiri:

-**CrossBar**

- permite transferuri multiple si simultane, Ucmd trebuie sa rezolve toate posibilele conflicte.
- Este greu de proiectat fizic
- exista implementari care necesita k ≈ log2n comutatoar elementare

- **Trunchi K**

- prezinta conectivitatea comutatorului CB
- costuri reduse.
- k linii simultane, nu mereu min(m, n). Necesita determinarea unui k minim.

25. De ce nu se poate realiza un procesor de 10TFlops(minim 3 motive) ?

(2009)

- **Distanta dintre memorie si procesor** trebuie sa fie atat de mica incat nu avem inca tehnologia necesara (doar in **quantum computing** e posibil sa se ajunga la acea dimensiune).

-- **Viteza datelor** ar trebui sa fie egala cu viteza luminii, frecventa procesorului sa fie 10^5 GHz

- Consum prea mare de **energie** (proportional cu frecventa)

- **viteza memoriei trebuie sa fie prea mare** pentru timpurile actuale

Pe scurt: **tehnlogia** inca nu face fata unei asemenea provocari.

PROBLEME

1.Se dadea cazul Carrefour care primeste de la fiecare magazin o lista ce conținea un fel de inventar cu numele produsului, cantitatea vândută și prețul produsului.

a) Se cerea să se proiecteze un sistem care realizează interpretarea acestor date folosind doua arhitecturi studiate la laborator. Punctajul se acorda în funcție de argumentele bine formulate în favoarea sau detrimentul arhitecturilor alese.

b) Pentru una dintre arhitecturile de la punctul a, să se schițeze un pseudocod. Se punea accent la formatul fișierelor de intrare/ieșire, cum se transmit datele, chestii specifice arhitecturii în principiu.

2. Se dau doua matrici A,B de dimensiune 4096x4096 .Sa se scrie un preudocod care calculeze produsul celor doua matrici folosind arhitectura Cell:

inmultirea normala a celor doua matrici

inmultirea folosind BMM

La fiecare subpunct, pe langa pseudocod trebuia sa descriem modul in care se vor retine datele in PPU/SPU

3. Ai N bile punctiforme pentru care ti se dau :

x

y coordinate initiale (coordonateme extremitatii initiala a segmentului pe care se deplaseaza bila)

v viteza de deplasare

x_ext

y_ext coordonateme extremitatii finale a segmentului pe care se deplaseaza bila

D

ca doua bile se ciocnesc atunci dispar din sistem.

Se cere sa se afiseze coordonatele la care o bila dispare din system (daca s-a ciocnit cu a olta bila) sau coordonatele extremitatii finale pentru bilele care au ajuns in extremitati in perioada T de timp (data).

a) utilizand instr SIMD calculate distanta parcursa de fiecare bila (se dadea formula distantei dintre 2 pct)

b) sa se modeleze sistemul cu threaduri (pthreads/python threads)

c) sa se modeleze sistemul in Cell

4. De implementat si in Python si in CELL ceva cu o imagine si se impartea la 4 SPU-uri. SPU-urile randau imaginea, o trimiteau inapoi la PPU care o afisa. Nu prea am idee despre ce era vorba exact aici :).
Cam asta era. Ai o animatie, si ti se cere sa o randezi in paralel (nu ti se cer detalii despre randarea propriu-zisa, ci doar despre modul in care imparti sarcinile catre thread-uri(Python) sau SPU-uri (Cell)).

5. Avem o imagine reprezentata printr-o matrice cu elemente de la 0 la 255. Se cere sa se mareasca dimensiunea matricei pana la un X dat astfel:
Exemplu:
Pas 0:
10 30
15 25
Pas1:
10 20 30
12.5 20 30
15 20 25
...
20 = media aritemtica a lui 10 si 30
12.5 = media aritmetica a lui 10 si 15
etc.
1)0.2p Sa se descrie cum paralelizati algoritmul pentru CELL.
2)0.4p Sa se discute despre performantele programului: comunicare, instructiuni SIMD. Se poate folosi Double Buffering? Ce avantaje aduce?
3)0.2p Sa se crie pseudocod SPU si PPU
4)0.2p Este algoritmul viabil pentru CELL? Ce a?i putea folosi altceva pentru a implementa algoritmul?

6. Se da un teren de NxN ?i un nor de MxM cu M < N, norul deasupra terenului ?i incepe sa ploua. Terenul e denivelat si fiecare element din matrice contine inaltimea acelui punct.
Cand ploua apa de pe zonele inalte se revarsa in mod egal pe zonele vecine mai joase

1)Explicati cum se poate paraleliza.
2)Pseudocod CELL.
3)Se pot folosi instructiuni SIMD ? Cum?
4)Se poate folosi double buffering? Cum?
5)Se merita utilizarea CELL pentru aceasta problema? Daca nu, ce ai folosi in schimb?

Caracteristici GPU

Procesorul grafic (GPU - graphics processing unit)

Termenul GPGPU (general purpose graphics processing unit) felxibilitate ridicata la programare, capabil sa rezolve probleme generale

Termenul de GPGPU apare atunci cand unitatea GPU este folosita ca si coprocesor matematic

Arhitectura GPU foloseste paradigma SIMD (taxonomia Flynn):

- **schimb rapid de context intre thread-uri**
- **planificarea in grupuri de thread-uri**
- **orientare catre prelucrari masive de date**

Unitatile tip GPU sunt potrivite pentru paralelismul de date, intensiv computationale. Datorita faptului ca aceleasi instructiuni sunt executate pentru fiecare element, nu sunt necesare mecanisme complexe pentru controlul fluxului.

Domenii ce folosesc procesare GPGPU: prelucrari video si de imagini, simulari de fizica, finante, dinamica fluidelor, criptografie, design electronic

GPU integrat in SoC:

- **RAM Host <-> RAM Device 24GB/s**
- **SoC formta din CPU_SoC si GPU_SoC**

GPU dedicat:

- **Comunicare cu CPU via PCIe**
- **RAM Host 24GB/s**
- RAM Device ⇔ VRAM 300GB/s
- **PCIe 16GB/s**

Pentru a putea executa functia gpuProcess pe DEVICE (GPU), pe partea de HOST (CPU) efectuam urmatorii pasi:

- **selectare unitatea de executie, PLATFORM/DEVICE, interogare mediu OpenCL**
- **initializare context si coada executie comenzi GPU, pregatire mediu OpenCL / GPU**
- **compilare program kernel pentru GPU, selectare kernel - pregatire mediu OpenCL / GPU**
- (1) alocare memorie pentru GPU, efectuare transferuri memorie DEVICE(GPU) ⇔ HOST(CPU)
- **(2) executia de kernel prin modelul NDRANGE pe DEVICE (GPU)**

- (3) efectuare transferuri memorie DEVICE(GPU) ⇔ HOST(CPU)

- **eliberare resurse / deinitializare mediu OpenCL**

Caracteristici CELL

Magistrala EIB

- Comunica SPE si PPE prin ea
- Structura bazata pe 4 inele pentru transfer de date
    - 2 in sens orar si 2 in anti-orar
- Latime de bande = 96 bytes / cycle
- Suporta > 100 cereri DMA intre SPE si Main Storage

PPE (Power Processor Element)

- Procesor pe arhitectura Power (64bit)
- Compatibil cu Linux de unde se pornesc aplicatii care ruleaza pe SPE-uri
- RISC + 32 registri de 128 bits + calcul vectorial
- PPE = PPU (Power Processing Unit) + PPSS (Power Processor Storage Subsystem)
    - PPSS
        - se ocupa de cereri din partea PPE sau a altor dispozitive I/O
        - Contine cache nivel 2 (512KB)
        - Contine cozi si unintate de interfata cu magistrala
    - PPE
        - O versiune modificata in Xbox360

SPE (Synergistic Processor Element)

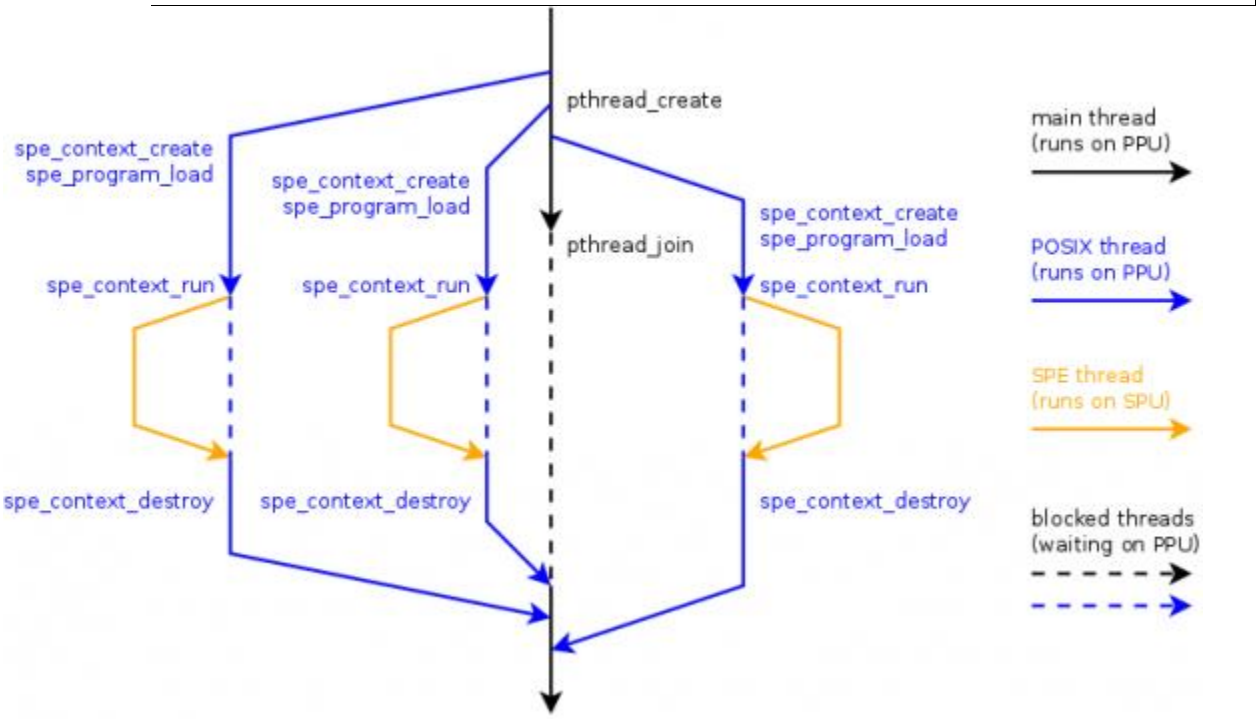
- RISC 128 bits
- SPE = SPU (Synergistic Processor Unit) + Memory Flow Controller
    - SPU
        - Procesoare simple fara cache
        - Local Store de 256KB SRAM
        - 128 reg de 128 biti + 1 x virgula fixa + 2 x virgula mobila

PPE-ul:

- procesor de uz general
- pe el se lansează în execuție programul
- suporta și instrucțiuni vectoriale

SPE-ul:

- execută numai instrucțiuni vectoriale
- lucrează cu datele și execută instrucțiunile din memoria sa locală, Local Store
- nu puteți executa direct programe pe SPE, acestea sunt controlate din programul de pe PPE.



De ce este nevoie de thread-uri pe PPE? **pentru că funcția ce pornește contextele, spe_context_run, este blocantă!**

MIRI PEPI:

1. Organizarea datelor în structurile SIMD și MIMD.

Organizarea datelor in structurile **SIMD**:

Probleme pe care le putem intampina: repartizarea in memoriile Mi & accesul paralel, citirea vectorilor intr-o singura instructiune sau accesul la linii si la coloane in cazul matricelor.

Memoria poate face cel mult un acces la un ciclu de lucru astfel n operanzi in n module pot fi cititi simulan ceea ce face defavorabila plasarea in acelasi modul al operanzilor.

Reprezentarea directa pe linii : acces paralel la linii si secvential la coloane;

Reprezentarea directa pe coloane: acces secvential la linii si paralel la coloane;

Reprezentarea deplasata ciclica: acces paralel la linii & coloanele matricei;

Organizarea deplasata face ca accesul la linii & coloane sa se faca intr-un singur ciclu si costul platit paralelismului este consistenta datelor;

Organizarea datelor in structurile **MIMD**:

Fiecare procesor trebuie sa aiba acces la toata memoria sistemului si capacitatea de adresare a fiecarui procesor e mai mica ca dimensiunea memoriei

Diferenta fiind facuta de faptul ca MIMD lucreaza cu matrice nu cu vectori. Matricile sunt 2D iar memoria 1D asa ca maparea este realizata prin row-major (c/c++, liniile sunt stocate continuu) sau column-major (coloanele sunt stocate continuu).

2. Să se scrie programul de împărțire a doi vectori de dimensiune n pentru o structură SIMD și MIMD. Comentați performanțele.

Presupunem N = numar de procesoare

SIMD:

K = nr procesorului

Ck = Ak/Bk (0 <= K < N);

Performanta este de O(1) deoarece programul este lansat in mod paralel de catre arhitectura;

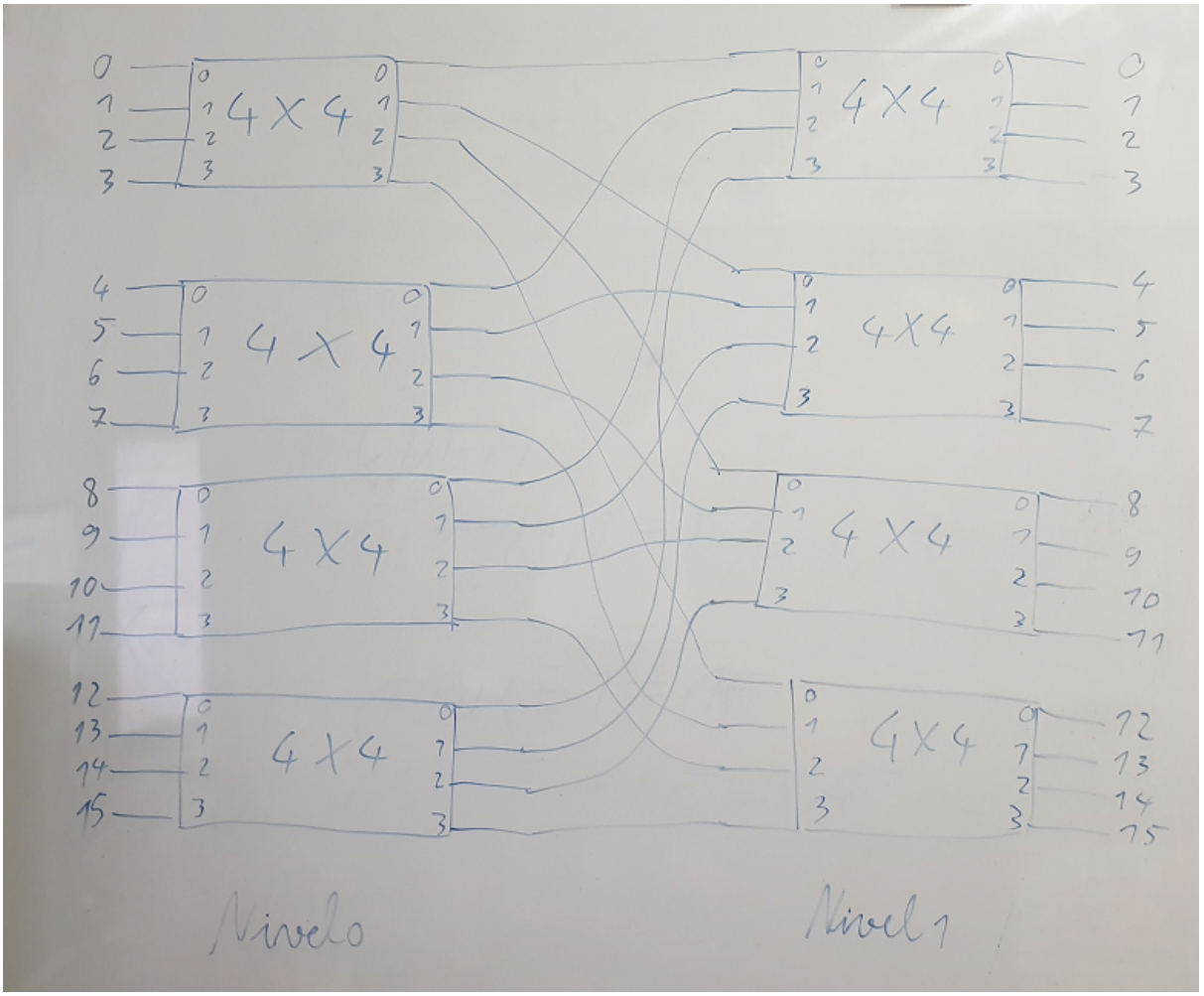
MIMD:

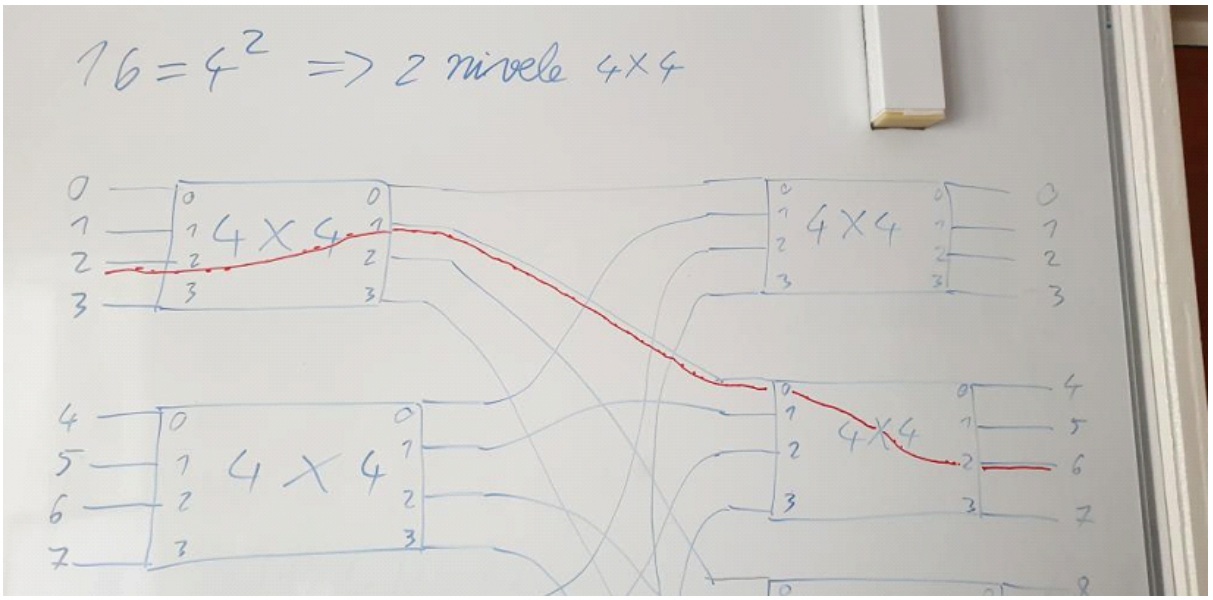
```
For k = 0; k < n-1
        Fork label
Label:
        Ck = Ak/Bk (0<= k < N)
        join();
```

Performanta este O(n) deoarece procesele sunt lansate manual;

3. Să se cupleze 16 procesoare cu 16 memorii utilizând rețea de tip Delta; implicit procesorul i să fie conectat la memoria i. Indicați conexiunea de la procesorul 2 la memoria 6.

16 = 4^2 => 2 nivele 4x4





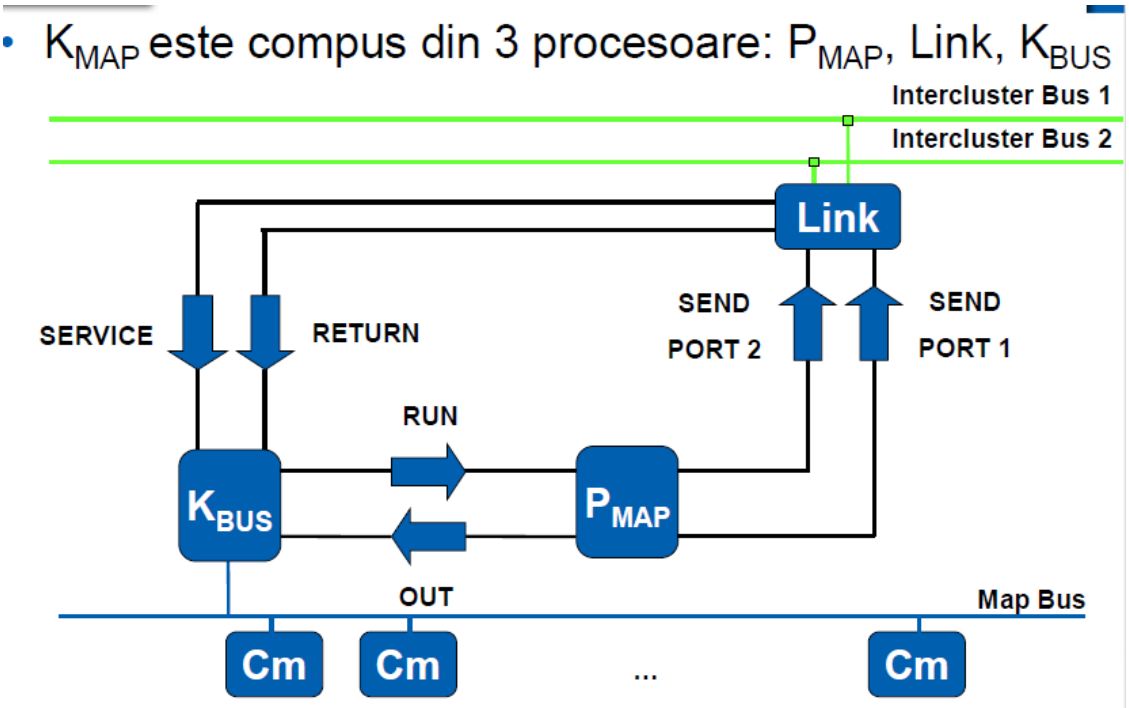
4. Descrieți primitiva P de la calculatorul personal. (CPU) //aici sa va descurcati, revin cu update mai tarziu

5. Considerând o structură multiprocesor pe o magistrală comună descrieți unitatea de acces la magistrala sistemului.

Kbus este procesorul local in relatie cu magistrala externa,

1. Se verifica daca adresa e locala, daca da se adreseaza resurse locale daca nu se trece in starea 2;
2. Se face cerere de acces la magistrala, iar daca cererea este acceptata se trece la pasul 3;
3. Se face bus busy astfel automatul preia functia procesorului, raman in starea asta pentru un nou ciclu daca e cazul.

6. Structura generală Kmap :



7. Comparație între crossbar și delta privind performanțele.

Crossbar are mai multe basic switch-uri decat Delta, dar scaleaza mai bine din punct de vedere al request-urilor servite cu succes, la numar mare de procesoare

10. Comparatie intre intercalarea perfecta si reteaua butterfly.

Intercalarea perfecta(shuffle):

Se foloseste la FFT, algoritmi de sortare si transpuneri, aceasta asigura conectarea intre procesoarele unei structuri SIMD de n procesoare cu n-2 comunicatii simultane, dupa log in baza 2 din n iteratii datele ajung pe procesorul sursa;

Reteaua butterfly:

Presupune interschimbarea celui mai semnificativ bit cu cel mai nesemnificativ.

P_butterfly(an .. a1 a0) = (a0 a(n-1) … a1 a0) - **ASTIA-S INDICI**

P_butterfly(a2 a1 a0) = (a0, a1, a2) - **TOT INDICI SURPRINZATOR**

Permutarea Butterfly este simetrica. Exista doua tipuri de permutare butterfly ce ofera posibilitatea de conectare intre resursele clasei.

Permutarea Butterfly k inferioara:

P_butterfly_k_inf(an … ak a(k-1) … a1 a0) = (an … ak a0 … a1 a(k-1)) **INDICI BBY**

Permutarea Butterfly k superioara:

P_butterfly_k_sup(an … a(n-k+1) a(n-k) … a1 a0) = (a(n-k+1) a(n-1) … an a(n-k) … a0)

11. Descrieti folosind PMS structura cu care ati lucrat la laborator.
//bonus

12. Sa se scrie programul de insumare a elementelor unei matrice A(n, n) pentru o structura SIMD si pentru o structura MIMD. Comentati performantele. Obs. Trebuie specificata organizarea datelor.

SIMD:

```
sumk = 0 ( asta e local sau nu ? )
for i = 0 to n – 1
        sumk += aik (suma coloana k)
end i loop

local_sum = 0 (local procesorului, in registru poate?)
for i = 0 to n – 1
        local_sum += sum[(i + k) mod n] (ca sa nu acceseze toate simultan acelasi modul de
        memorie)
end i loop
```

O(n)

MIMD:

```
for k = 0 to n – 2 (nu si pt el insusi)
   FORK Adr
end k loop
Adr:
   sumk = 0 (0 ≤ k ≤ n – 1)
   for i = 0 to n – 1
         sumk += aki (suma pe linii pentru acces contiguu in memorie)
   end i loop
   JOIN

for k = 0 to n – 1
   sum_final += sumk
```

O(n), ms Vlad Ivan, un scumpic

14. Clasificarea Flynn a sistemelor de calcul.
SISD - un singur flux de date si un singur flux de instructiuni. Nu exista paralelism
SIMD - mai multe fluxuri de date, dar un singur flux de instructiuni. Exista paralelism. S.n. si sisteme de procesare vectoriala.
MISD - paralelism reduntant. E un tip de calculator atipic. Se foloseste in sisteme de calcul unde e nevoie de precizie maxima folosite cand vrei sa faci chestii diferite asupra la aceleasi date si dadea exemplu de chestii meteorologice.
MIMD - clustere, MPI, toate procesoarele participa la acelasi program:
-shared memory/ distributed memory

//daca vreti desene va duceti la pag 150 in asc merged!!!

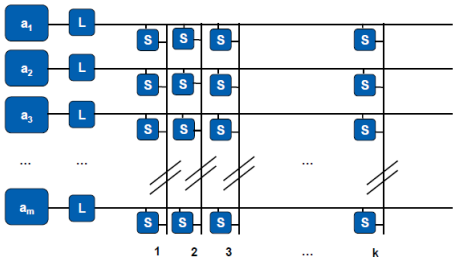
15. Descrieti caracteristicile generale CELL:
Cell e o arhitectura de calculator

Un Cell este compus din:
- 1 Power Processor Element (PPE).
- 8 Synergistic Processor Elements (SPEs). or APU = additional processing unit
- Element Interconnect Bus (EIB).
- Direct Memory Access Controller (DMAC).
- 2 Rambus XDR memory controllers.
- Rambus FlexIO (Input / Output) interface.

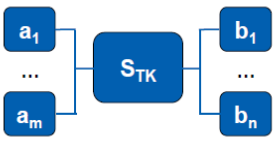
16. Retea de comutare de tip intercalare perfecta si comparatie cu reteaua cu ordine inversa (permutarea de baza). **Vezi ex 36.**

17.Trunchi K ierarhic si neierarhic

Trunchi K neierarhic: face legatura intre elemente de acelasi tip (PP - MM);



Trunchi K ierarhic: se pot face conexiuni intre componente de tip diferit (PM - MP)
exista o singura componenta S care leaga elentele de tip A cu elemente de tip B



20. Organizarea datelor la SIMD

Probleme pe care le putem intampina: repartizarea in memoriile Mi & accesul paralel, citirea vectorilor intr-o singura instructiune sau accesul la linii si la coloane in cazul matricelor.

Memoria poate face cel mult un acces la un ciclu de lucru astfel n operanzi in n module pot fi cititi simulan ceea ce face defavorabila plasarea in acelasi modul al operanzilor.

Reprezentarea directa pe linii : acces paralel la linii si secvential la coloane;

Reprezentarea directa pe coloane: acces secvential la linii si paralel la coloane;

Reprezentarea deplasata ciclica: acces paralel la linii & coloanele matricei;

Organizarea deplasata face ca accesul la linii & coloane sa se faca intr-un singur ciclu si costul platit paralelismului este consistenta datelor;

21. Comparatie SIMD MIMD la algoritmi //faci comparatie pe 2 coduri ?
(poti sa iei exercitiul 2)

22. Probabilitatea ca o cerere sa fie acceptata la un comutator CB (formula)
B(p, m) = rata de transfer

PA = B(p, m) / r * p = (m / r * p) - [m * (1 - r/m) ^p / r * p];

24. Block placement in cache.

Mapare directă - un bloc din cache poate fi așezat într-un singur loc

Mapare (full) asociativă - un bloc din cache poate fi așezat oriunde

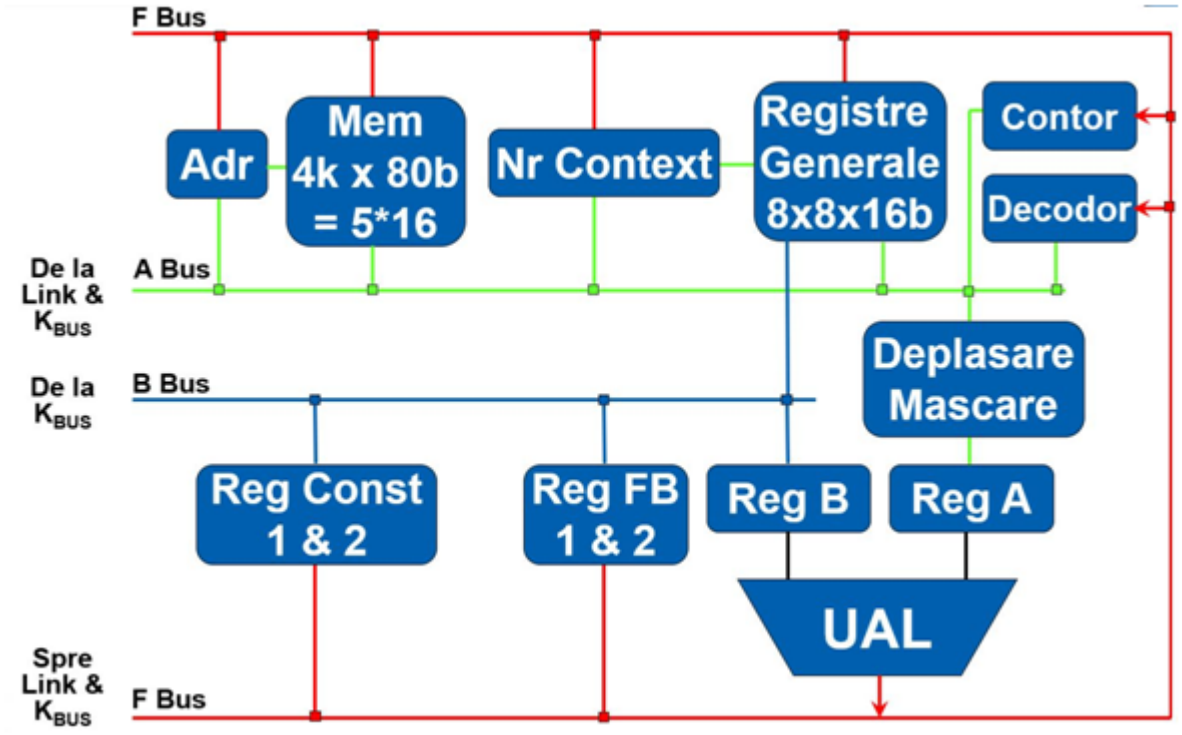
Mapare set asociativă - un blo din cache poate fi așezat într-un subset de poziții

Maparea n-way set asociativă devine din ce în ce mai dificilă și costisitoare de implementat pentru un n mare. Cu cât n-ul este mai mare, cu atât este mai mică probabilitatea de apariție a fenomenului de thrashing(momentul în care două sau mai multe regiuni de memorie sunt accesate în mod repetat și pot încăpea împreună în același bloc din cache.

25. Arhitectura procesorului PMAP.

PMAP este un procesor dedicat pe 16 biți, microprogramat, cu codificare orizontală. Memoria de control este formată din 4k * 80 de biți, iar memoria de control RAM este foarte rapidă, având cuvântul de 80 de biți - Writable Control Store.
Funcțiile fundamentale ale procesorului PMAP sunt:

- Translatarea adreselor nelocale din cluster
- Implementarea primitivelor OS critice dpdv al timpului de execuție
- Culegerea informațiilor statistice pentru îmbunătățirea soluțiilor de proiectare



În registrele generale este salvat contextul execuției în curs.

Este organizat în jurul a 3 magistrale:

A, B pentru operanzi

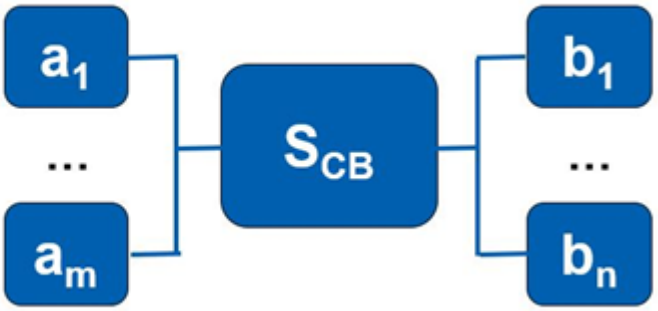
F pentru rezultate

Pentru a putea mapa adresele PMAP, trebuie să aibă acces rapid la informație pentru translatarea adreselor virtuale în adrese fizice. Prelucrarea este în banda de asamblare - operații UAL și operanzii ce vin simultan, separat, prin registrii tampon. Accesul la o memorie în același cluster este mult mai rapid decât în alt cluster. UAL are un operator la nivel de bit pentru registrul A ce asigură deplasarea, rotirea și mascarea(cu 30 de măști predefinite, permițând astfel preluarea descriptorului de context).

26. Comutator legatura multipla (CrossBar) ierarhic. (voia 3 cuv, maxim o fraza: InfiniBand, Linux, Cluster) (ce pula mea?)

Asigură comunicarea între m componente de tip a și n componente de tip b, purtând numele de Cross-Bar-Switch. Oricare A poate comunica cu oricare B, pot avea loc transferuri multiple și simultane. Cel mai dificile de implementat Ucmd pentru Scb - ea trebuie sa rezolve toate posibilele conflicte.

– S {CB, ma, nb, c = min(m, n), m*nSp}



ANDREI:
27. Benchmark-uri, probleme si rezolvari legate de acestea (ceva de genu).
(nu trebuia sa zici de HPCC si LINPACK)
Benchmark-urile reprezintă teste create pentru a putea compara performanțele între diverse sisteme și arhitecturi. Testele constau în seturi de programe menite să scoată în evidență performanțele anumitor componente dintr-un sistem de calcul(Floating Point CPU Performance, Caracteristici de interconectare, Memorii externe).

Probleme:

- Producătorii HW fac “tuning”
- Performanțele mașinilor se degradează când load-ul depășește valoarea de prag
- Securitatea, disponibilitatea, fiabilitatea, serviceabilitatea sau scalabilitatea nu sunt obiectul unui Benchmark

(nu scrie nimic in curs de rezolvarea lor și nici pe net nu găsesc)

Tipuri de benchmark-uri:

- Aplicații reale: procesoare de text, aplicații utilizator
- Kernel: conține doar părți esențiale dintr-o varietate de programe
- Micro-benchmark-uri: dezvoltate de utilizatori pentru a testa componentele sistemului
- Sintetice: se strând date statistice asupra operațiilor efectuate de aplicații și se construiesc programe care să respecte proporția aflată astfel
- Pentru sisteme I/O
- Pentru sisteme paralele

MIRI + PEPI:
28. Tipuri de paralelizare la sistemele cu prelucrare paralela.

1. Paralelism la nivel de bloc/job, intre joburi diverse avem nevoie de, mecanism de salvare de context, ceas de sincronizare, canale de I/O pt transfer date;

   Anumite faze ale unui job pot fi suprapuse

2. Paralelism la nivel de subansamble hardware: intre elem de prelucrare a vectorilor, intre componentele logice ale dispozitivelor aritmetico/logice (gen carry lookahead)
3. Paralelism la nivel de program: intre diverse sectiuni independente ale unui program, intre loopurile unui program; ambele au nevoie de analiza dependentelor de date
4. Paralelism la nivel de instructiune (mancati cacatu cu pipeline si branch predictor):

   Seriala - fara pipeline

   Seriala paralela - gen pipeline da poa sa fie si inseriate instructiunile nuj

   Paralela - full pipeline

29. Structuri SN. P i/e structura (ce plm SN? SISTEME NUMERICE)

PMS(Processor Memory Switches)

- Pc = procesor central
- M = memoria
- S = comutatorul
- L = linia de legatura
- K = unitatea de comanda
- P i/e = procesorul de intrare/iesire
- D = operatorul de date
- T = terminalul

Structura Procesor de i/e:

Primitiva ce poate fi incorporata in PC-ul in care are functie de I/E sau poate fi privita independent;

Are urmatoarele atribute:

- Fucntie: DMA/canal I/E
- Implementare: conventional, microprogramat
- Rata de transfer: Mb/s / Gb/s
- Mod de transfer : prin furt de ciclu sau prin rafala
- Lungimea cuvantului 8/16 biti

ANDREI:

30. Block replacement

Atunci când are loc un miss, controlerul de cache trebuie “sa facă ceva”. Pe un cache mapat direct, lucrurile sunt simple - se va suprascrie conținutul unui block-frame cu date noi aduse din memorie. Într-o implementare n-set-asociativă însă, trebuie să alegem care dintre cele n block-frame-uri trebuie suprascrise.

În practică, există 3 strategii implementate pentru acest lucru:

- Random - foarte ușor de implementat
- FIFO - un pic mai dificil de implementat, dar este mai bun în cazul cache-urilor mici
- LRU - este cel mai dificil de implementat, în special pentru cache-urile de dimensiuni mari, dar este mai bun în cazul cache-urilor de dimensiuni mici. Totuși, trebuie avut în vedere costul cache-ului față de latența oferită de aceasta: overhead-ul computațional și hardware-ul adițional necesar calculării valorilor LRU.

PEPI:

31. Determinarea dimensiunii cache-ului

Pentru a determina dimensiunea cache-ului, vom tine cont de faptul ca adresele sunt pe n biti. Acesti n biti sunt compusi din t biti de de tag, s biti pentru index set si b biti pentru offset. Un bloc va avea deci 2^b cuvinte, iar memoria cache va avea 2^s linii. Dimensiunea cache-ului va fi de 2^(s+b) cuvinte.

Daca am considera L lungimea liniei de cache, cu ajutorul codului:

```
char x[1024];
for ( step = 0; step < 1000; step++)
        for (i = 0; i < 1024; i+= L)
                x[i]++;
```

Cazul 1: Cache > 1024 bytes

Dupa prima iteratie a buclei step, tot vectorul e in cache si nu vor mai exista miss-uri
Exemplu : Cache destul de mare pentru a contine 16 liniii, vectorul intra in 11 linii de cache. Astfel, vor fi 11 miss-uri si apoi vor exista doar hit-uri.Pentru un numar mare de iteratii, hit-rate-ul va fi aproape de 100%

Cazul 2: Cache < 1024 bytes

Vor aparea miss-uri
Exemplu: Cache-ul are 16 linii. Vectorul ocupa 19 linii de cache. Initial vor fi 19 miss-uri. Apoi s-ar citi aceleasi 19 linii de cache din care doar 16 intra in cache. Vom avea astfel 13 hit-uri si 3 miss-uri pentru fiecare iteratie. Pentru un numar mare de iteratii, se ajunge la un hit-rate de 81%.

Daca vectorii sunt mici si intra in cache hit-rate = 100%

Daca vectorii sunt mai mari decat cache-ul hit-rate < 100%

32. Comunicatia Kbus Pmap

KBUS asigura controlul in cadrul unui cluster prin excluderea mutuala a acceselor la MapBus. Functiile de baza ale PMAP sunt : maparea de adrese, comunicarea, sincronizarea.

PE SCURT:

Kbus este unitatea de comanda a map bus.Pmap este un procesor de mapare care face translatia de adrese.KBUS memoreaza STAREA contextelor si le aloca cereri. Le trimite prin RUN la PMAp. PMAP proceseaza cererile de la KBUS care activeaza schimbarile de context cerute de KBUS. PMAP pune iesirea la OUT.

PE LUNG:

KBUS mentine starea a 8 contexte si le aloca cereri de serviciu (fiecare va fi rezident in PMAP). Numarul contextului si alte informatii sunt plasate in coada RUN. Din RUN sunt preluate de PMAP care activeaza contextul si eventual translateaza adresa si o plaseaza in OUT. In acest timp KBUS poate citi alte cereri sau efectua alte functii cerute de PMAP. In PMAP se executa o schimbare de context prin intermediul celor doua cozi (RUN/OUT). KBUS preia acum cererea din OUT si transfera adresa si datele la S-ul local al destinatiei; apoi se ocupa de alte cereri. Cand accesul la memorie este terminat, KBUS citeste confirmarea de terminare sau citeste datele si plaseaza iara contextul in RUN, reactivandu-l. KBUS trimite apoi confirmarea la procesul ce a initiat cererea de serviciu. Contextul poate fi utilizat acum pentru o noua cerere de serviciu

33. Permutarea Butterfly

Presupune interschimbarea celui mai semnificativ bit cu cel mai nesemnificativ.
P_butterfly(an .. a1 a0) = (a0 a(n-1) … a1 a0) - **ASTIA-S INDICI**
P_butterfly(a2 a1 a0) = (a0, a1, a2) - **TOT INDICI SURPRINZATOR**

Permutarea Butterfly este simetrica. Exista doua tipuri de permutare butterfly ce ofera posibilitatea de conectare intre resursele clasei.

Permutarea Butterfly k inferioara:
P_butterfly_k_inf(an … ak a(k-1) … a1 a0) = (an … ak a0 … a1 a(k-1)) **INDICI BBY**

Permutarea Butterfly k superioara:
P_butterfly_k_sup(an … a(n-k+1) a(n-k) … a1 a0) = (a(n-k+1) a(n-1) … an a(n-k) … a0)

34. Benchmark-uri pentru sisteme seriale

Benchmarking – nevoia de a compara sisteme de calcul

SPEC (Standard Performance Evaluation Corporation) realizeaza benchmark-uri pentru sisteme de calcul si le publica pe web. Incearca sa simuleze situatii reale: – SPECweb testeaza performantele unui server de web pentru diferite tipuri de cereri HTTP paralele – SPEC CPU testeaza CPU-ul ruland. SPEC sunt scrise in C sau Fortran (compilatoarele pot avea optimizari pentru acest tip de aplicatie). Se distribuie cu o licenta. Exista un conflict cu GPL (foloseste cod GPL).
CA SA TE DAI JMEQUER:

TPC (Transaction Processing Performance Council) a definit benchmark-uri pentru procesarea tranzactiilor si a bazelor de date.

Consortiul BAPCo (Business Application Performance Corporation) este format din Intel, AMD, Dell, HP, NVidia, Apple & Microsoft, etc: – WEBmark, SYSmark 2004SE, si MobileMark.

SiSoftware Sandra (System ANalyser, Diagnostic and Reporting Assistant) este un program de testare si diagnoza

35 Cell BE bariere si solutii.

Toate APU-urile pot funcționa independent. Sincronizarea se face în hardware. Se evită overheadul software-ul. Memoriile de la ambele capete au informații suplimentare despre stare.
Fiecare bucată de memorie adresabilă de 1024 biți în DRAM are:

Bit complet / gol

Memorie pentru APU ID și adresa APS LS
Fiecare APU are un bit ocupat pentru fiecare parte adresabilă a stocării locale.
Bit complet / gol - datele sunt curente dacă bitul e 1:

APU nu poate citi datele dacă este 0

APU isi va lasa ID-ul și adresa de stocare locală => a doua APU va fi refuzată
Bit ocupat:

Dacă 1, APU poate utiliza pentru a scrie informații din DRAM

Dacă este 0, APU poate scrie date

36. Comparatie intre permutarea de baza si perfect shuffle
Pemutarea de baza:

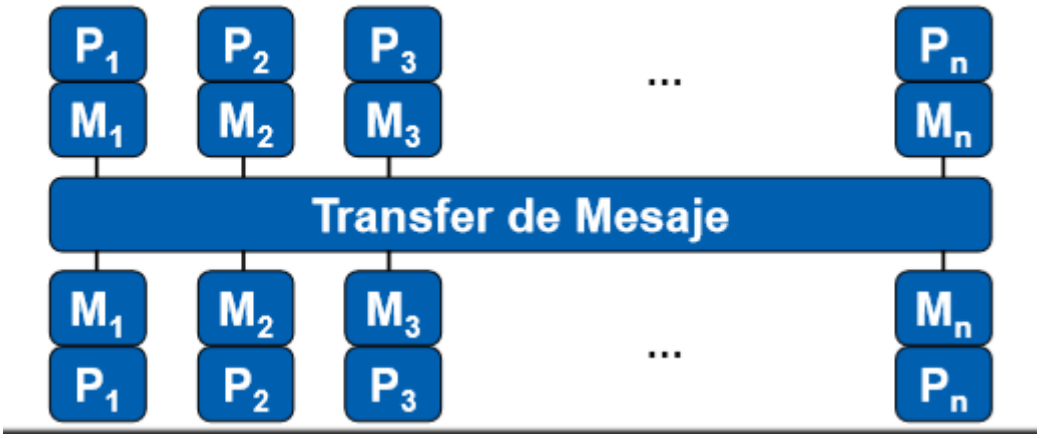
Adecvat pentru operatii recurente, calculabile in log2n pasi.
Bitul cel mai nesimnificativ devine cel mai semnificativ.
P_base(an a(n-1) … a1 a0) = (a0 an a(n-1) … a1) INDICI PRIETENI PESTE TOOT
Permutarea de baza este intercalarea perfecta inversa.
Pentru 8 resurse avem P_base (a2, a1, a0) = (a0 a2 a1)
Variante posibile:
Pemutarea de baza k inferioara
- cei mai nesimnificativi k biti din adresa
- asigura conexiunea IN submultimi de resurse
P_b_kinf = (an … ak a(k-1) … a1 a0) = (an … ak a0 a(k-1) … a1)
Pemutarea de baza k superioara
- cei mai seminificativi k biti din adresa
- asigura conexiunea INTRE submultimi de resurse
P_b_ksup = (an … a(n-k+1) a(n-k) … a0) = (a(n-k+1) an … a(n-k+2) a(n-k) … a0)

Perfect Shuffle:
Adecvat pentru Transformata Fourier (FFT – Fast Fourier Transformation), Algoritmi de sortare, Transpuneri de matrice.
Asigura conectarea intre procesoarele unei structuri SIMD de n procesoare cu n-2 comunicatii simultane, dupa log2n iteratii datele ajung pe procesorul sursa
P_shuffle(an a(n-1) … a1 a0) = (a(n-1) … a1 a0 an) INDICI PRIETENI PESTE TOOT

Bitul cel mai semnificativ devine cel mai nesimnificativ.
Pentru 8 resurse avem P_shuffle (a2 a1 a0) = (a1 a0 a2)
Variante posibile:
Pemutarea shuffle k inferioara
- cei mai nesimnificativi k biti din adresa
- asigura conexiunea memoriilor - adresa de baza si offset intr-un modul
P_shuffle_kinf = (an … ak a(k-1) … a1 a0) = (an … ak … a1 a1 a(k-1)))
Pemutarea shuffle k superioara
- cei mai seminificativi k biti din adresa
P_shuffle_ksup = (an … a(n-k+1) a(n-k) … a0) = (a(n-1) … a(n-k+1) an a(n-k) … a0)

37. Memorii distribuite
Din punct de vedere al accesului la memorie, structurile MIMD sunt:
1. Sisteme cu memorie partajata (Shared Memory)
2. Sisteme cu memorie distribuita (Distributed Memory)
    2.1. Gestiune dinamica a accesului la memoria distribuita
    2.2. Gestiune statica a accesului la memoria distribuita

Accesul se face prin transfer de mesaje. Sistemele au la baza mai multe noduri legate printr-o retea de comutatie de mesaje (nu neaparat fizica). Conexiunea este statica, punct la punct intre noduri. Memoria locala este adresabila direct doar de catre procesorul local.



38. Optimizari programe (trebuia vb de cache)
Aici merge zis de C++/Java ca sunt row major si ca exploatează localitatea spatiala.
Localitate spatiala +/- localitate temporala: Ce sunt, ce înseamnă, când apar.Cum ajuta cache: cache hit + evitare trashing.

Localitatea spatiala: cand se acceseaza o locatie anume de memorie, este foarte probabil ca urmatorul acces sa fie in apropierea acestei locatii in memorie - Ar fi astfel ideala aducerea “aproape” de procesor, a blocurilor contigue de date pentru ca instructiunile urmatoare sa le gaseasca disponibile.

Localitatea temporala: cand se acceseaza o locatie anume de memorie, este foarte probabil ca aceasta locatie sa mai fie accesata din nou - Tinerea datelor accesate de curand, “aproape” de procesor, pentru ca instructiunile urmatoare sa le gaseasca disponibile

Putem folosi maparea asociativa, in loc de cea directa pentru evitarea thrashingului (O linie de memorie poate fi plasata oriunde in Cache). Exista mai multe moduri de a imbunatatii performanta cache-urilor si anume

- Reducerea penalitatilor unui Cache Miss - Reducerea ratei de Cache Miss-uri

- Reducerea timpului de rezolvare a unui Hit - Imbunatatirea memoriei

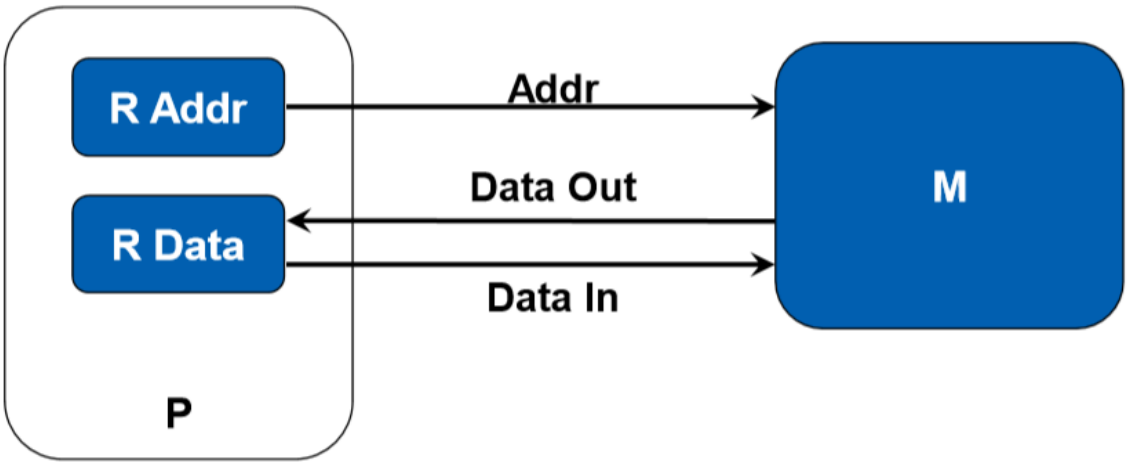
- Performanta crescuta

- Cost redus

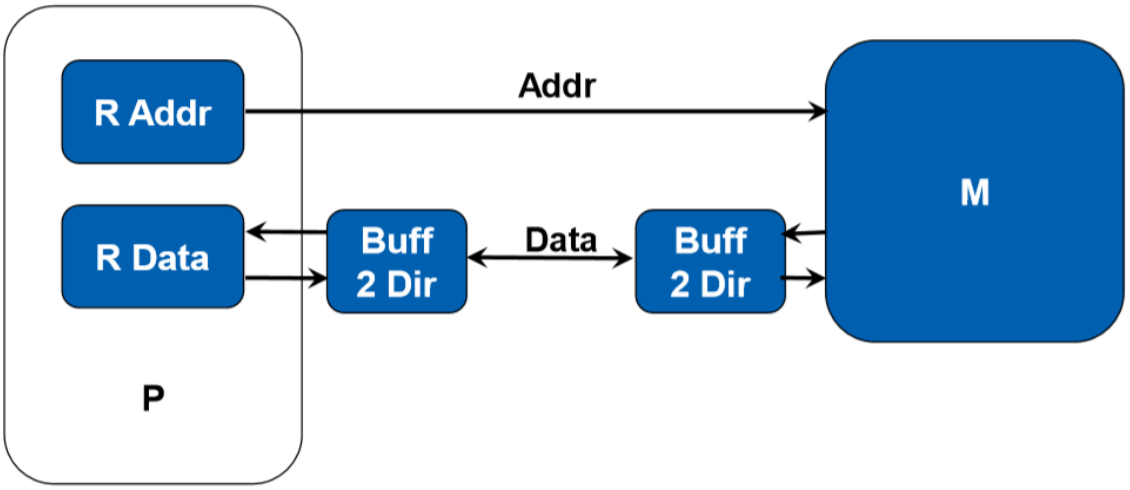
Optimizări în cod: reorganizarea buclelor, folosirea inteligenta a variabilelor (ca în lab la sum pe matrice).

39 Comutatoare in cuplarea memorie procesor.

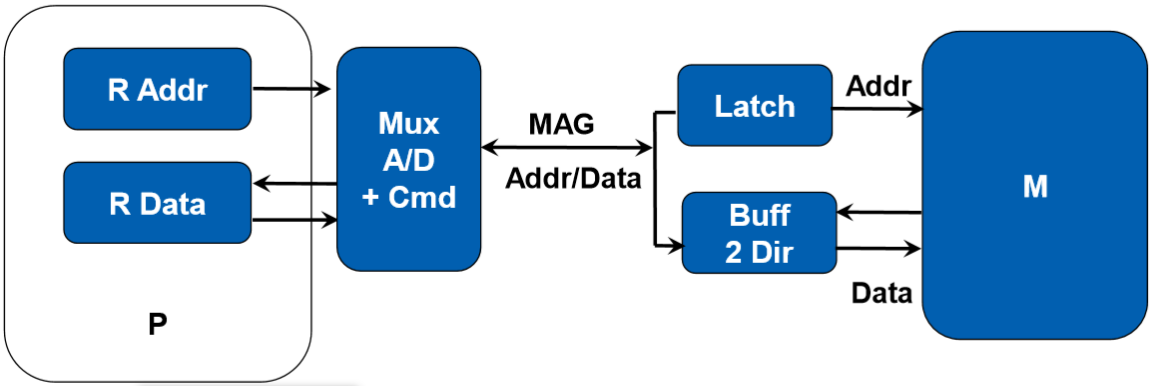
Switch Processor Memory → SPM
Liniile de adrese si date sunt complet separate
R/W – Disjuncte → No Switch! (nu se justifica ambele linii)



DIn si DOut sunt multiplexate prin magistrala de date
Bufferele sunt fie MUX fie TS.



Addr, DIn si DOut sunt multiplexate prin aceeasi magistrala
Bufferele sunt fie MUX fie TS



40 Retea de comutare de tip interclasare perfecta.

Perfect Shuffle:

Adecvat pentru Transformata Fourier (FFT – Fast Fourier Transformation), Algoritmi de sortare, Transpuneri de matrice. Asigura conectarea intre procesoarele unei structuri SIMD de n procesoare cu n-2 comunicatii simultane, dupa log2n iteratii datele ajung pe procesorul sursa

P_shuffle(an a(n-1) … a1 a0) = (a(n-1) … a1 a0 an) INDICI PRIETENI PESTE TOOT

Bitul cel mai semnificativ devine cel mai nesimnificativ.

Pentru 8 resurse avem P_shuffle (a2 a1 a0) = (a1 a0 a2)

Variante posibile:

Pemutarea shuffle k inferioara

- cei mai nesimnificativi k biti din adresa

- asigura conexiunea memoriilor - adresa de baza si offset intr-un modul

P_shuffle_kinf = (an … ak a(k-1) … a1 a0) = (an … ak … a1 a1 a(k-1)))

Pemutarea shuffle k superioara

- cei mai seminificativi k biti din adresa

P_shuffle_ksup = (an … a(n-k+1) a(n-k) … a0) = (a(n-1) … a(n-k+1) an a(n-k) … a0)

41 Sa se scrie programul SIMD si MIMD care aduna 2 vectori (pe elemente). Sa se analizeze performantele. Specificati organizarea datelor (Vezi ex 18)

SIMD: se lanseaza in mod paralel by default din arhitectura

ck = ak + bk (0 ≤ k ≤ n – 1) => O(1)

MIMD: trebuie sa lansam noi fiecare proces O(n)

for k = 0 to n – 2 (nu si pt el insusi)

FORK Adr

end k loop

Adr: ck = ak + bk (0 ≤ k ≤ n – 1)

JOIN

Organizare datelor: vezi ex 1

Presupunem N = numar de procesoare

SIMD:

K = nr procesorului

Ck = Ak + Bk (0 <= K < N);

Performanta este de O(1) deoarece programul este lansat in mod paralel de catre arhitectura;

MIMD:

```
For k = 0; k < n-1
        Fork label
Label:
        Ck = Ak + Bk (0<= k < N)
        join();
```

Performanta este O(n) deoarece procesele sunt lansate manual;

42 Comparatia generala intre SIMD si MIMD

Diferente SIMD/MIMD pentru codul A x B =C:

- In SIMD procesoarele se sincronizau instructiune cu instructiune
- In MIMD nu exista (neaparat) sincronizari intre FI (flux de instructiuni) ale P-urilor din structura
- La SIMD se calculeaza elementele lui C pe linie si coloana (FI unic)
- La MIMD orice procesoare pot calcula orice elemente din C (fiecare P are un FI propriu!)
- MIMD: mai usor de programat & utilizat decat SIMD – dar, mai scump

Organizarea datelor in structurile **SIMD**:

Probleme pe care le putem intampina: repartizarea in memoriile Mi & accesul paralel, citirea vectorilor intr-o singura instructiune sau accesul la linii si la coloane in cazul matricelor.

Memoria poate face cel mult un acces la un ciclu de lucru astfel n operanzi in n module pot fi cititi simulan ceea ce face defavorabila plasarea in acelasi modul al operanzilor.

Reprezentarea directa pe linii : acces paralel la linii si secvential la coloane;

Reprezentarea directa pe coloane: acces secvential la linii si paralel la coloane;

Reprezentarea deplasata ciclica: acces paralel la linii & coloanele matricei;

Organizarea deplasata face ca accesul la linii & coloane sa se faca intr-un singur ciclu si costul platit paralelismului este consistenta datelor;

Organizarea datelor in structurile **MIMD**:

Fiecare procesor trebuie sa aiba acces la toata memoria sistemului si capacitatea de adresare a fiecarui procesor e mai mica ca dimensiunea memoriei

Diferenta fiind facuta de faptul ca MIMD lucreaza cu matrice nu cu vectori. Matricile sunt 2D iar memoria 1D asa ca maparea este realizata prin row-major (c/c++, liniile sunt stocate continuu) sau column-major (coloanele sunt stocate continuu).

43 Organigrama KML de la SBC-uri + schema logica

KML – asigura accesul la Mem locala a PL prin intermediul SD1 si SA1 & a proc externe prin SD2 si SA2

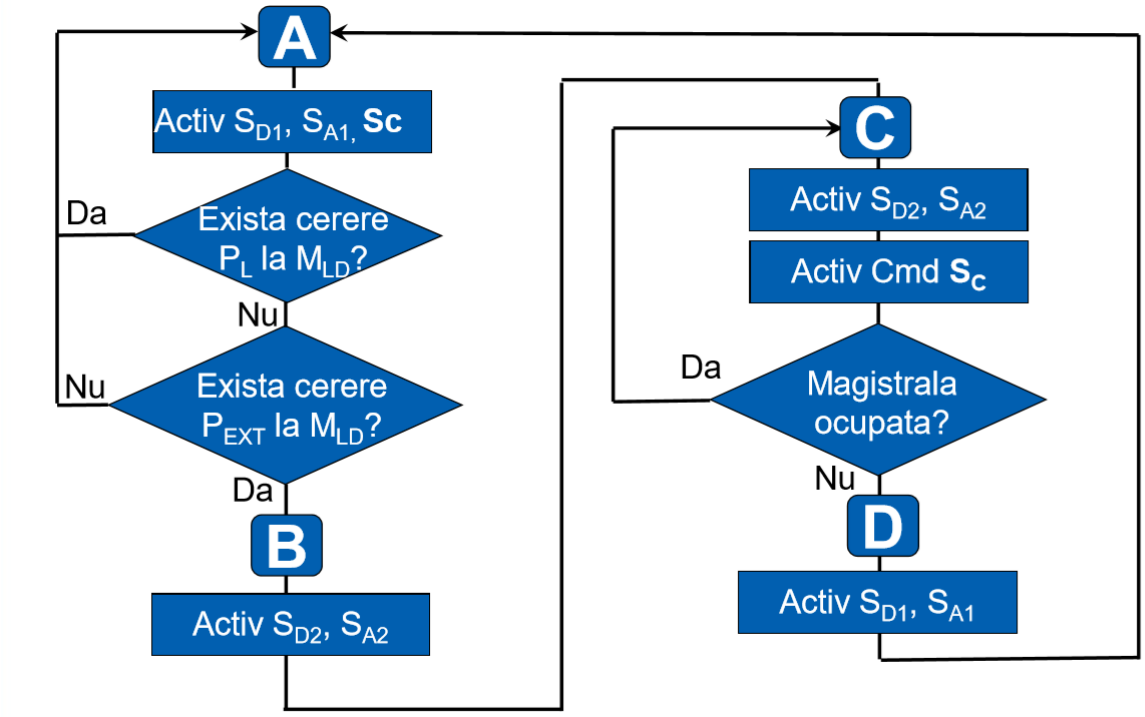
• PL are prioritate fata de proc externe (PEXT) == daca vrea PL la MLD NU ma intereseaza ce vor celelalte PEXT

• Starea A: PL poate avea comenzi locale

• Starea B: activez accesul ext la MLD – semnalele asociate starii

• Obs: Mem locala e vazuta de fiec proc intr-un ac spatiu de mem, iar de proc ext in spatiul complementar → trebuie sa existe o unitate de translatare a adreselor (TA)

• Magistrala ocupata: nu stii ciclii masina ai unui proc extern. Numai cand magistrala e libera un PEXT a terminat ciclul masina curent → abordare top-down



44 Sa se scrie programul ce determina produsul elementelor unei matrice pe SIMD si MIMD. Sa se analizeze performanta. Reprezentarea datelor. (Vezi ex 12 doar ca faci * in loc de +)

//TODO UITAT-TE INCA O DATA

SIMD:

prodk = 0 (asta e local sau nu ?)

for i = 0 to n – 1

    prodk += aik (produs coloana k)

end i loop

local_prod = 0 (local procesorului, in registru poate?)

for i = 0 to n – 1

    local_prod += prod[(i + k) mod n] (ca sa nu acceseze toate simultan acelasi modul de memorie)

end i loop

O(n)

MIMD:

```
for k = 0 to n – 2 (nu si pt el insusi)
   FORK Adr
end k loop
Adr:
   prodk = 0 (0 ≤ k ≤ n – 1)
   for i = 0 to n – 1
         prodk *= aki (suma pe linii pentru acces contiguu in memorie)
   end i loop
   JOIN

for k = 0 to n – 1
   prod_final += prodk
```

O(n), ms Vlad Ivan, un scumpic

45 Exemplificati primitivele P si M de pe calculatorul propriu
(spui ce CPU si ce Memorie ai);

46 Comunicatia intercluster intr-un sistem MIMD slab cuplat

Comunicarea intre procesoare se face prin transfer de mesaje si comutare de circuite.

1. Cm Master trimite o cerere de transfer la Kmap Master
2. Kmap Master pregateste mesaj intercluster si codifica cererea
3. Mesajul intercluster e transmis pe Intercluste Bus pe baza alg. De rutare
4. Kmap Slave decodifica cererea primita si o trimitre catre cluster sau local
5. Cererea de ciclu de memorie este transmisa la Cm Slave
6. Cm Slave transmite rezultatul catre Kmap Slave
7. Kmap Slave pregateste mesajul
8. Kmap Slave transmite Kmap Master
9. Kmap Master receptioneaza si interpreteaza mesajul
10. Rezultatul trimis catre Cm Master.

Sistemele slab cuplate sunt eficiente atunci cand interactiunea intre procesoare este minima(granularitate mare)

50 Organizarea datelor in SIMD. Realizarea inmultirii unei matrici cu ea insasi (Vezi ex 1 + ex 12)

51 Care sunt cele 4 elemente de care trebuie tinut cont cand se realizeaza designul unui cache.

Q1 : Block placement - unde o sa fie scris in cache
Q2: Block identification - cum se gaseste o linie in cache
Q3: Block replacement - cum se inlocuieste o linie in cache la un miss
Q4: Write strategy - ce se intampla cand se scrie in cache (write through / write back)

52 De ce nu se poate realiza un procesor de 10TFlops(minim 3 motive) ?

- Am avea nevoie de un procesor de 10.000 GHZ
- Datele ar trebui sa circule cu viteza luminii
- O celula de memorie ar trebui sa fie de dimensiunea a cativa atomi

53 Caracteristici HPC Challenge Benchmark.

Avantajul major a HPCC este ca arhitecturile pot fi descrise cu mult mai multe metrici decat Flopsurile LINPACK.

Scopurile sunt de a oferi benchmark uri care exprima performanta aplicatiilor, sa permita optimizari sa arhiveze rezultatele. Acesta e format din 7 benchmark uri diferite si e un framework pentru adaugarea benchmark-urilor relevante.

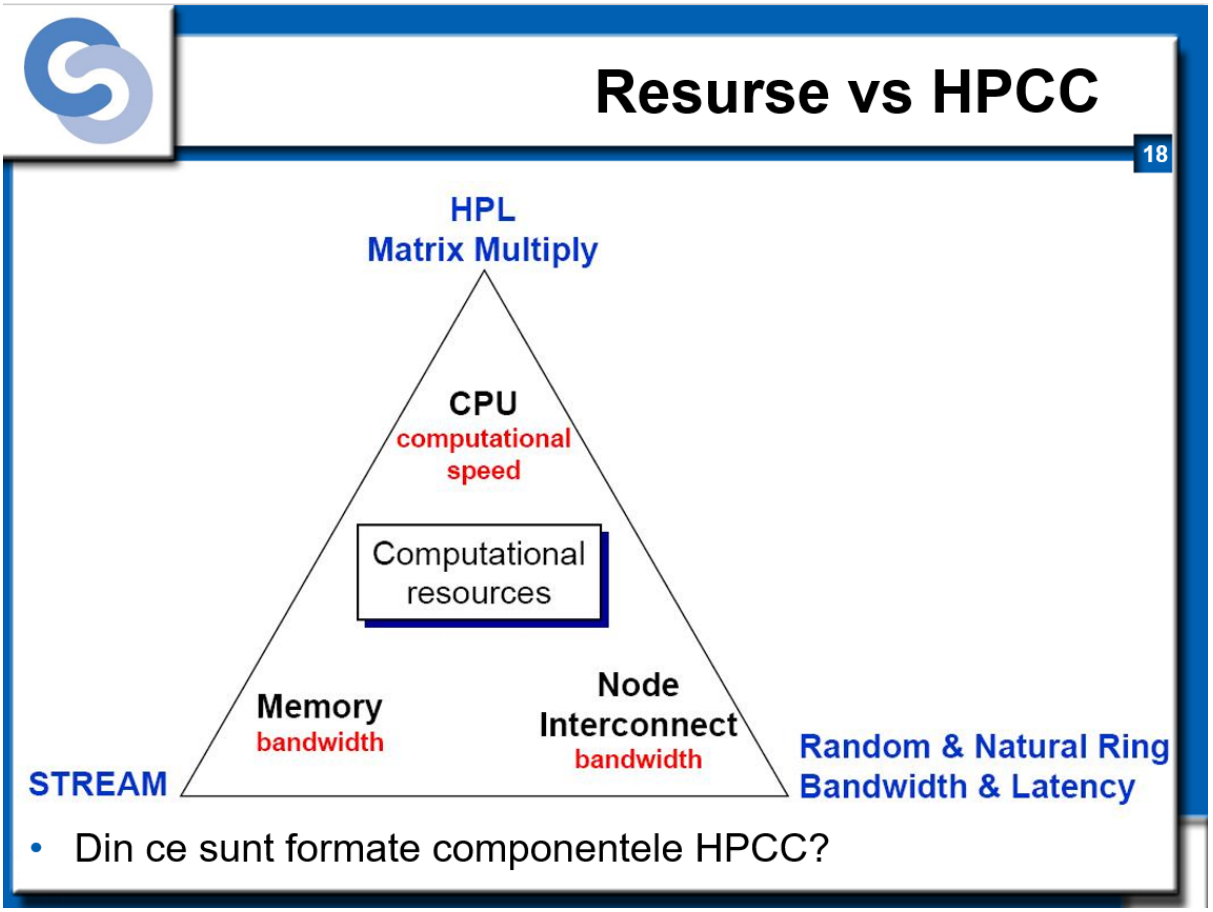
HPCC este format dintr-un “Base Run”, urmat de “Optimization Runs”

-> in Base Run e rularea “out of the box”

-> trebuie sa instaleze pe masinile de test MPI si Blas

-> in O.R. se pot inlocui anumite subrutine pentru a creste performantele unei anume componente

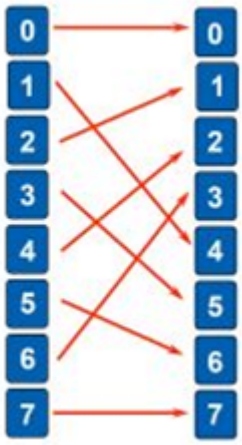
Pentru nivelele la care lucreaza:



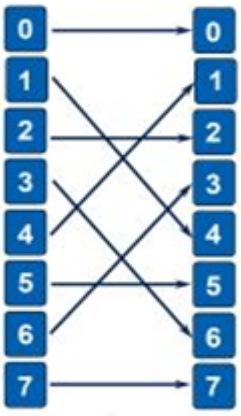
54 Comparatie Base Line Permutation - Bit Reversal Permutation

În cazul Base Line Permutation, bitul cel mai nesemnificativ devine cel mai semnficativ. Permutarea de bază este intercalarea perfectă inversă. Pe de altă parte, la Bit Reversal Permutation se interschimbă complet ordinea biților. Prin această permutare putem obține echivalențe.

BLP - P_base(an, an-1, … a1, a0) = (a0, an, an-1, … , a1)



BRP - P_reversal(an, … , a1, a0) = (a0, a1, …, an)

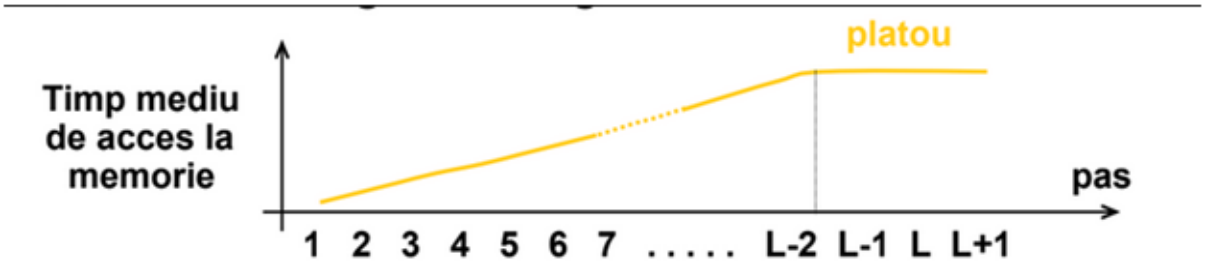


55 Cum se masoara dimensiunea liniei unui cache? Exemplu de program si grafic. (pentru inlocuirea oricarui alt subiect :)

Începem cu pasul 1 pentru i, și îl creștem incremental (+=2, +=3 etc) până când performanța codului nu se mai înrăutățește.

```
char a[N];
for (int i = 0; i < N; i++)
        a[N]++;
```

La pasul 1, avem cache miss la fiecare indice care se împarte exact la L. Pasul de la care nu se mai înrăutățește performanța reprezintă dimensiunea liniei de cache.



56 Arhitectura unui calculator din top 500 (arhitectura, numar procesoare, so, ...)

Oricare de aici: https://www.top500.org/lists/2019/11/

57. **Instructiuni masina SIMD**

- Incarcare vector: load A (ACC[k] <- A[1, k]), unde ACCi este un reg acumulator care contine info locala pentru Pi
- Memorare vector: STO A
- Adunare vector: ADD A[i]
- Inmultire vector: MUL A[i]
- Difuzie scalara: BCAST i

58. **Implementarea salturilor conditionate SIMD**

Salturile conditionate se implementeaza prin divizarea fluxului de instructiuni: un flux pentru conditia indeplinita si altul pentru conditia neindeplinita. Cele doua FI nu pot fi executate simultan, ci la momente diferite de timp si in mod secvential. Se folosesc operatii de mascare a activitatii procesoarelor PI, fiecare procesor avand asociat cate un bit de mascare intr-un registru masca: Pi - MASK[i]; Astfel, daca MASK[i]=0 => procesorul PI nu functioneaza, daca MASK[i] = 1 => Pi nu functioneaza. Operatiile de testare a masitolor se realizeaza in fiecare Pi.

59. **SIMD vs MIMD**

Clasificarea Flynn a sistemelor de calcul
--Taxonomia lui Flynn

• Impartirea sistemelor de calcul in functie de:

– Fluxul de Instructiuni – secventa de instructiuni executate de procesor

– Fluxul de Date – secventa de operanzi manipulata de procesor

• Bazate pe acest criteriu se desprind:

• I – SISD = Single Instruction Single Data Stream (structura Von Neumann)

• II – SIMD = Single Instruction Multiple Data Stream

• III – MISD = Multiple Instruction Single Data Stream

• IV – MIMD = Multiple Instruction Multiple Data Stream

SIMD e de obicei mai ieftin (+ less hardware)

MIMD e mai flexibil, se preteaza la o clasa mai larga de probleme

SIMD nu merge bine pe cod cu multe branchuri

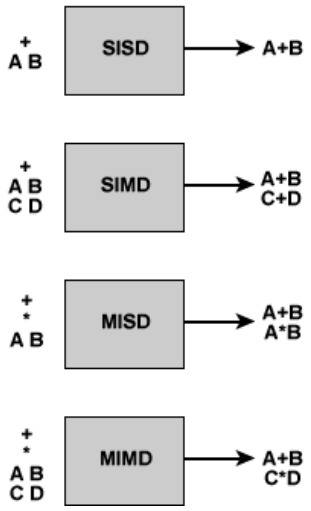
(ziceti si de definitie)

REPREZENTAREA DATELOR PE SIMD SI MIMD

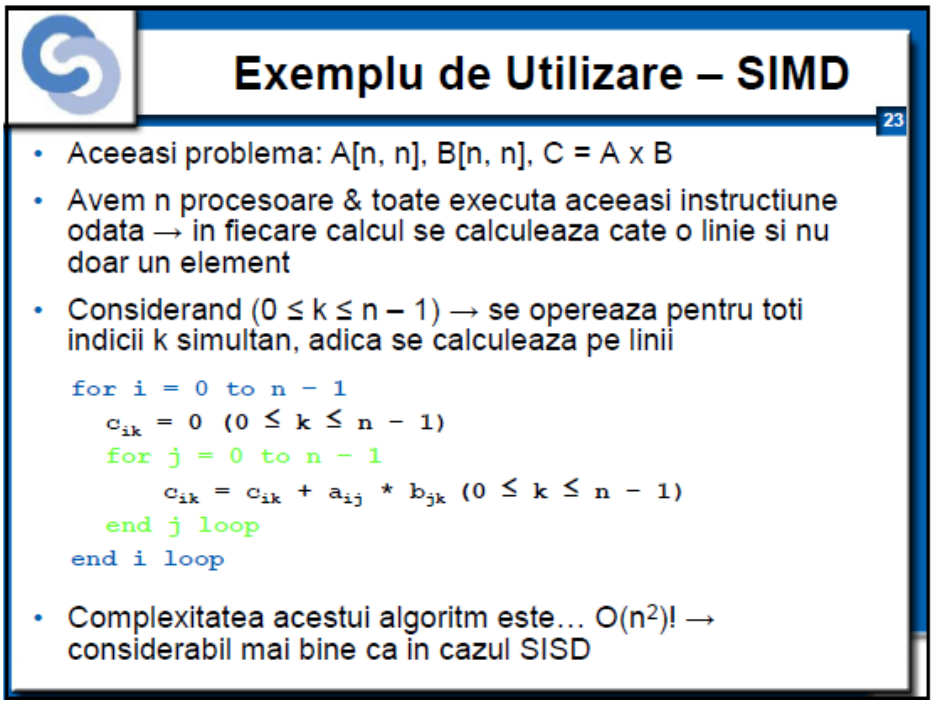
MIMD -> memorii separate , UMA, NUMA,<CC-NUMA>, COMA

SIMD -> memorie comuna, procesare batch, < bus larg >

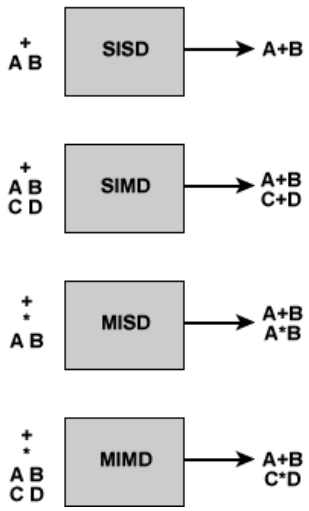
Flynn.
Capabilitatile celor 4 clase de sisteme de calcul.



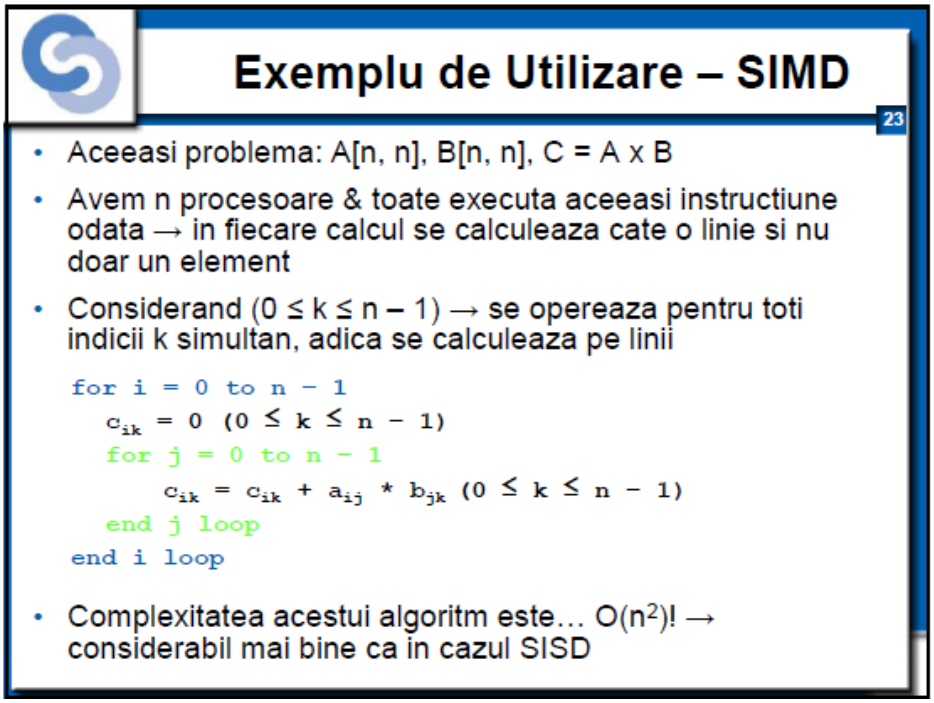
60. Comparatie SIMD MIMD la algoritmi

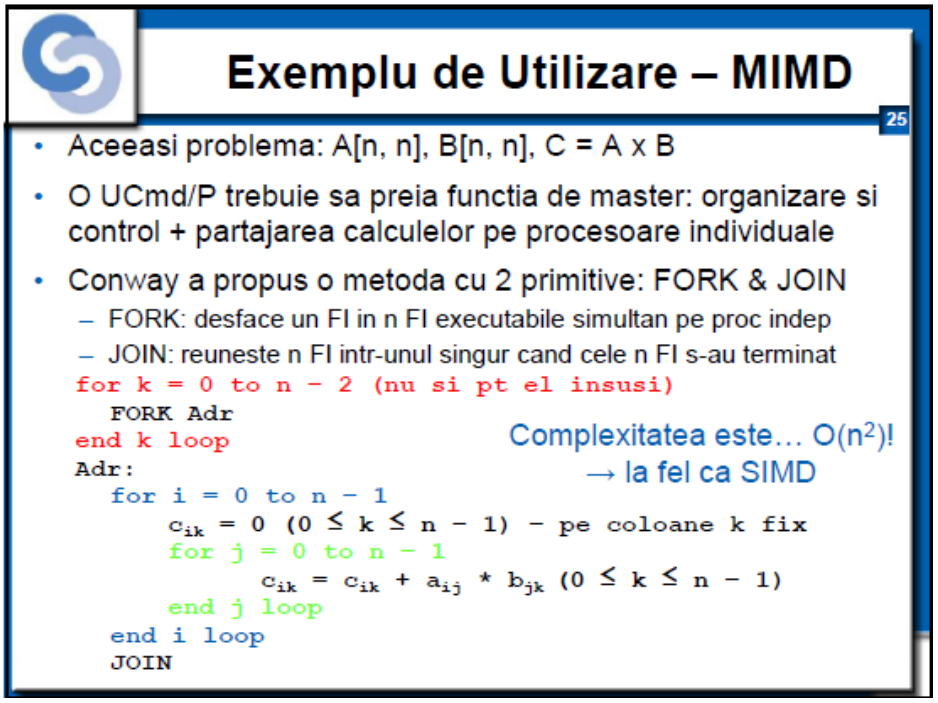


Flynn.
Capabilitatile celor 4 clase de sisteme de calcul.



61. Comparatie SIMD MIMD la algoritmi





Pentru Tavi:

bOgDaN:

61. Topologii si caracteristici ale retelelor bazate pe rutare

Topologii:

- Ortogonale: noduri aranjate intr-un spatiu n-dimensional ortogonal, fiecare legatura produce o deplasare intr-o singura dimensiune
- Strict ortogonale: fiecare nod are cel putin o legatura in fiecare dimensiune; trecerea intr-o noua dimensiune se poate face din orice nod; rutare mai simpla, implementare hardware mai eficienta;

Exemple: Hipercub (2 cuburi unu in altu), Tor, Plasa n-dimensionala

62. Sisteme cu procesoare omogene simetrice si asimetrice

In sistemele MIMD procesoarele pot fi : omogene si eterogene

Sistemele cu procedoare omogene pot fi :

- Cu procesoare simetrice (identice functional si I/O)
    - Sunt mai usor de programat, programarea resurselor e similara pt toate procesoarele
    - Ofera facilitati de prelucrare al proceselor in caz de defectiune al unor resurse
    - In general include elemente de redundanta
- Cu procesoare asimetrice (identice dpdv functional dar diferite ca subsisteme I/O)

63. Rolul KBus in Kmap + comunicatia Kbus - MapBus

Kmap este compus din 3 procesoare:Pmap, Link, KBus

Kbus reprezinta unitatea de comanda a Map Bus care controleaza toate tranzactiile de pe aceasta magistrala (Kmap prelucreaza tranzactii concurente)

Comunicatia Kbus-Mapbus:

-Kbus executa o mare varietate de task-uri printre care si tratarea inteligenta a erorilor

-Kbus este un un procesor microprogramat cu omemorie de control ROM

-Map Bus este o magistrala de comunicatii adrese/date intre Kbus si S-urile locale

-Kbus controleaza toate accesele pe Map Bus precum si cozile si registrele pentru comunicarea cu Pmap

-Mapaarea Map Bus este sincrona cu Kbus

64. Sisteme multiprocessor si multicalculator

Sistem multiprocesor: sistem de calcul unitar unde o colectie de procesoare autonome functioneaza concurent si coopereaza la realizarea unei functii logice si de calcul de sub control unui “executiv” unic

- Are un singur spatiu de adresare (comunicare prin shared memory)
- Se impart in:
    - Cu memorie distribuita (scalabilitate)
    - Ce memorie partajata (nu exista scalabilitate)

Sistem multicalculator: mai multe calculatoare autonome, interconectate, ce colaboreaza la realizarea unor task-uri din cadrul unei lucrari complexe

- Are spatii multiple de adresare (comunicare prin transfer de mesaje)
- Se impart in:
    - Distribuite
    - Centralizate

65. 3 probleme ce pot influenta performanta codului multi-threaded

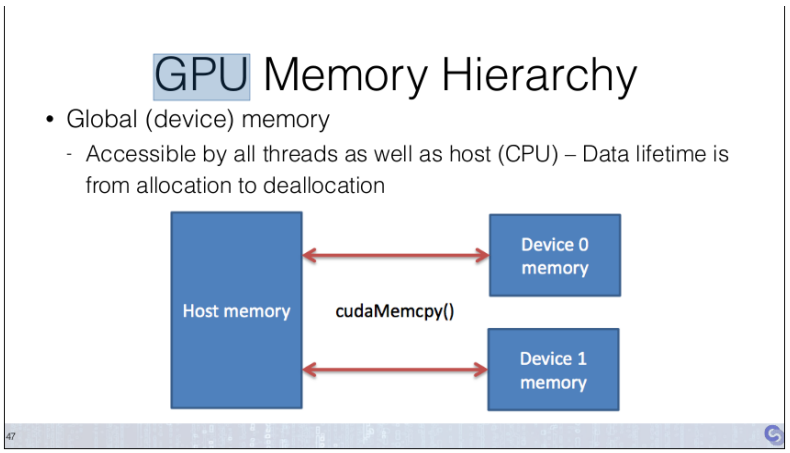
Acces simultan la date

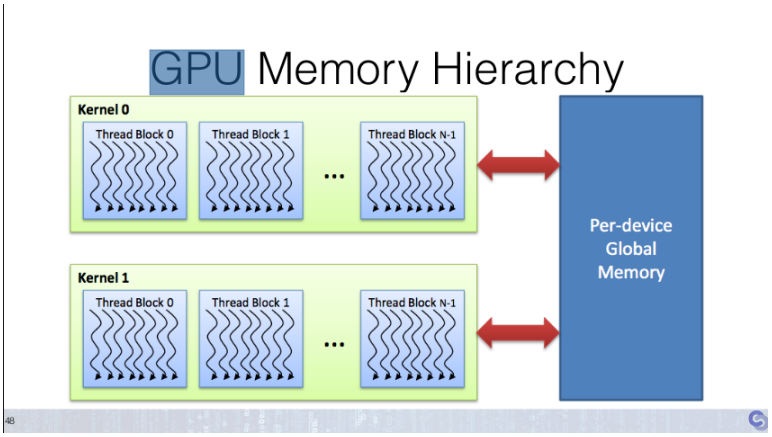
Lock-urile incetinesc executia programului

Modul de acces la date (pe module de memorie diferite, acces pe linii/coloane, etc)

66. Cum arata ierarhia de memorie pe GPU?

In GPU memoria este impartita in blocuri, si in general 2 mem de cache.





bl 10 sc 3 ap 57 str 16 februarie VA ASTEPT <3

67. Ce speedup ai daca 60% din cod e paralelizabil?

T = f*G* + *(1-f)*(G/P) // calculati si voi

A se cauta legea lui Amdahl : 1/S = f+ (1-f)/P (pentru siguranta mai cautati o data formula)

68. REMARCILE ???

ORA DE LIMBA ROMANA CU RALUCA DOGARU

TOTUL DESPRE CELL:

https://docs.google.com/document/d/1TOawGZW5rHlGrFfOzh_2MQbSYR2sr0FkxeBcfvuRApY/edit?usp=sharing

1. **Permutarea de baza**
2. **Permutarea perfecta**
3. **Permutare cu bit negat**
4. **Permutare butterfly**
5. **Permutare reverse bit**
6. **Permutare cu incrementare**

https://docs.google.com/document/d/1oajkqAoiJUynUkJ1YWCWErhnH8pe1AQaFREHeOYMfDM/edit?usp=sharing

X1. Organizarea datelor în structurile SIMD și MIMD
X2. Să se scrie programul de împărțire a doi vectori de dimensiune n pentru o structură SIMD și MIMD. Comentați performanțele.
X3. Să se cupleze 16 procesoare cu 16 memorii utilizând rețea de tip Delta; implicit procesorul i să fie conectat la memoria i. Indicați conexiunea de la procesorul 2 la memoria 6.
X4. Descrieți primitiva P de la calculatorul personal.
X5. Considerând o structură multiprocesor pe o magistrală comună descrieți unitatea de acces la magistrala sistemului.-
X6. Structura generală Kmap.
X7. Comparație între crossbar și delta privind performanțele.
X8. Subiect la alegere din materia de ASC.
X9. Considerand o structura multiprocesor pe un bus comun, descrieti unitatea de acces la magistrala sistemului al fiecarui modul. (Vezi ex 5)
X10. Comparatie intre intercalarea perfecta si reteaua butterfly. (Vezi ex 33 si 36)
X11. Descrieti folosind PMS structura cu care ati lucrat la laborator.
X12. Sa se scrie programul de insumare a elementelor unei matrice A(n, n) pentru o structura SIMD si pentru o structura MIMD. Comentati performantele. Obs. Trebuie specificata organizarea datelor.
X13. Considerand o structura multiprocesor pe o magistrala comuna, descrieti unitatea de acces la magistrala sistemului. (Vezi ex 5)
X14. Clasificarea Flynn a sistemelor de calcul.
X15. Descrieti caracteristicile generale CELL
X16. Retea de comutare de tip intercalare perfecta si comparatie cu reteaua cu ordine inversa (permutarea de baza). Vezi ex 36.
X17.Trunchi K ierarhic si neierarhic
X18.Adunarea elementelor a doi vectori in SIMD si MIMD (cod + cum retii datele)
X19.Subiect cu permutari (parca era descrierea permutarilor de tip perfect shuffle). Vezi ex 36.
X20. Organizarea datelor la SIMD (Vezi ex 1)
X21. Comparatie SIMD MIMD la algoritmi (Vezi ex 2 :-?)
X22. Probabilitatea ca o cerere sa fie acceptata la un comutator CB (formula)
X23. Comparatie permutare baza, permutare perfecta (Vezi ex 36)
X24. Block placement in cache.
X25. Arhitectura procesorului PMAP.
X26. Comutator legatura multipla (CrossBar) ierarhic. (voia 3 cuv, maxim o fraza: InfiniBand, Linux, Cluster) (ce pula mea?)
X27. Benchmark-uri, probleme si rezolvari legate de acestea (ceva de genu).
(nu trebuia sa zici de HPCC si LINPACK)
X28. Tipuri de paralelizare la sistemele cu prelucrare paralela.
X29. Structuri SN. P i/e structura (ce plm SN?)
X30. Block replacement
X31. Determinarea dimensiunii cache-ului
X32. Comunicatia Kbus Pmap
X33. Permutarea Butterfly
X34. Benchmark-uri pentru sisteme seriale

BAG PULA IN MATERIA ASTA

X35 Cell BE bariere si solutii.
X36. comparatie intre permutarea de baza si perfect shuffle
X37. Memorii distribuite
X38. optimizari programe (trebuia vb de cache)
X39 Comutatoare in cuplarea memorie procesor.
X40 Retea de comutare de tip interclasare perfecta. (Vezi ex 36)
X41 Sa se scrie programul SIMD si MIMD care aduna 2 vectori (pe elemente). Sa se analizeze performantele. Specificati organizarea datelor (Vezi ex 18)
X42 Comparatia generala intre SIMD si MIMD
X43 Organigrama KML de la SBC-uri + schema logica
X44 Sa se scrie programul ce determina produsul elementelor unei matrice pe SIMD si MIMD. Sa se analizeze performanta. Reprezentarea datelor. (Vezi ex 12 doar ca faci * in loc de +)
X45 Exemplificati primitivele P si M de pe calculatorul propriu
X46 Comunicatia intercluster intr-un sistem MIMD slab cuplat
X47 Adunare elemente dintr-o matrice cod SIMD MIMD plus organizarea datelor si concluzii (Vezi ex 12)
X48 Comparatie intercalare perfecta si permutarea de baza. (Vezi ex 36)
X49 Switch K ierarhic si neierarhic [se repeta deci il distrugem (vezi 17)]
X50 Organizarea datelor in SIMD. Realizarea inmultirii unei matrici cu ea insasi (Vezi ex 1)
X51 Care sunt cele 4 elemente de care trebuie tinut cont cand se realizeaza designul unui cache.
X52 De ce nu se poate realiza un procesor de 10TFlops(minim 3 motive) ?
X53 Caracteristici HPC Challenge Benchmark.
X54 Comparatie Base Line Permutation - Bit Reversal Permutation
X55 Cum se masoara dimensiunea liniei unui cache? Exemplu de program si grafic.
(pentru inlocuirea oricarui alt subiect :)
X56 Arhitectura unui calculator din top 500 (arhitectura, numar procesoare, so, ...)

Daca vreti sa puneti rezolvari, puneti-le de pe pagina urmatoare

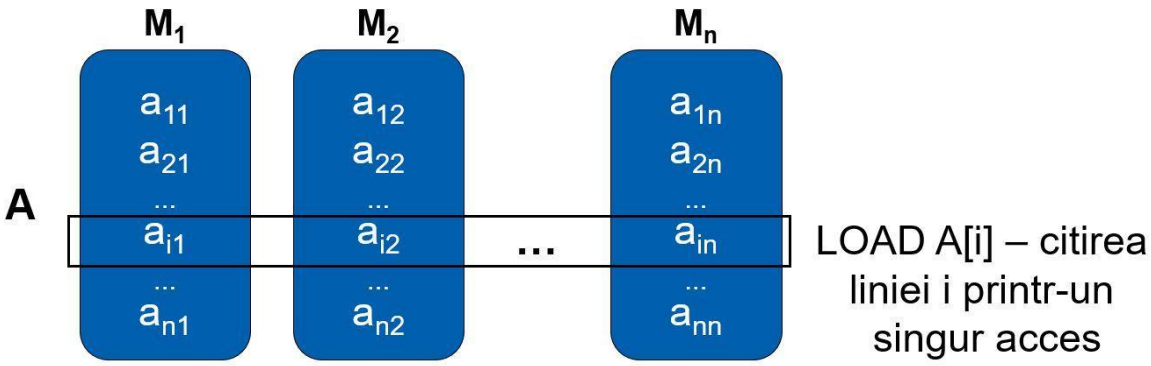
Rezolvari

1. Organizarea datelor în structurile SIMD și MIMD

SIMD:

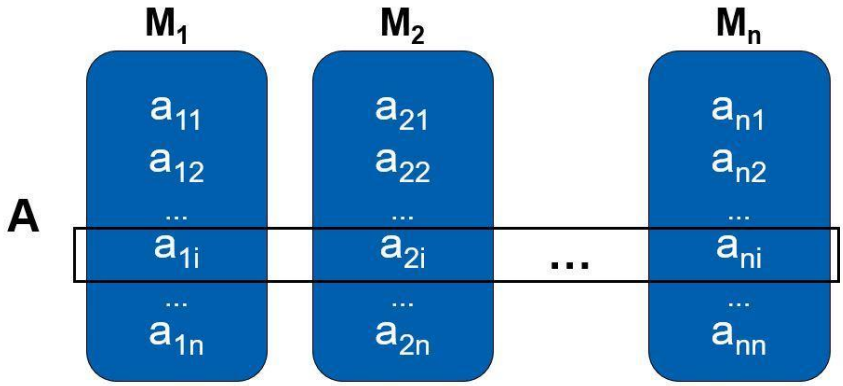
- Probleme:
  - Repartizarea in memoriile M_i & accesul paralel
  - Citirea vectorilor intr-o singura instructiune
  - La matrice: acces la linii si la coloane → necesara transpunerea/deplasarea datelor?
- Memoria:
  - Poate face cel mult un acces la un ciclu de lucru
  - n operanzi in n module diferite pot fi cititi simultan
  - Defavorabila plasarea in acelasi modul al operanzilor → citirea va fi secventiala in n cicli de acces!

- Matrice A[n,n] & structura SIMD (n procs/memorii)
- Cu acces simultan la liniile matricei:
  - Acces paralel la linii
  - Acces secvential la coloane



LOAD A[i] – citirea liniei i printr-un singur acces

- Matrice A[n,n] & structura SIMD (n procs/memorii)
- Cu acces simultan la coloanele matricei:
  - Acces secvential la linii
  - Acces paralel la coloane

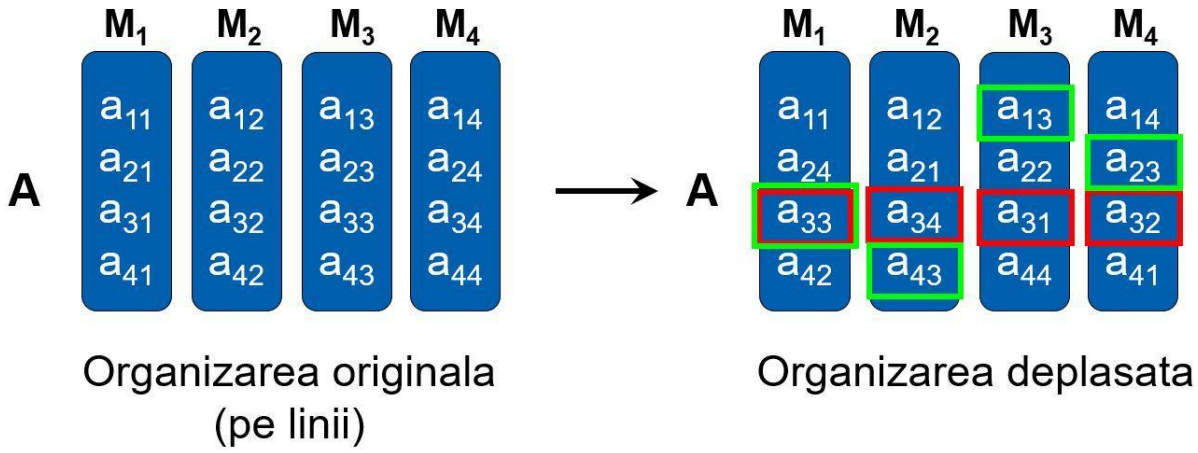


LOAD A[i] – citirea coloanei i printr-un singur acces

Reprezentarile directe pe linii/coloane sunt incompatibile la acelasi moment de timp

Inmultirea unei matrici cu ea insasi necesita acces paralel la linii si coloane pentru performanta maxima:

- Matrice A[4,4] & structura SIMD (4 procs/memorii)
- Cu acces paralel la liniile & coloanele matricei:
  - Exemplu valid pt inmultirea matricelor (linia & coloana 3)



Organizarea originala (pe linii)

Organizarea deplasata

Necesar un shuffling cu reteaua de comutatie (a_{i1} nu e pe P_1)

MIMD:

Organizarea memoriei:

- Fiecare procesor trebuie sa aiba acces la toata memoria sistemului
- Capacitatea de adresare a fiecarui procesor e mai mica ca dimensiunea memoriei → mecanisme speciale
- Asigura protectia memoriei pentru asigurarea integritatii datelor din sistem

Diferența la MIMD e că lucrează cu matrici, nu cu vectori. Reprezentarea matricilor (care sunt 2D) în memorie (care este 1D) - row-major, column-major, în funcție de limbaj.

2. Să se scrie programul de împărțire a doi vectori de dimensiune n pentru o structură SIMD și MIMD. Comentați performanțele.

$\pi\pi$ n procesoare sa fie mai laba

SIMD: se lanseaza in mod paralel by default din arhitectura
k -> index propriu procesorului
ck = ak / bk ($0 \le k \le n - 1$)
O(1)

MIMD: trebuie sa lansam noi fiecare proces + sa fakem joint <3
for k = 0 to n – 2 (nu si pt el insusi)
FORK Adr

BAG PULA IN MATERIA ASTA

end k loop
Adr:

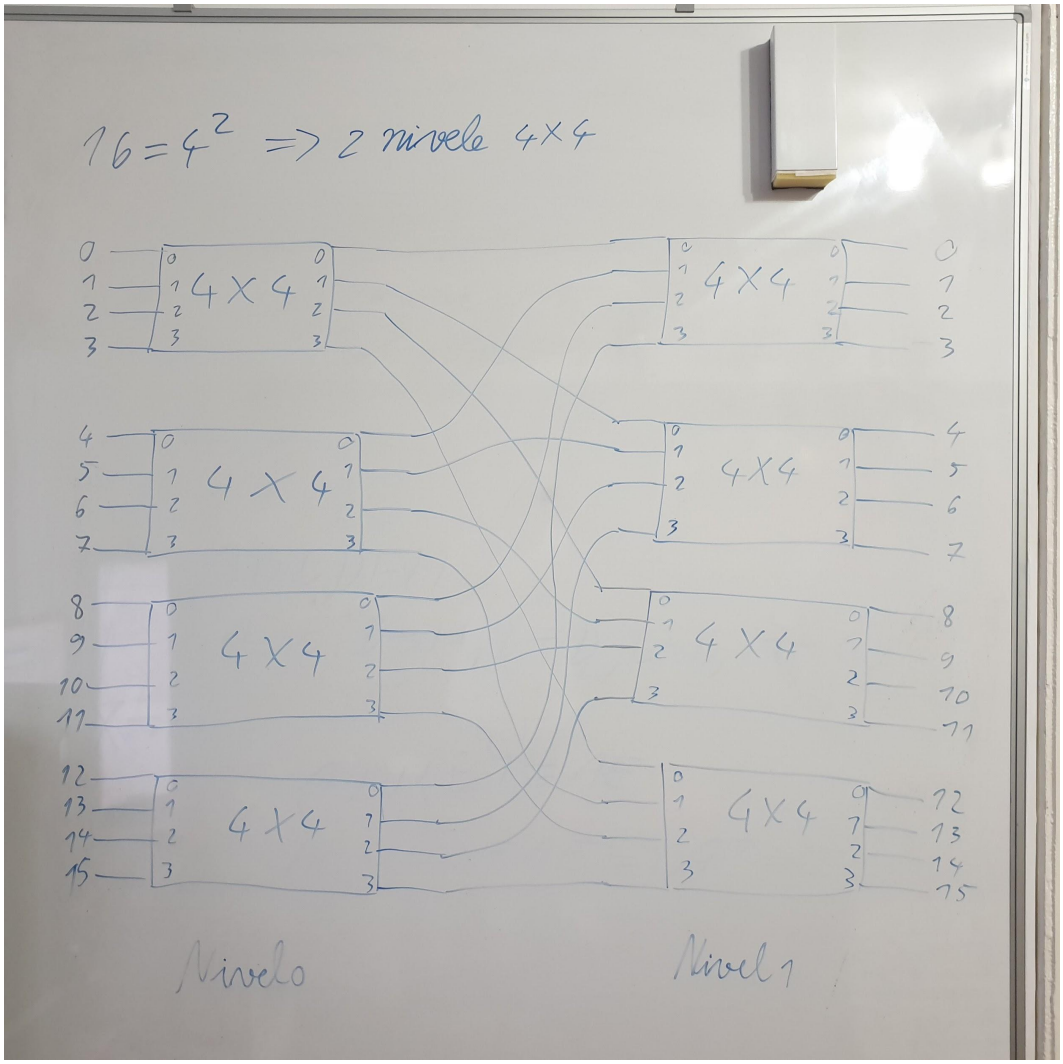
ck = ak/bk ($0 \leq k \leq n - 1$)

JOIN

O(n) pentru ca trebuie sa stai sa lansezi n procese.

3. Să se cupleze 16 procesoare cu 16 memorii utilizând rețea de tip Delta; implicit procesorul i să fie conectat la memoria i. Indicați conexiunea de la procesorul 2 la memoria 6.

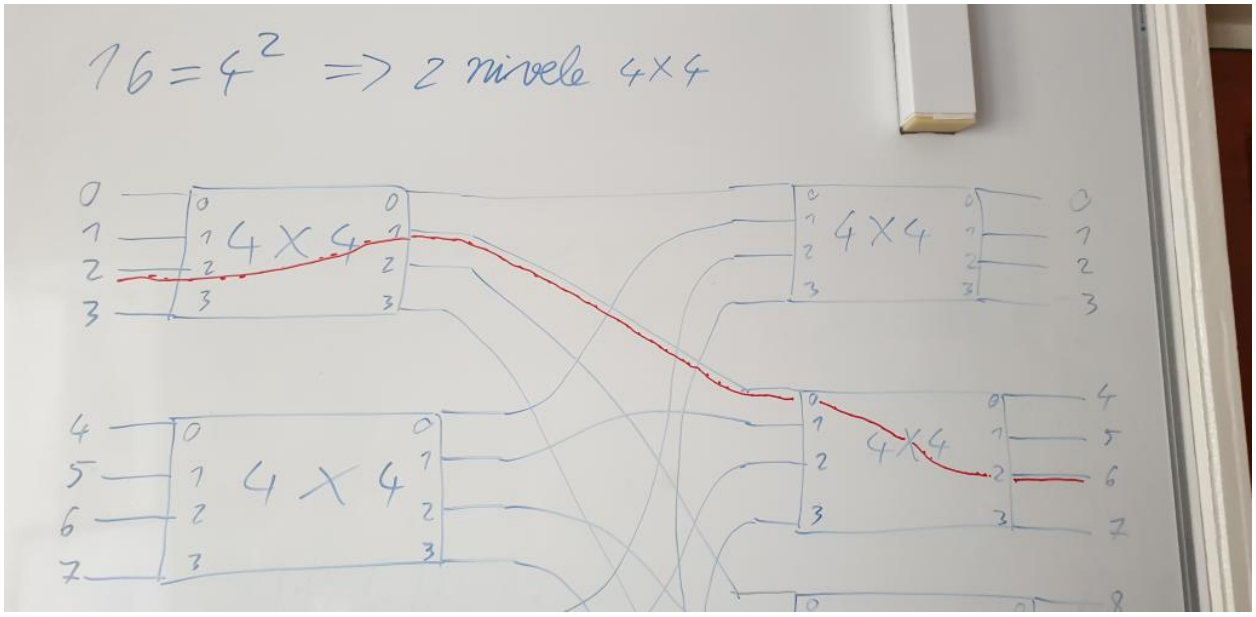
BAG PULA IN MATERIA ASTA



Adresa ar trebui sa fie in baza b, aka in dreapta sa avem 00, 01, 02, 03, 10...33

Conexiunea de la 2 la 6

BAG PULA IN MATERIA ASTA



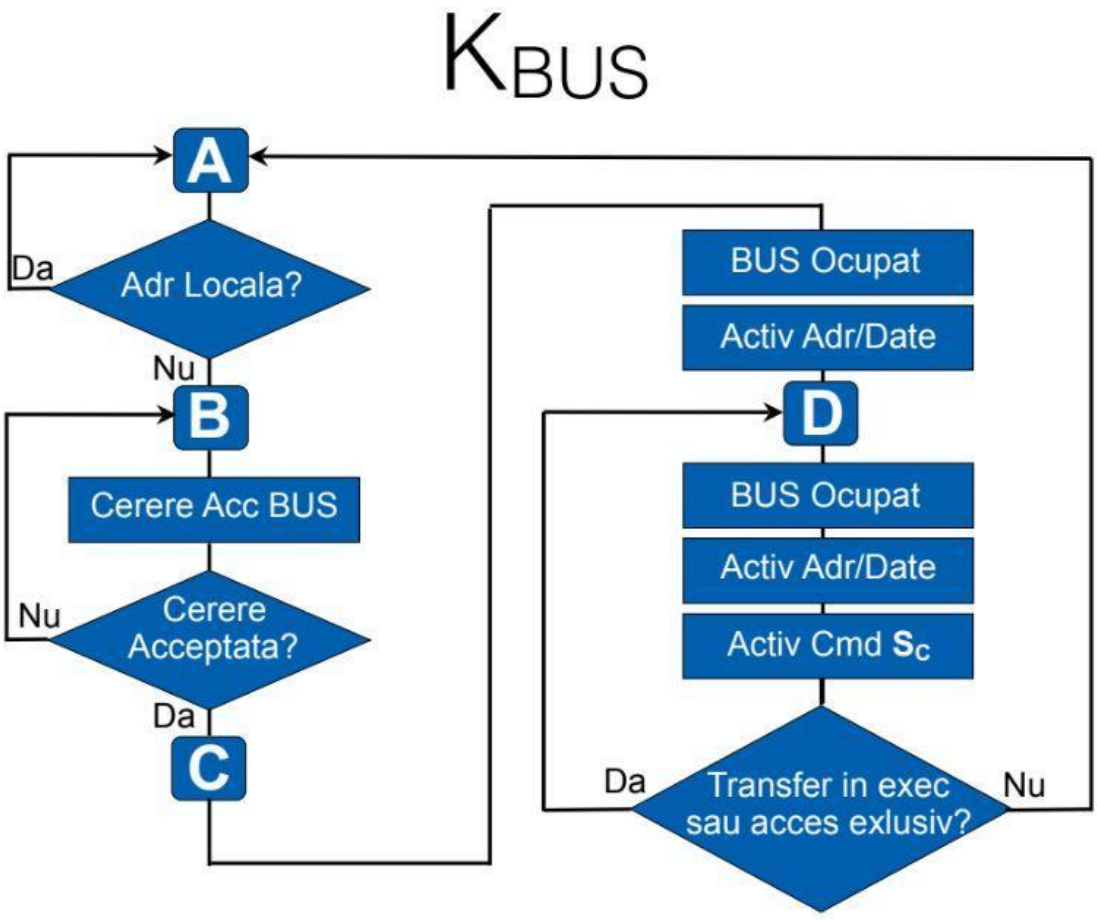
4. Descrieți primitiva P de la calculatorul personal.
Aka CPU, subiect bonus.
Ex:
Citeste, interpreteaza si executa instructiuni
In acest proces are loc generarea comenzilor spre toate resursele sistemului + citirea starilor acestora
Atribute:
Functie: universal
Implementare: microprogramata ?? [“Intel Sandy Bridge: hardwired pipelined CISC (x86) machine (with some microcode support)”; https://inst.eecs.berkeley.edu/~cs152/sp12/lectures/L02SimpleImps.pdf]
Formatul instructiunilor: variabil (de obicei sunt arhitecturi CISC)
Lungime instructiuni: 64 biti (eu cred că e 1 octet > mulți (câți?) octeți) (cred ca instructiunile sunt pe 832 biti sau ceva) [la 8086 nu erau 1 octet 6 octeti?]
[nu stiu daca este vorba despre asta http://en.wikipedia.org/wiki/Advanced_Vector_Extensions, atunci ar fi 256 biti de la sandy bridge] conform http://www.intel.com/content/dam/www/public/us/en/documents/manuals/64ia32architecturessoftwaredeveloperinstructionsetreferencemanual325383.pdf, sunt lungimi variabile pt instrucțiuni și în Intel i7. Eu cred că un răspuns gen "lungimea variabilă, de la instrucțiuni scurte pe 1 octet, până la instrucțiuni complexe pe mai mulți octeți" ar merge.
Ciclu instructiune: ciclu varibil(S cicli masina)
Tehnologie: VLSI Very Large Scale Integration

5. Considerând o structură multiprocesor pe o magistrală comună descrieți unitatea de acces la magistrala sistemului.

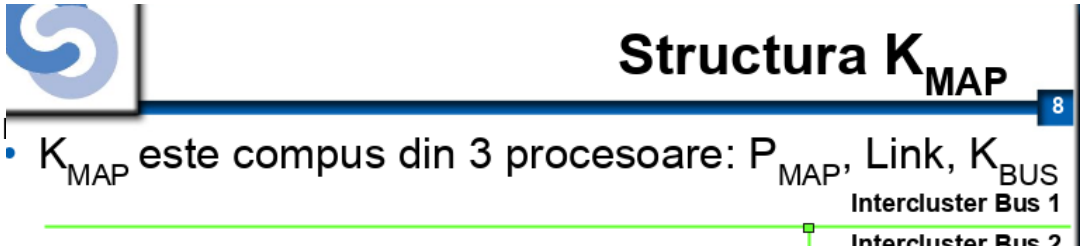
BAG PULA IN MATERIA ASTA

Descriere K_{BUS}

- K_{BUS} – reprezinta procesorul local in relatie cu magistrala externa
- Starea A: adresa e locala?
  - Da = se adreseaza resurse locale, nu trebuie iesit pe magistrala
  - Nu = adr nu e locala → se trece in starea B
- Starea B: cerere de acces la magistrala; daca cererea e acceptata se trece in starea C
- Starea C: se emite Bus Busy (mag ocupata) – doar in C & D; accesul pe BUS e exclusiv & cu rafala (overwrite) raman in D pt un nou ciclu daca mai e cazul; automatul preia functia procesorului; Dupa preluarea comenzii, procesorul face functia de secventiere a Adr, Date, Cmd
- Intrebare: de ce si starea C si starea D? Din cauza ciclului instructiune curent. Trebuie sincronizate comenzile cu decodificarea adreselor & stabilirea datelor pe magistrala (e un Δ_t acolo = T_{Wait} datorat interpretarii adreselor) → se introduce starea D pentru a compensa acest timp de latenta: starea de overwrite
- Obs: K_{BUS} trebuie sa dispuna de unitati logice combinationale ce identifica:
  - Daca adresa mem/port I/O implicate in transfer sunt locale
  - Semnalul de cerere acceptata e combinat din mai multe conditii: mag libera & modul SBC activ



6. Structura generală Kmap.



k

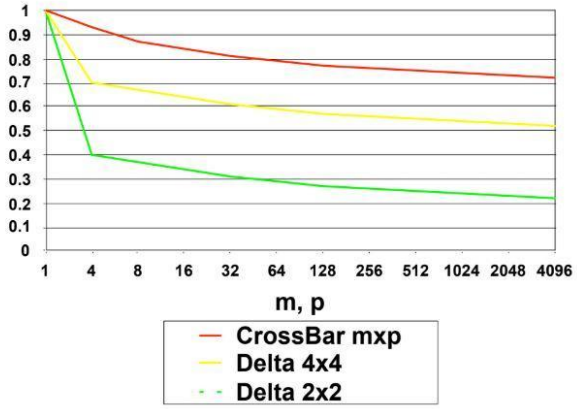
7.Comparație între crossbar și delta privind performanțele.

Analiza Comparativa Delta / CB

- Probabilitatea ca o cerere sa fie acceptata este:

$$P_A = \frac{b^n \cdot r_n}{a^n \cdot r}$$

- Ce influenta au m si p-ul unei retele?
  - CrossBar **m**x**p** cu r=1 rata de cereri a fiecarui procesor
  - Delta 4^nx4^n cu 4x4
  - Delta 2^nx2^n cu 2x2
- Pentru m mare, P_A-ul scade puternic
  - Multe rejectii si conflicte
  - Limita la 16^nx16^n este 0.6



11. Descrieti folosind PMS structura cu care ati lucrat la laborator.

Aka pc-ul de la lab, subiect bonus. ??????????

BAG PULA IN MATERIA ASTA

12. Sa se scrie programul de insumare a elementelor unei matrice A(n, n) pentru o structura SIMD si pentru o structura MIMD. Comentati performantele. Obs. Trebuie specificata organizarea datelor (Pp n procesoare.)

Standard

```
for i = 0 to n-1
        for j = 0 to n-1
                sum += aij
        end j loop
end i loop
```

SIMD vNicu:

```
sum[k] = 0
sum_final = 0
for i = 0 to n - 1
        sum[k] += a[i][k]
end i loop
sum_final += sum[k]
```

SIMD vVladIvan:

```
sumk = 0 ( asta e local sau nu ? )
for i = 0 to n – 1
        sumk += aik (suma coloana k)
end i loop

local_sum = 0 (local procesorului, in registru poate?)
for i = 0 to n – 1
        local_sum += sum[(i + k) mod n] (ca sa nu acceseze toate simultan acelasi modul de
memorie)
end i loop
O(n)
```

MIMD vVladIVan:

```
for k = 0 to n – 2 (nu si pt el insusi)
        FORK Adr
end k loop
Adr:
        sumk = 0 (0 ≤ k ≤ n – 1)
        for i = 0 to n – 1
                sumk += aki (suma pe linii pentru acces contiguu in memorie)
        end i loop
        JOIN

for k = 0 to n – 1
        sum_final += sumk
```

BAG PULA IN MATERIA ASTA

O(n)

Organizare datelor: vezi ex 1

12bis. Sa se calculeze suma a doua matrici in SIMD si MIMD. Comentati performantele. Specificati org. datelor:

SIMD:

for i = 0 to n – 1
    cik = aik + bik (0 ≤ k ≤ n – 1)
end i loop

MIMD:

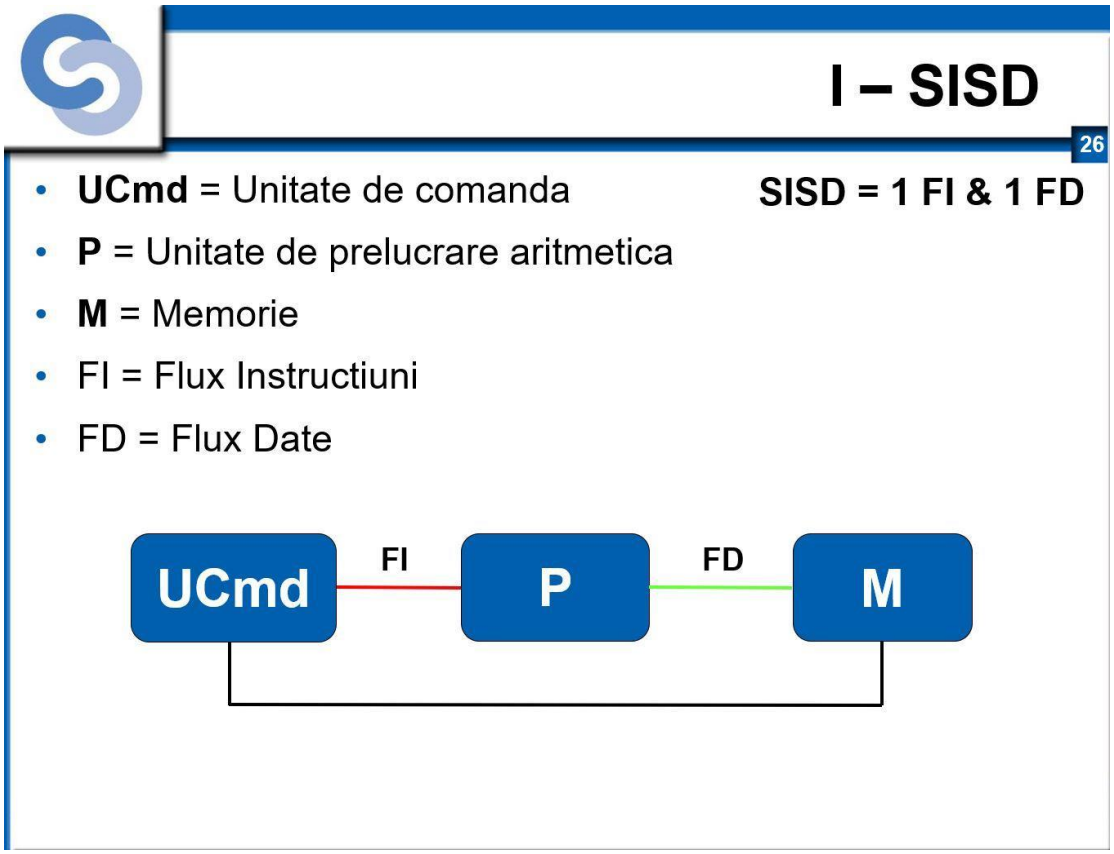
for k = 0 to n – 2 (nu si pt el insusi)
    FORK Adr
end k loop
Adr:
    for i = 0 to n – 1
        cik = aik + bik (0 ≤ k ≤ n – 1) – pe coloane k fix
    end i loop
    JOIN

14. Clasificarea Flynn a sistemelor de calcul.

SISD - un singur flux de date si un singur flux de instructiuni. Nu exista paralelism
SIMD - mai multe fluxuri de date, dar un singur flux de instructiuni. Exista paralelism. S.n. si sisteme de procesare vectoriala.

BAG PULA IN MATERIA ASTA

MISD - paralelism reduntant. E un tip de calculator atipic. Se foloseste in sisteme de calcul unde e nevoie de precizie maxima (pe wiki scrie ca de exemplu la avioane)
Mai scria in curs si ca sunt folosite cand vrei sa faci chestii diferite asupra la aceleasi date si dadea exemplu de chestii meteorologice.
MIMD - clustere, MPI (scria ceva prin curs de cuplat Slab/Puternic respectiv MPI/OpenMP), Ceva gen MPI da pe retea/socketi si OpenMP pe shared memory -- Slab ca procesoarele Chiar sunt super separate vs. sunt doar cores diferite care ruleaza cod diferit)



BAG PULA IN MATERIA ASTA

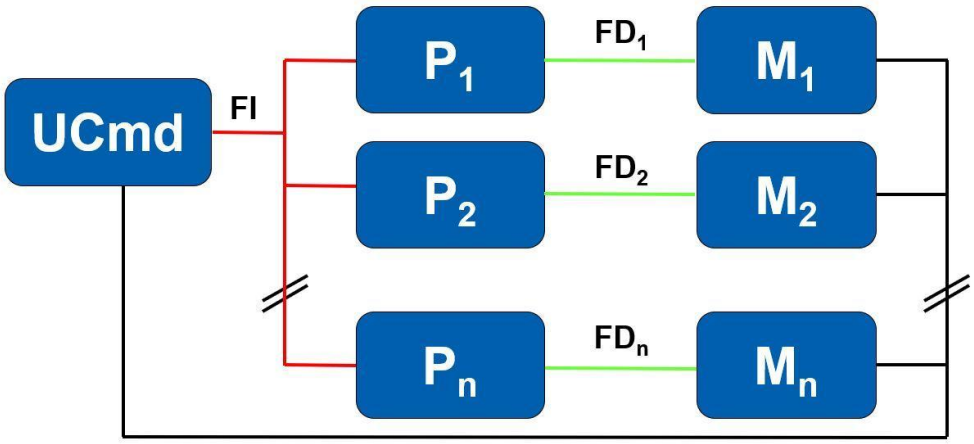
II – SIMD

27

- SIMD sunt masini vectoriale:
  - O singura UCmd
  - Mai multe procesoare si module de memorie (orice procesor vede orice memorie)

SIMD = 1 FI & n FD

- Toate procesoarele fac aceeasi op impusa de UCmd prin FI



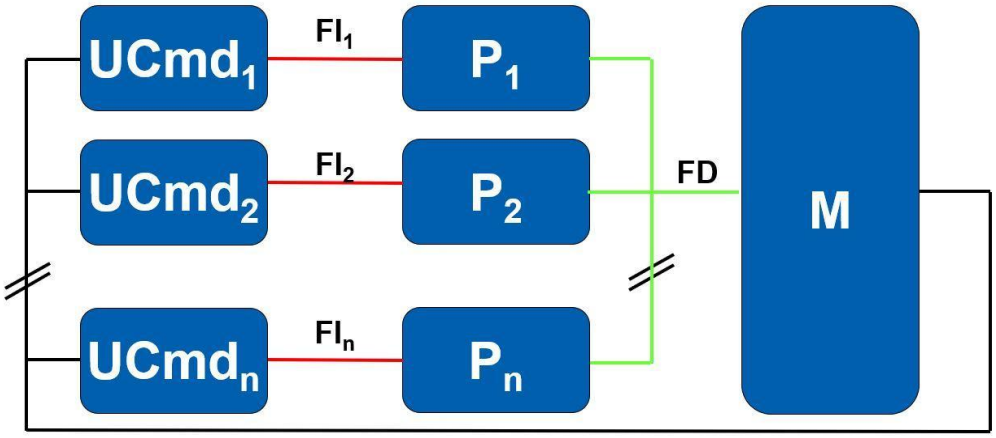
III – MISD

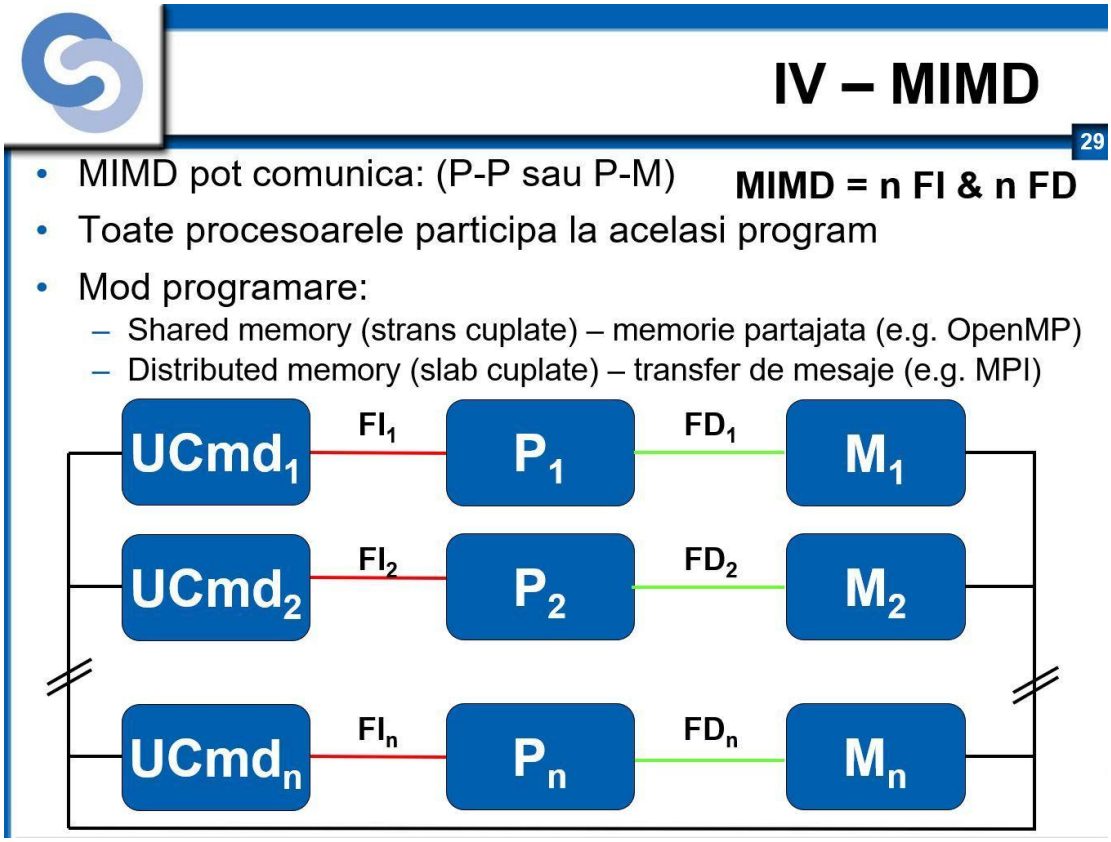
28

- MISD au:
  - Mai multe UCmd si procesoare
  - Un singur modul de memorie

MISD = n FI & 1 FD

- Domeniu de aplicatie restrans si special: aplicarea altor algoritmi pe aceleasi date (Apps: meteo/evidenta populatiei)




IV – MIMD
29
MIMD pot comunica: (P-P sau P-M)
MIMD = n FI & n FD
Toate procesoarele participa la acelasi program
Mod programare:
Shared memory (strans cuplate) – memorie partajata (e.g. OpenMP)
Distributed memory (slab cuplate) – transfer de mesaje (e.g. MPI)
UCmd1
FI1
P1
FD1
M1
UCmd2
FI2
P2
FD2
M2
UCmdn
FIn
Pn
FDn
Mn

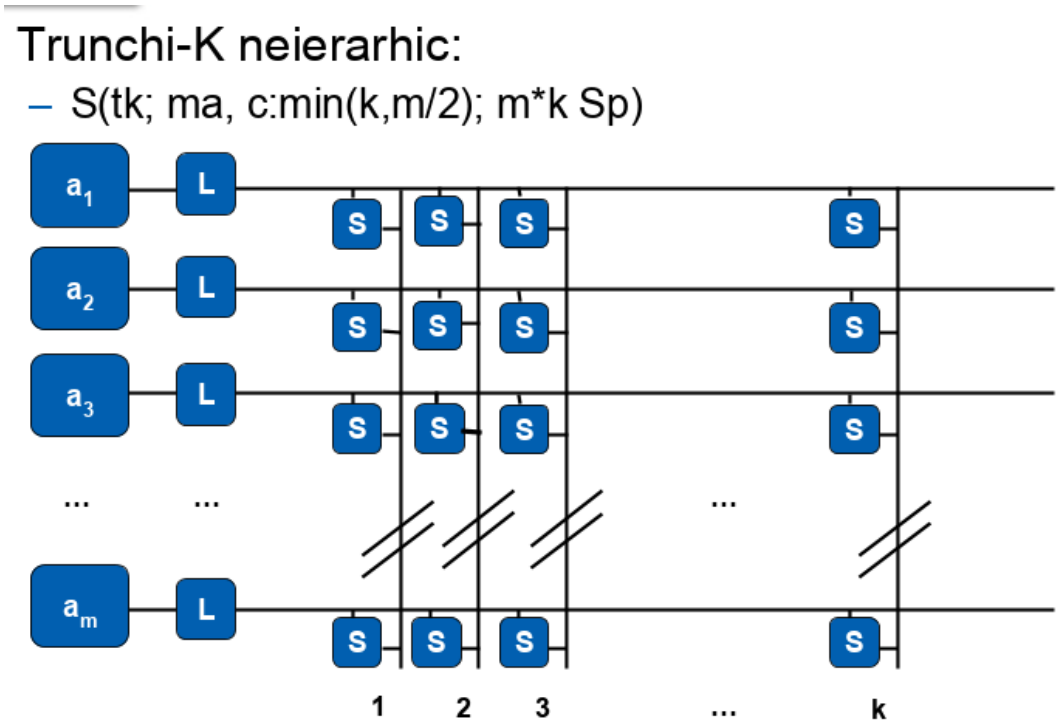
15. Descrieti caracteristicile generale CELL

A hardware Cell is made up of:

- 1 Power Processor Element (PPE).
- 8 Synergistic Processor Elements (SPEs). or APU = additional processing unit
- Element Interconnect Bus (EIB).
- Direct Memory Access Controller (DMAC).
- 2 Rambus XDR memory controllers.
- Rambus FlexIO (Input / Output) interface.

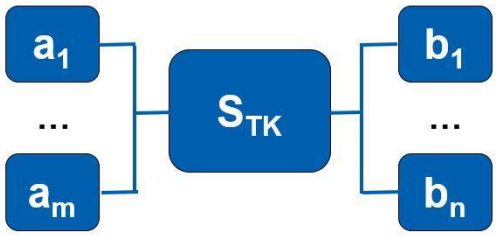
17. Trunchi K

Neierarhic: Face legatura intre P-P si M-M (elemente de acelasi tip).



Ierarhic: Asigura comunicarea intre P-M si M-P. E o singura unitate S ce leaga mai multe A-uri de mai multe B-uri. Dimensiunea (K) trebuie aleasa in functie de nr de A-uri si nr de B-uri).

- Imbina conectivitatea oferita de S legatura multipla cu costul redus al celorlalte tipuri de comutatoare
- S {TK, ma, nb, c = k, k*(m+n)Sp}



- Orice a poate comunica cu orice b
- Nu totdeauna vei avea m x n comunicatii simultane
  - Asa a aparut idea de a folosi k linii pentru comunicatie
- In functie de m si n si de trafic, se va determina un k optim
  - 8 procs & 4 memorii: pot fi maxim 4 accese la memorii → k=4, nu are sens mai mult; k < n si k < m

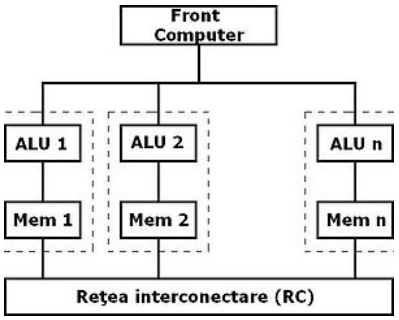
18.Adunarea elementelor a doi vectori in SIMD si MIMD (cod + cum retii datele)

SIMD: se lanseaza in mod paralel by default din arhitectura
ck = ak + bk (0 ≤ k ≤ n – 1)
O(1)

MIMD: trebuie sa lansam noi fiecare proces
for k = 0 to n – 2 (nu si pt el insusi)
    FORK Adr
end k loop
Adr:
    ck = ak + bk (0 ≤ k ≤ n – 1)
JOIN
O(n) pentru ca trebuie sa stai sa lansezi n procese.

Organizare datelor: vezi ex 1

20. Organizarea datelor la SIMD



Nu cred ca trebuie mai fancy de atat. Practic, ai ALU multiple, Mem 1, …. Mem n si o retea de interconectare la baza. (Vezi ex 1)

22. Probabilitatea ca o cerere sa fie acceptata la un comutator CB (formula)

- Consideram un CB (pxm) si:
  - P_A = probabilitatea de acceptare a unei cereri oarecare
    - E data de rata de transfer B(p,m), de r si de p:

$$P_A = \frac{B(p,m)}{r \cdot p} = \frac{m}{r \cdot p} - m\frac{\left(1-\frac{r}{m}\right)^p}{r \cdot p}$$

    - Obs: se considera ca cererile sunt independente; in realitate cererile rejectate se transfera ciclului urmator → marind rata de transfer
    - Daca numarul de resurse interconectate creste: $\lim_{m\to\infty}\left(1-\frac{r}{m}\right)^p = e^{-r\frac{p}{m}}$
    - Daca p si m sunt foarte mari atunci :

$$B(p,m) \cong m\left(1-e^{-r\frac{p}{m}}\right) \qquad P_A \cong \frac{m}{r \cdot p}\left(1-e^{-r\frac{p}{m}}\right)$$

    - Formele simplificate sunt acceptabile in urmatoarele limite:
      - Daca p si m ≥ 30 atunci ele sunt in limita unei erori de 1%
      - Daca p si m ≥ 8 atunci ele sunt in limita unei erori de 5%

24. Block placement in cache.

Mapare directa - un bloc din cache poate fi asezat intr-un singur loc
Mapare (full) asociativa - un bloc din cache poate fi asezat oriunde
Mapare set asociativa - un bloc din cache poate fi asezat intr-un subset de pozitii

Trade offs:
Maparea n-way set associativa devine din ce in ce mai dificila si costisitoare de implementat pentru un n mare:

- Majoritatea cache-urilor sunt 1, 2 sau cel mult 4 set asociative

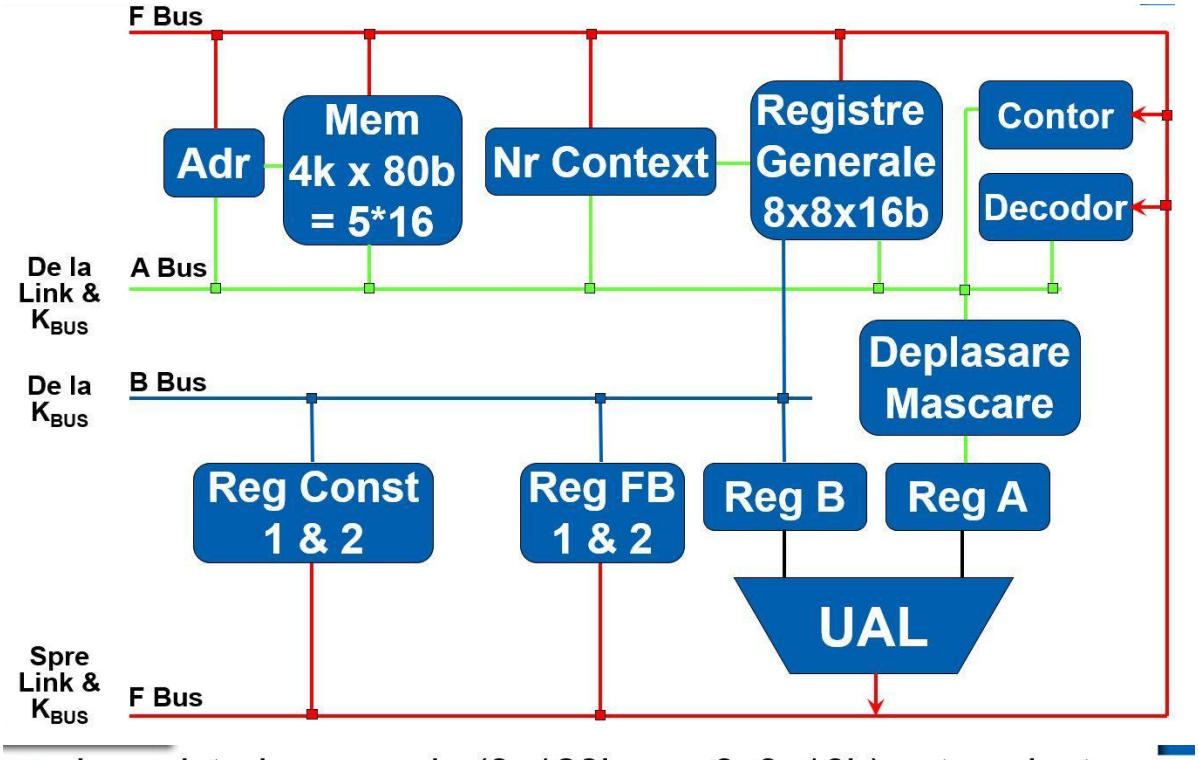
BAG PULA IN MATERIA ASTA

Cu cat n-ul este mai mare, cu atat este mai mica probabilitatea de aparitie a fenomenului de thrashing:

- Momentul in care doua (sau mai multe) regiuni de memorie sunt accesate in mod repetat si pot incapea impreuna in acelasi bloc din cache

25. Arhitectura procesorului PMAP.

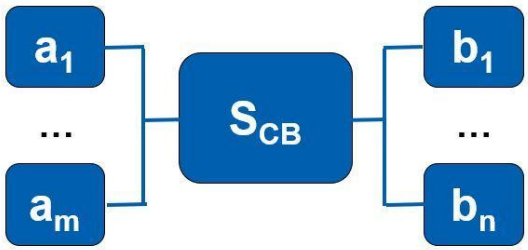
- P_{MAP} este un procesor dedicat pe 16 biti, microprogramat, cu codificare orizontala
- Memoria de control este formata din 4k * 80 de biti
- Memoria de control RAM foarte rapida are cuvantul de 80 biti (5*16) – Writable Control Store
- Functiile fundamentale ale procesorului P_{MAP} sunt:
  - Translatarea adreselor nelocale din cluster
  - Implementarea primitivelor OS critice din punctul de vedere al timpului de executie
  - Culegerea informatiilor statistice pentru imbunatatirea solutiilor de proiectare



- In registrele generale (8x128b sau 8x8x16b) este salvat contextul executiei in curs
- Este organizat in jurul a 3 magistrale:
  - A, B pentru operanzi
  - F pentru rezultate
- Pentru a putea mapa adresele P_{MAP} trebuie sa aibe acces rapid la informatia pt translatarea adr virtuale in adr fizice
- UAL are un operator la nivel de bit pentru Reg A ce asigura:
  - Deplasarea
  - Rotirea
  - Mascarea (cu 30 de masti predefinite), permitand preluarea descriptorului de context
- Prelucrarea este in banda de asamblare – operatii UAL si operanzii ce vin simultan, separat, prin registrii tampon
- Accesul la o memorie in acelasi cluster este mult mai rapid decat in alt cluster

26. Comutator legatura multipla (CrossBar) ierarhic. (voia 3 cuv, maxim o fraza: InfiniBand, Linux, Cluster) (ce pula mea?)

- Asigura comunicatia intre m componente de tip a si n componente de tip b
- Este cunoscut si ca Cross-Bar-Switch
  - S {CB, ma, nb, c = min(m, n), m*nSp}



- Orice a poate comunica cu orice b
- Pot avea loc transferuri multiple si simultane!
- Cel mai dificil este de implementat U_{Cmd} pentru S_{CB} – ea trebuie sa rezolve toate posibilele conflicte

27. Benchmark-uri, probleme si rezolvari legate de acestea (ceva de genu).

- Lectura specificatiilor unor sisteme de calcul nu este suficienta pentru a ne oferi o imagine completa asupra performantelor acestuia
- Asadar, a fost necesara crearea unor teste (benchmark-uri) care sa permita compararea performantelor intre sisteme/arhitecturi diverse
- Testele constau din seturi de programe care sunt menite sa scoata in evidenta performantele diverselor componente ale unui SC:
  - Floating Point CPU Performance
  - Caracteristici de interconectare (Bandwidth/Latenta)
  - Memorii externe (Hard-Drives)

Caracteristici & Probleme

4

- Sunt create pentru a “simula” comportamentul aplicatiilor reale (sintetice)
- Cele mai bune benchmark-uri sunt aplicatiile utilizator in sine…
- Asadar, trebuie sa fim intotdeauna circumspecti cand utilizam benchmark-uri!
- Cateva probleme:
  - Producatorii HW fac “tuning”
  - Performantele masinilor se degradeaza cand load-ul depaseste o valoare de prag
  - Securitatea, disponibilitatea, fiabilitatea, serviceabilitatea sau scalabilitatea NU sunt obiectul unui benchmark

Tipuri de Benchmark-uri

5

1. Aplicatii reale: procesoare de text, aplicatii utilizator
2. Kernel: contine doar parti esentiale dintr-o varietate de programe (i.e. Linpack), cu rezultate in MFlops
3. Micro-benchmark-uri: dezvoltate de utilizatori pentru a testa componente ale sistemului
4. Sintetice: se strang date statistice asupra operatiilor efectuate de aplicatii si se construiesc programe care sa respecte proportia aflata astfel (Whetstone/Dhrystone)
5. Pentru sisteme I/O
6. Pentru sisteme paralele

28. Tipuri de paralelizare la sistemele cu prelucrare paralela.

1. Paralelism la nivel de bloc/job, intre joburi diverse avem nevoie de, mecanism de salvare de context, ceas de sincronizare, canale de I/O pt transfer date;
   Anumite faze ale unui job pot fi suprapuse
2. Paralelism la nivel de subansamble hardware: intre elem de prelucrare a vectorilor, intre componentele logice ale dispozitivelor aritmetico/logice (gen carry lookahead)
3. Paralelism la nivel de program: intre diverse sectiuni independente ale unui program, intre loopurile unui program; ambele au nevoie de analiza dependentelor de date
4. Paralelism la nivel de instructiune (mancati cacatu cu pipeline si branch predictor):
   Seriala - fara pipeline
   Seriala paralela - gen pipeline da poa sa fie si inseriate instructiunile nuj
   Paralela - full pipeline

29. Structuri SN. P i/e structura

Procesorul de I/E – $P_{I/E}$

- Rolul $P_{I/E}$:
  - Primitiva ce poate fi incorporata in PC-ul in care are functie de I/E sau poate fi privita independent
  - Daca este o primitiva independenta are urmatoarele atribute
- Atribute:
  - Functie: DMA | canal I/E | procesor specializat I/E
  - Tip implementare: conventional | μprogramat | procesor specializat (coprocesoare de I/E)
  - Rata de transfer: Mb/s | Gb/s
  - Mod de transfer: cum rezolva conflictul de acces la UC
    - Prin furt de ciclu
    - Prin rafala
  - Lungimea cuvantului: 8 | 16 biti (in general caractere)

30. Block replacement

Cand are loc un miss, controlerul de cache trebuie sa “faca ceva”

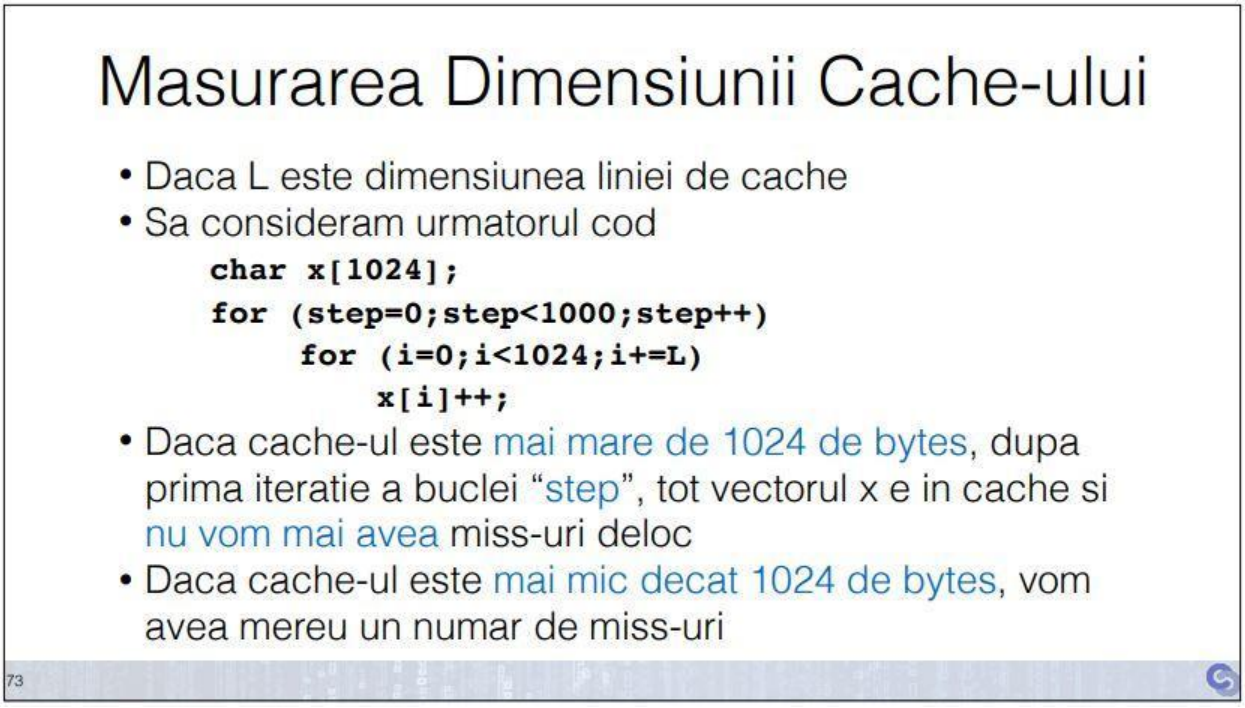
Pe un cache mapat direct, este foarte simplu – se va suprascrie continutul unui block-frame cu date noi aduse din memorie
Intr-o implementare n-set-asociativa insa, trebuie sa facem o alegere: care dintre cele n blockframe-uri trebuie suprascrise?

In practica, exista trei strategii implementate pentru acest lucru

- Random: foarte usor de implementat
- First-In-First-Out (FIFO):
  - Un pic mai dificil de implementat
  - Aduce beneficii la cache-uri de dimensiuni mai mici
- Least-Recently Used (LRU):
  - Si mai dificil de implementat, mai ales pentru cache-uri de dimensiuni mari
  - Aduce beneficii la cache-uri de dimensiuni mai mici
  - Atentie, trebuie sa avem mereu in vedere costul cache-ul vs latenta oferita de acesta: overhead-ul computational si hardware-ul aditional necesar calcularii valorilor LRU

31. Determinarea dimensiunii cache-ului

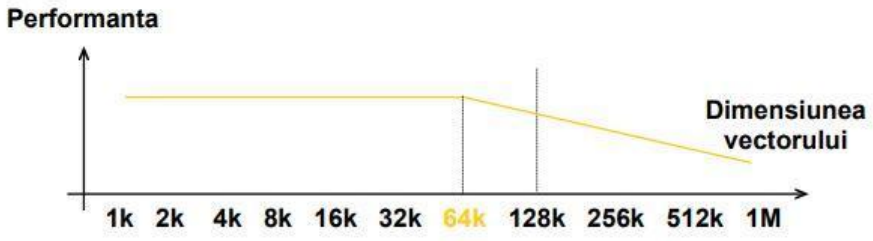
Adresa pe n biti: t biti tag, s biti set index, b biti block offset, fiec adresa are loc unui cuvant
Fiecare bloc: 2^b cuvinte; Memoria cache are 2^s linii;
Dimensiune cache = 2 ^(s+b) cuvinte



BAG PULA IN MATERIA ASTA

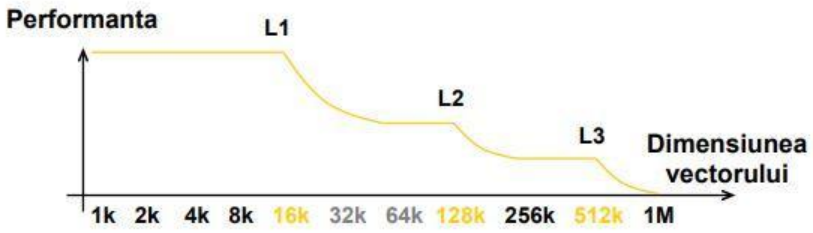
Masurarea Dimensiunii Cache-ului

- Astfel:
  - Daca vectorii sunt mici si intra in cache hit-rate = 100%
  - Daca vectorii sunt mai mari decat cache-ul hit-rate < 100%



Mai multe Nivele de Cache?

- In general insa, exista mai multe nivele de cache: L1, L2, L3
- Avand in vedere acest fapt, graficul anterior devine



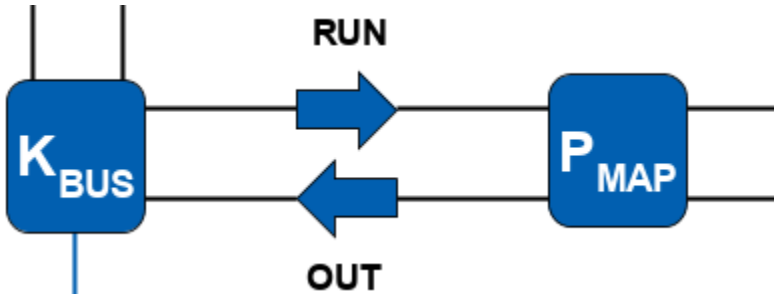
BAG PULA IN MATERIA ASTA

32. Comunicatia Kbus Pmap

KBUS asigura controlul in cadrul unui cluster prin excluderea mutuala a acceselor la MapBus.
Comunicarea intre clustere prin InterCluster Bus.
Functii de baza Pmap:

- Maparea de adrese
- Comunicare
- sincronizare

O serie de primitive OS sunt implementate in Pmap.



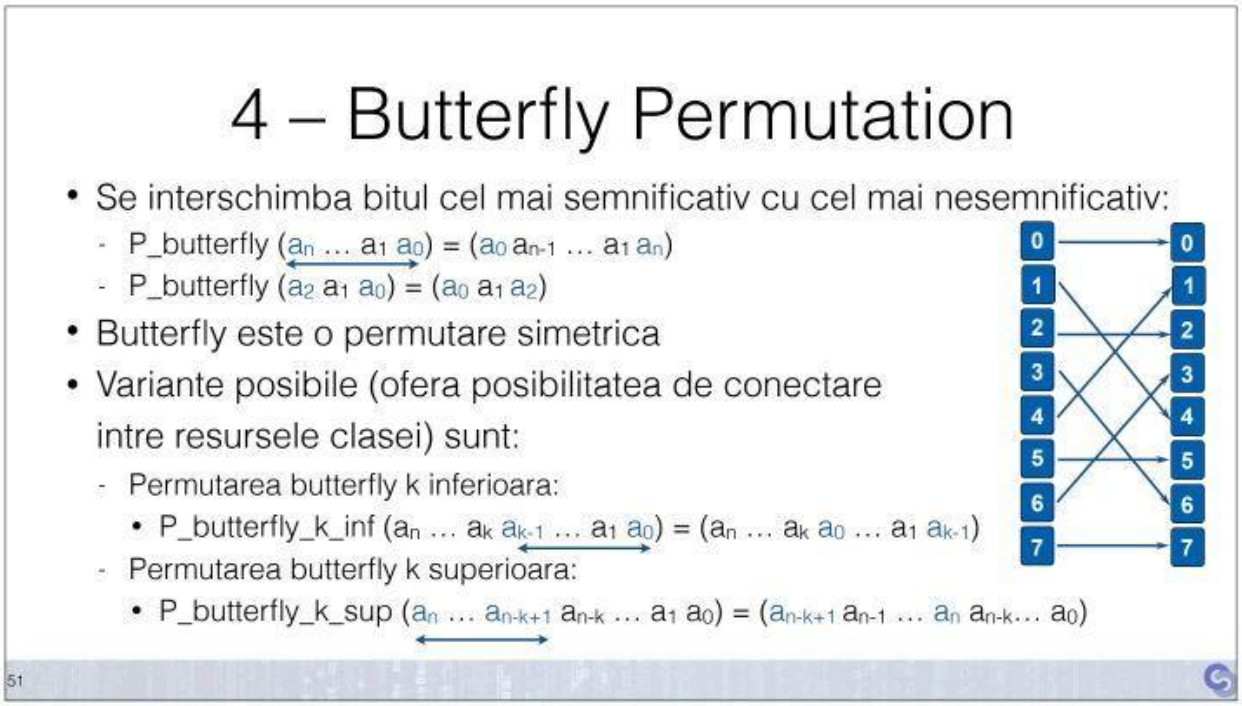
Kbus este unitatea de comanda a map bus.
Pmap este un procesor de mapare care face translatia de adrese.
KBUS memoreaza STAREA contextelor si le aloca cereri. Le trimite prin RUN la PMAp.
PMAP proceseaza cererile de la KBUS care activeaza schimbarile de context cerute de KBUS.
PMAP pune iesirea la OUT

- K_{BUS} mentine starea a 8 contexte si le aloca cereri de serviciu (fiecare va fi rezident in P_{MAP})
- Numarul contextului si alte informatii sunt plasate in coada RUN
- Din RUN sunt preluate de P_{MAP} care activeaza contextul si eventual translateaza adresa si o plaseaza in OUT
- In acest timp K_{BUS} poate citi alte cereri sau efectua alte functii cerute de P_{MAP}
- In P_{MAP} se executa o schimbare de context prin intermediul celor doua cozi (RUN/OUT)

Comunicatia K_{BUS} – P_{MAP}

- K_{BUS} preia acum cererea din OUT si transfera adresa si datele la S-ul local al destinatiei; apoi se ocupa de alte cereri
- Cand accesul la memorie este terminat, K_{BUS} citeste confirmarea de terminare sau citeste datele si plaseaza iara contextul in RUN, reactivandu-l
- K_{BUS} trimite apoi confirmarea la procesul ce a initiat cererea de serviciu
- Contextul poate fi utilizat acum pentru o noua cerere de serviciu

33. Permutarea Butterfly



34. Benchmark-uri pentru sisteme seriale

- SPEC (Standard Performance Evaluation Corporation) realizeaza benchmark-uri pentru sisteme de calcul si le publica pe web
- Incearca sa simuleze situatii reale:
  - SPECweb testeaza performantele unui server de web pentru diferite tipuri de cereri HTTP paralele
  - SPEC CPU testeaza CPU-ul ruland
- SPEC sunt scrise in C sau Fortran (compilatoarele pot avea optimizari pentru acest tip de aplicatie)
- Se distribuie cu o licenta
- Exista un conflict cu GPL (foloseste cod GPL)…
- http://www.spec.org/

- TPC (Transaction Processing Performance Council) a definit benchmark-uri pentru procesarea tranzactiilor si a bazelor de date:
  - http://www.tpc.org/
- Consortiul BAPCo (Business Application Performance Corporation) este format din Intel, AMD, Dell, HP, NVidia, Apple & Microsoft, etc:
  - WEBmark, SYSmark 2004SE, si MobileMark
  - http://www.bapco.com/
- **SiSoftware Sandra** (**S**ystem **AN**alyser, **D**iagnostic and **R**eporting **A**ssistant) este un program de testare si diagnoza:
  - http://www.sisoftware.net/

35 Cell BE bariere si solutii.

All APUs can work independently
- Sounds like a memory nightmare

Synchronization is done in hardware
- Avoids software overhead

Memories on both ends have additional status information

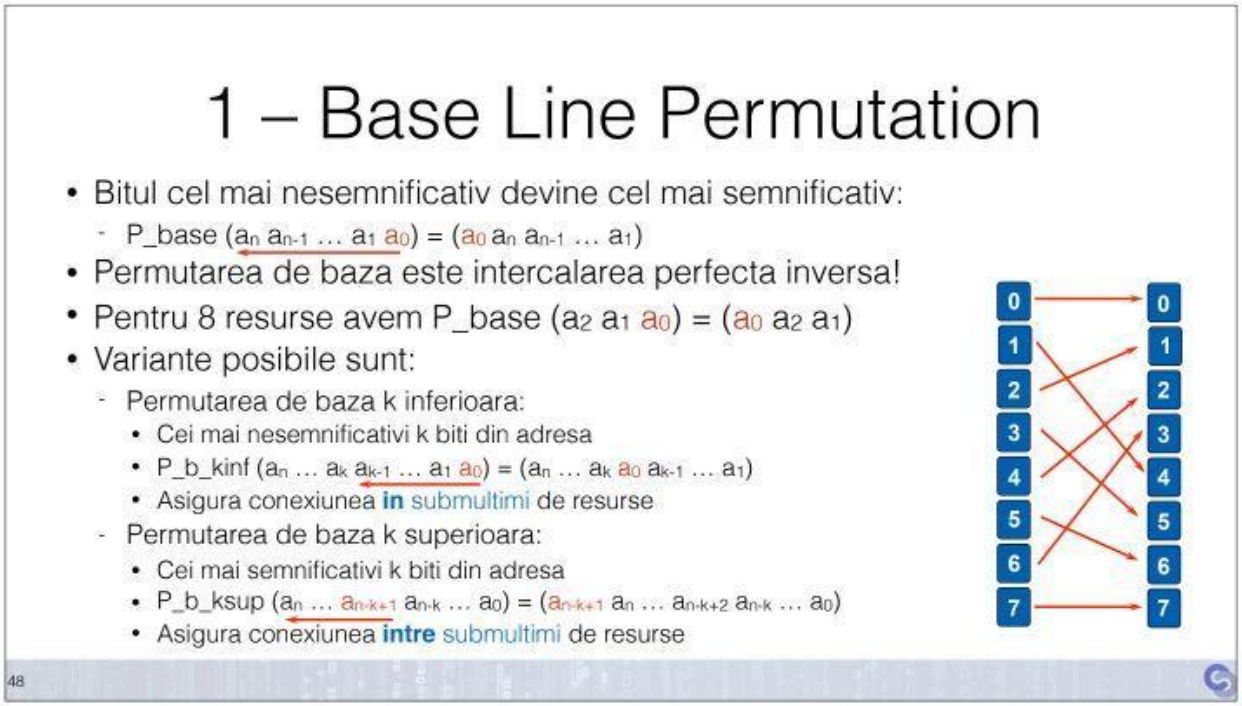
Each 1024 bit addressable memory chunk in DRAM has
- Full/Empty bit
- Memory for APU ID and APU LS address

Each APU has
- Busy bit for each addressable part of local storage

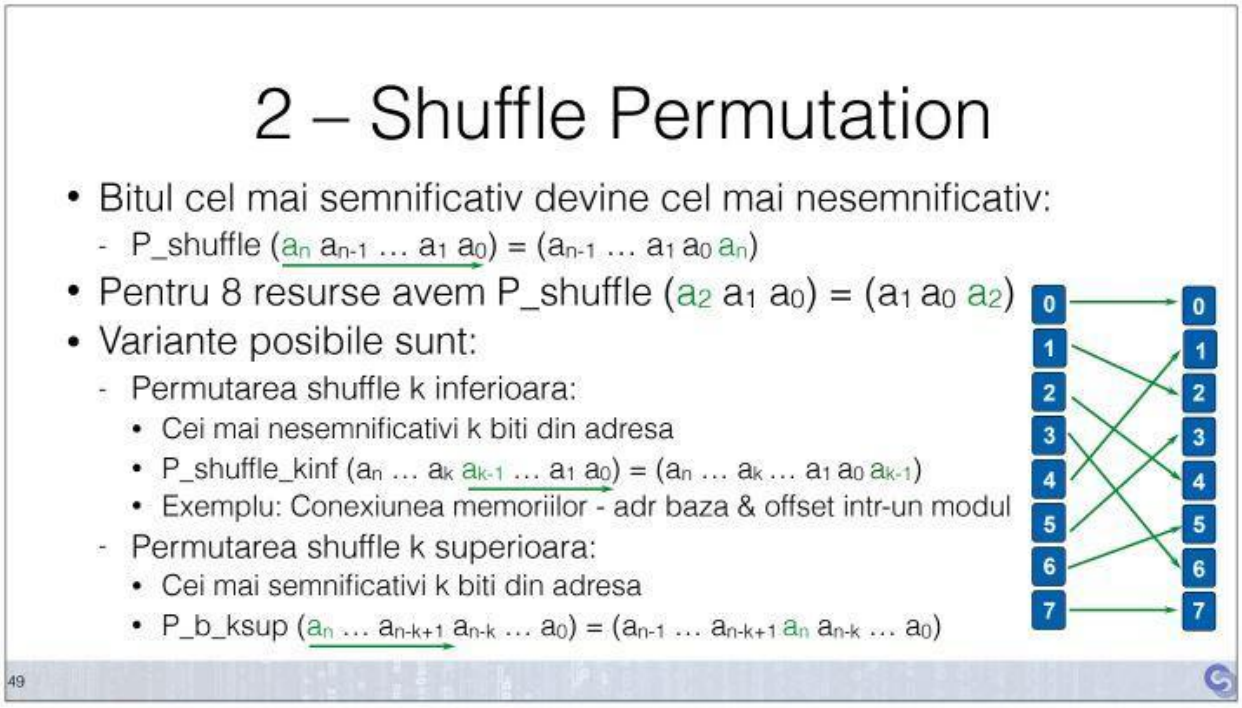
Full/Empty Bit – data is current if equals 1
- APU cannot read the data if it is 0
- APU leaves its ID and local storage address
  - Second APU would be denied

Busy bit
- If 1, APU can use to write info from DRAM
- If 0, APU can write any data

36. Comparatie intre permutarea de baza si perfect shuffle



Adecvat pentru operatii recurente, calculabile in log2n pasi.



(aka perfect shuffle)
Este adecvata pentru:

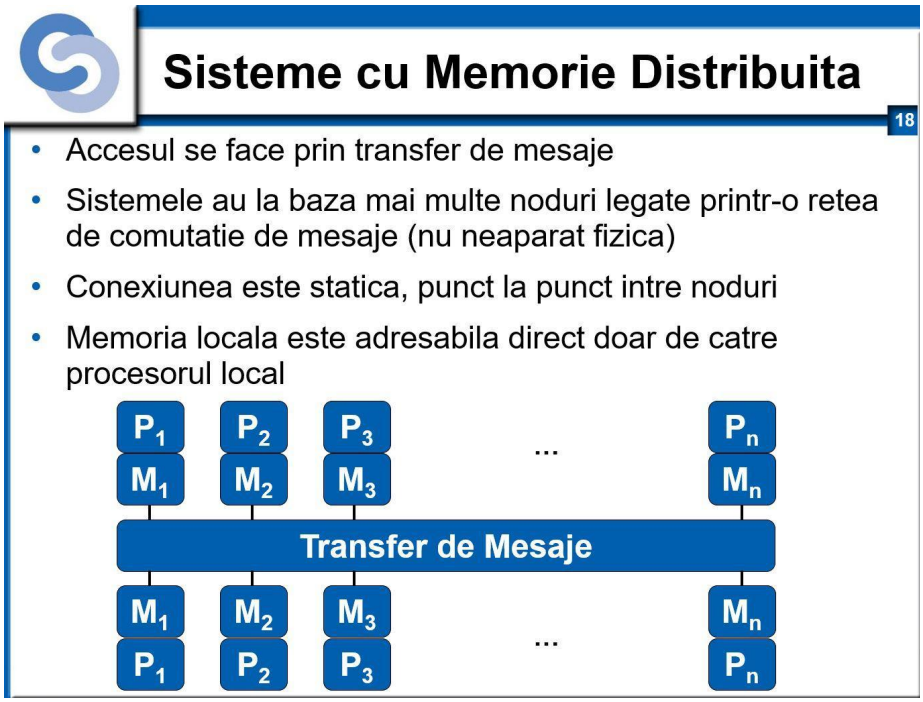
- Transformata Fourier (FFT – Fast Fourier Transformation)

BAG PULA IN MATERIA ASTA

- Algoritmi de sortare
- Transpuneri de matrice

Asigura conectarea intre procesoarele unei structuri SIMD de n procesoare cu:
n-2 comunicatii simultane
dupa log2n iteratii datele ajung pe procesorul sursa

37. Memorii distribuite



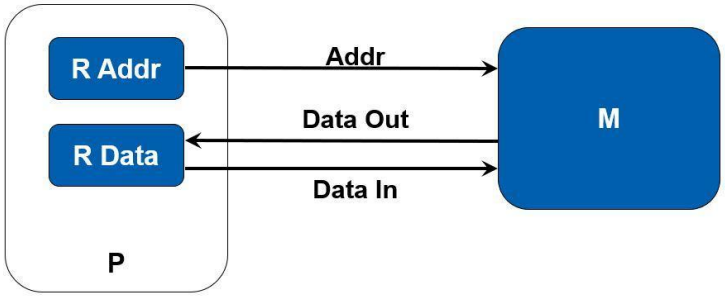
Ex: SUN Enterprise 10000 - Starfire -> Reteaua de comutatie este de tip Gigaplane XB (102.4GB/s; p2p: 1.6GB/s)

38 Optimizari programe (trebuia vb de cache)
Aici merge zis de C++/Java ca sunt row major si ca exploatează localitatea spatiala.
Localitate spatiala +/- localitate temporala: Ce sunt, ce înseamnă, când apar.
Cum ajuta cache: cache hit + evitare trashing.
Optimizări în cod: reorganizarea buclelor, folosirea inteligenta a variabilelor (ca în lab la sum pe matrice).

39 Comutatoare în cuplarea memorie procesor - Cursul 4a, ultimele 3 slideuri

BAG PULA IN MATERIA ASTA

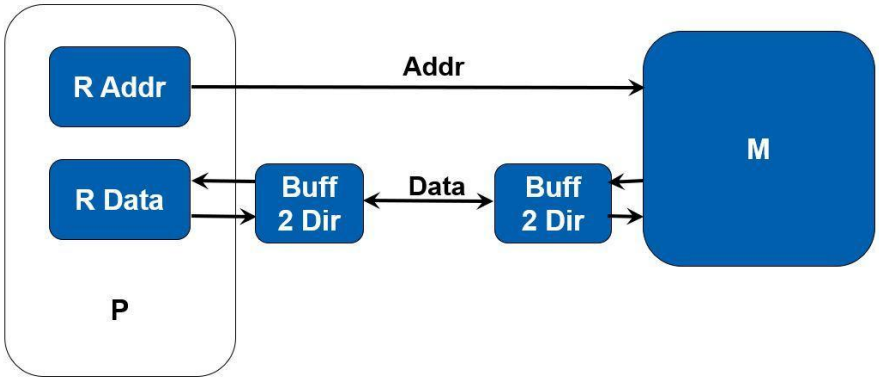
- Switch Processor Memory → $\mathbf{S_{PM}}$
- Liniile de adrese si date sunt complet separate



R/W – Disjuncte → No Switch!

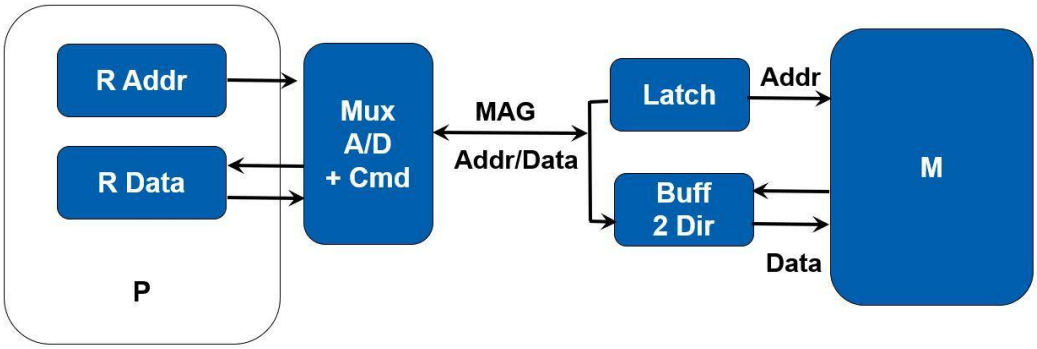
(nu se justifica ambele linii)

- Switch Processor Memory → $\mathbf{S_{PM}}$
- DIn si DOut sunt multiplexate prin magistrala de date



Bufferele sunt fie MUX fie TS

- Switch Processor Memory → $\mathbf{S_{PM}}$
- Addr, DIn si DOut sunt multiplexate prin aceeasi magistrala



Bufferele sunt fie MUX fie TS

42 Comparatia generala intre SIMD si MIMD

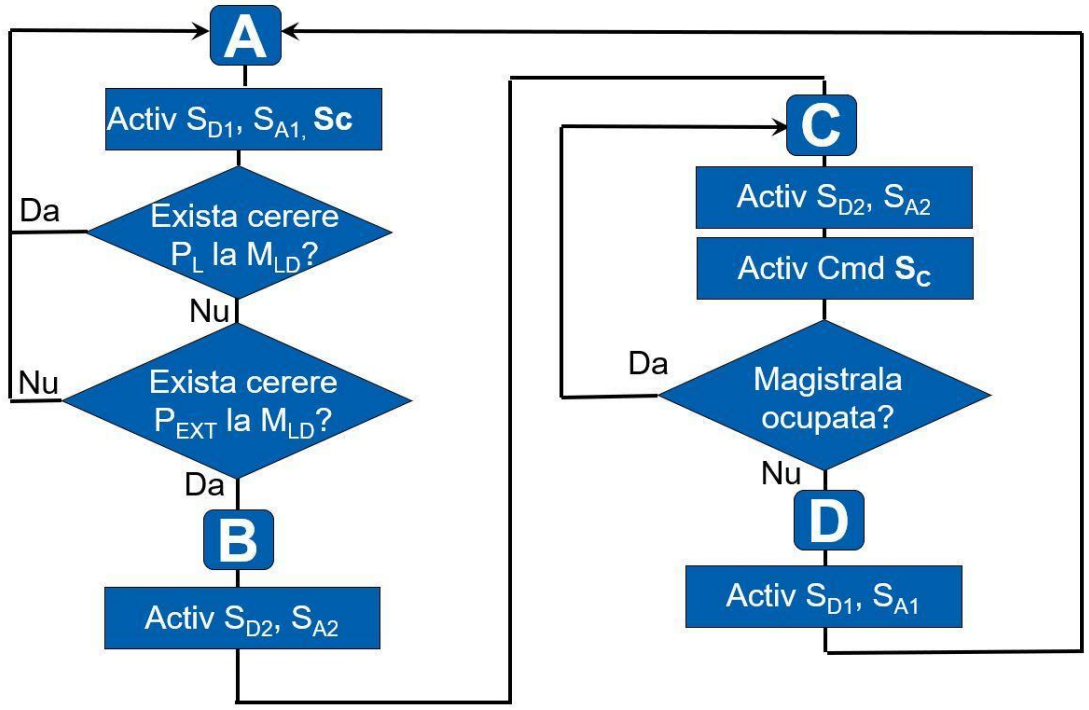
Diferente SIMD/MIMD pentru codul A x B =C:

- In SIMD procesoarele se sincronizau instructiune cu instructiune
- In MIMD nu exista (neaparat) sincronizari intre FI (flux de instructiuni) ale P-urilor din structura
- La SIMD se calculeaza elementele lui C pe linie si coloana (FI unic)
- La MIMD orice procesoare pot calcula orice elemente din C (fiecare P are un FI propriu!)

MIMD: mai usor de programat & utilizat decat SIMD – dar, mai scump

Compartie organizare date: vezi subiectul 1.

43 Organigrama KML de la SBC-uri + schema logica



- K_{ML} – asigura accesul la Mem locala a P_L prin intermediul S_{D1} si S_{A1} & a proc externe prin S_{D2} si S_{A2}
- P_L are prioritate fata de proc externe (P_{EXT}) == daca vrea P_L la M_{LD} NU ma intereseaza ce vor celelalte P_{EXT}
- Starea A: P_L poate avea comenzi locale
- Starea B: activez accesul ext la M_{LD} – semnalele asociate starii
- Obs: Mem locala e vazuta de fiec proc intr-un ac spatiu de mem, iar de proc ext in spatiul complementar → trebuie sa existe o unitate de translatare a adreselor (TA)
- Mag ocupata: nu stii ciclii masina ai unui proc extern. Numai cand magistrala e libera un P_{EXT} a terminat ciclul masina curent → abordare top-down

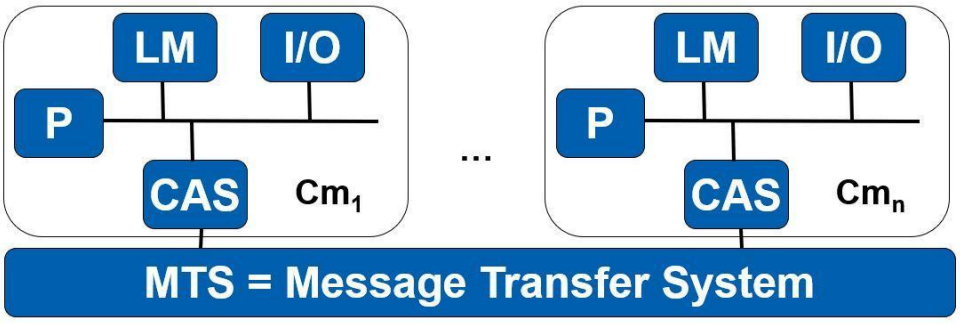
45. Exemplificati primitivele P si M de pe calculatorul propriu

Asta e din aia la misto pt dat puncte la vrajeala, cred. Zici ce CPU si ce MEM ai, right?

46. Comunicatia intercluster intr-un sistem MIMD slab cuplat

- De tip cluster → calculatoarele au resurse locale necesare functionarii independente
- Procesele se executa pe procesoare care dispun de toate resursele pentru implementarea task-ului respectiv
- Comunicarea intre procesoare se face prin
  - Transfer de mesaje
  - Comutare de circuite
- Sistemele slab cuplate (distribuite) sunt eficiente atunci cand interactiunea intre procesoare este minima (granularitatea problemei este mare)

- Resurse locale ale CM_i = Computer Module:
  - LM = local memory (acces din interior/exterior)
  - I/O = magistrala de intrare/iesire (acces din interior/exterior)
  - MTS = Message Transfer System – poate fi implementata cu comutare de circuite sau transfer de mesaje
  - MTS dicteaza performantele prin lungimea mesajelor & viteza de lucru
  - CAS = Collision Avoidance System – legatura si accesul la MTS se fece prin excludere mutuala:
    - un singur transfer odata daca MTS este magistrala
    - mai multe transferuri simultane daca MTS este de tip retea



51 Care sunt cele 4 elemente de care trebuie tinut cont cand se realizeaza designul unui cache.

Block placement - unde trb scris un bloc in cache,
Block identification - cum se gaseste o linie in cache
Block replacement - cum se inlocuieste o linie in cache la un miss
Write strategy - ce se int cand se scrie in cache (write through / write back)

52 De ce nu se poate realiza un procesor de 10TFlops(minim 3 motive) ?

- Am avea nevoie de un procesor de 10.000 GHZ
- Datele ar trebui sa circule cu viteza luminii
- O celula de memorie ar trebui sa fie de dimensiunea a cativa atomi

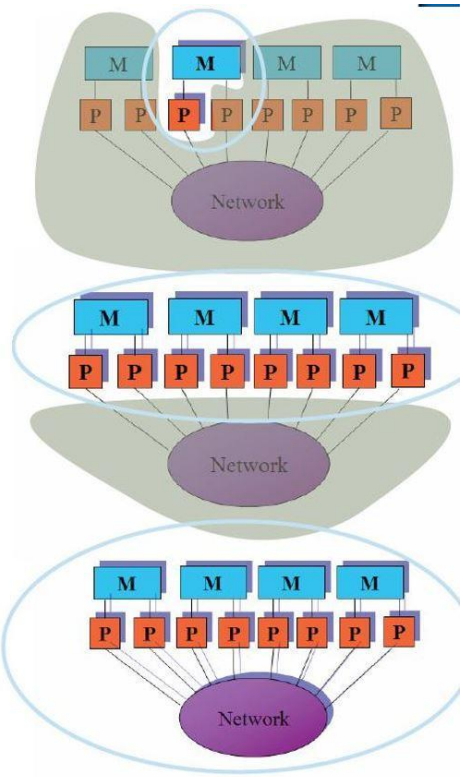
53 Caracteristici HPC Challenge Benchmark.

- Un Benchmark ce evidentiaza
  - Procesoarele
  - Memoria
  - Reteaua de interconectare

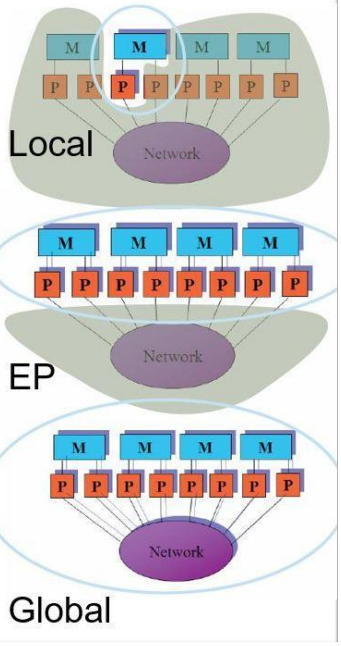
HPCS = High Productivity Computing Systems
DARPA = Defense Advanced Research Projects Agency

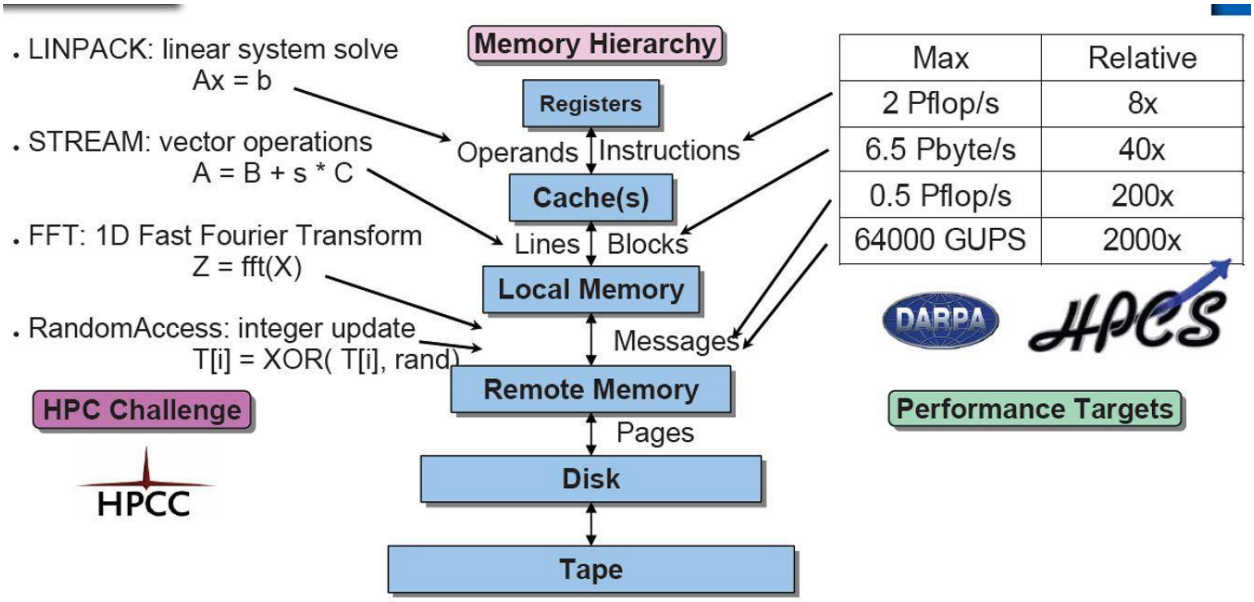
- Avantajul major al HPCC – arhitecturile pot fi descrise cu mult mai multe metrici decat FLOPS-urile LINPACK
- Scopuri ale HPCC
  - Sa fie complementar cu TOP 500
  - Sa ofere benchmark-uri care exprima performata aplicatiilor ca o functie a caracteristilor accesului la memoria sistemului (ca localitate spatiala si temporala)
  - Sa permita optimizari
    - Sa inregistreze efortul necesar pentru tuning-ul aplicatiilor
    - Rularea necesita MPI si BLAS
  - Rezultatele obtinute sa poata fi verificate
  - Sa arhiveze rezultatele

- Local: un singur procesor participa la calculele dorite
- Embarrasingly Parallel: fiecare procesor din sistem participa la calcule, insa ele nu comunica explicit intre ele
- Global: toate procesoarele din sistem participa la calcule si comunica explicit intre ele



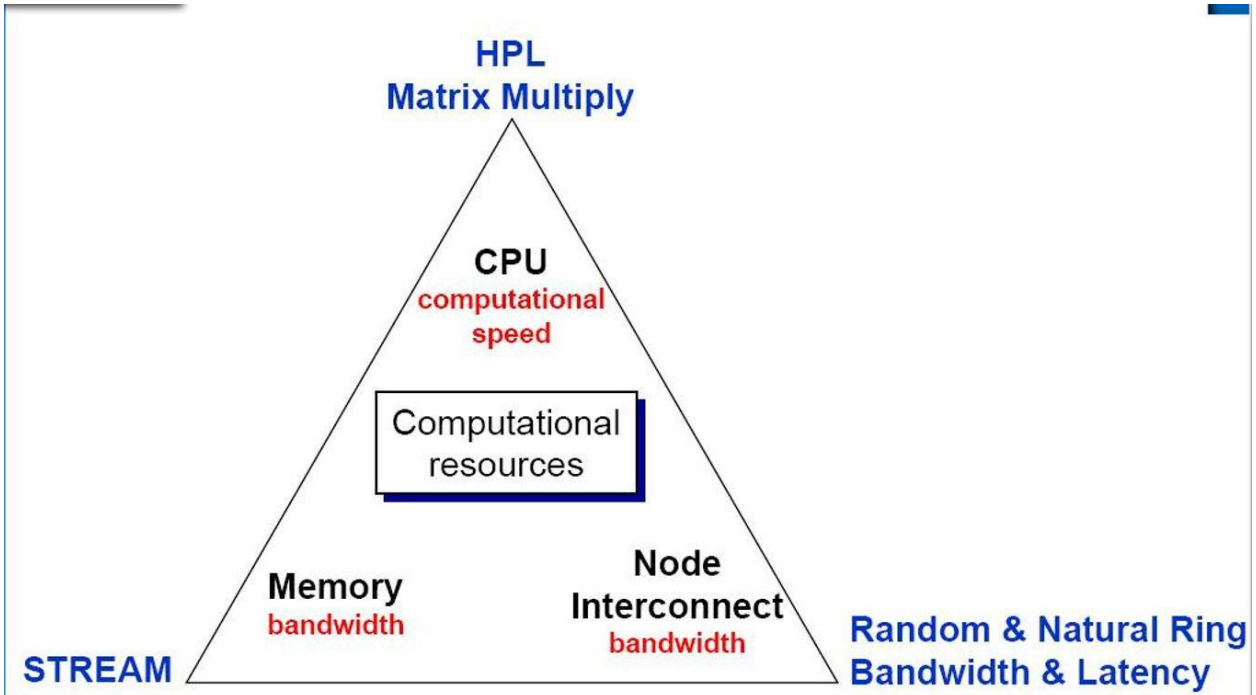
- HPCC e format din 7 benchmark-uri diferite si este de fapt un framework pentru adaugarea de benchmark-uri relevante:
  1. HPL (LINPACK) – MPI Global (Ax=b)
  2. STREAM – Local
     *STREAM – EP
  3. PTRANS – MPI Global ($A \leftarrow A + B^T$)
  4. RandomAccess – Local, MPI Global
     *RandomAccess – EP
  5. BW and Latency – MPI
  6. FFT – Local, Global & EP
  7. Matrix Multiply – Local & EP





- HPCS incearca prin HPCC sa:
  - Uniformizeze diferentele existente in ierarhia de memorie
  - Imbunatateasca performantele aplicatiilor reale
  - Usureze programarea sistemelor (paralele) de calcul

- Benchmark-ul functioneaza ca un program de sine statator
- Similar ca fisier de intrare/specificare cu HPL
- HPCC este format dintr-un “Base Run”, urmat de eventuale “Optimization Runs”:
  - Base Run este rularea Benchmark-ului “out of the box”
  - Utilizatorul trebuie sa instaleze pe masinile de test MPI si BLAS
  - In Optimized Runs se pot inlocui anumite subrutine, pentru a creste performantele unei anume componente
- Rezultatele se upload-eaza pe web la: http://icl.cs.utk.edu/hpcc/
- Se pot genera apoi tabele html sau Excel cu rezultatele obtinute:
  - In mod intentionat nu se va oferi un singur criteriu de evaluare
  - Astfel NU se vor obtine clasificari generale!
- Scopul declarat al HPCC: sa NU necesite mai mult de dublul unei rulari HPL

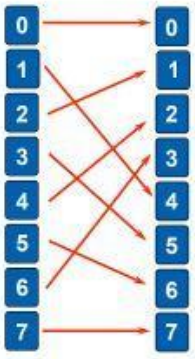


- Din ce sunt formate componentele HPCC?

54 Comparatie Base Line Permutation - Bit Reversal Permutation

1 – Base Line Permutation

- Bitul cel mai nesemnificativ devine cel mai semnificativ:
  - P_base ($a_n\ a_{n-1} \dots a_1\ a_0$) = ($a_0\ a_n\ a_{n-1} \dots a_1$)
- Permutarea de baza este intercalarea perfecta inversa!
- Pentru 8 resurse avem P_base ($a_2\ a_1\ a_0$) = ($a_0\ a_2\ a_1$)
- Variante posibile sunt:
  - Permutarea de baza k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_b_kinf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0\ a_{k-1} \dots a_1$)
    - Asigura conexiunea **in** submultimi de resurse
  - Permutarea de baza k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_b_ksup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_0$) = ($a_{n-k+1}\ a_n \dots a_{n-k+2}\ a_{n-k} \dots a_0$)
    - Asigura conexiunea **intre** submultimi de resurse

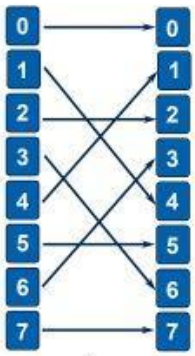


48

BAG PULA IN MATERIA ASTA

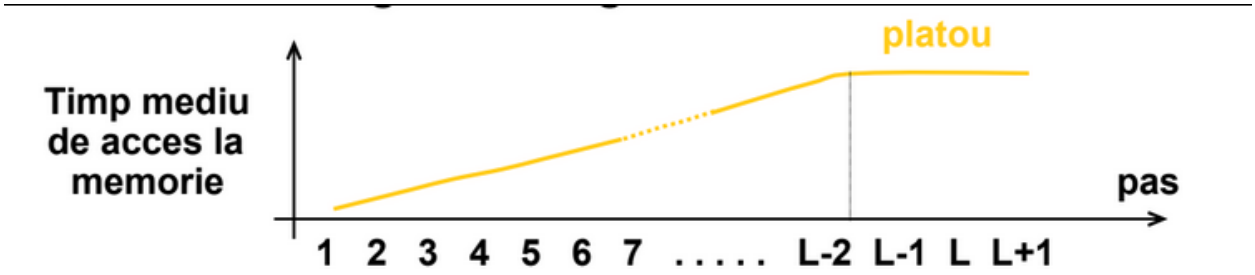
5 – Bit Reversal Permutation

- Se interschimba complet ordinea bitilor:
  - P_reversal ($a_n \dots a_1\ a_0$) = ($a_0\ a_1 \dots a_{n-1}\ a_n$)
  - P_reversal ($a_2\ a_1\ a_0$) = ($a_0\ a_1\ a_2$)
- Cu aceasta permutare se pot obtine echivalente
- Variante posibile sunt:
  - Permutarea reversal k inferioara:
    - P_reversal_k_inf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0\ a_1 \dots a_{k-1}$)
  - Permutarea reversal k superioara:
    - P_reversal_k_sup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_1\ a_0$) = ($a_{n-k+1} \dots a_n\ a_{n-k} \dots a_0$)



52

55 Cum se masoara dimensiunea liniei unui cache? Exemplu de program si grafic.
(pentru inlocuirea oricarui alt subiect :))
Char a[N];
for(i = 0; i < N; i++)
a[N]++;
Marim pasul i +=2, i +=3, pana performanta codului nu se mai inrautateste.
La pasul 1 avem cache miss la fiecare indice care se imparte exact la L
Pasul de la care nu se mai inrautateste performanta, e dimensiunea liniei de cache



BAG PULA IN MATERIA ASTA

56 Arhitectura unui calculator din top 500 (arhitectura, numar procesoare, so, ...)
https://www.top500.org/system/179397

Summit - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband

Site:	DOE/SC/Oak Ridge National Laboratory
System URL:	http://www.olcf.ornl.gov/olcf-resources/compute-systems/summit/
Manufacturer:	IBM
Cores:	2,397,824
Memory:	2,801,664 GB
Processor:	IBM POWER9 22C 3.07GHz
Interconnect:	Dual-rail Mellanox EDR Infiniband
Performance	
Linpack Performance (Rmax)	143,500 TFlop/s
Theoretical Peak (Rpeak)	200,795 TFlop/s
Nmax	16,693,248
HPCG [TFlop/s]	2,925.75
Power Consumption	
Power:	9,783.00 kW (Submitted)
Power Measurement Level:	3
Measured Cores:	2,397,824
Software	
Operating System:	RHEL 7.4
Compiler:	XLC, nvcc
Math Library:	ESSL, CUBLAS 9.2
MPI:	Spectrum MPI

BAG PULA IN MATERIA ASTA

Problema (anul acesta NU a fost openbook):
Se dadea cazul Carrefour care primeste de la fiecare magazin o lista ce conținea un fel de inventar cu numele produsului, cantitatea vândută și prețul produsului.

a) Se cerea să se proiecteze un sistem care realizează interpretarea acestor date folosind doua arhitecturi studiate la laborator. Punctajul se acorda în funcție de argumentele bine formulate în favoarea sau detrimentul arhitecturilor alese.

b) Pentru una dintre arhitecturile de la punctul a, să se schițeze un pseudocod. Se punea accent la formatul fișierelor de intrare/ieșire, cum se transmit datele, chestii specifice arhitecturii în principiu.

La problema se cerea implementarea in pe arhitectura Cell a unui
Client
de BitTorrent. Se cerea explicarea functionalitatii clientului si
aplicabilitatea pe arhitectura Cell precum si pseudocod. (se punea mai
mult accent pe explicatii nu neaparat pe pseudo-cod)
Se explica in enuntul functionarea unui client de torrent

Problema clasica anul trecut:
Ai N bile punctiforme pentru care ti se dau :
x
y coordinate initiale (coordonateme extremitatii initiala a
segmentului pe care se deplaseaza bila)
v viteza de deplasare
x_ext
y_ext coordonateme extremitatii finale a segmentului pe care se
deplaseaza bila
Daca doua bile se ciocnesc atunci dispar din sistem.
Se cere sa se afiseze coordonatele la care o bila dispare din system
(daca s-a ciocnit cu a olta bila) sau coordonatele extremitatii finale
pentru bilele care au ajuns in extremitati in perioada T de timp
(data).
a) utilizand instr SIMD calculate distanta parcursa de fiecare bila
(se dadea formula distantei dintre 2 pct)
b) sa se modeleze sistemul cu threaduri (pthreads/python threads)
c) sa se modeleze sistemul in Cell

La problema:
De implementat si in Python si in CELL ceva cu o imagine si se
impartea la 4 SPU-uri. SPU-urile randau imaginea, o trimiteau inapoi
la PPU care o afisa. Nu prea am idee despre ce era vorba exact
aici :).

Cam asta era. Ai o animatie, si ti se cere sa o randezi in paralel (nu ti se cer detalii despre randarea propriu-zisa, ci doar despre modul in care imparti sarcinile catre thread-uri(Python) sau SPU-uri (Cell)).
2.

Problema 1h:
Avem o imagine reprezentata printr-o matrice cu elemente de la 0 la 255. Se cere sa se mareasca dimensiunea matricei pana la un X dat astfel:
Exemplu:
Pas 0:
10 30
15 25
Pas1:
10 20 30
12.5 20 30
15 20 25

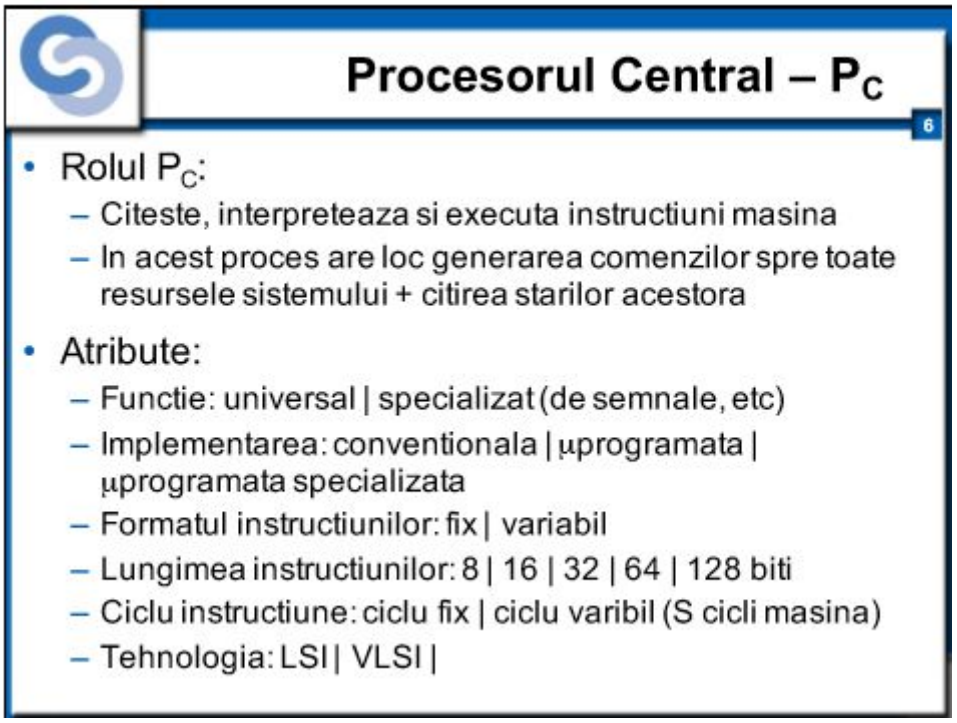
Problema :

Se dau doua matrici A,B de dimensiune 4096x4096 .Sa se scrie un preudocod care calculeze produsul celor doua matrici folosind arhitectura Cell:
inmultirea normala a celor doua matrici
inmultirea folosind BMM
La fiecare subpunct, pe langa pseudocod trebuia sa descriem modul in care se vor retine datele in PPU/SPU
La problema sa nu uitati ca aveti voie cu orice fel de materiale.

Problema:
Se da un teren de NxN ?i un nor de MxM cu M < N, norul deasupra terenului ?i incepe sa ploua. Terenul e denivelat si fiecare element din matrice contine inaltimea acelui punct.
Cand ploua apa de pe zonele inalte se revarsa in mod egal pe zonele vecine mai joase
1)Explicati cum se poate paraleliza.
2)Pseudocod CELL.
3)Se pot folosi instructiuni SIMD ? Cum?
4)Se poate folosi double buffering? Cum?
5)Se merita utilizarea CELL pentru aceasta problema? Daca nu, ce ai folosi in schimb?

PMS: (curs 2) - subiect sigur

- Descrieti primitiva P (si M sau cu privire la PMS) de la calculatorul personal. (**2015**, **2014t**, cu M in 2009, 2010, 2009, 2009)



- Descrieti folosind PMS structura cu care ati lucrat la laborator. (**2014**, 2011)
- Nivelul structural de organizare a sistemelor numerice. Exemplu de modelare PMS a unui sistem multiprocesor. (2009)
- Structuri SN. P i/e structura (2010)

SBC: (curs 3a) - subiect sigur

- Considerand o structura multiprocesor pe un bus/magistrala comun, descrieti unitatea de acces la magistrala sistemului al fiecarui modul. (**2015**, **2014, 2014t**)
- Organigrama Kbus; Descrierea schemei logice Kbus (2010, 2010, 2009, 2009)
- Schema logica Kml sau Organigrama KML de la SBC-uri. (2011, 2009)

Comutatoare: (curs 3b)

- Comutator de trunchi K ierarhic (**2014**, 2011, 2010, 2009)
- Comutator legatura multipla (CrossBar) ierarhic. (2010)
- Neierarhice (2009)
- Comutatoare in cuplarea memorie procesor (2010)

Cache cu memorie: (curs 4): - nu cred ca mai da asa ceva … nici macar nu l-a predat el

- Block placement/replacement in cache. (2010, 2010)
- Determinarea dimensiunii cache-ului (2010)

- Cum se masoara dimensiunea liniei unui cache? Exemplu de program si grafic. (pentru inlocuirea oricarui alt subiect :) (2009)
- optimizari programe (trebuia vb de cache) (2010)
- Care sunt cele 4 elemente de care trebuie tinut cont cand se realizeaza designul unui cache. (2009)

CELL: (curs 5 // lab 7,8,9)

- Descrieti caracteristicile generale CELL (**2014t**)
- Mituri despre Cell BE.(2010)
- Cell BE bariere si solutii.(2010) - **ce e mai jos scrie in curs si aici: https://docs.google.com/document/d/1y1B0GEFnlDzzlnDlUe2BX8FccWWqat08VVlEyG1G1Hc/edit tot asta au scris**

Memory Wall,

-more slower threads

-asynchronous loads

Efficiency Wall ,

-more slower threads

-specialized functions

Power Wall

-reduce transistor power

-reduce switching function

SIMD/MIMD (curs 7-8-9): -- doua subiecte sigur

- Organizarea/reprezentarea datelor in structurile SIMD/MIMD (**2015**, **2014, 2014t,** 2010, 2009)
  - la SIMD curs 7 final cu chestiile alea de acces linie/coloana sau cu shufle pe ambele; **(curs 8 final)**
  - la MIMD ..(???) un pic in curs **8a final** cred + un pic **curs 9 slide 4, slide 8 + tot de la slide 12 cred pana la final**

- Clasificarea Flynn a sistemelor de calcul. (**2014t**, 2011, 2009, 2009) **(curs 7)**

- Comparatie SIMD vs SISD pe exemplu de cod. (2010) **(curs 7)**
- Comparatie SIMD/MIMD
  - generala intre SIMD si MIMD (**2014**, 2009, 2009) **(curs 7)**
  - dpdv al structurii si organizarii/reprezentarii datelor (nu garantez) (2011, 2009)
  - Comparatie SIMD MIMD la algoritmi (2010) **(curs 7 cred)**
  - Comparatie salturi conditionate SIMD MIMD (2011, 2010, 2010, 2009, 2009) **(la SIMD: curs 8a, MIMD = ?,d esi intuitiv este evident)**

- De ce nu se poate realiza un procesor de 10TFlops(minim 3 motive) ? (2009) **(curs 7 inceput)**

Distanta dintre memorie si procesor trebuie sa fie atat de mica incat nu avem inca tehnologia necesara (doar in quantum computing e posibil sa se ajunga la acea dimensiune).

zice si de viteza datelor ca ar trebui sa fie egala cu viteza luminii, frecventa procesorului sa fie 10^5 GHz

Consum prea mare de energie?
vviteza memoriei prea mare pentru timpurile actuale
…?

- Tipuri de paralelizare la sistemele cu prelucrare paralela.(2010) **(curs 7 inceput)**

-paralelism la nivel de bit (operatii floating point)
-paralelism la nivel de instructiune (mai multe instr / ciclu <-pipeline)
-paralelism la nivelul memoriei (acces concurential)
-paralelism al sistemului de operare (thread-uri, procese) in aceeasi masina
-paralelism distribuit

aici nu erau:
- **paralelism la nivel de bloc**
- **paralelism la nivel de subansamble hardware**
- **paralelism la nivel de instructiune**
- **paralelism la nivel de program**

??

- **Programul: (curs 7)**
  - Sa se scrie programul de impartire a doi vectori de dimensiune n pe o structura si SIMD si MIMD. Comentati performantele. (**2015**, **2014**) **(curs 7)**

luat din documentul de la calapod (cel postat de miruna):
SIMD:
Fie vectorii u si v si distribuim u[k] si v[k] in modulul de memorie k. Fiecare procesor imparte cele 2 elemente corespunzatore din modulul sau.
w[k] = u[k] / v[k] (0 <= k <= n 1) k indexul procesorului, n numarul total de procesoare
O (1)
Nu e complet, trebuie tinut seama de impartire la 0 (eventual cu masti).
MIMD:
Acelasi lucru, doar ca faci cu fork si join, e prin cursul 6.

  - Sa se scrie programul de insumare a elementelor unei matrice A(n, n) pentru o structura SIMD si pentru o structura MIMD. Comentati performantele. Obs. Trebuie specificata organizarea datelor. (**2014t**, 2009) (curs 7 + 8 la SIMD)

    - Adunarea elementelor a doi vectori in SIMD si MIMD (cod + cum retii datele Sa se analizeze performantele. Specificati organizarea datelor) (2011, 2009)
    - program impartire vectori SIMD + MIMD, comparatii intre ele (2010)
    - MIMD, Exemplu pe progr cu dependinte de date (2010) **(curs 7 final)**
    - Sa se scrie programul ce determina produsul elementelor unei matrice pe SIMD si MIMD. Sa se analizeze performanta. Reprezentarea datelor. (2009, 2009) **(curs 7)**

Retele de comutare/permutari SIMD (curs 8b): - subiect sigur

- Reteaua de comutare de tip intercalare perfecta si comparatie cu reteaua butterfly. (**2015, 2014,** 2009)
- Retea de comutare de tip intercalare perfecta si comparatie cu reteaua cu ordine inversa (**2014t**)
- Retea de comutare de tip interclasare perfecta (2009)
- Comparatie permutare baza, permutare perfecta (2010, 2010)
- Comparatie intercalare perfecta si permutarea de baza (2009)
- Subiect cu permutari (parca era descrierea permutarilor de tip perfect shuffle) (2011)
- Comparatie Base Line Permutation - Bit Reversal Permutation (2009)
- Permutarea Butterfly (2010)

- Memorii distribuite (2010) (cred ca UMA/NUMA) **(curs 9)**

Comunicatia intercluster/intracluster: (curs 10) - 2 subiecte sigur

- Comunicatia intercluster în structurile slab cuplate. (**2015, 2014, 2014t,** 2011, 2010, 2009, 2009)

aici ce ati scrie? ca in curs nu scrie exact cum e comunicatia la structurile slab cuplate. spune doar ca se face prin transfer de mesaje si comutare de circuite.

- Comunicare in cadrul unui cluster (2010)

- Structura generala Kmap (diagrama). (**2015**, **2014, 2014t,** 2010, 2009, 2009) **(curs 10)**
- Arhitectura procesorului PMAP. (2010)
- Comunicatia Kbus â?? Pmap (2010)

Retele de comutare MIMD (curs 12): - subiect sigur

- Comparatie între crossbar si delta privind performantele (sau in general).(**2015**, 2009, 2009)

- 16 procesoare cu 16 memorii utilizând retea de tip Delta; implicit procesorul i sa fie conectat la memoria i. Indicati conexiunea de la procesorul 2 la memoria 6. (**2015**, **2014, 2014t,** 2009)
- Retele de comutatie de tip Delta. Exemplu pentru 9 procesoare si 4 memorii. // intre 4 procesoare si 9 module de memorie(2009, 2009)
- Retea Delta 16x9 (dimensiunea nu o mai stiu exact) (2011, 2010)
- Retea delta 16x16, default i , memory i (2010)
- Retele Delta 8x27. (2010)

- Probabilitatea ca o cerere sa fie acceptata la un comutator CB (formula) (2010)

j

Benchmark: (curs 13): - parca a zis ca nu da dinasta

- Benchmark-uri, probleme si rezolvari legate de acestea (ceva de genu). (nu trebuia sa zici de HPCC si LINPACK) (2010)
- Benchmark-uri pentru sisteme seriale (2010)
- Caracteristici HPC Challenge Benchmark. (2009)
- Pe ce nivele se face benchmark la HPC. De ce e bine ca sunt mai multe metrici? (2010)

Top 500 (curs 14): - asta nici macar nu a predat-o, deci nu cred ca da

- Cel mai popular mod de conectare in top500. Cel mai popular SO. Cea mai populara arhitectura(arhitectura, numar procesoare, so, ...) --- (vroia 3 cuv, maxim o fraza: InfiniBand, Linux, Cluster) (2010, 2010, 2009, 2009)
- Sa se aleaga un calculator din TOP 500 si sa vorbim despre el. (2009)

Chestii ramase neatribuite:

- Subiect la alegere din materia de ASC (**2015**) --- despre Tianhe 2 o sa scriu daca imi pica asa ceva :D

PROBLEME:

2015 / Prima sesiune serie / grupa: CA / 2015-06-06

----Problema----

(anul acesta NU a fost openbook):

Se dadea cazul Carrefour care primeste de la fiecare magazin o lista ce con?inea un fel de inventar cu numele produsului, cantitatea vânduta ?i pre?ul produsului.

a) Se cerea sa se proiecteze un sistem care realizeaza interpretarea acestor date folosind doua arhitecturi studiate la laborator. Punctajul se acorda în func?ie de argumentele bine formulate în favoarea sau detrimentul arhitecturilor alese.

b) Pentru una dintre arhitecturile de la punctul a, sa se schi?eze un pseudocod. Se punea accent la formatul fi?ierelor de intrare/ie?ire, cum se transmit datele, chestii specifice arhitecturii în principiu.

2014 / Prima sesiune serie / grupa: CA /, data: 2014-05-26
2014 / Sesiunea de restante serie / grupa: CA /, 2014-09-05

2011 / Prima sesiune serie / grupa: CA /, 1 Iunie 2011
---Problema---
Se dau doua matrici A,B de dimensiune 4096x4096 .Sa se scrie un preudocod care calculeze produsul celor doua matrici folosind arhitectura Cell:
inmultirea normala a celor doua matrici
inmultirea folosind BMM
La fiecare subpunct, pe langa pseudocod trebuia sa descriem modul in care se vor retine datele in PPU/SPU
La problema sa nu uitati ca aveti voie cu orice fel de materiale.

2010 / Prima sesiune serie / grupa: CA / 332/333, data aduagarii: 2011-09-21, 332 si

333CA
335CC

331CC
---Problema---
clasica anul trecut:
Ai N bile punctiforme pentru care ti se dau : x y coordinate initiale (coordonateme extremitatii initiala a segmentului pe care se deplaseaza bila) v viteza de deplasare x_ext y_ext coordonateme extremitatii finale a segmentului pe care se deplaseaza bila Daca doua bile se ciocnesc atunci dispar din sistem.
Se cere sa se afiseze coordonatele la care o bila dispare din system (daca s-a ciocnit cu a olta bila) sau coordonatele extremitatii finale pentru bilele care au ajuns in extremitati in perioada T de timp (data).
a) utilizand instr SIMD calculate distanta parcursa de fiecare bila
(se dadea formula distantei dintre 2 pct)
b) sa se modeleze sistemul cu threaduri (pthreads/python threads)
c) sa se modeleze sistemul in Cell

2009 / Prima sesiune serie / grupa: CA/CB /

1. 331CA, 2009
---Problema---

De implementat si in Python si in CELL ceva cu o imagine si se impartea la 4 SPU-uri. SPU-urile randau imaginea, o trimiteau inapoi la PPU care o afisa. Nu prea am idee despre ce era vorba exact aici :).
Cam asta era. Ai o animatie, si ti se cere sa o randezi in paralel (nu ti se cer detalii despre randarea propriu-zisa, ci doar despre modul in care imparti sarcinile catre thread-uri(Python) sau SPU-uri (Cell)).

2. 15.06.09 - 333CA
---Problema--- 1h:
Avem o imagine reprezentata printr-o matrice cu elemente de la 0 la 255. Se cere sa se mareasca dimensiunea matricei pana la un X dat astfel:
Exemplu:
Pas 0:
10 30
15 25
Pas1:
10 20 30
12.5 20 30
15 20 25
...
20 = media aritemtica a lui 10 si 30
12.5 = media aritmetica a lui 10 si 15
etc.
1)0.2p Sa se descrie cum paralelizati algoritmul pentru CELL.
2)0.4p Sa se discute despre performantele programului: comunicare, instructiuni SIMD. Se poate folosi Double Buffering? Ce avantaje aduce?
3)0.2p Sa se crie pseudocod SPU si PPU
4)0.2p Este algoritmul viabil pentru CELL? Ce a?i putea folosi altceva pentru a implementa algoritmul?
3. 332CA 14.06.09:
---Problema---
Se da un teren de NxN ?i un nor de MxM cu M < N, norul deasupra terenului ?i incepe sa ploua. Terenul e denivelat si fiecare element din matrice contine inaltimea acelui punct. Cand ploua apa de pe zonele inalte se revarsa in mod egal pe zonele vecine mai joase
1)Explicati cum se poate paraleliza.
2)Pseudocod CELL.
3)Se pot folosi instructiuni SIMD ? Cum?
4)Se poate folosi double buffering? Cum?
5)Se merita utilizarea CELL pentru aceasta problema? Daca nu, ce ai folosi in schimb?

4. 332 CB + 334 CB
---Problema---

Se cerea implementarea in pe arhitectura Cell a unui Client de BitTorrent. Se cerea explicarea functionalitatii clientului si aplicabilitatea pe arhitectura Cell precum si pseudocod. (se punea ai mult accent pe explicatii nu neaparat pe pseudo-cod) Se explica in enuntul functionarea unui client de torrent.
Examenul a durat 50 de minute teoria, 45 problema. Corectarea s-a facut lejer, notele de la oral au fost incurajatoare (de la 8.5 in sus , desi doar 3 candidati).

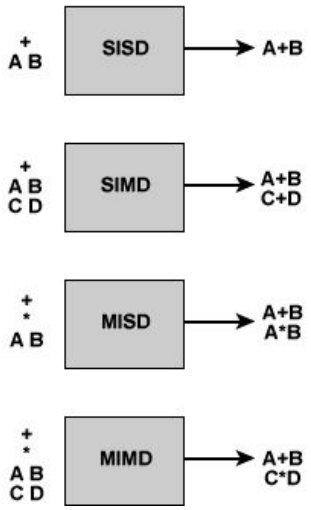
DIN DOC DE PE SERIE:

Propun sa postam aici rezolvari la subiectele pe care le gasim / le-am gasit.
// TODO: rezolvari

https://drive.google.com/file/d/0B_IrjBH5rVDYczZyWHcyY0JaS28/edit?usp=sharing
^ curs complet (sa dati cu ctrl-f prin el)
curs complet seria CC (diferit de cel de mai sus)

Teorie:
SIMD vs MIMD
9) Clasificarea Flynn a sistemelor de calcul
--Taxonomia lui Flynneu ma refer la cine da 'acordul' ca cineva sa
• Impartirea sistemelor de calcul in functie de:
– Fluxul de Instructiuni – secventa de instructiuni executate de procesor
– Fluxul de Date – secventa de operanzi manipulata de procesor
• Bazate pe acest criteriu se desprind:
• I – SISD = Single Instruction Single Data Stream (structura
Von Neumann)
• II – SIMD = Single Instruction Multiple Data Stream
• III – MISD = Multiple Instruction Single Data Stream
• IV – MIMD = Multiple Instruction Multiple Data Stream

Flynn.
Capabilitatile celor 4 clase de sisteme de calcul.



1) Implementarea salturilor conditionate in SIMD si MIMD - comparatie

La MIMD e ca si cum ai rula mai multe thread-uri. Ideea e ca fiecare core are propriul program counter, flag-uri, etc. (o parte de registre proprii) asa ca salturi se petrec ca si cum ar fi un procesor separat fiecare core.

La SIMD, ai o inst, de genu' cum erau spu_add(), numai ca atunci cand "faci if" se activeaza un flag daca conditia a fost sau nu adevarat pt fiecare data. De exemplu poate exista inst m = spu_greater_or_equal(v, 0). Ei in urma executarii in m o sa existe pentru fiecare element din v, rezultatul operatiei, adica pt fiecare element, in m o sa fie o valoare diferita de zero pentru true, si 0 pentru false. Acum poti sa folosesti alta instructiune SIMD care il primeste pe m si executa ceva pe un core sau nu, daca elem specific din m e diferit de 0.

Modelul abstractizat la curs, e ca se activeaza niste flag-uri intr-un registru, si fiecare core primeste bit-ul sau, in loc de bytes intregi de date

CatalinV

Bonus: CLSS A, i; CEQL A, i; CMP i; LDMSK i; STMSK i;

8) Comparatia generala intre SIMD si MIMD

SIMD e de obicei mai ieftin (+ less hardware)

MIMD e mai flexibil, se preteaza la o clasa mai larga de probleme

SIMD nu merge bine pe cod cu multe branchuri

(ziceti si de definitie)

11) Reprezentarea datelor pe SIMD si MIMD

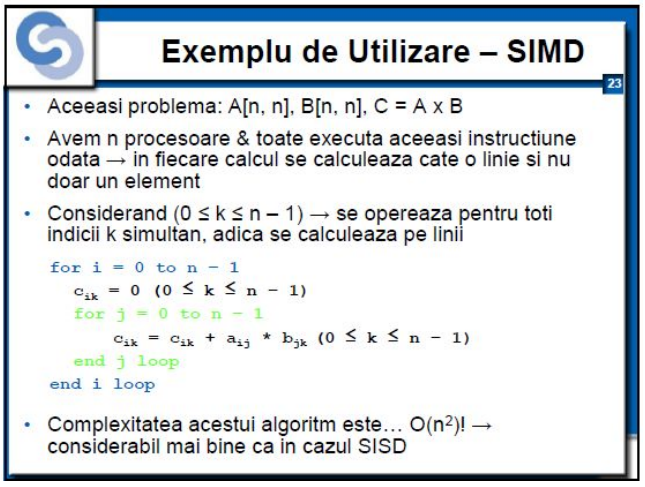
MIMD -> memorii separate , UMA, NUMA,<CC-NUMA>, COMA

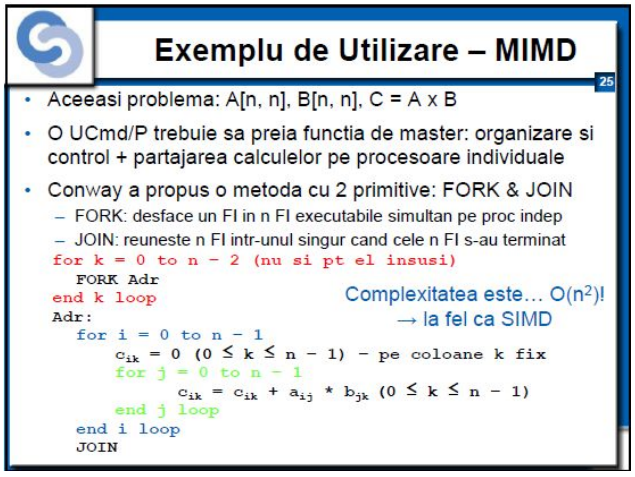
SIMD -> memorie comuna, procesare batch, < bus larg >

79) Comparatie SIMD si MIMD dpdv al structurii si organizarii datelor (nu garantez ca imi amintesc bine)
-- ca 11

28.Organizarea datelor in SIMD + Realizarea inmultirii unei matrici cu ea insasi
Organizarea datelor SIMD: cand consideram matricea ca fiind pe linii/coloane. Limbajele de programare care folosesc reprezentarea pe linii sunt C/C++, Java. Limbajele care folosesc reprezentarea pe coloane sunt: Fortran (TODO: de completat daca mai gasesc altele)
Realizarea inmultirii unei matrici cu ea insasi : pseudocod instr simd

40. Comparatie SIMD MIMD la algoritmi
aici merge ex din curs cu inmultirea matricilor.. nimic fancy





Sa se scrie program SIMD si MIMD

47. program impartire vectori SIMD + MIMD, comparatii intre ele

52. Comparatie SIMD vs SISD pe exemplu de cod.

Inmultirea a doua matrice: O(N^3) -> SISD vs O(n^2) -> SIMD

66) MIMD, Exemplu pe progr cu dependinte de date

Aflarea diametrului unui graf.

Calculul mediei aritmetice a elementelor unui vector.

78) Adunarea elementelor a doi vectori in SIMD si MIMD (cod + cum retii datele)

6) Sa se scrie programul SIMD si MIMD care aduna 2 vectori (pe elemente). Sa se analizeze performantele. Specificati organizarea datelor

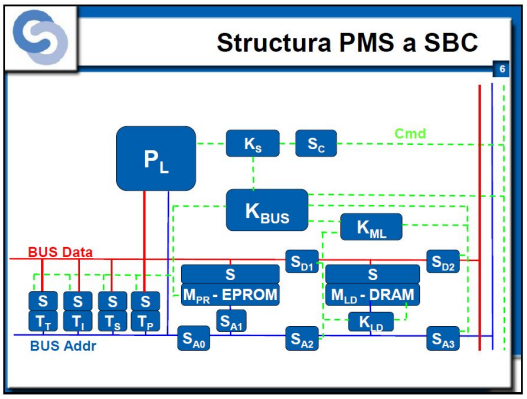
20)Adunare elemente dintr-o matrice cod SIMD MIMD plus organizarea datelor si concluzii

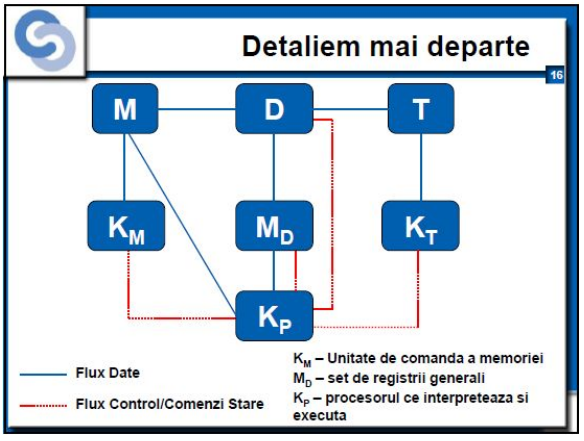
12) Sa se scrie programul ce determina produsul elementelor unei matrice pe SIMD si MIMD. Sa se analizeze performanta. Reprezentarea datelor.

PMS + primitive

2) Structura PMS

Ceva general despre primitive + (una dintre scheme)



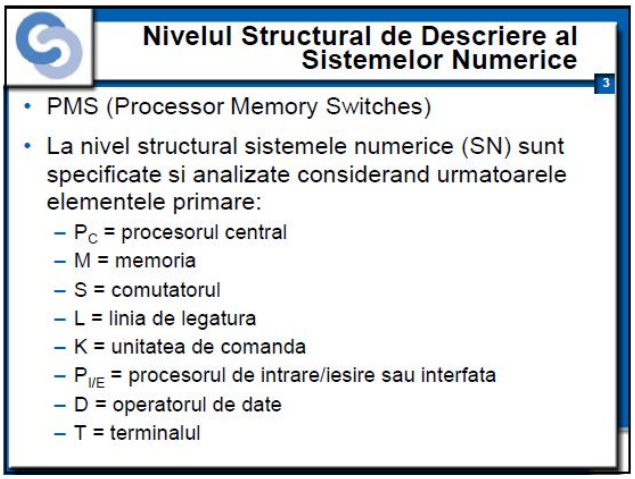


077) Descrieti in metoda PMS resursele de la laborator.

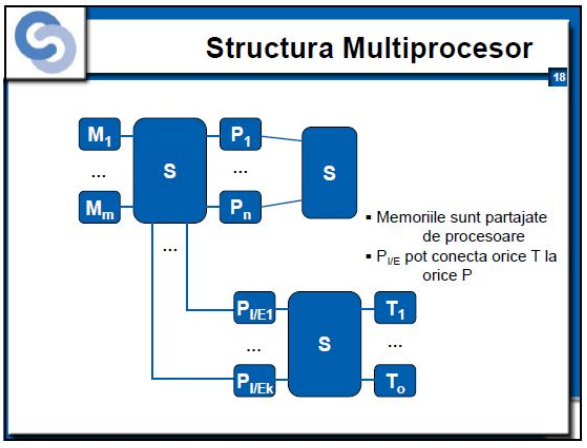
--- vezi 25 (dedesubt), ca singurele resurse din laborator sunt calculatoarele

25) Descrie calculatorul tau folosind primitive PMS

-- vedeti si atributele, sunt primele ~7 slide-uri din cursul de PMS



27. Nivelul structural de organizare a sistemelor numerice. Exemplu de modelare PMS a unui sistem multiprocesor.



15) Exemplificati primitivele P si M de pe calculatorul propriu

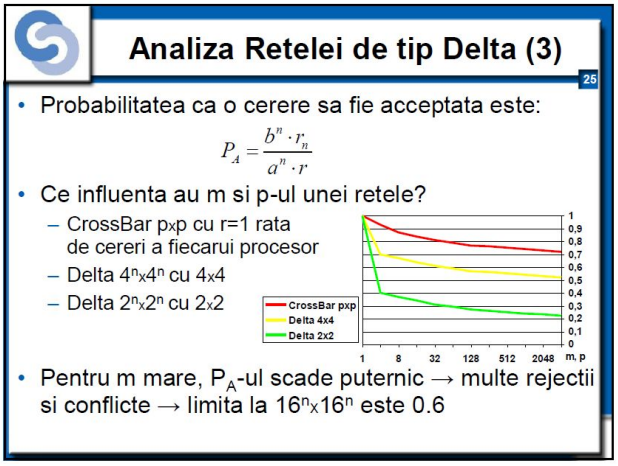
--- [presupun] P = CPU si M = Cache + RAM + HDD?

de retinut ca RAM-ul e memorie principala, though

Comutatoare (Switch) - permutari

13) Comparatie RC Cross Bar si RC Delta

--- CB are mai multe basic switch-uri decat Delta, dar scaleaza mai bine dpdv al request-urilor servite cu succes, la numar mare de procesoare (e un grafic in pdf-ul 12)



16) Comparatie intre intercalare perfecta si butterfly

21)Comparatie intercalare perfecta si permutarea de baza

34. Comparatie Base Line Permutation - Bit Reversal Permutation

bit reversal -> a3 a2 a1 a0 → a0 a1 a2 a3 (reverse de la ai la aj (caz pentru 3 si 0))

BLP -> a3 a2 a1 a0 → a0 a3 a2 a1 (muta ai la sfarsit (cazul simplu pentru i = 0))

43. Comparatie permutare baza, permutare perfecta

38. Subiect cu permutari (parca era descrierea permutarilor de tip perfect shuffle)

68) comparatie intre permutarea de baza si perfect shuffle

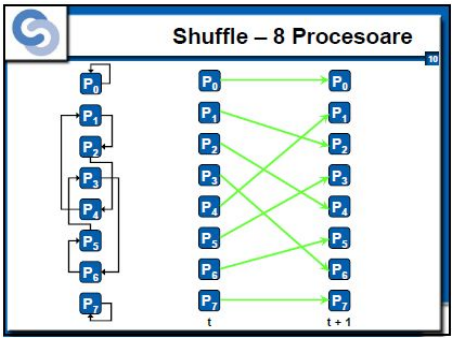
63. Permutarea Butterfly

--- a3 a2 a1 a0 -> a0 a2 a1 a3

se aleg a0 si an-1 si se interschimba

//diferit de bit reversal permutation :)

3) Retea de comutare de tip interclasare perfecta



1 – Base Line Permutation

22

- Bitul cel mai nesemnificativ devine cel mai semnificativ:
  P_base ($a_n\ a_{n-1} \dots a_1\ a_0$) = ($a_0\ a_n\ a_{n-1} \dots a_1$)
- Permutarea de baza este intercalarea perfecta inversa!
- Pentru 8 resurse avem P_base ($a_2\ a_1\ a_0$) = ($a_0\ a_2\ a_1$)
- Variante posibile sunt
  - Permutarea de baza k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_b_kinf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0\ a_{k-1} \dots a_1$)
    - Asigura conexiunea in submultimi de resurse
  - Permutarea de baza k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_b_ksup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_0$) = ($a_{n-k+1}\ a_n \dots a_{n-k+2}\ a_{n-k} \dots a_0$)
    - Asigura conexiunea intre submultimi de resurse
- TA: Variantele p_base pentru 8 resurse

2 – Shuffle Permutation

23

- Bitul cel mai semnificativ devine cel mai nesemnificativ:
  P_shuffle ($a_n\ a_{n-1} \dots a_1\ a_0$) = ($a_{n-1} \dots a_1\ a_0\ a_n$)
- Pentru 8 resurse avem P_shuffle ($a_2\ a_1\ a_0$) = ($a_1\ a_0\ a_2$)
- Variante posibile sunt
  - Permutarea shuffle k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_shuffle_kinf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k \dots a_1\ a_0\ a_{k-1}$)
    - Utilizare posibila: conexiunea memoriilor – adresa de baza a unui modul + adresa in cadrul acelui modul
  - Permutarea shuffle k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_shuffle_ksup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_0$) = ($a_{n-1} \dots a_{n-k+1}\ a_n\ a_{n-k} \dots a_0$)
  - Permutarea shuffle q ≠ c, intre q·c resurse (nu putere a lui 2):
    - P_shuffle_q#c (ADR) = q · V_{ADR} mod (q·c -1) pentru $0 \le V_{ADR} < q \cdot c-1$
    - P_shuffle_q#c (ADR) = V_{ADR} pentru $V_{ADR} = q \cdot c-1$
    - V_{ADR} = 0, 1, 2, etc, n-1 succesiv

3 – Negate Bit Permutation

24

- Bitul cel mai nesemnificativ este negat
  P_neg_bit ($a_n\ a_{n-1} \dots a_1\ a_0$) = ($a_n\ a_{n-1} \dots a_1\ \overline{a_0}$):
- P_neg_bit ($a_2\ a_1\ a_0$) = ($a_2\ a_1\ \overline{a_0}$)

- Varianta posibila este negarea bitului k:
  P_neg_bit_k ($a_n\ a_{n-1} \dots a_{k-1} \dots a_1\ a_0$) =
  ($a_n\ a_{n-1} \dots \overline{a_{k-1}} \dots a_1\ a_0$)
- P_neg_bit ($a_2\ a_1\ a_0$) = ($a_2\ \overline{a_1}\ a_0$)

4 – Butterfly Permutation

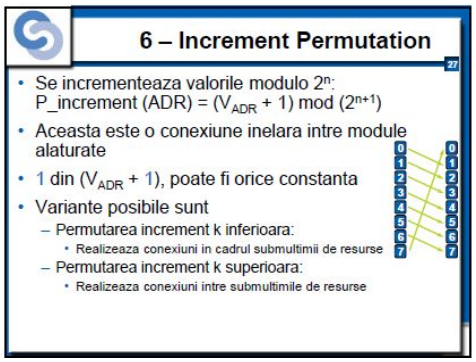
25

- Se interschimba bitul cel mai semnificativ cu cel mai nesemnificativ:
  P_butterfly ($a_n \dots a_1\ a_0$) = ($a_0\ a_{n-1} \dots a_1\ a_n$)
- P_butterfly ($a_2\ a_1\ a_0$) = ($a_0\ a_1\ a_2$)
- Butterfly este o permutare simetrica
- Variante posibile sunt
  - Permutarea butterfly k inferioara:
    - P_butterfly_kinf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0 \dots a_1\ a_{k-1}$)
    - Ofera posibilitatea de conectare intre resursele clasei
  - Permutarea butterfly k superioara:
    - P_butterfly_ksup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_1\ a_0$) = ($a_{n-k+1}\ a_{n-1} \dots a_n\ a_{n-k} \dots a_0$)
    - Ofera posibilitatea de conectare intre clase

5 – Bit Reversal Permutation

26

- Se interschimba complet ordinea bitilor:
  P_reversal ($a_n \dots a_1\ a_0$) = ($a_0\ a_1 \dots a_n$)
- P_reversal ($a_2\ a_1\ a_0$) = ($a_0\ a_1\ a_2$)
- Cu aceasta permutare se pot obtine echivalente
- Variante posibile sunt
  - Permutarea reversal k inferioara:
    - P_reversal_kinf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0\ a_1 \dots a_{k-1}$)
  - Permutarea reversal k superioara:
    - P_reversal_ksup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_1\ a_0$) = ($a_{n-k+1} \dots a_n\ a_{n-k} \dots a_0$)



72) Comutatoare in cuplarea memorie procesor

simplex, duplex, dual duplex, cross-bar-switch, trunchi K

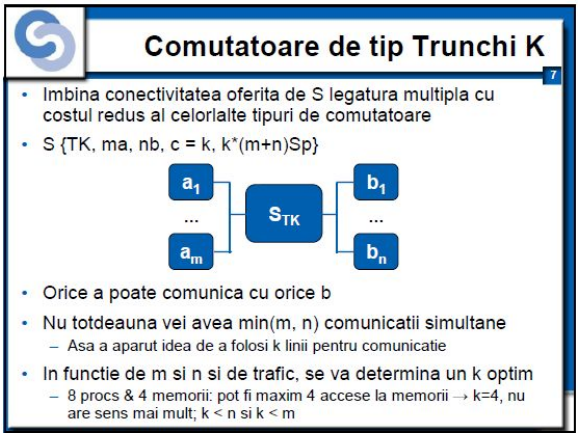
4) Comutator de trunchi K ierarhic si neierarhic

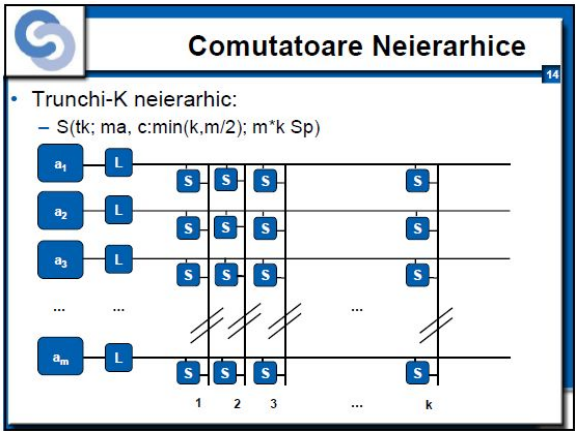
Comutator de tip k -> MAXIM k conexiuni, e mai simplu de implementat dar e posibil sa nu se paralelizeze pe toata lumea

ierarhic -> conexiuni diferite (ex: procesor cu memorie)

neierarhic -> conecteaza acelasi chestii (PP, MM, etc)

Bonus: faceti desenele :)





41. Probabilitatea ca o cerere sa fie acceptata la un comutator CB (formula)

--- fuck :(((. much fuck. so formula

read 214 - 216 din PDFul merged :)

read ca read, dar ce write?

7. Sa se proiecteze o Retea de Comutare de tip Delta intre P procesoare si M module de memorie.

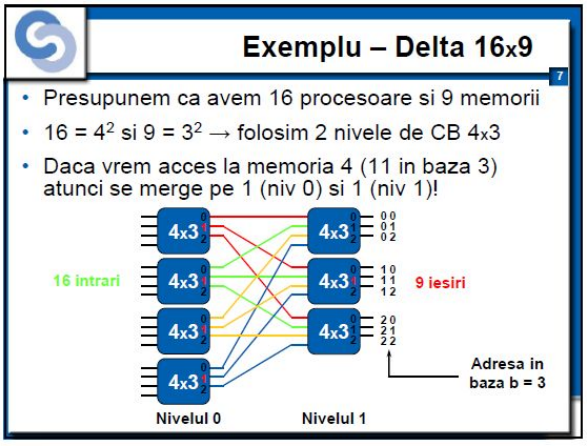
a) 4P x 9M

- tot 2 nivele: nivel 0 - 2, nivel 1 - 3

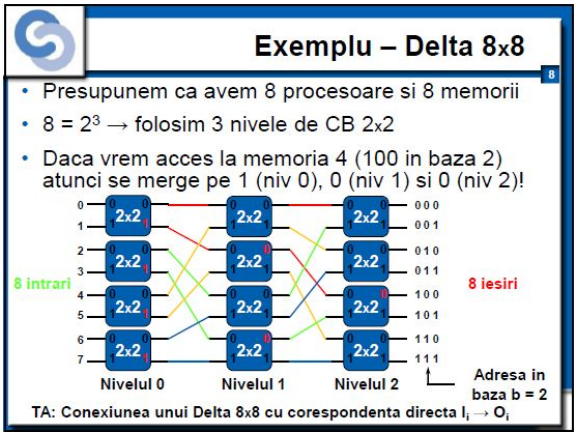
b) 16P x 9M -> exemplu dat

c) 16P x 16M, default i , memory i

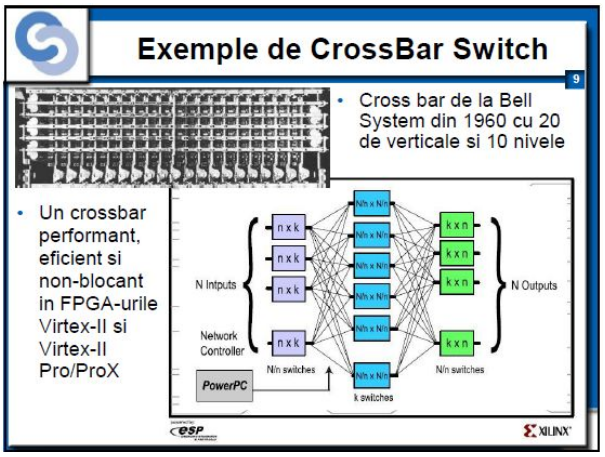
- 4 nivele de cate 2? sau
- 2 nivele de cate 4



d) 8P x 27M



51. Comutator legatura multipla (CrossBar) ierarhic



Clusters

5) Comunicatia intercluster in MIMD slab cuplate

--- Mesaje (MPI) / comutare circuite

74) Comunicatia intercluster

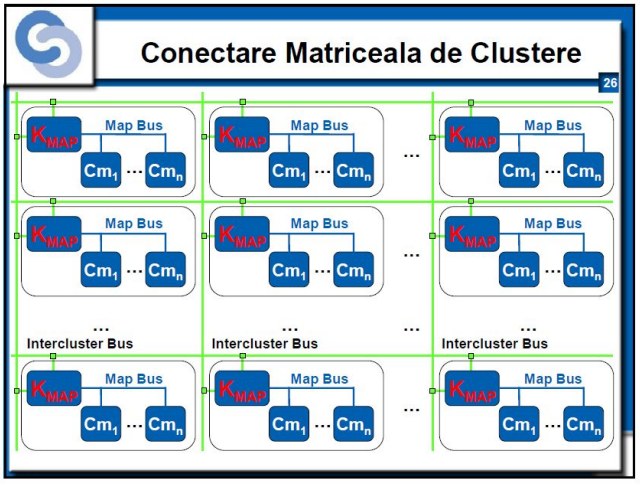
intercluster bus si prin KMAP

44. Comunicatia intercluster

42. Structura Kmap

23)Diagrama Kmap

17) Kmap - descriere generala

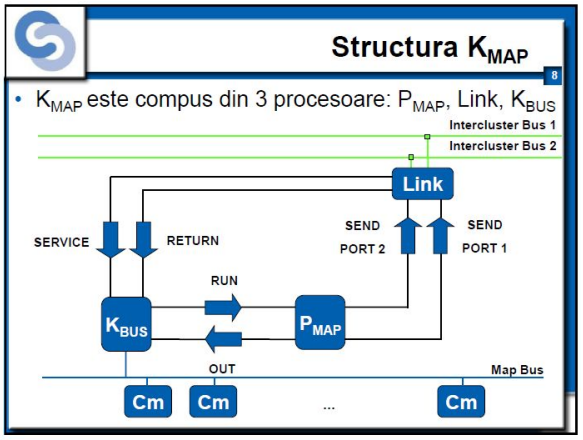


Kmap gestioneaza magistrala Map Bus si *magistralele verzi*

-- accesul pe magistrala verde e exclusiv, nu pot sa fie doua mesaje in acelasi timp

(?) citez din curs: “Sursa de bottleneck intr-un cluster este Map Bus ↔ posibila doar o singura tranzactie la un moment dat” + “KMAP asigura: Controlul in cadrul unui cluster – prin excluderea mutuala a accesului la MapBus (arbitrare intre Cm-urile din acelasi cluster)”

Inteleg ca accesul exclusiv este pe Map Bus, si nu pe Intercluster Bus, altfel ar fi chiar a pain in the ass xD



KBUS: unitate de comanda a Map Bus si va controla toate tranzactiile de pe aceasta magistrala

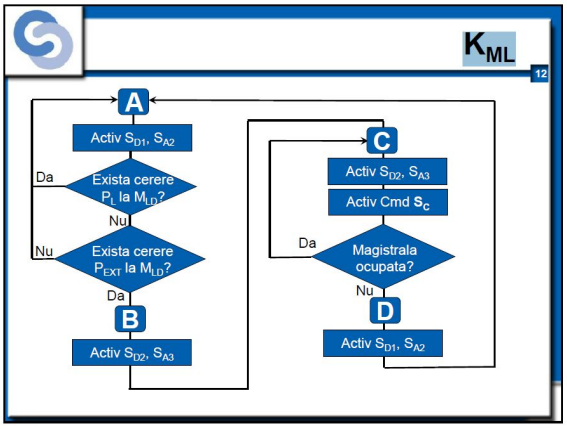
•PMAP: procesor de mapare care face toate translatarile de adresa (maxim 8 procese simultan)

•Link: este cel care gestioneaza si supravegheaza transmiterea mesajelor intercluster (send/recv)

Cm - nod intern de cluster

75) Schema logica Kml

10) Organigrama KML de la SBC-uri.



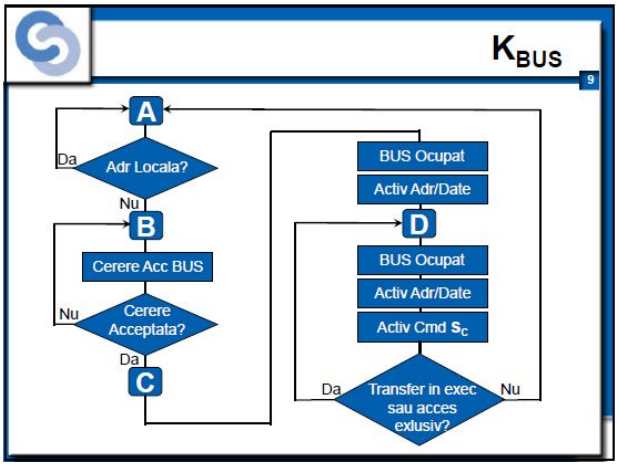
46. Organigrama Kbus

hop, sari peste astea

59. Descrierea schemei logice Kbus

29. Logica si arhitectura Kbus(SBC)

22)Diagrama KBUS



Cache

30. Care sunt cele 4 elemente de care trebuie tinut cont cand se realizeaza designul unui cache.

block placement, block identification, block replacement, write strategy

71) optimizari programe (trebuia vb de cache)

Row major vs Column major

36. Cum se masoara dimensiunea liniei unui cache? Exemplu de program si grafic.

--- Se face cu un vector la care crestem pasul pana cand performanta nu se mai degradeaza => --- => pasul == lungimea liniei de cache

61. Determinarea dimensiunii cache-ului

exploatam scaderea performantei la cache miss-uri

pentru linii : iteram prin vector marind stride-ul mereu, la un moment dat performanta scade f mult pentru ca ‘sarim’ din dimensiunea cache line si fortacem un cache miss la fiecare apel

pentru tot cache-ul : iteram de mai multe ori cu strideul de CACHE_LINE_SIZE, incrementand dimensiunea vectorului ce il tinem in cache. La un moment dat incepem sa facem trashing si stim ca am depasit dimensiunea cacheului

48. Block placement in cache.

direct, set asociativ (2, 4, 8, etc), complet asociativ

60. Block replacement

Random - cel mai simplu, surprinzator de eficient

LRU - greu de implementat

FIFO - usor de implementat

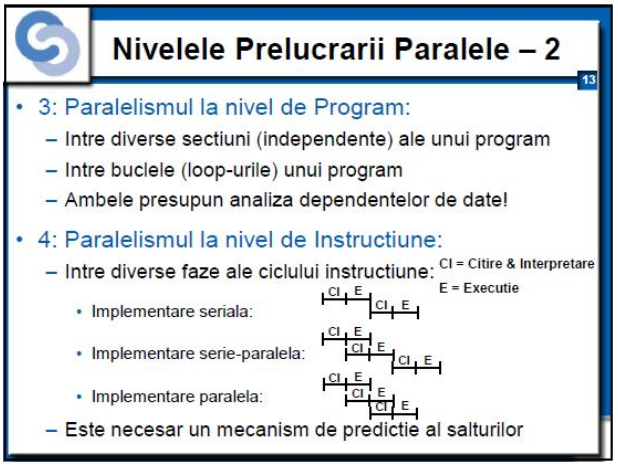
Others

31. De ce nu se poate realiza un procesor de 10TFlops(minim 3 motive) ?
--- a) viteza memoriei e cu 10^20 mai mica decat ar trebui
--- b) pentru a permite dateleor sa ajunga avem nevoie de fiecare cuvant de memorie sa ocupe 3.7 Angstrom^3 (~ dim unei molecule mici)
--- c) no quantum yet

50. Arhitectura procesorului PMAP.

54. Mituri despre Cell BE.
--- nu stie double precision => fals, dar e semnificativ mai lent decat SP
--- se comporta ca un FPU

56. Tipuri de paralelizare la sistemele cu prelucrare paralela.



57. Structuri SN. P i/e structura
100. Nivelul structural de organizare a sist numerice

- Sisteme Matriceale (Processor Array):
    - Unitatea de baza a informatiei este vectorul
    - Dispun de instructiuni similare SC Von Neumann – operatiile asupra vectorilor sunt efectuate in aceeasi instructiune
- Sisteme Multiprocesor (Multiprocessor Systems):
    - Formate din N unitati de prelucrare interconectate printr-o retea de comutare (Strans/Slab cuplata)
    - Sistemele lucreaza independent la realizarea aceluiasi Job
- Sisteme Pipeline:
    - Dispun de mai multe unitati de prelucrare asezate in banda de asamblare (RISC):
    - Fiecare UC executa o prelucrare specifica si transfera rezultatul subansamblului adiacent

62. Comunicatia Kbus â Pmap

67) Cell BE bariere si solutii.
Memory Wall,
-more slower threads
-asynchronous loads
Efficiency Wall ,
-more slower threads
-specialized functions

Power Wall
-reduce transistor power
-reduce switching function

69) Memorii distribuite
70) comunicare in cadrul unui cluster

Top 500
18) Sa se aleaga un calculator din TOP 500 si sa vorbim despre el.
-- http://www.top500.org/system/177975
33. Caracteristicile arhitecturale pentru arhitectura cea mai des intalnita in top 500
-- cluster
65. Cel mai utilizat SO din top500

--- Linux

37. Arhitectura unui calculator din top 500 (arhitectura, numar procesoare, so, ...)

-- https://www.olcf.ornl.gov/wp-content/themes/olcf/titan/images/titan-infographic.png + Cray Linux

53. Cel mai popular mod de conectare in top500. Cel mai popular SO. Cea mai populara arhitectura.

-- InfiniBand, Linux, Cluster

Benchmark

64. Benchmark-uri pentru sisteme seriale

SPEC, TPC, BAPCo, SANDRA

--- cursul de benchmarking, slide-urile 7 si 8

32. Caracteristici HPC Challenge Benchmark.

--- compara sistemele in mai mult de o categorie

55. Benchmark-uri, probleme si rezolvari legate de acestea (ceva de genu). (nu trebuia sa zici de HPCC si LINPACK)

73) Pe ce nivele se face benchmark la HPC. De ce e bine ca sunt mai multe metrici?

el

Probleme:

Problema clasica anul trecut:

Ai N bile punctiforme pentru care ti se dau :

x

y coordinate initiale (coordonateme extremitatii initiala a

segmentului pe care se deplaseaza bila)

v viteza de deplasare

x_ext

y_ext coordonateme extremitatii finale a segmentului pe care se

deplaseaza bila

Daca doua bile se ciocnesc atunci dispar din sistem.

Se cere sa se afiseze coordonatele la care o bila dispare din system

(daca s-a ciocnit cu a olta bila) sau coordonatele extremitatii finale

pentru bilele care au ajuns in extremitati in perioada T de timp

(data).

a) utilizand instr SIMD calculate distanta parcursa de fiecare bila

(se dadea formula distantei dintre 2 pct)

b) sa se modeleze sistemul cu threaduri (pthreads/python threads)

c) sa se modeleze sistemul in Cell

din ce inteleg, trebuie sa se verifice daca 2 cate 2 segmentele se intersecteaza
daca 2 segmente se intersecteaza trebuie sa determinam la ce distanta
apoi sa vedem daca distanta respectiva este suficient de aproape sa fie ajunsa la timp de ambele bile (portiunea de segment e parcursa cu viteza v in timp <= T)
in caz, afirmativ, retinem coordonatele coliziunii

s0. Cuprins

Organizarea datelor în structurile SIMD și MIMD

1. Să se scrie programul de împărțire a doi vectori de dimensiune n pentru o structură SIMD și MIMD. Comentați performanțele.
2. Să se cupleze 16 procesoare cu 16 memorii utilizând rețea de tip Delta; implicit procesorul i să fie conectat la memoria i. Indicați conexiunea de la procesorul 2 la memoria 6.
3. Descrieți primitiva P de la calculatorul personal.
4. Considerând o structură multiprocesor pe o magistrală comună descrieți unitatea de acces la magistrala sistemului.-
5. Structura generală Kmap.
6. Comparație între crossbar și delta privind performanțele.
7. Comparatie intre intercalarea perfecta si reteaua butterfly. (Vezi ex 29 si 32)
8. Sa se scrie programul de insumare a elementelor unei matrice A(n, n) pentru o structura SIMD si pentru o structura MIMD. Comentati performantele. Obs. Trebuie specificata organizarea datelor.
9. Clasificarea Flynn a sistemelor de calcul.
10. Descrieti caracteristicile generale CELL
11. Retea de comutare de tip intercalare perfecta si comparatie cu reteaua cu ordine inversa (permutarea de baza). Vezi ex 32.
12. Trunchi K ierarhic si neierarhic
13. Adunarea elementelor a doi vectori in SIMD si MIMD (cod + cum retii datele)
14. Subiect cu permutari (parca era descrierea permutarilor de tip perfect shuffle). Vezi ex 32.
15. Organizarea datelor la SIMD (Vezi ex 1)
16. Comparatie SIMD MIMD la algoritmi (Vezi ex 2 :-?)
17. Probabilitatea ca o cerere sa fie acceptata la un comutator CB (formula)
18. Comparatie permutare baza, permutare perfecta (Vezi ex 32)
19. Block placement in cache.
20. Arhitectura procesorului PMAP.
21. Comutator legatura multipla (CrossBar) ierarhic.
22. Benchmark-uri, probleme si rezolvari legate de acestea (ceva de genu).
23. Tipuri de paralelizare la sistemele cu prelucrare paralela.
24. Structuri SN. P i/e structura (ce plm SN?)
25. Block replacement
26. Determinarea dimensiunii cache-ului
27. Comunicatia Kbus Pmap
28. Permutarea Butterfly
29. Benchmark-uri pentru sisteme seriale
30. Cell BE bariere si solutii.
31. comparatie intre permutarea de baza si perfect shuffle
32. Memorii distribuite

33. optimizari programe (trebuia vb de cache)
34. Comutatoare in cuplarea memorie procesor.
35. Retea de comutare de tip interclasare perfecta. **(Vezi ex 32)**
36. Sa se scrie programul SIMD si MIMD care aduna 2 vectori (pe elemente). Sa se analizeze performantele. Specificati organizarea datelor **(Vezi ex 14)**
37. Comparatia generala intre SIMD si MIMD
38. Organigrama KML de la SBC-uri + schema logica
39. Sa se scrie programul ce determina produsul elementelor unei matrice pe SIMD si MIMD. Sa se analizeze performanta. Reprezentarea datelor. **(Vezi ex 9 doar ca faci * in loc de +)**
40. Comunicatia intercluster intr-un sistem MIMD slab cuplat
41. Adunare elemente dintr-o matrice cod SIMD MIMD plus organizarea datelor si concluzii **(Vezi ex 9)**
42. Comparatie intercalare perfecta si permutarea de baza. **(Vezi ex 32)**
43. Care sunt cele 4 elemente de care trebuie tinut cont cand se realizeaza designul unui cache.
44. De ce nu se poate realiza un procesor de 10TFlops(minim 3 motive) ?
45. Caracteristici HPC Challenge Benchmark.
46. Comparatie Base Line Permutation - Bit Reversal Permutation
47. Cum se masoara dimensiunea liniei unui cache? Exemplu de program si grafic.
48. Arhitectura unui calculator din top 500 (arhitectura, numar procesoare, so, ...)
49. Implementare de salturi conditionala in MIMD siSIMD. Comparati-le
50. Să se cupleze 16 procesoare cu 9 memorii utilizând rețea de tip Delta. Indicați conexiunea de la procesorul 8 la memoria 3.
51. Comunicatia intercluster

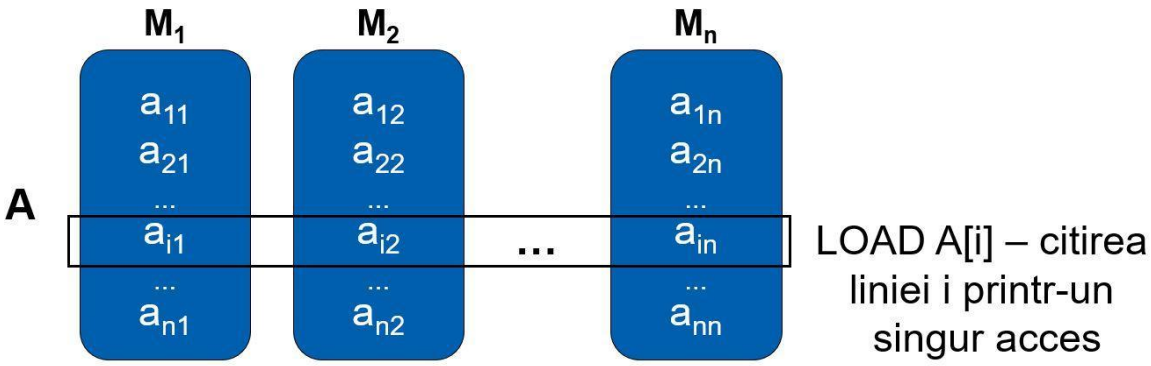
Rezolvari

1. Organizarea datelor în structurile SIMD și MIMD

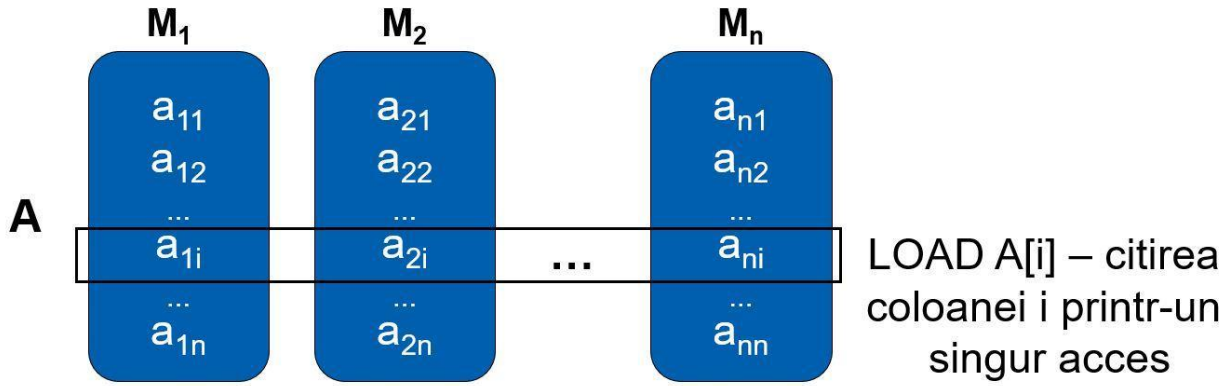
SIMD:

- Probleme:
  - Repartizarea in memoriile M_i & accesul paralel
  - Citirea vectorilor intr-o singura instructiune
  - La matrice: acces la linii si la coloane → necesara transpunerea/deplasarea datelor?
- Memoria:
  - Poate face cel mult un acces la un ciclu de lucru
  - n operanzi in n module diferite pot fi cititi simultan
  - Defavorabila plasarea in acelasi modul al operanzilor → citirea va fi secventiala in n cicli de acces!

- Matrice A[n,n] & structura SIMD (n procs/memorii)
- Cu acces simultan la liniile matricei:
  - Acces paralel la linii
  - Acces secvential la coloane



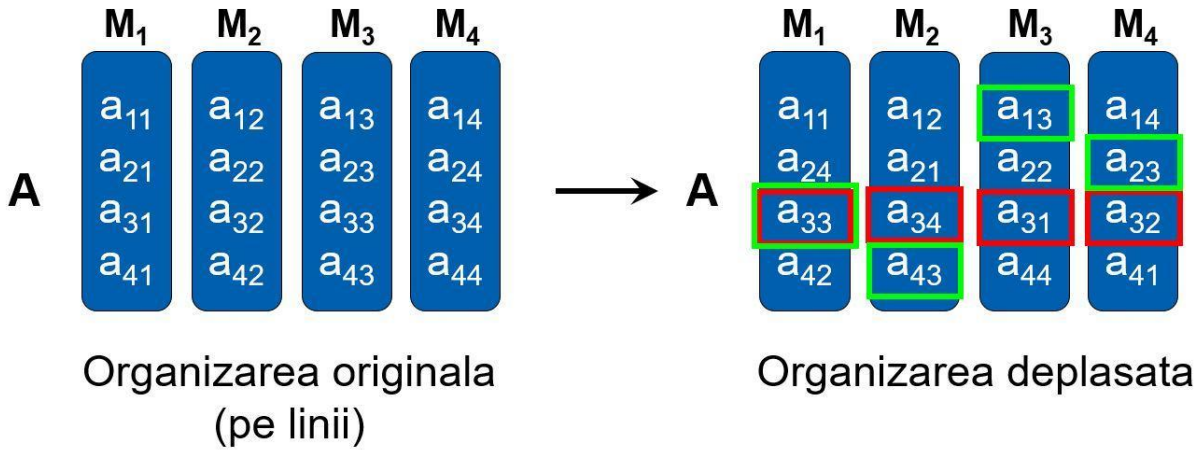
- Matrice A[n,n] & structura SIMD (n procs/memorii)
- Cu acces simultan la coloanele matricei:
  - Acces secvential la linii
  - Acces paralel la coloane



Reprezentarile directe pe linii/coloane sunt incompatibile la acelasi moment de timp

Inmultirea unei matrici cu ea insasi necesita acces paralel la linii si coloane pentru performanta maxima:

- Matrice A[4,4] & structura SIMD (4 procs/memorii)
- Cu acces paralel la liniile & coloanele matricei:
  - Exemplu valid pt inmultirea matricelor (linia & coloana 3)



Organizarea originala (pe linii)

Organizarea deplasata

Necesar un shuffling cu reteaua de comutatie (a_{i1} nu e pe P_1)

MIMD:

Organizarea memoriei:

- Fiecare procesor trebuie sa aiba acces la toata memoria sistemului
- Capacitatea de adresare a fiecarui procesor e mai mica ca dimensiunea memoriei → mecanisme speciale
- Asigura protectia memoriei pentru asigurarea integritatii datelor din sistem

Diferența la MIMD e că lucrează cu matrici, nu cu vectori. Reprezentarea matricilor (care sunt 2D) în memorie (care este 1D) - row-major, column-major, în funcție de limbaj.

UMA

Reprezentarea datelor pe SIMD si MIMD:

MIMD -> memorii separate , UMA, NUMA,<CC-NUMA>, COMA

SIMD -> memorie comuna, procesare batch, < bus larg >

2. Să se scrie programul de împărțire a doi vectori de dimensiune n pentru o structură SIMD și MIMD. Comentați performanțele.

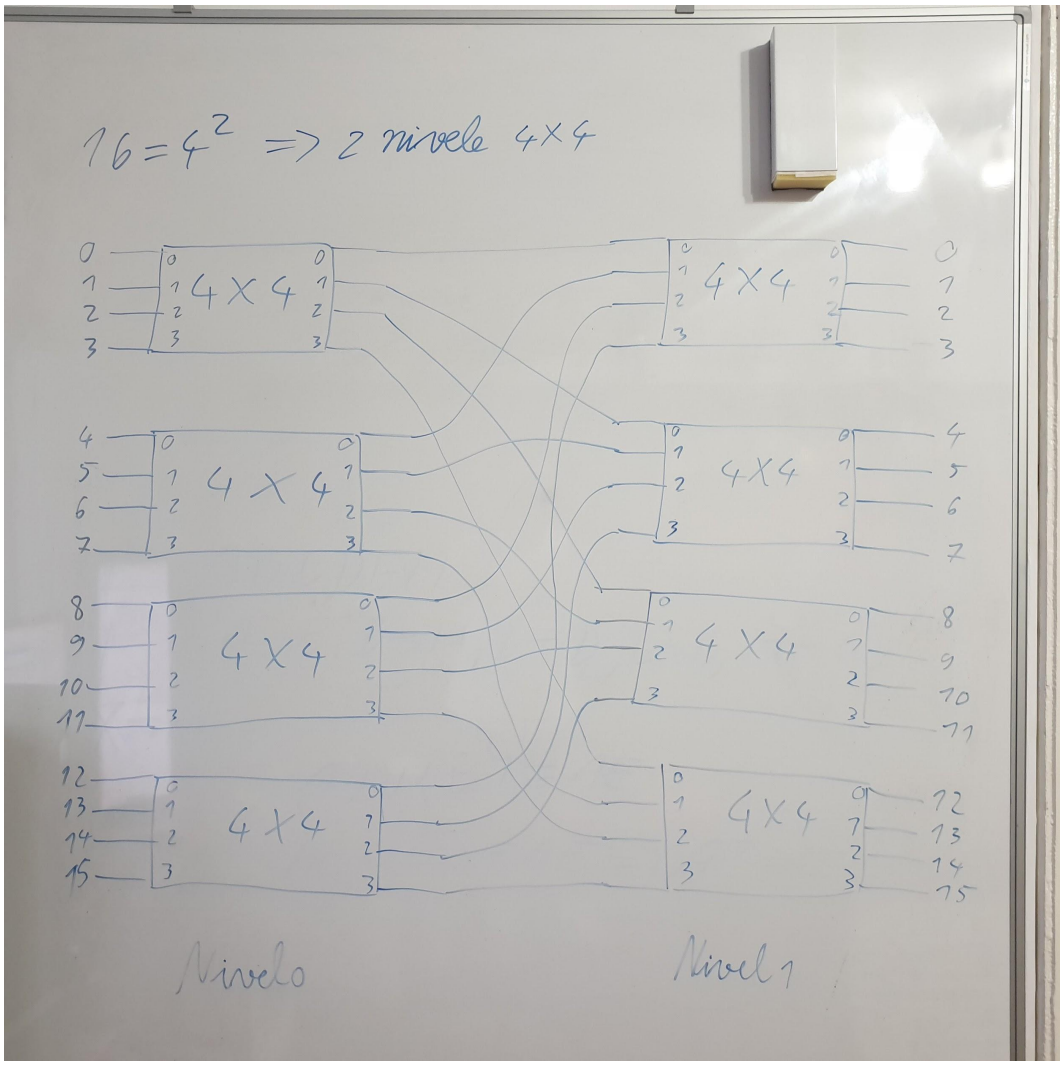
n procesoare sa fie mai laba

SIMD: se lanseaza in mod paralel by default din arhitectura

k -> index propriu procesorului
ck = ak / bk ($0 \leq k \leq n - 1$)
O(1)

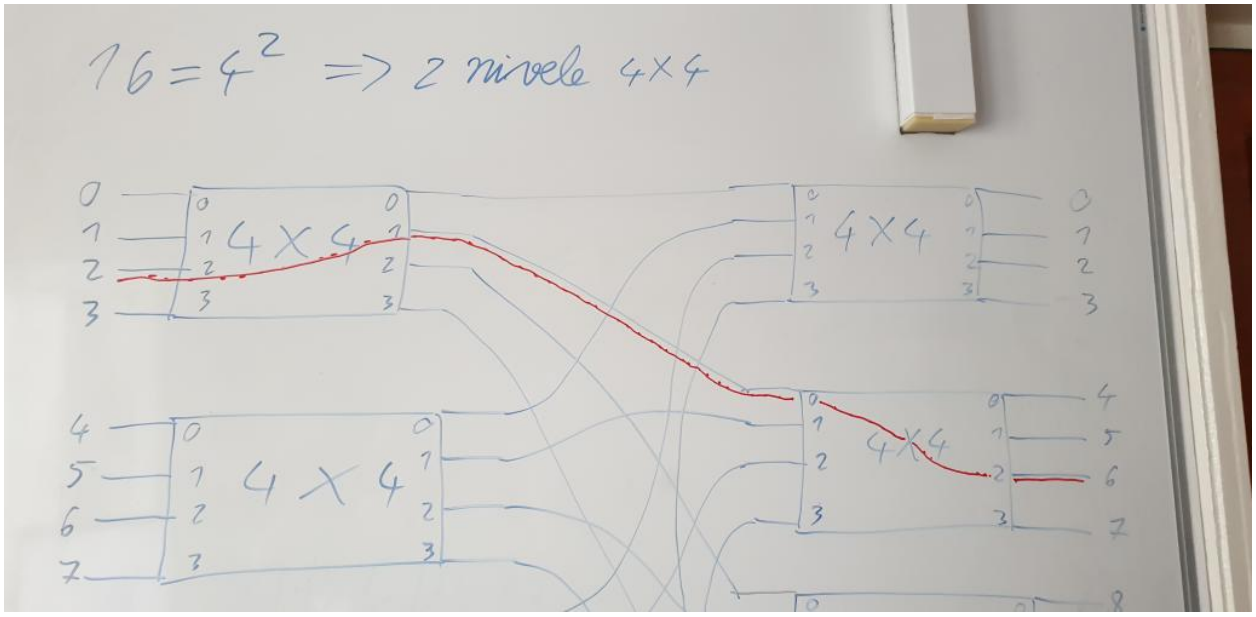
MIMD: trebuie sa lansam noi fiecare proces + sa fakem joint <3
for k = 0 to n – 2 (nu si pt el insusi)
    FORK Adr
end k loop
Adr:
    ck = ak/bk ($0 \leq k \leq n - 1$)
    JOIN
O(n) pentru ca trebuie sa stai sa lansezi n procese.

3. Să se cupleze 16 procesoare cu 16 memorii utilizând rețea de tip Delta; implicit procesorul i să fie conectat la memoria i. Indicați conexiunea de la procesorul 2 la memoria 6.



Adresa ar trebui sa fie in baza b, aka in dreapta sa avem 00, 01, 02, 03, 10...33

Conexiunea de la 2 la 6



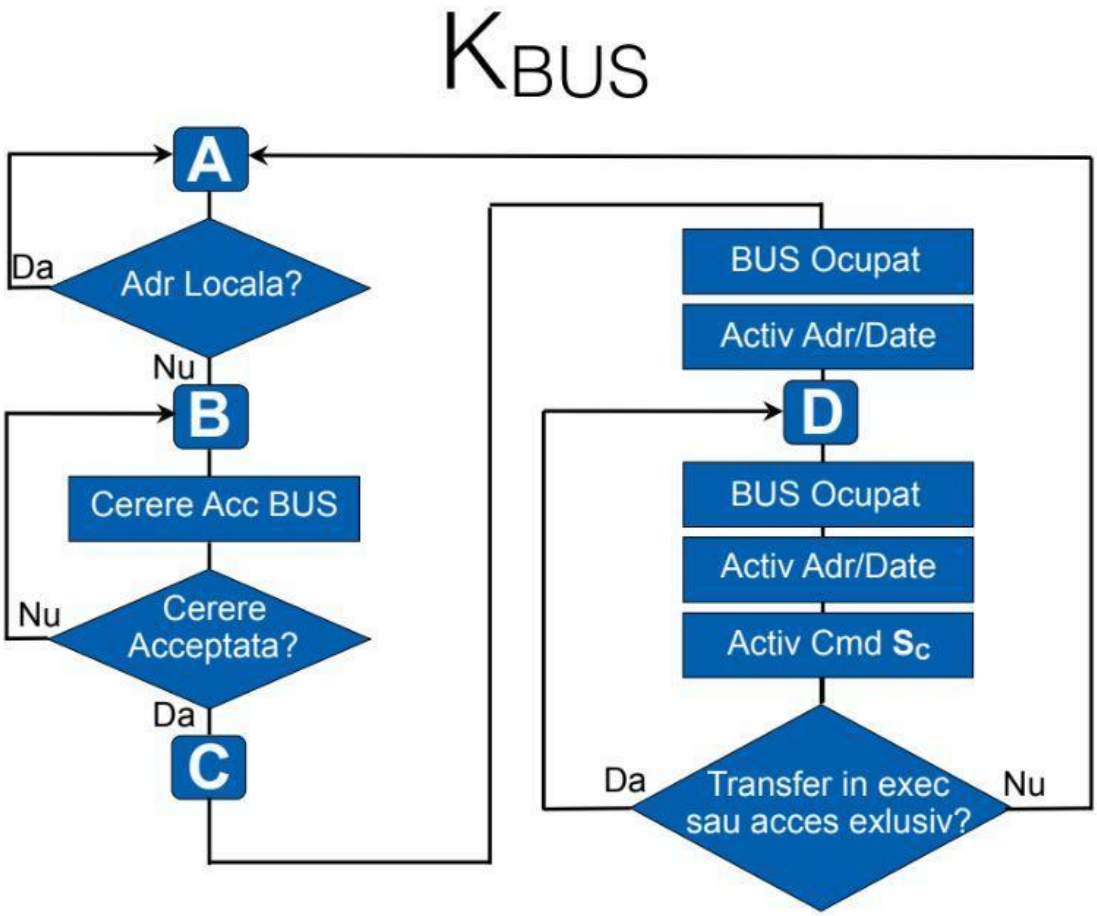
4. Descrieți primitiva P de la calculatorul personal

Aka CPU, subiect bonus.
Ex:
Citeste, interpreteaza si executa instructiuni
In acest proces are loc generarea comenzilor spre toate resursele sistemului + citirea starilor acestora
Atribute:
Functie: universal
Implementare: microprogramata
Formatul instructiunilor: variabil (de obicei sunt arhitecturi CISC)
Lungime instructiuni: 64 biti (eu cred că e 1 octet > mulți (câți?) octeți) (cred ca instructiunile sunt pe 832 biti sau ceva) [la 8086 nu erau 1 octet 6 octeti?]
[nu stiu daca este vorba despre asta http://en.wikipedia.org/wiki/Advanced_Vector_Extensions, atunci ar fi 256 biti de la sandy bridge] conform http://www.intel.com/content/dam/www/public/us/en/documents/manuals/64ia32architecturessoftwaredeveloperinstructionsetreferencemanual325383.pdf, sunt lungimi variabile pt instrucțiuni și în Intel i7. Eu cred că un răspuns gen "**lungimea variabilă, de la instrucțiuni scurte pe 1 octet, până la instrucțiuni complexe pe mai mulți octeți**" ar merge.
Ciclu instructiune: ciclu varibil(S cicli masina)
Tehnologie: F

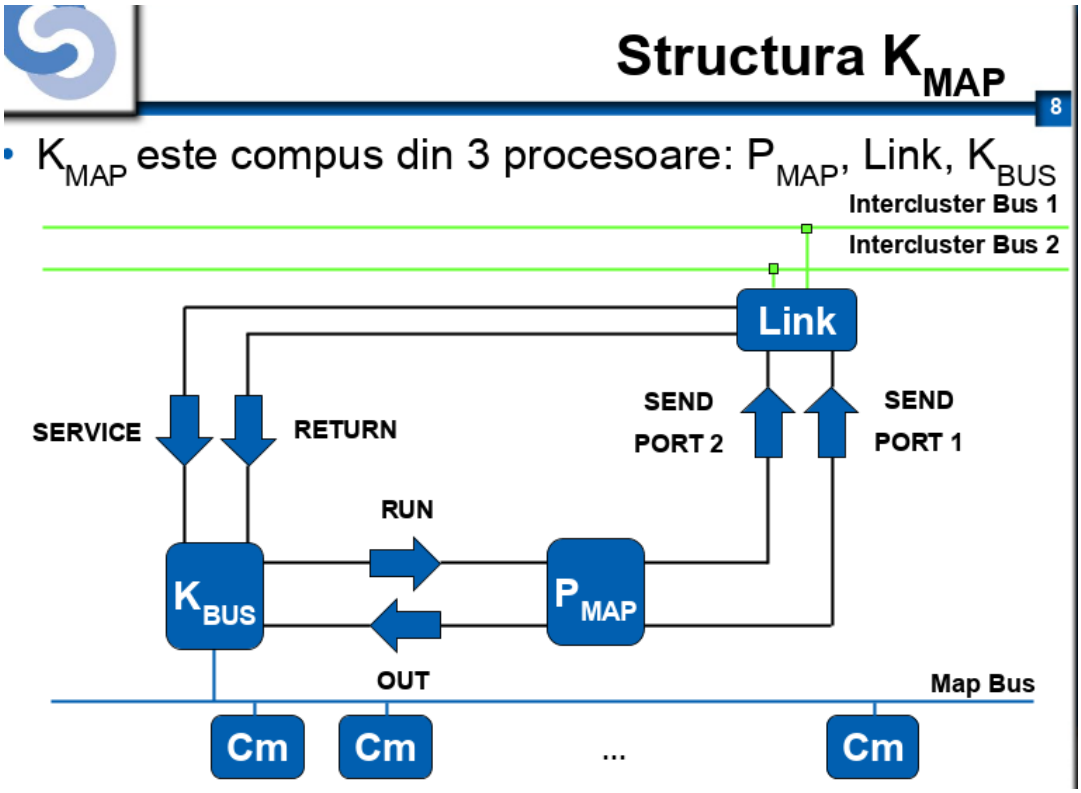
5. Considerând o structură multiprocesor pe o magistrală comună descrieți unitatea de acces la magistrala sistemului.

Descriere K_{BUS}

- K_{BUS} – reprezinta procesorul local in relatie cu magistrala externa
- Starea A: adresa e locala?
  - Da = se adreseaza resurse locale, nu trebuie iesit pe magistrala
  - Nu = adr nu e locala → se trece in starea B
- Starea B: cerere de acces la magistrala; daca cererea e acceptata se trece in starea C
- Starea C: se emite Bus Busy (mag ocupata) – doar in C & D; accesul pe BUS e exclusiv & cu rafala (overwrite) raman in D pt un nou ciclu daca mai e cazul; automatul preia functia procesorului; Dupa preluarea comenzii, procesorul face functia de secventiere a Adr, Date, Cmd
- Intrebare: de ce si starea C si starea D? Din cauza ciclului instructiune curent. Trebuie sincronizate comenzile cu decodificarea adreselor & stabilirea datelor pe magistrala (e un Δ_t acolo = T_{Wait} datorat interpretarii adreselor) → se introduce starea D pentru a compensa acest timp de latenta: starea de overwrite
- Obs: K_{BUS} trebuie sa dispuna de unitati logice combinationale ce identifica:
  - Daca adresa mem/port I/O implicate in transfer sunt locale
  - Semnalul de cerere acceptata e combinat din mai multe conditii: mag libera & modul SBC activ



6. Structura generală Kmap.



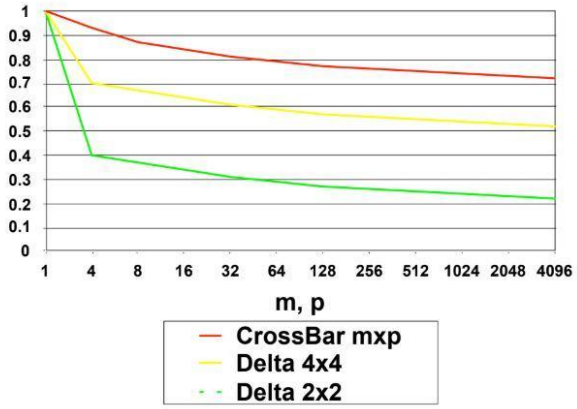
7.Comparație între crossbar și delta privind performanțele.

Analiza Comparativa Delta / CB

- Probabilitatea ca o cerere sa fie acceptata este:

$$P_A = \frac{b^n \cdot r_n}{a^n \cdot r}$$

- Ce influenta au m si p-ul unei retele?
  - CrossBar **m**x**p** cu r=1 rata de cereri a fiecarui procesor
  - Delta 4^nx4^n cu 4x4
  - Delta 2^nx2^n cu 2x2
- Pentru m mare, P_A-ul scade puternic
  - Multe rejectii si conflicte
  - Limita la 16^nx16^n este 0.6



8. Comparatie intre intercalarea perfecta si reteaua butterfly. (Vezi ex 29 si 32)

9. Sa se scrie programul de insumare a elementelor unei matrice A(n, n) pentru o structura SIMD si pentru o structura MIMD. Comentati performantele. Obs. Trebuie specificata organizarea datelor

Pp n procesoare.
SIMD:
sumk = 0 (asta e local sau nu ?)
for i = 0 to n – 1
	sumk += aik (suma coloana k)
end i loop

local_sum = 0 (local procesorului, in registru poate?)
for i = 0 to n – 1
	local_sum += sum[(i + k) mod n] (ca sa nu acceseze toate simultan acelasi modul de memorie)
end i loop
O(n)

MIMD:
for k = 0 to n – 2 (nu si pt el insusi)
	FORK Adr
end k loop
Adr:
	sumk = 0 ($0 \leq k \leq n - 1$)
	for i = 0 to n – 1
		sumk += aki (suma pe linii pentru acces contiguu in memorie)
	end i loop
	JOIN

for k = 0 to n – 1
	sum_final += sumk

O(n)

Organizare datelor: vezi ex 1

9bis. Sa se calculeze suma a doua matrici in SIMD si MIMD. Comentati performantele. Specificati org. datelor:

SIMD:

```
for i = 0 to n – 1
        cik = aik + bik (0 ≤ k ≤ n – 1)
end i loop
```

MIMD:

```
for k = 0 to n – 2 (nu si pt el insusi)
        FORK Adr
end k loop
Adr:
        for i = 0 to n – 1
                cik = aik + bik (0 ≤ k ≤ n – 1) – pe coloane k fix
        end i loop
        JOIN
```

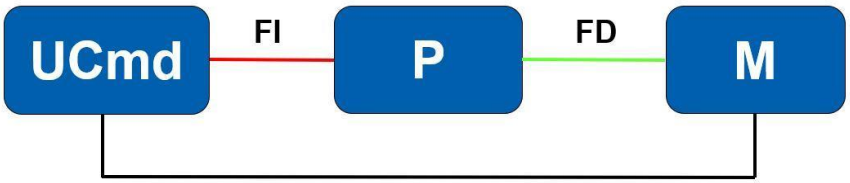
10. Clasificarea Flynn a sistemelor de calcul.

SISD - un singur flux de date si un singur flux de instructiuni. Nu exista paralelism
SIMD - mai multe fluxuri de date, dar un singur flux de instructiuni. Exista paralelism. S.n. si sisteme de procesare vectoriala.
MISD - paralelism reduntant. E un tip de calculator atipic. Se foloseste in sisteme de calcul unde e nevoie de precizie maxima (pe wiki scrie ca de exemplu la avioane)
Mai scria in curs si ca sunt folosite cand vrei sa faci chestii diferite asupra la aceleasi date si dadea exemplu de chestii meteorologice.
MIMD - clustere, MPI (scria ceva prin curs de cuplat Slab/Puternic respectiv MPI/OpenMP),
Ceva gen MPI da pe retea/socketi si OpenMP pe shared memory -- Slab ca procesoarele
Chiar sunt super separate vs. sunt doar cores diferite care ruleaza cod diferit)

I – SISD

- **UCmd** = Unitate de comanda
- **P** = Unitate de prelucrare aritmetica
- **M** = Memorie
- FI = Flux Instructiuni
- FD = Flux Date

SISD = 1 FI & 1 FD

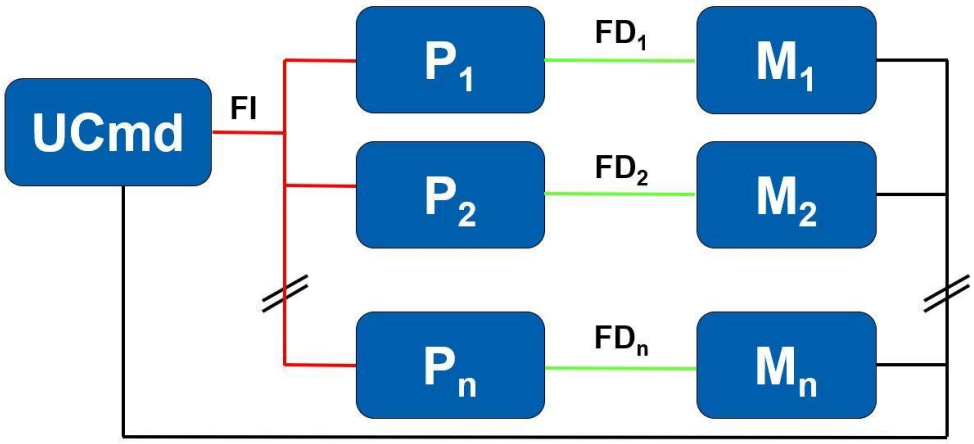


II – SIMD

- SIMD sunt masini vectoriale:
  - O singura UCmd
  - Mai multe procesoare si module de memorie (orice procesor vede orice memorie)

SIMD = 1 FI & n FD

- Toate procesoarele fac aceeasi op impusa de UCmd prin FI

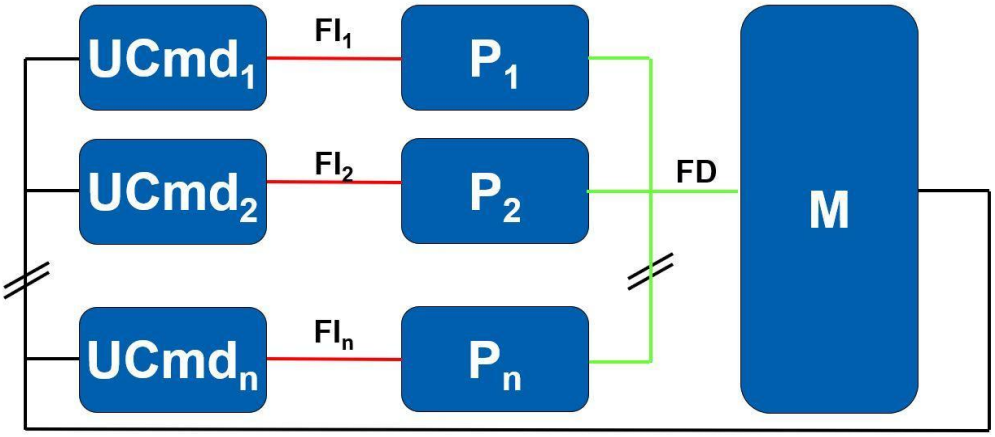


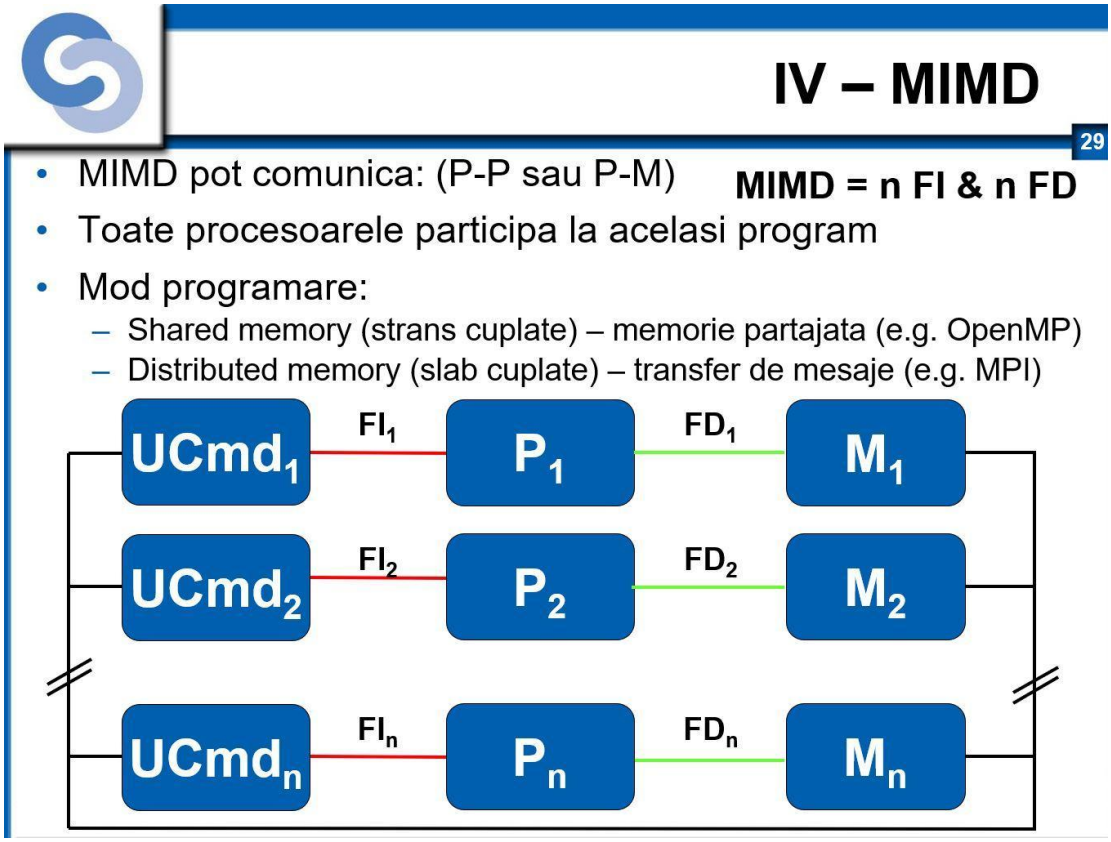
III – MISD

- MISD au:
  - Mai multe UCmd si procesoare
  - Un singur modul de memorie

MISD = n FI & 1 FD

- Domeniu de aplicatie restrans si special: aplicarea altor algoritmi pe aceleasi date (Apps: meteo/evidenta populatiei)





11. Descrieti caracteristicile generale CELL

A hardware Cell is made up of:

- 1 Power Processor Element (PPE).
- 8 Synergistic Processor Elements (SPEs). or APU = additional processing unit
- Element Interconnect Bus (EIB).
- Direct Memory Access Controller (DMAC).
- 2 Rambus XDR memory controllers.
- Rambus FlexIO (Input / Output) interface.

12. Retea de comutare de tip intercalare perfecta si comparatie cu reteaua cu ordine inversa (permutarea de baza).

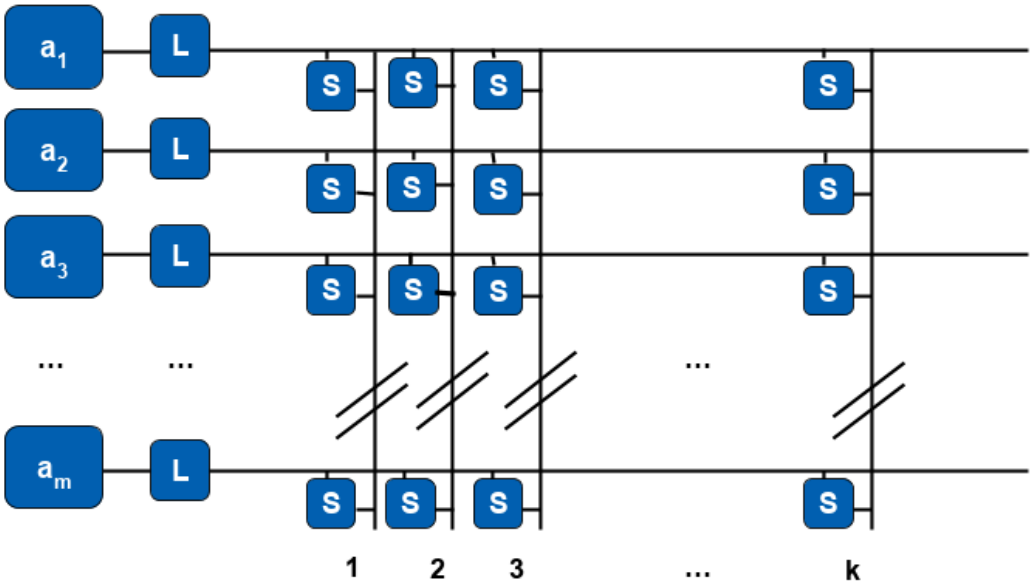
Vezi ex 32.

13. Trunchi K ierarhic / trunchi k neierarhic

Neierarhic: Face legatura intre P-P si M-M (elemente de acelasi tip).

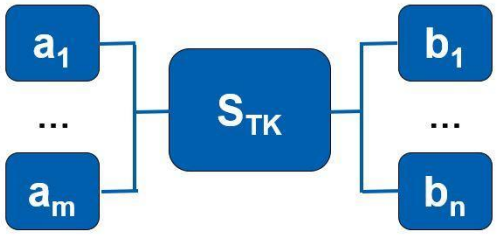
Trunchi-K neierarhic:

– S(tk; ma, c:min(k,m/2); m*k Sp)



Ierarhic: Asigura comunicarea intre P-M si M-P. E o singura unitate S ce leaga mai multe A-uri de mai multe B-uri. Dimensiunea (K) trebuie aleasa in functie de nr de A-uri si nr de B-uri).

- Imbina conectivitatea oferita de S legatura multipla cu costul redus al celorlalte tipuri de comutatoare
- S {TK, ma, nb, c = k, k*(m+n)Sp}



- Orice a poate comunica cu orice b
- Nu totdeauna vei avea m x n comunicatii simultane
  - Asa a aparut idea de a folosi k linii pentru comunicatie
- In functie de m si n si de trafic, se va determina un k optim
  - 8 procs & 4 memorii: pot fi maxim 4 accese la memorii → k=4, nu are sens mai mult; k < n si k < m

14.Adunarea elementelor a doi vectori in SIMD si MIMD (cod + cum retii datele)

SIMD: se lanseaza in mod paralel by default din arhitectura
ck = ak + bk (0 ≤ k ≤ n – 1)
O(1)

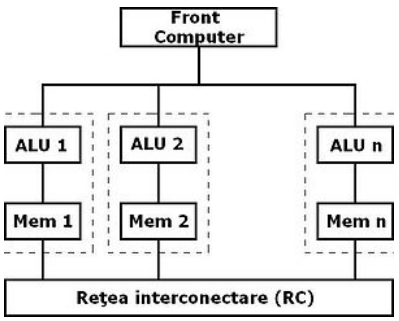
MIMD: trebuie sa lansam noi fiecare proces
for k = 0 to n – 2 (nu si pt el insusi)
    FORK Adr
end k loop
Adr:
    ck = ak + bk (0 ≤ k ≤ n – 1)
JOIN
O(n) pentru ca trebuie sa stai sa lansezi n procese.

Organizare datelor: vezi ex 1

15. Subiect cu permutari (parca era descrierea permutarilor de tip perfect shuffle).

Vezi ex 32.

16. Organizarea datelor la SIMD



Nu cred ca trebuie mai fancy de atat. Practic, ai ALU multiple, Mem 1, …. Mem n si o retea de interconectare la baza.
(Vezi ex 1)

17. Comparatie SIMD MIMD la algoritmi

Vezi ex 2

18. Probabilitatea ca o cerere sa fie acceptata la un comutator CB (formula)

- Consideram un CB (pxm) si:
  - P_A = probabilitatea de acceptare a unei cereri oarecare
    - E data de rata de transfer B(p,m), de r si de p:

$$P_A = \frac{B(p,m)}{r \cdot p} = \frac{m}{r \cdot p} - m\frac{\left(1-\frac{r}{m}\right)^p}{r \cdot p}$$

    - Obs: se considera ca cererile sunt independente; in realitate cererile rejectate se transfera ciclului urmator → marind rata de transfer
    - Daca numarul de resurse interconectate creste: $\lim_{m\to\infty}\left(1-\frac{r}{m}\right)^p = e^{-r\frac{p}{m}}$
    - Daca p si m sunt foarte mari atunci :

$$B(p,m) \cong m\left(1-e^{-r\frac{p}{m}}\right) \qquad P_A \cong \frac{m}{r \cdot p}\left(1-e^{-r\frac{p}{m}}\right)$$

    - Formele simplificate sunt acceptabile in urmatoarele limite:
      - Daca p si m ≥ 30 atunci ele sunt in limita unei erori de 1%
      - Daca p si m ≥ 8 atunci ele sunt in limita unei erori de 5%

19. Comparatie permutare baza, permutare perfecta

(Vezi ex 32)

20. Block placement in cache.

Mapare directa - un bloc din cache poate fi asezat intr-un singur loc
Mapare (full) asociativa - un bloc din cache poate fi asezat oriunde
Mapare set asociativa - un bloc din cache poate fi asezat intr-un subset de pozitii

Trade offs:
Maparea n-way set associativa devine din ce in ce mai dificila si costisitoare de implementat pentru un n mare:

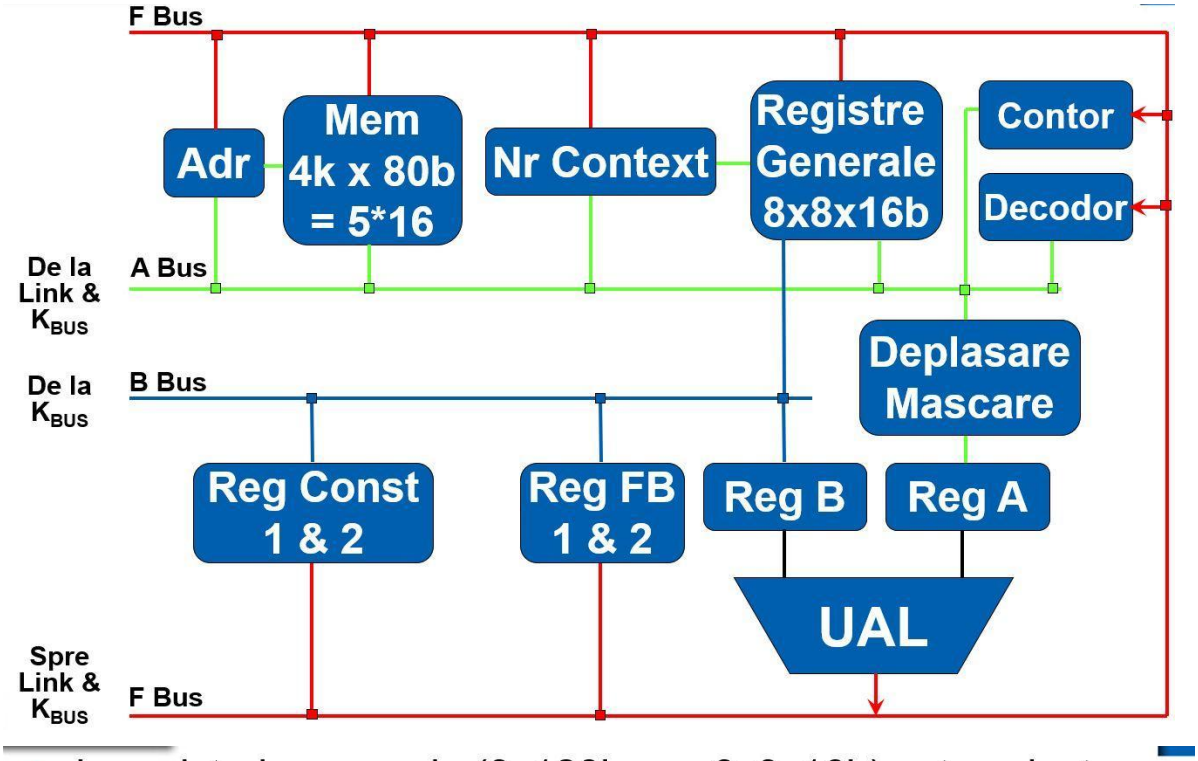
- Majoritatea cache-urilor sunt 1, 2 sau cel mult 4 set asociative

Cu cat n-ul este mai mare, cu atat este mai mica probabilitatea de aparitie a fenomenului de thrashing:

- Momentul in care doua (sau mai multe) regiuni de memorie sunt accesate in mod repetat si pot incapea impreuna in acelasi bloc din cache

21. Arhitectura procesorului PMAP.

- P_{MAP} este un procesor dedicat pe 16 biti, microprogramat, cu codificare orizontala
- Memoria de control este formata din 4k * 80 de biti
- Memoria de control RAM foarte rapida are cuvantul de 80 biti (5*16) – Writable Control Store
- Functiile fundamentale ale procesorului P_{MAP} sunt:
  - Translatarea adreselor nelocale din cluster
  - Implementarea primitivelor OS critice din punctul de vedere al timpului de executie
  - Culegerea informatiilor statistice pentru imbunatatirea solutiilor de proiectare

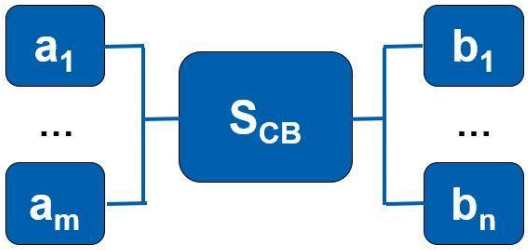


- In registrele generale (8x128b sau 8x8x16b) este salvat contextul executiei in curs
- Este organizat in jurul a 3 magistrale:
  - A, B pentru operanzi
  - F pentru rezultate
- Pentru a putea mapa adresele P_{MAP} trebuie sa aibe acces rapid la informatia pt translatarea adr virtuale in adr fizice
- UAL are un operator la nivel de bit pentru Reg A ce asigura:
  - Deplasarea
  - Rotirea
  - Mascarea (cu 30 de masti predefinite), permitand preluarea descriptorului de context
- Prelucrarea este in banda de asamblare – operatii UAL si operanzii ce vin simultan, separat, prin registrii tampon
- Accesul la o memorie in acelasi cluster este mult mai rapid decat in alt cluster

22. Comutator legatura multipla (CrossBar) ierarhic.

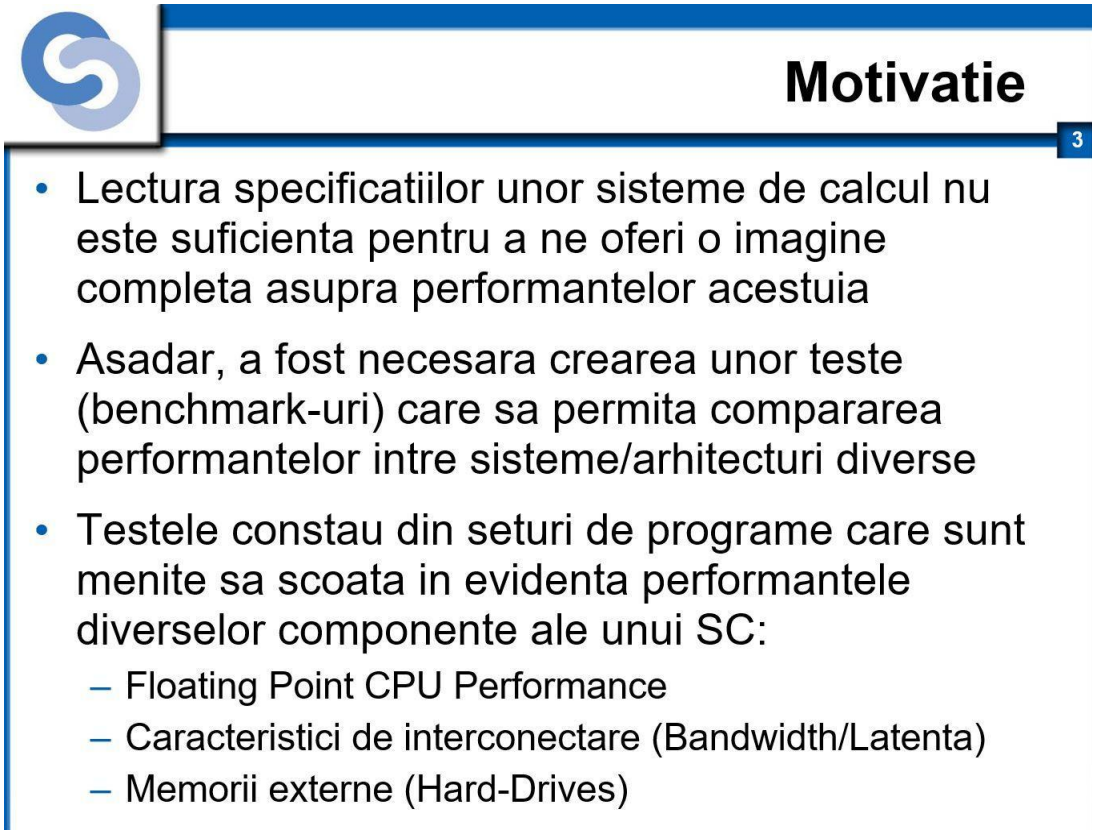
(voia 3 cuv, maxim o fraza: InfiniBand, Linux, Cluster) (ce pula mea?)

- Asigura comunicatia intre m componente de tip a si n componente de tip b
- Este cunoscut si ca Cross-Bar-Switch
  - S {CB, ma, nb, c = min(m, n), m*nSp}



- Orice a poate comunica cu orice b
- Pot avea loc transferuri multiple si simultane!
- Cel mai dificil este de implementat U_{Cmd} pentru S_{CB} – ea trebuie sa rezolve toate posibilele conflicte

23. Benchmark-uri, probleme si rezolvari legate de acestea (ceva de genu).



Caracteristici & Probleme

- Sunt create pentru a “simula” comportamentul aplicatiilor reale (sintetice)
- Cele mai bune benchmark-uri sunt aplicatiile utilizator in sine…
- Asadar, trebuie sa fim intotdeauna circumspecti cand utilizam benchmark-uri!
- Cateva probleme:
  - Producatorii HW fac “tuning”
  - Performantele masinilor se degradeaza cand load-ul depaseste o valoare de prag
  - Securitatea, disponibilitatea, fiabilitatea, serviceabilitatea sau scalabilitatea NU sunt obiectul unui benchmark

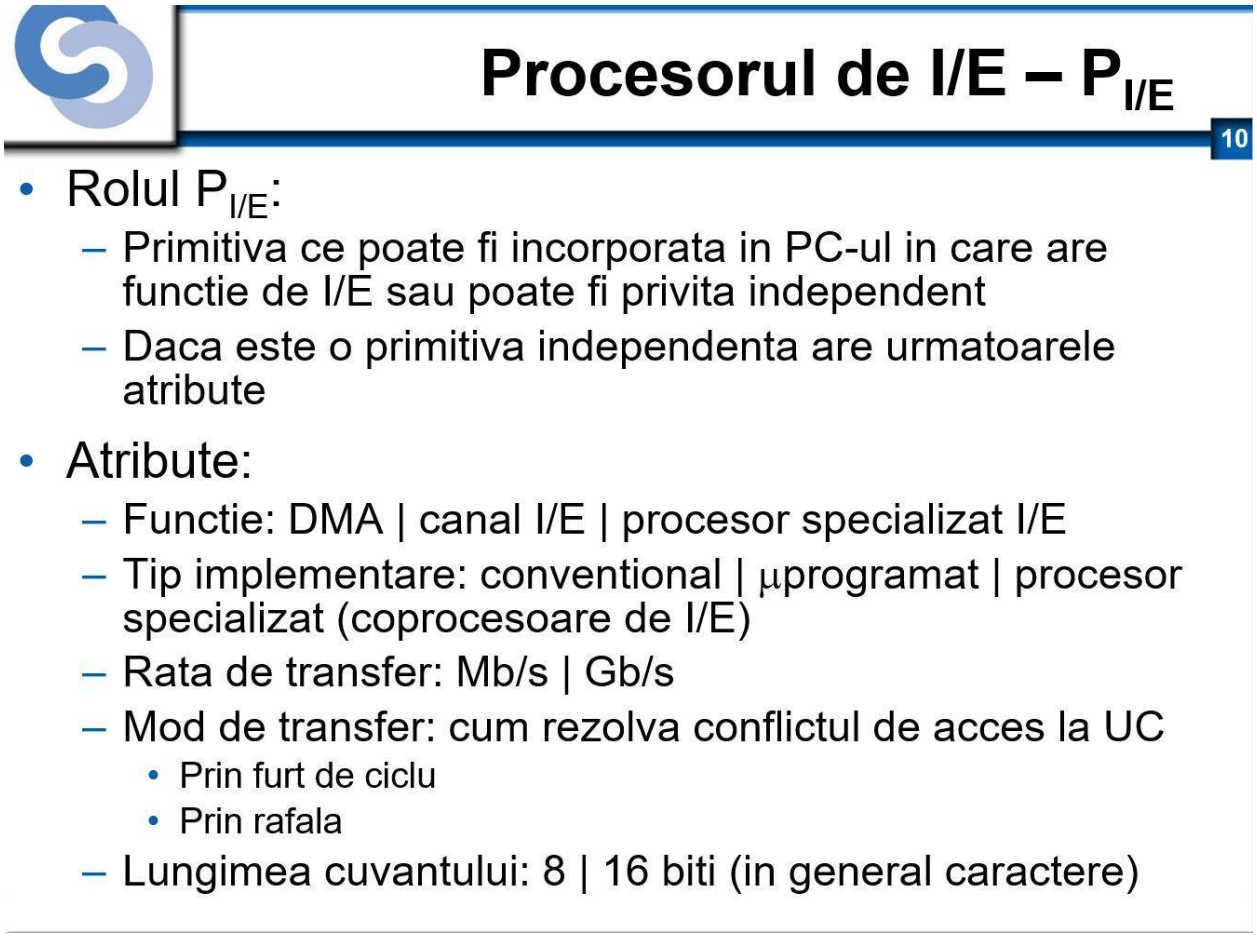
Tipuri de Benchmark-uri

1. Aplicatii reale: procesoare de text, aplicatii utilizator
2. Kernel: contine doar parti esentiale dintr-o varietate de programe (i.e. Linpack), cu rezultate in MFlops
3. Micro-benchmark-uri: dezvoltate de utilizatori pentru a testa componente ale sistemului
4. Sintetice: se strang date statistice asupra operatiilor efectuate de aplicatii si se construiesc programe care sa respecte proportia aflata astfel (Whetstone/Dhrystone)
5. Pentru sisteme I/O
6. Pentru sisteme paralele

24. Tipuri de paralelizare la sistemele cu prelucrare paralela.

1. Paralelism la nivel de bloc/job, intre joburi diverse avem nevoie de, mecanism de salvare de context, ceas de sincronizare, canale de I/O pt transfer date;
   Anumite faze ale unui job pot fi suprapuse
2. Paralelism la nivel de subansamble hardware: intre elem de prelucrare a vectorilor, intre componentele logice ale dispozitivelor aritmetico/logice (gen carry lookahead)
3. Paralelism la nivel de program: intre diverse sectiuni independente ale unui program, intre loopurile unui program; ambele au nevoie de analiza dependentelor de date
4. Paralelism la nivel de instructiune (mancati cacatu cu pipeline si branch predictor):
   Seriala - fara pipeline
   Seriala paralela - gen pipeline da poa sa fie si inseriate instructiunile nuj
   Paralela - full pipeline

25. Structuri SN. P i/e structura



26. Block replacement

Cand are loc un miss, controlerul de cache trebuie sa “faca ceva”

Pe un cache mapat direct, este foarte simplu – se va suprascrie continutul unui block-frame cu date noi aduse din memorie
Intr-o implementare n-set-asociativa insa, trebuie sa facem o alegere: care dintre cele n blockframe-uri trebuie suprascrise?

In practica, exista trei strategii implementate pentru acest lucru

- Random: foarte usor de implementat
- First-In-First-Out (FIFO):
  - Un pic mai dificil de implementat
  - Aduce beneficii la cache-uri de dimensiuni mai mici
- Least-Recently Used (LRU):
  - Si mai dificil de implementat, mai ales pentru cache-uri de dimensiuni mari
  - Aduce beneficii la cache-uri de dimensiuni mai mici
  - Atentie, trebuie sa avem mereu in vedere costul cache-ul vs latenta oferita de acesta: overhead-ul computational si hardware-ul aditional necesar calcularii valorilor LRU

27. Determinarea dimensiunii cache-ului

Adresa pe n biti: t biti tag, s biti set index, b biti block offset, fiec adresa are loc unui cuvant
Fiecare bloc: 2^b cuvinte; Memoria cache are 2^s linii;
Dimensiune cache = 2 ^(s+b) cuvinte

Masurarea Dimensiunii Cache-ului

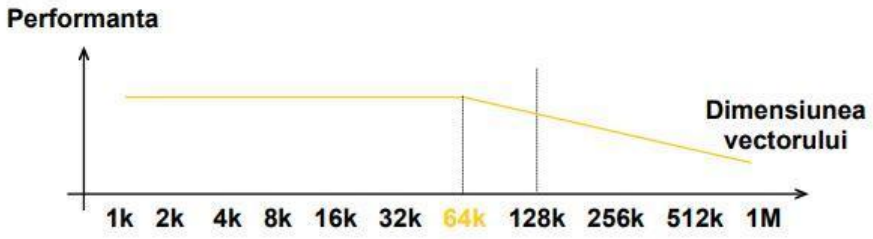
- Daca L este dimensiunea liniei de cache
- Sa consideram urmatorul cod

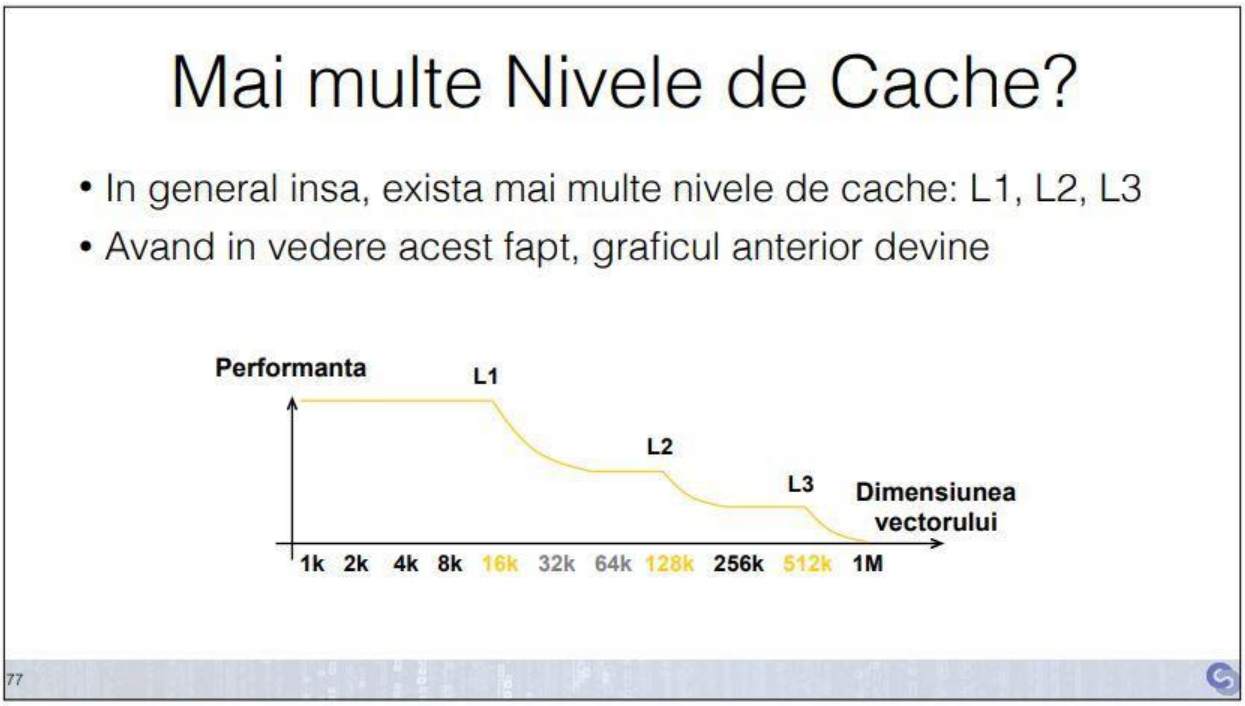
```
char x[1024];
for (step=0;step<1000;step++)
    for (i=0;i<1024;i+=L)
        x[i]++;
```

- Daca cache-ul este mai mare de 1024 de bytes, dupa prima iteratie a buclei "step", tot vectorul x e in cache si nu vom mai avea miss-uri deloc
- Daca cache-ul este mai mic decat 1024 de bytes, vom avea mereu un numar de miss-uri

Masurarea Dimensiunii Cache-ului

- Astfel:
  - Daca vectorii sunt mici si intra in cache hit-rate = 100%
  - Daca vectorii sunt mai mari decat cache-ul hit-rate < 100%



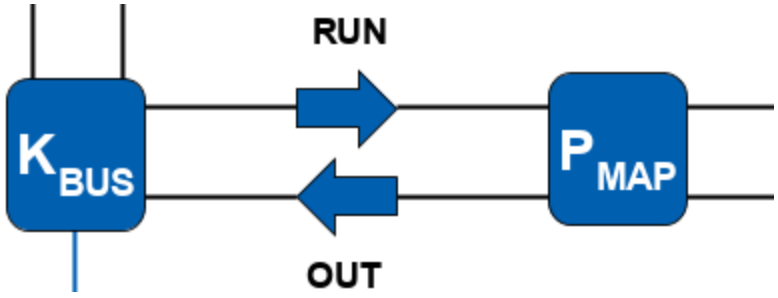


28. Comunicatia Kbus Pmap

KBUS asigura controlul in cadrul unui cluster prin excluderea mutuala a acceselor la MapBus.
Comunicarea intre clustere prin InterCluster Bus.
Functii de baza Pmap:

- Maparea de adrese
- Comunicare
- sincronizare

O serie de primitive OS sunt implementate in Pmap.



Kbus este unitatea de comanda a map bus.
Pmap este un procesor de mapare care face translatia de adrese.
KBUS memoreaza STAREA contextelor si le aloca cereri. Le trimite prin RUN la PMAp.
PMAP proceseaza cererile de la KBUS care activeaza schimbarile de context cerute de KBUS.

PMAP pune iesirea la OUT

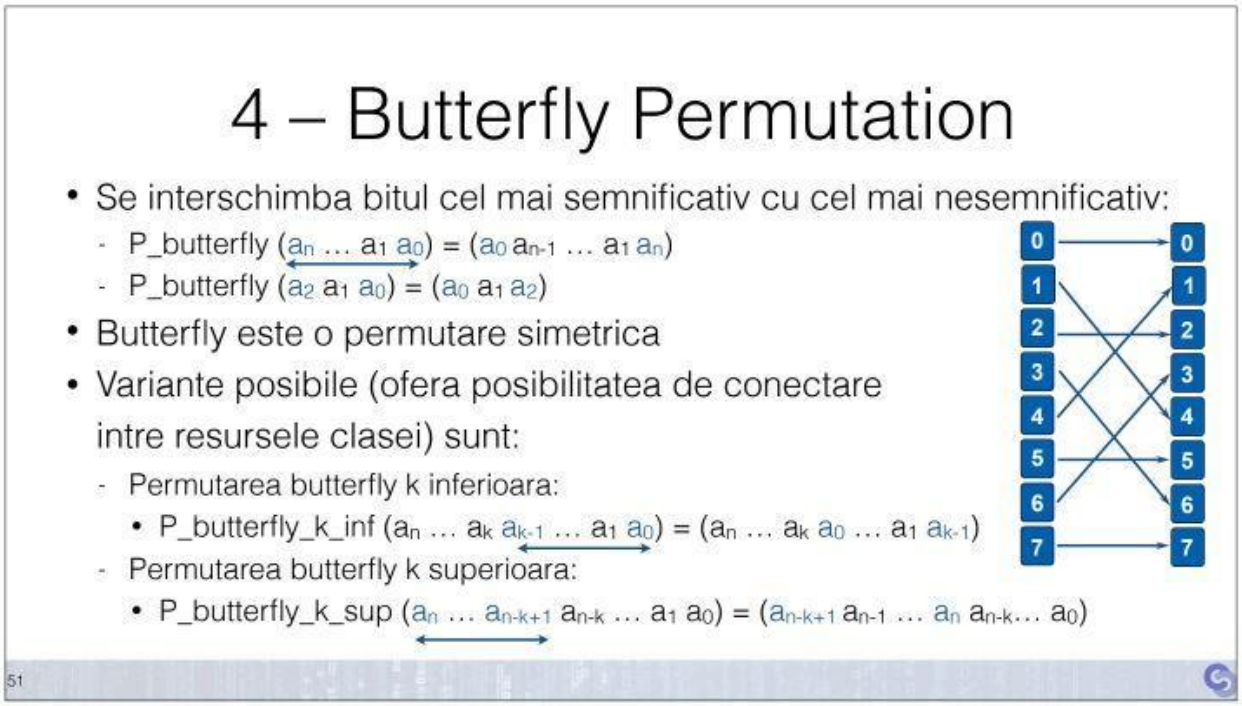
Comunicatia K_{BUS} – P_{MAP}

- K_{BUS} mentine starea a 8 contexte si le aloca cereri de serviciu (fiecare va fi rezident in P_{MAP})
- Numarul contextului si alte informatii sunt plasate in coada RUN
- Din RUN sunt preluate de P_{MAP} care activeaza contextul si eventual translateaza adresa si o plaseaza in OUT
- In acest timp K_{BUS} poate citi alte cereri sau efectua alte functii cerute de P_{MAP}
- In P_{MAP} se executa o schimbare de context prin intermediul celor doua cozi (RUN/OUT)

Comunicatia K_{BUS} – P_{MAP}

- K_{BUS} preia acum cererea din OUT si transfera adresa si datele la S-ul local al destinatiei; apoi se ocupa de alte cereri
- Cand accesul la memorie este terminat, K_{BUS} citeste confirmarea de terminare sau citeste datele si plaseaza iara contextul in RUN, reactivandu-l
- K_{BUS} trimite apoi confirmarea la procesul ce a initiat cererea de serviciu
- Contextul poate fi utilizat acum pentru o noua cerere de serviciu

29. Permutarea Butterfly



30. Benchmark-uri pentru sisteme seriale

- SPEC (Standard Performance Evaluation Corporation) realizeaza benchmark-uri pentru sisteme de calcul si le publica pe web
- Incearca sa simuleze situatii reale:
  - SPECweb testeaza performantele unui server de web pentru diferite tipuri de cereri HTTP paralele
  - SPEC CPU testeaza CPU-ul ruland
- SPEC sunt scrise in C sau Fortran (compilatoarele pot avea optimizari pentru acest tip de aplicatie)
- Se distribuie cu o licenta
- Exista un conflict cu GPL (foloseste cod GPL)…
- http://www.spec.org/

- TPC (Transaction Processing Performance Council) a definit benchmark-uri pentru procesarea tranzactiilor si a bazelor de date:
  - http://www.tpc.org/
- Consortiul BAPCo (Business Application Performance Corporation) este format din Intel, AMD, Dell, HP, NVidia, Apple & Microsoft, etc:
  - WEBmark, SYSmark 2004SE, si MobileMark
  - http://www.bapco.com/
- **SiSoftware Sandra** (**S**ystem **AN**alyser, **D**iagnostic and **R**eporting **A**ssistant) este un program de testare si diagnoza:
  - http://www.sisoftware.net/

31 Cell BE bariere si solutii.

All APUs can work independently
- Sounds like a memory nightmare

Synchronization is done in hardware
- Avoids software overhead

Memories on both ends have additional status information

Each 1024 bit addressable memory chunk in DRAM has
- Full/Empty bit
- Memory for APU ID and APU LS address

Each APU has
- Busy bit for each addressable part of local storage

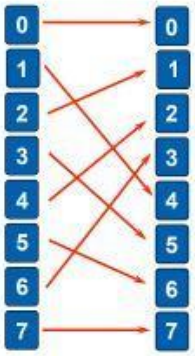
Full/Empty Bit – data is current if equals 1
- APU cannot read the data if it is 0
- APU leaves its ID and local storage address
  - Second APU would be denied

Busy bit
- If 1, APU can use to write info from DRAM
- If 0, APU can write any data

32. Comparatie intre permutarea de baza si perfect shuffle

1 – Base Line Permutation

- Bitul cel mai nesemnificativ devine cel mai semnificativ:
  - P_base ($a_n\ a_{n-1} \dots a_1\ a_0$) = ($a_0\ a_n\ a_{n-1} \dots a_1$)
- Permutarea de baza este intercalarea perfecta inversa!
- Pentru 8 resurse avem P_base ($a_2\ a_1\ a_0$) = ($a_0\ a_2\ a_1$)
- Variante posibile sunt:
  - Permutarea de baza k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_b_kinf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0\ a_{k-1} \dots a_1$)
    - Asigura conexiunea **in** submultimi de resurse
  - Permutarea de baza k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_b_ksup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_0$) = ($a_{n-k+1}\ a_n \dots a_{n-k+2}\ a_{n-k} \dots a_0$)
    - Asigura conexiunea **intre** submultimi de resurse

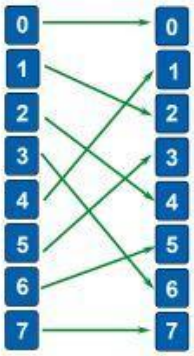


48

Adecvat pentru operatii recurente, calculabile in log2n pasi.

2 – Shuffle Permutation

- Bitul cel mai semnificativ devine cel mai nesemnificativ:
  - P_shuffle ($a_n\ a_{n-1} \dots a_1\ a_0$) = ($a_{n-1} \dots a_1\ a_0\ a_n$)
- Pentru 8 resurse avem P_shuffle ($a_2\ a_1\ a_0$) = ($a_1\ a_0\ a_2$)
- Variante posibile sunt:
  - Permutarea shuffle k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_shuffle_kinf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k \dots a_1\ a_0\ a_{k-1}$)
    - Exemplu: Conexiunea memoriilor - adr baza & offset intr-un modul
  - Permutarea shuffle k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_b_ksup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_0$) = ($a_{n-1} \dots a_{n-k+1}\ a_n\ a_{n-k} \dots a_0$)



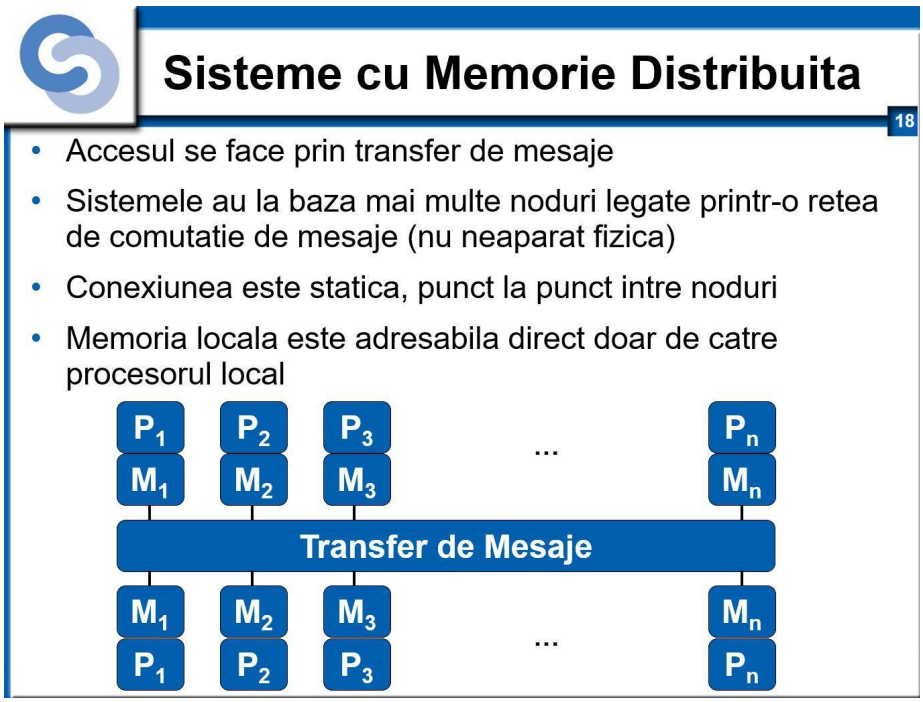
49

(aka perfect shuffle)
Este adecvata pentru:

- Transformata Fourier (FFT – Fast Fourier Transformation)
- Algoritmi de sortare
- Transpuneri de matrice

Asigura conectarea intre procesoarele unei structuri SIMD de n procesoare cu:
n-2 comunicatii simultane
dupa log2n iteratii datele ajung pe procesorul sursa

33. Memorii distribuite



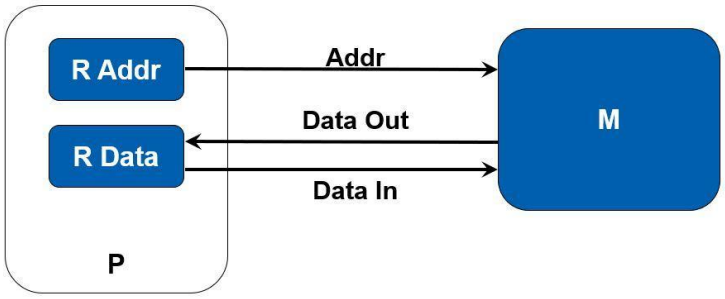
Ex: SUN Enterprise 10000 - Starfire -> Reteaua de comutatie este de tip Gigaplane XB (102.4GB/s; p2p: 1.6GB/s)

34 Optimizari programe (trebuia vb de cache)

Aici merge zis de C++/Java ca sunt row major si ca exploatează localitatea spatiala.
Localitate spatiala +/- localitate temporala: Ce sunt, ce înseamnă, când apar.
Cum ajuta cache: cache hit + evitare trashing.
Optimizări în cod: reorganizarea buclelor, folosirea inteligenta a variabilelor (ca în lab la sum pe matrice).

35. Comutatoare în cuplarea memorie procesor

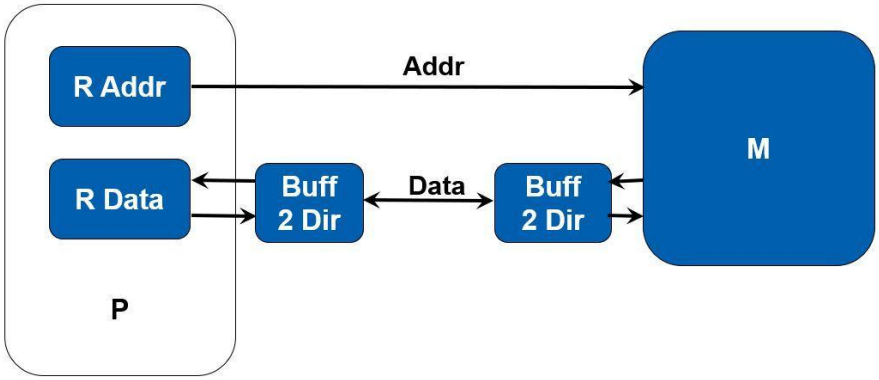
- Switch Processor Memory → $\mathbf{S_{PM}}$
- Liniile de adrese si date sunt complet separate



R/W – Disjuncte → No Switch!

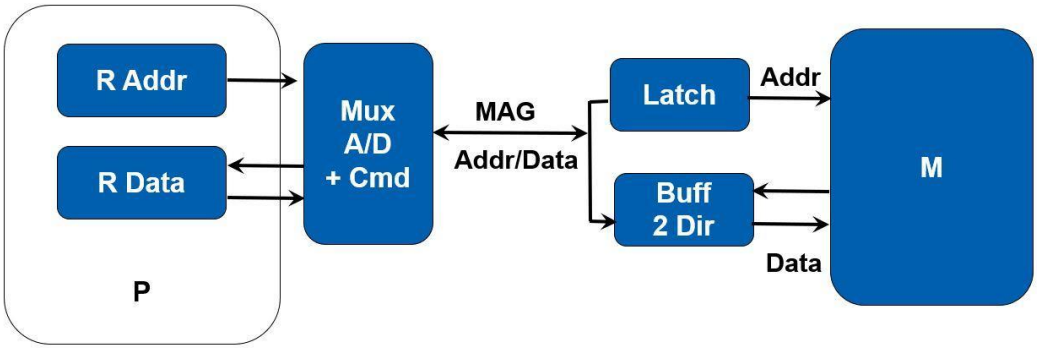
(nu se justifica ambele linii)

- Switch Processor Memory → $\mathbf{S_{PM}}$
- DIn si DOut sunt multiplexate prin magistrala de date



Bufferele sunt fie MUX fie TS

- Switch Processor Memory → S_{PM}
- Addr, DIn si DOut sunt multiplexate prin aceeasi magistrala



Bufferele sunt fie MUX fie TS

38 Comparatia generala intre SIMD si MIMD

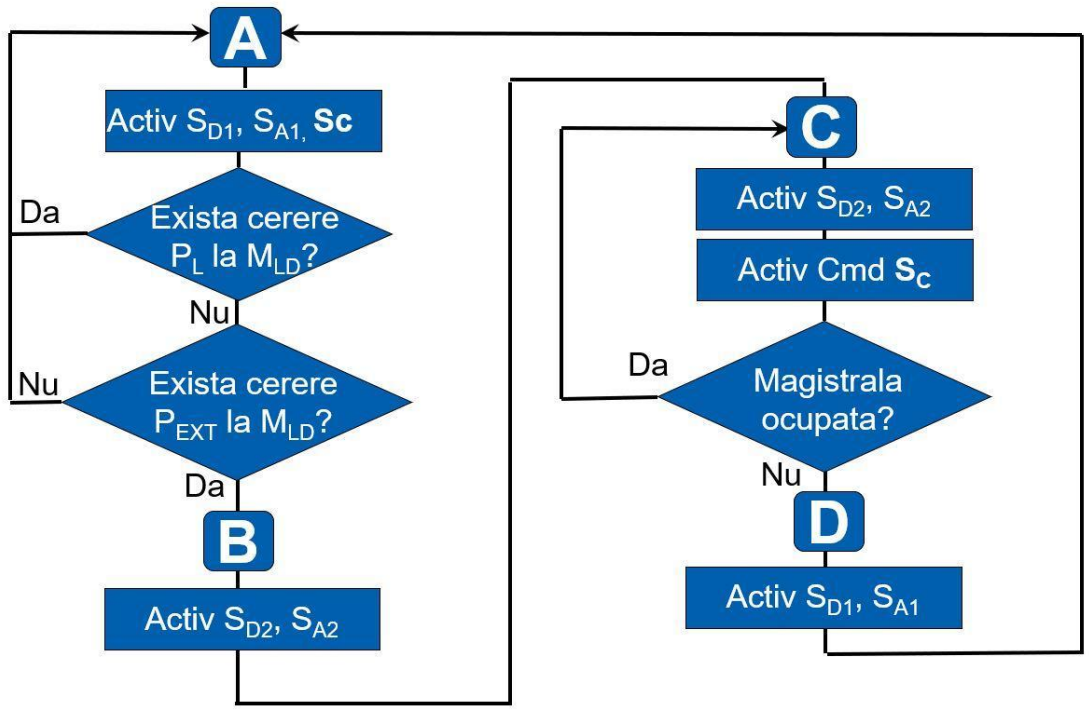
Diferente SIMD/MIMD pentru codul A x B =C:

- In SIMD procesoarele se sincronizau instructiune cu instructiune
- In MIMD nu exista (neaparat) sincronizari intre FI (flux de instructiuni) ale P-urilor din structura
- La SIMD se calculeaza elementele lui C pe linie si coloana (FI unic)
- La MIMD orice procesoare pot calcula orice elemente din C (fiecare P are un FI propriu!)

MIMD: mai usor de programat & utilizat decat SIMD – dar, mai scump

Compartie organizare date: **vezi subiectul 1.**

39. Organigrama KML de la SBC-uri + schema logica

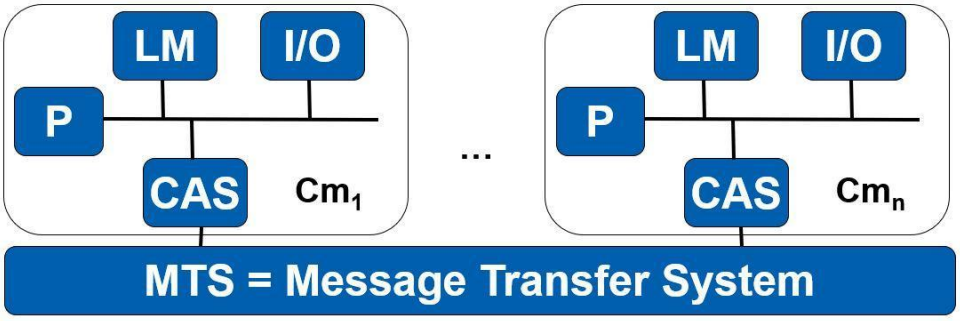


- K_{ML} – asigura accesul la Mem locala a P_L prin intermediul S_{D1} si S_{A1} & a proc externe prin S_{D2} si S_{A2}
- P_L are prioritate fata de proc externe (P_{EXT}) == daca vrea P_L la M_{LD} NU ma intereseaza ce vor celelalte P_{EXT}
- Starea A: P_L poate avea comenzi locale
- Starea B: activez accesul ext la M_{LD} – semnalele asociate starii
- Obs: Mem locala e vazuta de fiec proc intr-un ac spatiu de mem, iar de proc ext in spatiul complementar → trebuie sa existe o unitate de translatare a adreselor (TA)
- Mag ocupata: nu stii ciclii masina ai unui proc extern. Numai cand magistrala e libera un P_{EXT} a terminat ciclul masina curent → abordare top-down

46. Comunicatia intercluster intr-un sistem MIMD slab cuplat

- De tip cluster → calculatoarele au resurse locale necesare functionarii independente
- Procesele se executa pe procesoare care dispun de toate resursele pentru implementarea task-ului respectiv
- Comunicarea intre procesoare se face prin
  - Transfer de mesaje
  - Comutare de circuite
- Sistemele slab cuplate (distribuite) sunt eficiente atunci cand interactiunea intre procesoare este minima (granularitatea problemei este mare)

- Resurse locale ale CM_i = Computer Module:
  - LM = local memory (acces din interior/exterior)
  - I/O = magistrala de intrare/iesire (acces din interior/exterior)
  - MTS = Message Transfer System – poate fi implementata cu comutare de circuite sau transfer de mesaje
  - MTS dicteaza performantele prin lungimea mesajelor & viteza de lucru
  - CAS = Collision Avoidance System – legatura si accesul la MTS se fece prin excludere mutuala:
    - un singur transfer odata daca MTS este magistrala
    - mai multe transferuri simultane daca MTS este de tip retea



44. Care sunt cele 4 elemente de care trebuie tinut cont cand se realizeaza designul unui cache.

Block placement - unde trb scris un bloc in cache,
Block identification - cum se gaseste o linie in cache
Block replacement - cum se inlocuieste o linie in cache la un miss
Write strategy - ce se int cand se scrie in cache (write through / write back)

45. De ce nu se poate realiza un procesor de 10TFlops(minim 3 motive) ?

- Am avea nevoie de un procesor de 10.000 GHZ
- Datele ar trebui sa circule cu viteza luminii
- O celula de memorie ar trebui sa fie de dimensiunea a cativa atomi

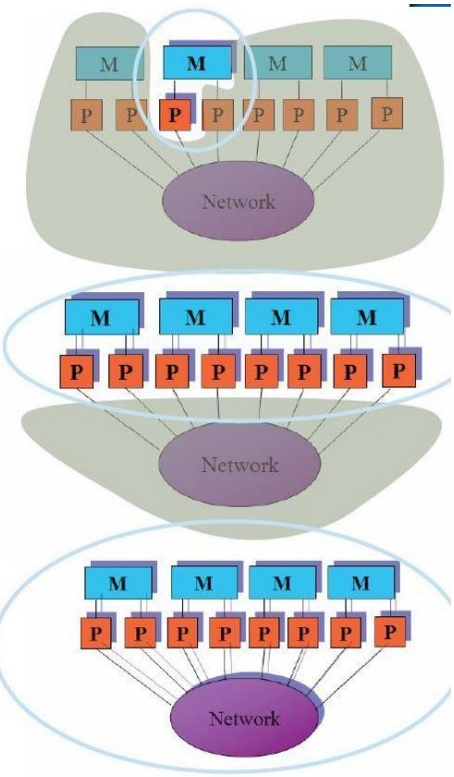
46. Caracteristici HPC Challenge Benchmark.

- Un Benchmark ce evidentiaza
  - Procesoarele
  - Memoria
  - Reteaua de interconectare

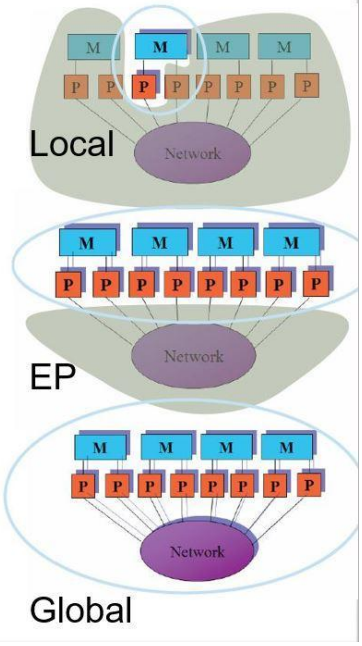
HPCS = High Productivity Computing Systems
DARPA = Defense Advanced Research Projects Agency

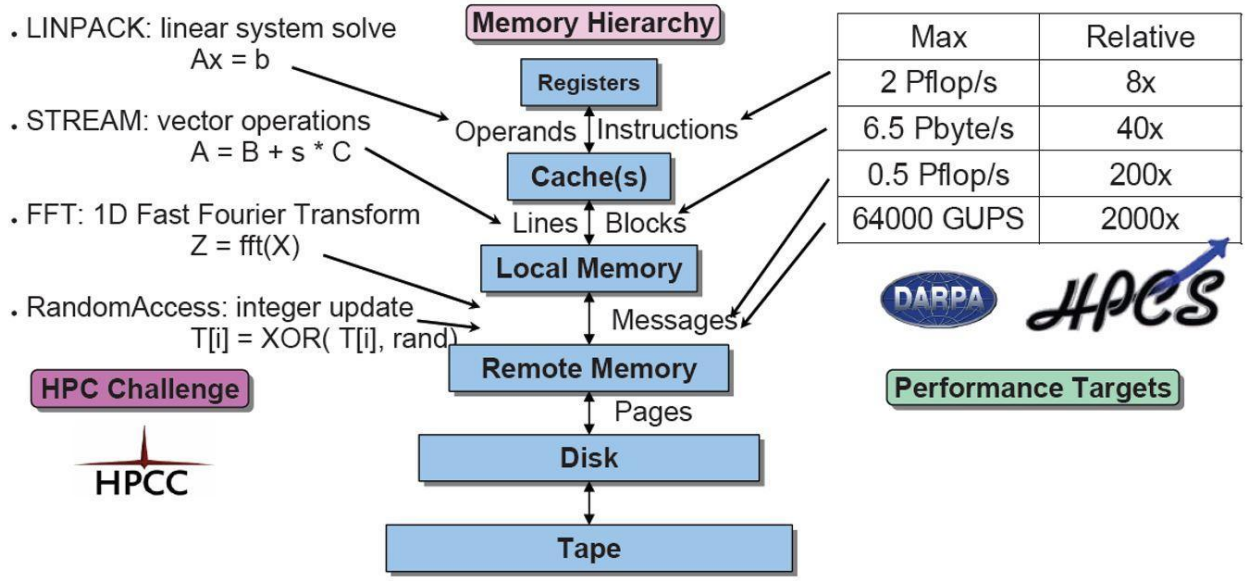
- Avantajul major al HPCC – arhitecturile pot fi descrise cu mult mai multe metrici decat FLOPS-urile LINPACK
- Scopuri ale HPCC
  - Sa fie complementar cu TOP 500
  - Sa ofere benchmark-uri care exprima performata aplicatiilor ca o functie a caracteristilor accesului la memoria sistemului (ca localitate spatiala si temporala)
  - Sa permita optimizari
    - Sa inregistreze efortul necesar pentru tuning-ul aplicatiilor
    - Rularea necesita MPI si BLAS
  - Rezultatele obtinute sa poata fi verificate
  - Sa arhiveze rezultatele

- Local: un singur procesor participa la calculele dorite
- Embarrasingly Parallel: fiecare procesor din sistem participa la calcule, insa ele nu comunica explicit intre ele
- Global: toate procesoarele din sistem participa la calcule si comunica explicit intre ele



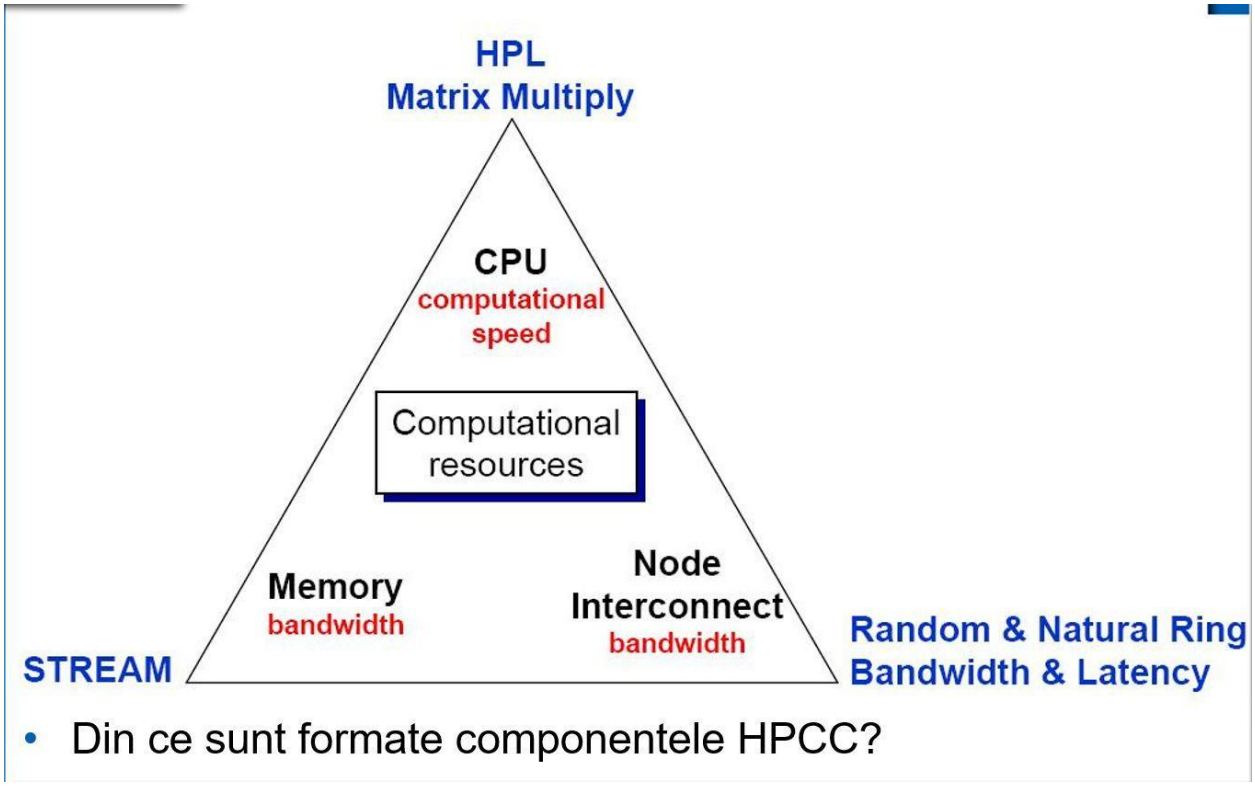
- HPCC e format din 7 benchmark-uri diferite si este de fapt un framework pentru adaugarea de benchmark-uri relevante:
  1. HPL (LINPACK) – MPI Global (Ax=b)
  2. STREAM – Local
     *STREAM – EP
  3. PTRANS – MPI Global ($A \leftarrow A + B^T$)
  4. RandomAccess – Local, MPI Global
     *RandomAccess – EP
  5. BW and Latency – MPI
  6. FFT – Local, Global & EP
  7. Matrix Multiply – Local & EP





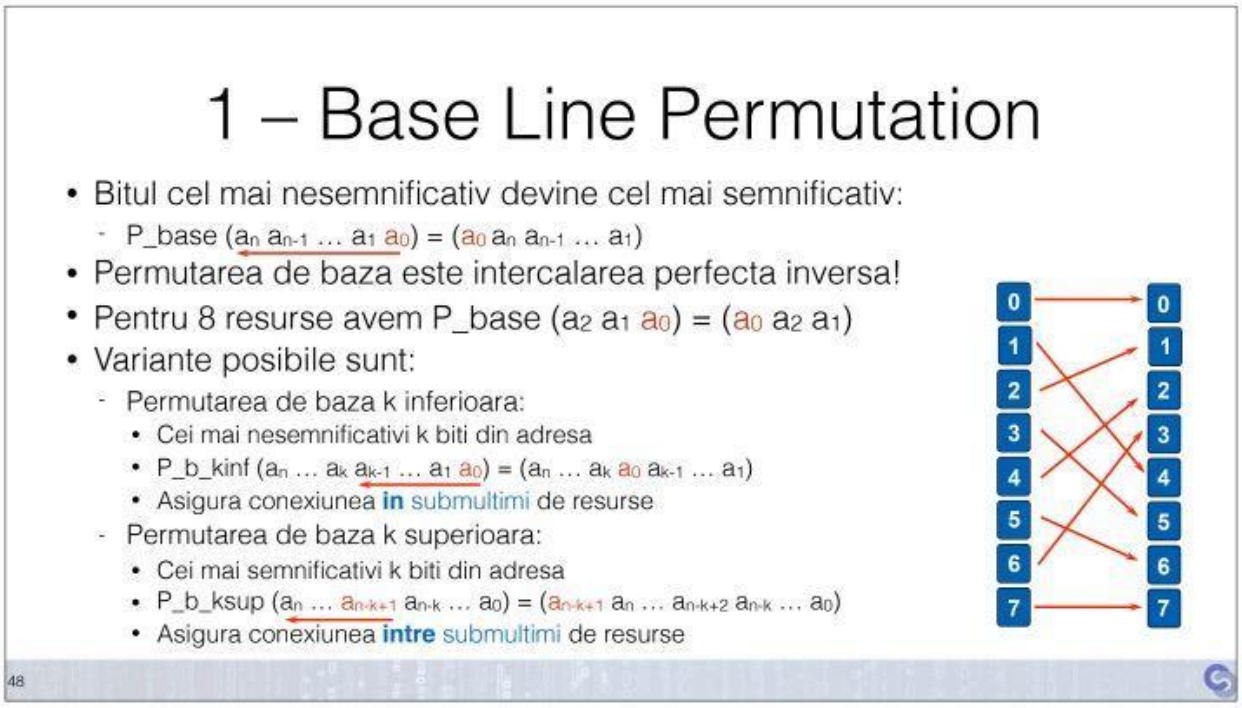
- HPCS incearca prin HPCC sa:
  - Uniformizeze diferentele existente in ierarhia de memorie
  - Imbunatateasca performantele aplicatiilor reale
  - Usureze programarea sistemelor (paralele) de calcul

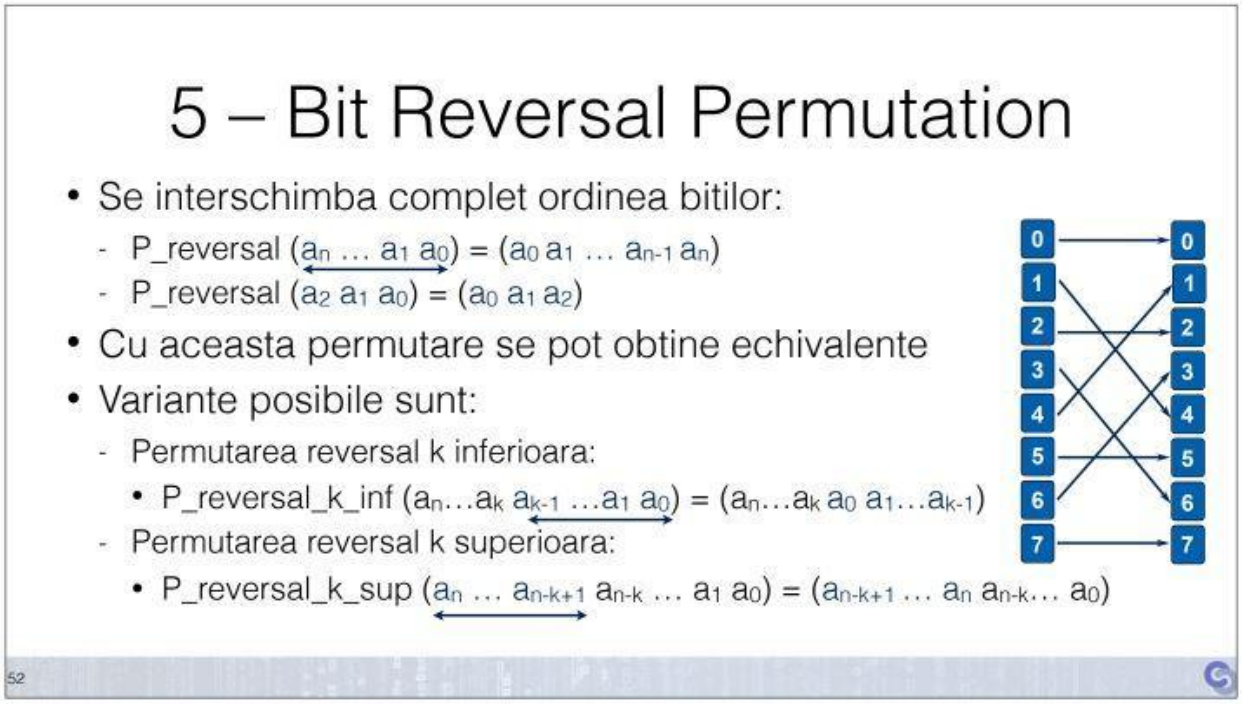
- Benchmark-ul functioneaza ca un program de sine statator
- Similar ca fisier de intrare/specificare cu HPL
- HPCC este format dintr-un “Base Run”, urmat de eventuale “Optimization Runs”:
  - Base Run este rularea Benchmark-ului “out of the box”
  - Utilizatorul trebuie sa instaleze pe masinile de test MPI si BLAS
  - In Optimized Runs se pot inlocui anumite subrutine, pentru a creste performantele unei anume componente
- Rezultatele se upload-eaza pe web la: http://icl.cs.utk.edu/hpcc/
- Se pot genera apoi tabele html sau Excel cu rezultatele obtinute:
  - In mod intentionat nu se va oferi un singur criteriu de evaluare
  - Astfel NU se vor obtine clasificari generale!
- Scopul declarat al HPCC: sa NU necesite mai mult de dublul unei rulari HPL



- Din ce sunt formate componentele HPCC?

47. Comparatie Base Line Permutation - Bit Reversal Permutation





48. Cum se masoara dimensiunea liniei unui cache? Exemplu de program si grafic.

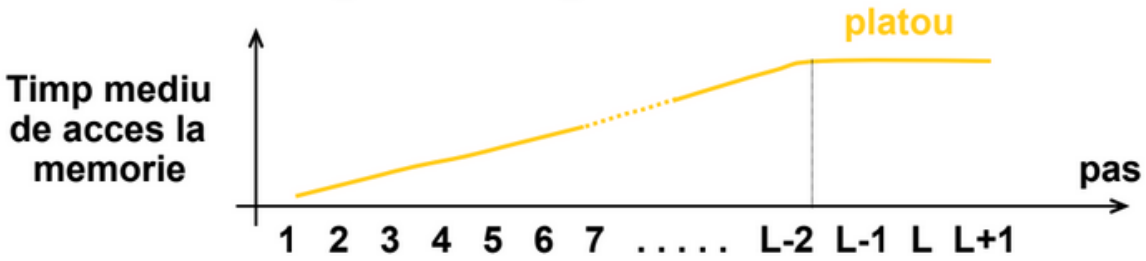
(pentru inlocuirea oricarui alt subiect :))

```
Char a[N];
for(i = 0; i < N; i++)
        a[N]++;
```

Marim pasul i +=2, i +=3, pana performanta codului nu se mai inrautateste.

La pasul 1 avem cache miss la fiecare indice care se imparte exact la L

Pasul de la care nu se mai inrautateste performanta, e dimensiunea liniei de cache



49. Arhitectura unui calculator din top 500 (arhitectura, numar procesoare, so, ...)

https://www.top500.org/system/179397

Summit - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband

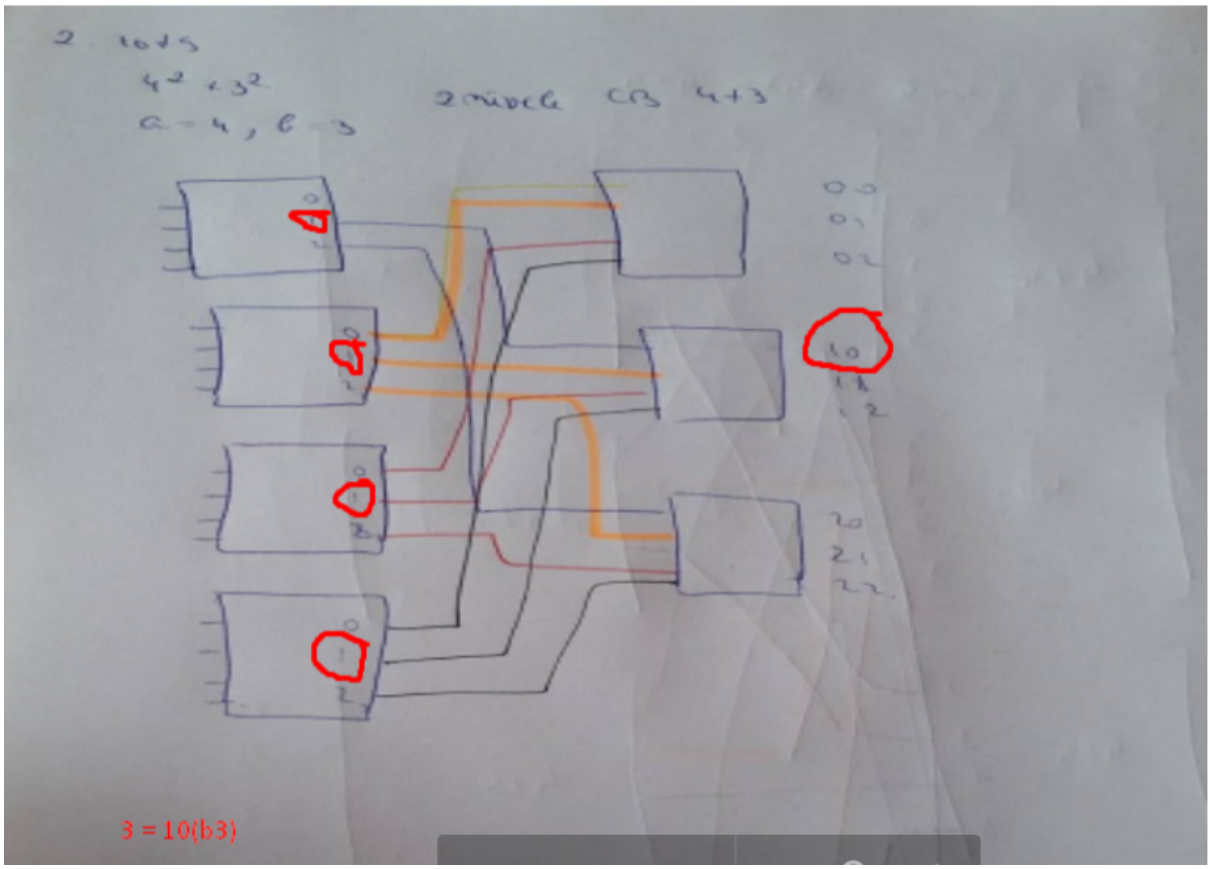
Site:	DOE/SC/Oak Ridge National Laboratory
System URL:	http://www.olcf.ornl.gov/olcf-resources/compute-systems/summit/
Manufacturer:	IBM
Cores:	2,397,824
Memory:	2,801,664 GB
Processor:	IBM POWER9 22C 3.07GHz
Interconnect:	Dual-rail Mellanox EDR Infiniband
Performance	
Linpack Performance (Rmax)	143,500 TFlop/s
Theoretical Peak (Rpeak)	200,795 TFlop/s
Nmax	16,693,248
HPCG [TFlop/s]	2,925.75
Power Consumption	
Power:	9,783.00 kW (Submitted)
Power Measurement Level:	3
Measured Cores:	2,397,824
Software	
Operating System:	RHEL 7.4
Compiler:	XLC, nvcc
Math Library:	ESSL, CUBLAS 9.2
MPI:	Spectrum MPI

50. Implementarea salturilor conditionate in SIMD si MIMD - comparatie

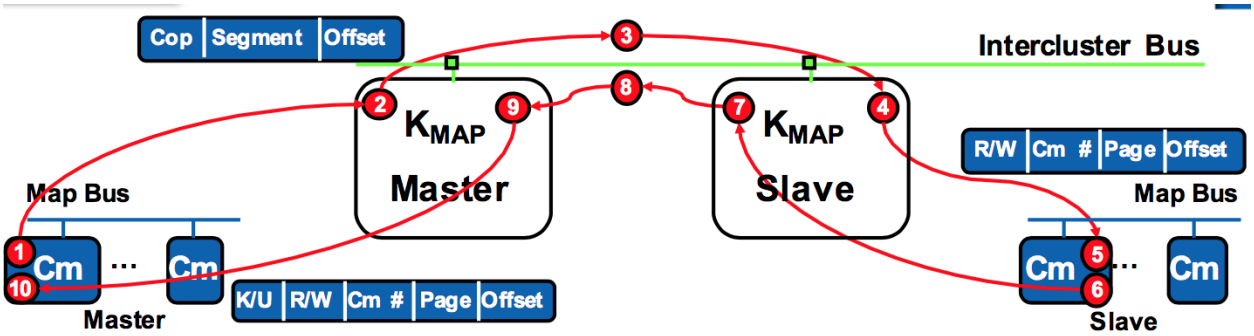
La MIMD e ca si cum ai rula mai multe thread-uri. Ideea e ca fiecare core are propriul program counter, flag-uri, etc. (o parte de registre proprii) asa ca salturi se petrec ca si cum ar fi un procesor separat fiecare core.

La SIMD, ai o inst, de genu' cum erau spu_add(), numai ca atunci cand "faci if" se activeaza un flag daca conditia a fost sau nu adevarat pt fiecare data. De exemplu poate exista inst m = spu_greater_or_equal(v, 0). Ei in urma executarii in m o sa existe pentru fiecare element din v, rezultatul operatiei, adica pt fiecare element, in m o sa fie o valoare diferita de zero pentru true, si 0 pentru false. Acum poti sa folosesti alta instructiune SIMD care il primeste pe m si executa ceva pe un core sau nu, daca elem specific din m e diferit de 0.

51.Să se cupleze 16 procesoare cu 9 memorii utilizând rețea de tip Delta. Indicați conexiunea de la procesorul 8 la memoria 3.



52. Comunicatia intercluster



1. Cm master trimite cerere de transfer la Kmap master.
2. Kmap master pregateste mesajul/ pachetul si codifica cererea
3. Mesajul este transmis pe intercluster bus folosind algoritmi de rutare
4. Kmap slave decodifica cererea si o trimite catre clusterul sau local
5. Cererea este trasmisa la Cm slave
6. Cm slave transmite rezultatul la Kmap slave
7. Kmap slave pregateste mesajul intercluster (reactivarea contextului respectiv)
8. Kmap slave transmite rezultatul prin intercluster bus catre Kmap master
9. Kmap master receptioneaza mesajul primit
10. Rezultatul este transmis catre Cm master

53. Problema cu speed up Amdahl

Amdahl's Law[1]

Amdahl's law is used to find the maximum expected improvement to an overall system when only part of the system is parallelized. That is, the speedup of a program using multiple processors in parallel computing is limited by the time needed for the sequential fraction of the program.

Amdahl's law states that if P is the proportion of a program that can be made parallel (i.e. benefit from parallelization), and $(1 - P)$ is the proportion that cannot be parallelized (remains sequential), then the maximum speedup that can be achieved by using N processors is:

$$SpeedUp(P, N) = \frac{1}{(1-P) + \frac{P}{N}}$$

As N tends to infinity, the maximum speedup tends to $1 / (1 - P)$.

Atentie daca zice ca avem un infinit de procesoare!!!!! -> 1/(1-P)

P = fractiunea de cod paralelizabila

0. Descrieti folosind PMS structura cu care ati lucrat la laborator.

Are cineva idee unde gasesc rezolvarea la asta?

in ce curs se gaseste clusterul descris ?

Nu se găsește în curs, you have to think it. E similar cu ce e în cursul 2, cred.

1. Organizarea datelor in structura SIMD si in MIMD.

curs7: 18-22

MIMD curs 9 - pg 1-3

La MIMD stie cineva ce trebuie sa scriem?

Ar fi practic modurile de organizare - pe linii, pe coloane, deplasată. Diferența la MIMD e că lucrează cu matrici, nu cu vectori, și aici poți să zici despre faza cu reprezentarea matricilor (care sunt 2D) în memorie (care sunt 1D) - row-major, column-major, în funcție de limbaj

2. Sa se scrie programul de insumare a elementelor unei matrice A(n, n) pentru o structura SIMD si pentru o structura MIMD. Comentati performantele. Obs. .Trebuie specificata oranizarea datelor

K Organigrama MG (KML poate) (MG de la magistrala, vrea sa zica Kbus aici) de la SBC-uri

Curs 3a - pag 6

16. Descrieti caracteristicile generale CELL

3. Sa se cupleze 16 procesoare cu 16 memorii utilizand retea de tip Delta. Indicati conexiunea de la procesorul 2 la memoria 6.

Trebuie 4^2 cu 4^2 sa legi acele procesoare de memorii prin niste nivele de permutari. Si ideea e sa aplici shuffle din nivel in nivel.

De ce nu merge 4x4 cu 4x4? Daca faci 4x2 ai prea putine iesiri.

16 procesoare cu 16 memorii, adica 16 in, 16 out, adica 4^2 (a^n) in, 4^2 (b^n) out.

Asta inseamna ca avem 2 nivele si folosim CB-uri de 4x4 (a*b).

4x4 cu doua nivele si folosesti reprezentare in baza 4. Cred ca e ok asa

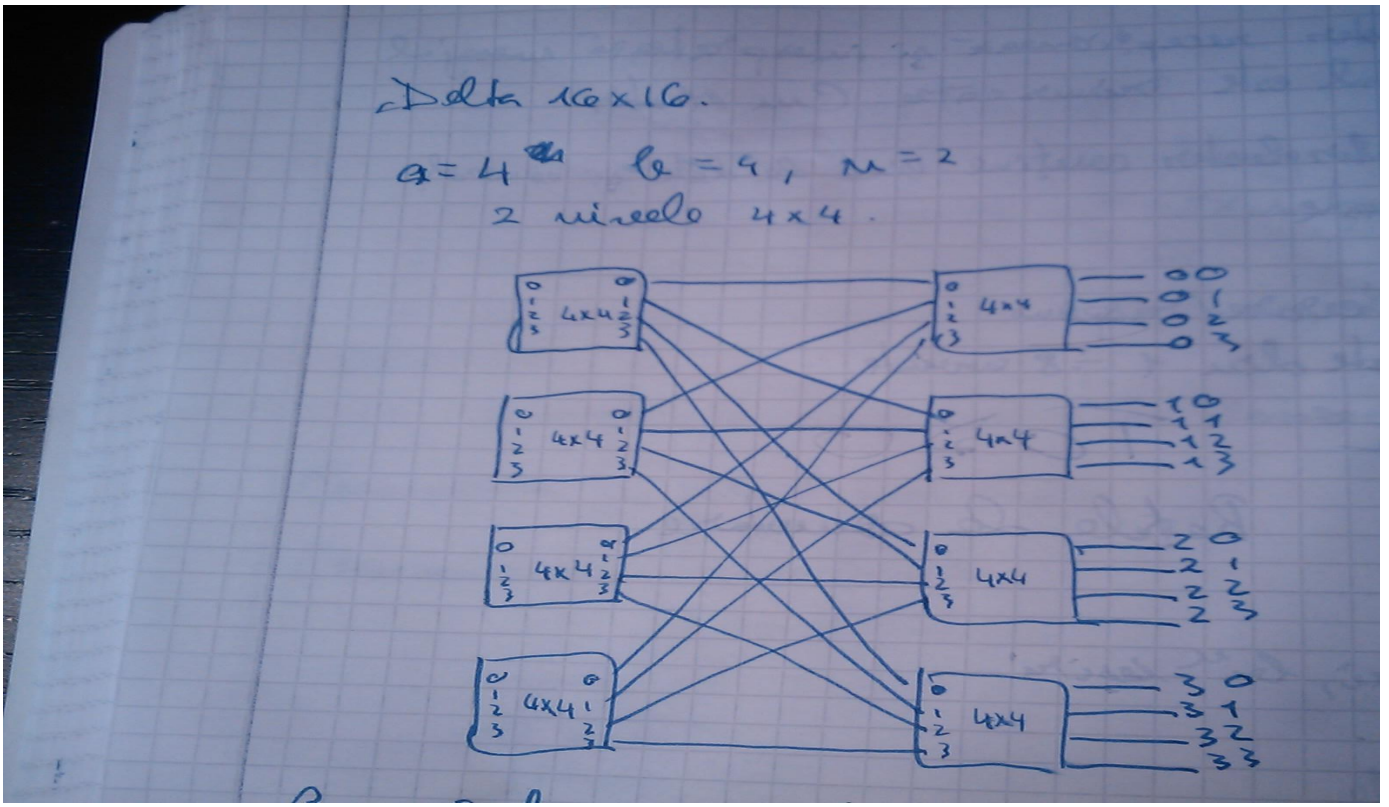
In curs vad ca e cu 4^2, adica n ar trebui sa fie mai mic ca a

cum indici conexiunea? adica ce trebuie sa facem exact?

Mai este si varianta asta :

3'. Sa se cupleze 16 procesoare la 16 memorii ulitizand o retea de tip Delta. Prin default procesorul i sa fie conectat la memoria i.

Poate pune cineva un desen sau sa explice cum se fac exact legaturile ? Trebuie luate module tot 4x4 , e necesar un shuffle ?

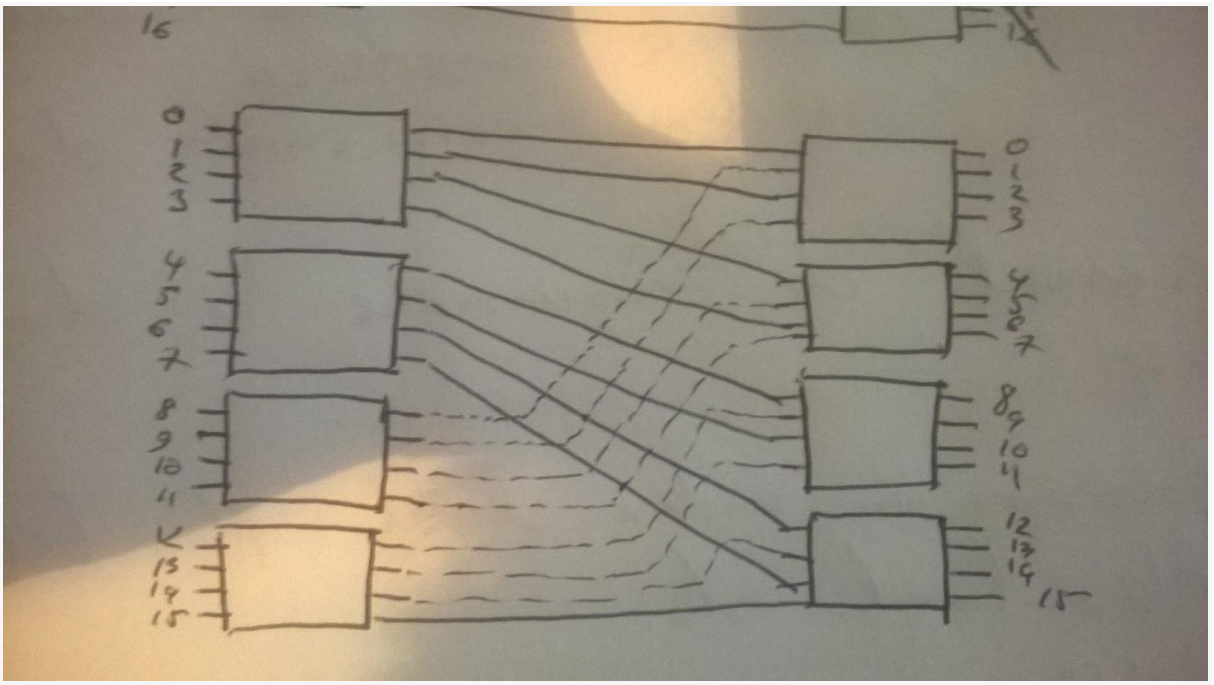


Ceva de genul

Actually, pentru că e putere a lui 2 (4x4 = 16 = putere a lui 2) cică ar trebui să faci shuffle, imediat pun și poza

Pai si matele alea acolo in mijloc nu-s shuffle? :))

Nu, ce ai făcut acolo e legătură 1 la 1 (legăturile de la primul switch se duc către prima legătură de la fiecare switch, etc). Pun imd poza



Ceva de genul ăsta. Legi la rând legăturile la valorile pare și după le pui și pe restu
Nu merge poza a 2-a. Nu poți conecta intrarea 0 cu ieșirea 15, spre exemplu. Poza 1 e ok. da..doh

Source?
Octa's Wisdom. :)) Și-a notat el la curs chestia asta că a zis-o proful.
much skill, very gresit => Care to explain why ?
asta e perfect shuffle, tu aici trebuie lucrezi in baza 4 - > e ce am făcut si eu :))

Actually e buna prima poza. daca vrei sa selectezi memoria 4 (10 -> 1 din fiecare modul de niv 0 duce in modulul al doilea de niv 1; apoi alegi iesirea 0 din acel modul si iti da 10) se merge pe 1 pe nivelul 0 si pe 0 pe nivelul 1.

Cred ca e perfect shuffle in baza 4. :D. Dap. 00 -> 00, 01 -> 10, 02 -> 20 etc etc
Yep. :)
Si cum se face rutarea in a doua poza? ^^^ asta incercam sa zic. tenks

aici e un link unde este descrisa destul de bine rutarea prin retele delta: link

4. Descrieti primitiva P la calculatorul personal
- Citeste, interpreteaza si executa instructiuni
- In acest proces are loc generarea comenzilor spre toate resursele sistemului + citirea starilor acestora

Atribute:
- Functie: universal
- Implementare: microprogramata ?? ["Intel Sandy Bridge: hardwired pipelined CISC (x86) machine (with some microcode support)"; https://inst.eecs.berkeley.edu/~cs152/sp12/lectures/L02-SimpleImps.pdf]
- Formatul instructiunilor: variabil (de obicei sunt arhitecturi CISC)
- Lungime instructiuni: 64 biti (eu cred că e 1 octet -> mulți (câți?) octeți) (cred ca instructiunile sunt pe 8-32 biti sau ceva) [la 8086 nu erau 1 octet - 6 octeti?] [nu stiu daca este vorba despre asta http://en.wikipedia.org/wiki/Advanced_Vector_Extensions, atunci ar fi 256 biti de la sandy bridge]
- conform http://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-instruction-set-reference-manual-325383.pdf, sunt lungimi variabile pt instrucțiuni și în Intel i7. Eu cred că un răspuns gen "lungimea variabilă, de la instrucțiuni scurte pe 1 octet, până la instrucțiuni complexe pe mai mulți octeți" ar merge.
- Ciclu instructiune: **ciclu varibil** (S cicli masina)
- Tehnologie: VLSI Very Large Scale Integration

5. Considerand o structura multiprocesor pe o magistrala comuna, descrieti unitatea de acces la magistrala sistemului. AKA Kbus curs 3a,

6. Structura generala a Kmap-ului. curs 10

7. Clasificarea Flynn a sistemelor de calcul. curs 7

8. Comunicatia intercluster in structurile MIMD slab cuplate. curs 9 defapt curs 10 -> Aici nu este de fapt ASC-Curs10.ppt slide-ul 26 ? Comunicarea dintre Kmap-uri practic

9. Descrieti caracteristicile generale CELL
Laborator de CELL.
Aici merge curs 5, slide 19.

10. Retea de comutare de tip intercalare perfecta si comparatie cu reteaua cu ordine inversa curs 8

11. Descrieti folosind PMS structura cu care ati lucrat la laborator.

Are cineva idee unde gasesc rezolvarea la asta?
Bănuiala mea ar fi să faci ce e în cursul 2 (elementele + whatever) + proprietățile de la CELL. Găsești pe wikipedia mai detaliat despre el

12. Comparatie/ generala intre SIMD si MIMD

13. Comunicatia intercluster in MIMD slab cuplate curs 9
http://cs.curs.pub.ro/wiki/asc/asc:extra:comutatoare

14. Comparatie intre intercalarea perfecta si reteaua butterfly curs8

15. Sa se scrie programul de impartire a doi vectori de dimensiune n pe o structura si SIMD si MIMD. Comentati performantele.

SIMD
Fie vectorii u si v si distribuim u[k] si v[k] in modulul de memorie k. Fiecare procesor imparte cele 2 elemente corespunzatore din modulul sau.
w[k] = u[k] / v[k] (0 <= k <= n - 1) k - indexul procesorului, n - numarul total de procesoare
O (1)
Nu e complet, trebuie tinut seama de impartire la 0 (eventual cu masti).
MIMD

Acelasi lucru, doar ca faci cu fork si join, e prin cursul 6.

16. Comutatorul de tip trunchi K ierarhic - curs 3b
http://cs.curs.pub.ro/wiki/asc/asc:extra:comutatoare

[Daca aveti completari/corecturi, postati-le cu alta culoare sub intrebare]

1.Organizarea datelor in structurile SIMD/MIMD``
Organizarea datelor in structura SIMD si in MIMD.
Comparatie SIMD si MIMD dpdv al structurii si organizarii datelor
Organizarea datelor la SIMD
Reprezentarea datelor pe SIMD si MIMD
2 x Structurarea datelor in sisteme SIMD
SIMD curs 7 - pg 9-11
MIMD curs 9 - pg 1-3? + slide 8

2. Sa se scie programul de impartire a doi vectori de dimensiune n pe o structura si SIMD si MIMD. Comentati performantele.`
Adunarea elementelor a doi vectori in SIMD si MIMD (cod + cum retii datele)
Program impartire vectori SIMD + MIMD, comparatii intre ele
Sa se scrie programul de insumare a elementelor unei matrice A(n, n) pentru o structura SIMD si pentru o structura MIMD. Comentati performantele. Obs. Trebuie specificata organizarea datelor.
Sa se scrie programul SIMD si MIMD care aduna 2 vectori (pe elemente). Sa se analizeze performantele. Specificati organizarea datelor
Sa se scrie programul ce determina produsul elementelor unei matrice pe SIMD si MIMD. Sa se analizeze performanta. Reprezentarea datelor.
Adunare elemente dintr-o matrice cod SIMD MIMD plus organizarea datelor si concluzii
2 x Sa se scrie programul SIMD si MIMD care aduna 2 vectori (pe elemente). Sa se analizeze performantele. Specificati organizarea datelor
2 x Inmultirea a doi vectori an si bn in SIMD si MIMD unde c[i]=a[i]*b[i]
Se cere implementarea unui algoritm ce determina maximul de pe coloane pentru o matrice A[n x n] pe o masina SIMD
Inmultirea matricelor
curs 7, slide 8, 10
curs 7, slide 18

3. Sa se cupleze 16 procesoare la 16 memorii ulitizand o retea de tip Delta. Prin default procesorul i sa fie conectat la memoria i.`
Sa se cupleze 16 procesoare cu 16 memorii utilizand retea de tip Delta. Indicati conexiunea de la procesorul 2 la memoria 6.
Retea delta 16x16, default i , memory i
Retea Delta 16x9 (dimensiunea nu o mai stiu exact)
Sa se proiecteze o retea de comutare de tip delta intre 4 procesoare si 9 module de memorie
Sa se proiecteze o retea de comutatie Delta cu 16 procesoare si 16 memorii.
Retea delta 16x9

5 x Sa se realizeze o retea de comutare Delta intre 16/25 procesoare si 16/25 memorii (procesorul i - memoria i)
2 x Sa se proiecteze o retea de comutare de tip delta intre 16 procesoare si 9 module de memorie (specificati calea intre procesorul 8 si memoria 3)
curs 12, slide 7

Pentru exemplul de 16 P x 9 M din curs a inteles cineva cum se fac legaturile dintre module ?
De la primul cross bar de pe nivelul 0, duci legaturi la toate 0-urile de pe nivelul 1, de la al doilea duci la toate 1-rile de pe nivelul 1 si tot asa

-4. Considerand o structura multiprocesor pe un bus comun, descrieti unitatea de acces la magistrala sistemului al fiecarui modul.`
Considerand o structura multiprocesor pe o magistrala comuna, descrieti unitatea de acces la magistrala sistemului.
Considerand o structura multiprocesor pe o magistrala comuna descrieti unitatea de acces la memoria locala\
Diagrama KBUS
Organigrama Kbus
Kmag (sch logica)
Curs 3a - pag 5

-5. Structura generala Kmap``
Structura generala a Kmap-ului.
Sructura Kmap
Kmap - descriere generala
Diagrama Kmap
Structura generala a Kmap-ului
curs 10 (cam paginile 4,5)

-6. -Comunicatia intercluster in MIMD slab cuplate.`
Comunicatia intercluster in structurile MIMD slab cuplate.
Comunicatia intercluster in MIMD slab cuplate
Comunica?ia intercluster intr-un sistem MIMD slab cuplat
3 x Comunicata intercluster in structurile MIMD slab cuplate
2 x Caracteristicile principale ale sistemelor strans cuplate si slab cuplate
curs 9 slide 22, 23

-Comunicatia intercluster
Comunicatia intercluster
Comunicarea intercluster
mec de referinta intercluster
curs 10 slide 26,27,28,29

7. Comparatie intre intercalarea perfecta si reteaua butterfly``
Retea de comutare de tip intercalare perfecta si comparatie cu reteaua cu ordine inversa.
Retea de comutare de tip interclasare perfecta

Comparatie permutare baza, permutare perfecta
Subiect cu permutari (parca era descrierea permutarilor de tip perfect shuffle)
Comparatie intre intercalare perfecta si butterfly
Comparatie intercalare perfecta si permutarea de baza
3 x Retea de comutare de tip intercalare perfecta
Comparatie intre permutarea de baza si intercalarea perfecta
Permutare fundamentala de tip intercalare perfecta
Curs 8

-8. Comutatorul de tip trunchi K ierarhic``
Trunchi K ierarhic si neierarhic
Comutator de trunchi K ierarhic si neierarhic
Switch K ierarhic si neierarhic
3 x Comutatorul de tip trunchi K ierarhic si neierarhic
Comparatie intre comutatorul Cross-Bar si Trunchi K
curs 3b, slide 7 (ierarhic), 14(neierarhic)

-9. Comparatie generala intre SIMD si MIMD``
Comparatie SIMD MIMD la algoritmi
Comparatia generala intre SIMD si MIMD
Comparatie generala SIMD MIMD
2 x Comparatia generala intre SIMD si MIMD
link
http://www.quora.com/Parallel-Computing/What-is-the-difference-between-SIMD-and-MIMD
link 2 http://cmsc411.com/simd-vs-mimd

10. Descrieti folosind PMS structura cu care ati lucrat la laborator.
Descrieti in metoda PMS resursele de la laborator.
Structura PMS, descriere generala. Prezentare calculator propriu cu primitive pms
Structura PMS
Exemplificati primitivele P si M de pe calculatorul propriu
Descrie calculatorul tau folosind primitive PMS
3 x Structura PMS la ce se intampla in laborator
2 x Definiti PMS si dati exemple de primitive
2 x Descrieti primitiva M (memorie) utillizand PMS
Descrieti primitiva P la calculatorul personal.
---curs 2, pag 7,8,9

aici pentru structura de cluster cred ca putem lua ce e in cursul 9, nu bag mana in foc

-11. Clasificarea Flynn a sistemelor de calcul.``
Clasificarea Flynn a sistemelor de calcul
Clasificarea Flynn a sistemelor de calcul
Clasificarea calc Flynn
2 x Clasificarea Flynn a sistemelor de calcul

2 x Clasificarea sistemelor de calcul avand in vedere fluxul de instructiuni si fluxul de date: SISD, SIMD, etc.
curs 6 pag 8,9,10

-12. Implementarea salturilor conditionate la SIMD si MIMD ``
Implementarea salturilor conditionate in SIMD si MIMD - comparatie
Comparatie salturi conditionate SIMD MIMD
5 x Implementarea salturilor conditionate in structura SIMD si in MIMD (comparatie)
SIMD - curs 7, slide 12
MIMD - sunt procesoare independente, deci practic salturile mimd ar trebui sa se execute normal, fiecare procesor ruleaza bucata lui de cod independent

13. Comparatie cross-bar si delta ``
Compara?ie RC Cross Bar si RC Delta
3 x Sa se calculeze probabilitatea de satisfacere a unei cereri intr-un switch cross-bar cu p procesoare si m memorii (1/(p*m) ?)
curs 3b , slide 6 si 9
curs 12 , slide 18(cross bar) slide 23(Delta)

-**14**.

- **15**. Schema logica Kml
Organigrama KML de la SBC-uri.
Organigrama KMG de la SBC-uri
Curs 3a - pag 6

16. Descrieti caracteristicile generale CELL``
linkF curs 5(?)

Curs1

Sistem de calcul:
P = elemente de procesare
M = elemente de memorie
L = elemente de interconectare tip magistrala
K = unitati de comanda
D = operatori de date
S = switchuri de interconectare
T = terminale

-Arhitectura masinii von neumann
UCP = {RG, UAL, Ucmd}
UC = {UCP, M}
MB = {UC, I/E}
SC = {MB, EP, soft:)}

evolutia spre SIMD, masini vectoriale, cu UAL multiple
UAL s-a extins catre coprocesorul matematic

Ucmd:
    -cicluri pe instructiune la RISC si CISC
    -codificarea instructiunilor
    -modul de adresare

subsistemul I/E
    -asigura o interfata electrica intr EP si UCP

Transfer programat
    -MEM-UCP-IE cu intreruperi intre ucp si ie
    -UCP se ocupa de fiecare caracter, Driverul din memorie de transfer
    Sincronizarea:
        -busy waiting, UCP lucreaza la viteza EP
        -intreruperi la start si stop, UCP nu lucreaza la viteza EP

Acces direct la memorie prin DMA
    -MEM-UCP-IE cu DMA intre MEM si IE (SI mentinut)
    -transfer fara interventia UCP, doar la start si stop, prin intreruperi
    -UCP initiaza sensul, datele si adresele
    -erorile sunt tratate de CPU
    -CPU nu poate initia alt transfer pana nu se termina cel curent
    -accesul CPU si DMA la memorie se face prin cycle stealing si burst
    -DMA prioritar, hardware

Canal de IE
    -ca la DMA, dar cu Canal de IE (SI mentinut)
    -CPU da adresa de inceput a programului din memorie
    -CPU poate inlantui transferuri

-Canalul se ocupa de erori

Calculator de IE
-interfata intre un sistem de calcul si mai multi utilizatori
-are functiile de transfer programat, dma si canal de IE
-in sisteme multiprocesor, nu iroseste resursele procesoarelor

---

CURS2

PMS
-primitivele PMS= atribut: valoare
M: rol, functii, tehnologie, operatii, lungime cuvant, mod acces, capacitate, timp acces
Functie: primara | auxiliara
Tehnologie: bipolara | MOS | statica | dinamica
Operatii: citire | citire/scriere
Mod acces: aleator | secvential | FIFO | LIFO | asociativ
Lungime cuvant: 8+1 | 16+2 | 32+4 | 64+8
Capacitatea: 4Mb | 8Mb | 16Mb | …| 1Gb | 2Gb | 4Gb
Ciclu de lucru: 2:4ms | …| 400ns

P: citeste, interpreteaza, executa instructiuni
functie, implementare (convent, uprogram), format instructiuni (fix, var, 80fp), cicli/instr, tech LSI, VLSI
Functie: universal | specializat (de semnale, etc)
Implementarea: conventionala | mprogramata | mprogramata specializata
Formatul instructiunilor: fix | variabil
Lungimea instructiunilor: 8 | 16 | 32 | 64 | 128 biti
Ciclu instructiune: ciclu fix | ciclu varibil (S cicli masina)
Tehnologia: LSI | VLSI |

S: m intrari, n iesiri, simplex, semiduplesx, duplex, paralelism min(m,n), concurenta C, (ne)ierarhic
Structura: ierarhica | neierarhica
Tip: simplex | semiduplex | duplex (e vorba de sensuri)
Numar de legaturi realizate:
m intrari | n iesiri din S
n intrari | m iesiri din S
Concurenta: c

L: legatura intre componente, bus sau intf ser/par, lungimea cuv, (a)sync, standard (acces serial, printr-un token sau un numarator da accesul pe rand la magistrala)
Functia: legatura seriala | legatura paralela | legatura de tip magistrala
Lungimea cuvantului: 5 | 6 | 7 | 8 | 16 | 32 | 64 + comanda

Mod de dialog: sincron | asincron | cu Q&A: cu/fara interblocare sau cu interblocare completa | fara A

Mod de control al accesului: inlantuire seriala (token) | interogare | cereri independente

Standard: paralel (SCSI) | serial (RS232; 485; 482) | magistrala (MultiBus; MCI(IBM); HPIBus; etc)

K: control, comanda resurse, implementare (conv, uprog), n stari

Pi/e: rata de transfer, intreruperi, rezolvarea accesului la memorie

D: operatii aritmetice, fixa si mobila, module independente, vectorial, matriceal, conventional/proc specializat, structuri de date asupra carora opereaza

Functie: prelucrare in virgula mobila | zecimala(BCD) | vectoriala | matriceala | cu liste

Tip implementare: conventionala | procesor specializat (coprocesor matematic)

Structuri de date asupra carora opereaza: scalari | vectori | matrice | liste

Operatii: + | - | * | / | cautare atomica | memorare/extragere atomica

T: adaptarea electrica a datelor, T sau (T,K,S)

Structura von neumann a PMS:

M - P - T

M - D - T
    |
    K

M - D - T
| | |
Km Md Kt
    |
    Kp

Md = registre generale

Structura multiprocesor: S intre P si M cu legatura la alt S intre Pi/e si T

Comutatoare ierarhice: componente diferite, neierarhice: aceleasi componente

S={poarta, 1a, 1b}

S {poarta, 1a, 1b, comutare la b} a - L - S - b

S {poarta, 1a, 1b, comutare la a} a - S - L - b

S {poarta, 1a, 1b, cu legatura la a si b} a - L - S - L - b

S poate conecta o resursa la magistrala sau o magistrala la alta magistrala

Comutatoare duplex: 1a, ++b
S {poarta, 1a, nb, concurenta 1, 1Sp} [name, ins, outs, simultaneous, distinct tracks]

Comutatoare dual duplex: 2a, ++b
S {poarta, 2a, nb, concurenta 2, 2nSp}

Comutatoare legatura multipla == Cross-Bar Switch
S {CB, ma, nb, c=min(m,n), m*nSp}

Comutatoare de tip trunchi K
S {TK, ma, nb, c=k, k(m+n)Sp}
8 proc, 4 mem, max 4 acc la mem, k=4 enough

CURS3

Comutatoare neierarhice:

Comutator duplex neierarhic (toate elem conectate pe o magistrala la S)
S {Dn, na, c=1, nSp}

Comutator legatura multipla - tip central (diag inf, graf neorientat, half duplex) S la fiecare intersectie
S {lm-neierarhica, na, c=n/2, n(n-1)/2 Sp}

Comutator trunchi-k
S {tk, na, c=min(k,n), k*n Sp) (diag inf, S la fiecare intersectie, conectare m vert la k oriz)

Comutatoare procesor-memorie Spm

intre P si M: linia de adrese (Radr), 2 linii de date (Rdata)
intre P si M: linia de adrese (Radr), 1 linie de date cu doua buffere bidirectionale =b--date--b=
intre P si M: 1 magistrala, cu MUX A/D la memorie + latch (adr), buffer 2 dir la proc

Localitatea spatiala -> cache, temporala -> registri
Localitate de instructiuni -> spatiala si temporala simultan

cache miss overhead = latency + block_size/bw
cache compus din seturi de blocuri
hit time = timp de accesare a cache-ului

page fault mult mai costisitor, de preferat non-blocante
cache block
-mic: low locality

    -mare: high miss penalty

mapare directa: un bloc poate fi pozitionat intr-o singura pozitie in cache
mapare total asociativa: un bloc poate fi pozitionat oriunde in cache
mapare set asociativa: un bloc poate fi pozitionat intr-un set de pozitii
seturi mari == trashing mic

index for set, tag for block, offset for word
Cache mapat direct: simplu, rapid, bit de validare pe fiecare linie, putin hw, trashing

cache mapat set asociativ: se identifica setul dupa index si se compara apoi tagurile

cache mapat complet asociativ: indexarea se face dupa bitii de index si de tag simultan

block replacement for set: random, LRU, FIFO

cache-ul face citirile rapide, nu scrierile
    citirea se poate face prin compararea simultana a tagurilor
    scrierea nu poate fi simultana cu compararea; first comp, then write
    write-through: simultan in cache si mem; easrier, consistency [multicore]
    write-back: write in mem only if modified or swapped, dirty bit; faster, cache speed, writes larger blocks; [multiproc]

write miss
    write allocate: aduce blocul din memorie si apoi il prelucreaza [write-back]
    no-write allocate: nu aduce blocul, se lucreaza cu el in memorie [blocurile scrise nu ajung in cache->write-through]

CURS4

Imbunatatirea performantelor memoriei, reduce: penalty, miss rate, hit/miss decision time
row-major, column-major

La vectori dinamici, vectori N-dimensionali sunt vectori N-1 dimensionali de vectori 1-dimensionali
Liste: alocarea dinamica a fiecarui element, cache miss la fiecare next
    -implementarea listelor ca vectori

masurarea dimensiunii liniei cache-ului: for cu step 1...L...

Maparea cache-urilor:
    -directa pentru cele apropate de procesor
    -asociativa pentru cele departate de procesor

Diferentierea cache-urilor:
    -separate pentru date si instructiuni aproape de procesor
    -unificate in rest

Cache-urile pot fi neblocante daca la un miss instructiunea urmatoare nu depinde de load-ul care a cauza miss-ul.

Timpul de acces la memorie: hit time + miss rate * miss penalty

Reprezentarea PMS a unei structuri multiprocesor pe o magistrala comuna
    -SBC: single board computer: procesor, memorie, IO, conectat la o magistrala de adrese, date si comenzi
        -Kbus==Kmag: unitatea de control a accesului la magistrala
        -Kml: unitate de control a accesului la memoria locala
        -Ks: control al intregului sistem
        -Kld: unitatea de comanda a memoriei locale (refresh?)

        -fiecare proc vede memoria locala de la 0 si ale celorlalte mapate in spatii superioare
    -PMS in SBC
        Pl: procesor local: furnizeaza date si adrese
            Tt: interfata cu ceasul in timp real
            Ti: interfata sistemului de intreruperi
            Ts: interfata seriala
            Tp: interfata paralela
        Ml: memoria poate fi accesata fie de Pl fie de alte procesoare, printr-un S
        Sd1, Sd2, S-uri locale de date, separa procesoarele intre ele
        Sa0, Sa1 - S-uri interne de adrese
        Sa2, Sa3 - S-uri externe de adrese
        Sc - S prin care se transmit comenzi catre exterior
    -Kbus
        Starea A: adresa locala, se acceseaza local, este starea B:
        Starea B: cerere de acces la magistrala, cerere acceptata, starea C
        Starea C-D: bus busy, scrierea burst
        Starea D: se ramane in ea daca mai e nevoie de cicli

        Kbus reprezinta procesorul local in relatie cu magistrala externa

    -Kml
        Asigura accesul la
            -memoria locala a Pl prin Sd1 si Sa2
            -mem procesoarelor externe prin Sd2 si Sa3
            -Pl are prioritate fata de procesoarele externe Pext

Starea A: comenzi locale
Starea B: acces extern la memorie
memoria locala e vazuta de fiecare proc in acelasi spatiu de memorie iar de procesoarele externe intr-un spatiu complementar, unitate de translatare a adreselor
Magistrala ocupata-semn ca alte procesoare lucreaza din moment ce nu cunoastem ciclii lor., abordare top-down
cicleaza in magistrala ocupata

CURS5

Paralelism la diferite niveluri
Cub cu procesoare, biti, frecventa
paralelism la nivel de job, hardware (ultim), program(cod, bucle), instructiune (serie, paralel intre citire si exec)
mecanism de predictie a salturilor

Clasificarea Flynn:
-sisteme matriceale, vectoriale
-sisteme multiprocesor, retea strans sau slab cuplata
-sisteme pipeline, banda de asamblare (RISC)

Taxonomia Flynn:
imparte sistemele in functie de fluxul de instructiuni si fluxul de date
-SISD (von neumann), SIMD, MISD, MIMD

-SISD: Ucmd---Finstr---P---Fdate---Mem
-SIMD: masini vectoriale: Ucmd---(split Finstr)---P1..n---...---M1...n
-MISD: Ucmd1..n---...---P1...n---(splitback)---Mem: diferiti algoritmi pe aceleasi date (data mining?)
-MIMD: Ucmd1..n---...---P1...n---...---Mem1...n
pot comunica P-P sau P-M
-strans cuplate, memorie partajata: OpenMP
-slab cuplate, memorie distribuita: MPI

matrix mutiply: SISD (3for), SIMD (whole last for), MIMD (for FORK; JOIN)
MIMD: master, fiecare proc cu FI propriu
FORK, JOIN: legatura dintre nr de procese si nr de procesoare

---
CURS6

SIMD: paralelism n/2, executie seriala, mascare
memoria face o citire/ciclu
n citiri din n module / ciclu
operanzi de plasat in module diferite

depl - diag, locindex[i]

CURS8
---
interconectare SIMD: inel, matrice
permutations
            k sup+intre submultimi, k inf in cadrul submultimii
    base line perm 0->n-1 + k
    shuffle perm n-1->0; q#c pt !putere2, q*c resurse
P_shuffle_q#c(adr) = q*Vadr mod (q*c-1) pt 0...<n-1 + k
        n-1 ramane pe loc
    negate bit permutation + k
    butterfly (interschimba n-1 cu 0), perm simetrica + k
    bit reversal + k
    increment perm + 1 mod 2^n

CURS9

MIMD pot comunica P-P sau P-M
    -multiprocesor
    -multicalculator

acces la memorie
    -mem partajata:
        -UMA: orice proc se poate conecta la un modul de memorie
        -NUMA: clustere conectate la un Global Interconnect Network cu
acces la Global Shared Memory
            intern, procesoarele sunt conectate la Cluster Shared
Memories
        -CC-NUMA
        -COMA: Proc--Cache--Data--Network
    -mem distribuita

Sisteme
    -strans cuplate: Proc, mem locale, magistrala rapida de
interconectare care ofera rata de transfer similara cu accesul la
memoria locala (yeah, right)
    -slab cuplate: tip cluster, fiecare modul are procesor, memorie
locala, IO si un mecanism de management al coliziunilor de pe
magistrala de transfer de mesaje

CM*
    un cluster format din Cm-uri, Kmap si map bus
    bottleneck-ul intr-un cluster: Map bus

CURS10

Kmap prelucreaza cererile de acces la memorie si decide daca adresa e locala sau globala, read-only/write si daca e nevoie de translatare
   arbitreaza comunicatia intre Cm*-urile din acelasi cluster si intre clustere prin intercluster bus

Componente: Kbus (tranzactii pe magistrala intra-cluster), Pmap (proc pentru translatari de adrese), Link (legatura cu intercluster)

CURS11

Mesaje intercluster:
    -mesaj direct:
        adresa cluster destinatie, adr clust sursa, numar context, camp indirectare, data (scriere)
    -mesaj raspuns:
        adresa cluster destinatie, camp nefolosit, numar context, camp de indirectare, data (read)
acces la memorie prin nume segment si drepturi de acces
Capabilitati:
    -mascare de biti (comparatie intre sirul stocat si cel al userului)
    -liste
    -criptare locala (parola)

MIMD:
    Honeywell: Central Processor, Command Switch, Mem, IO (2 din fiecare CS)
    Comutatoare elementare: 2ins, 2outs, 2cmd

CURS12

Retele de comutare ierarhice:
    n/2 (2x2) * 2 (n/2 x n/2) * n/2 (2x2)

Propun sa postam aici rezolvari la subiectele pe care le gasim / le-am gasit.
// TODO: rezolvari

https://drive.google.com/file/d/0B_IrjBH5rVDYczZyWHcyY0JaS28/edit?usp=sharing
^ curs complet (sa dati cu ctrl-f prin el) (**nu merge**)
curs complet seria CC (diferit de cel de mai sus)

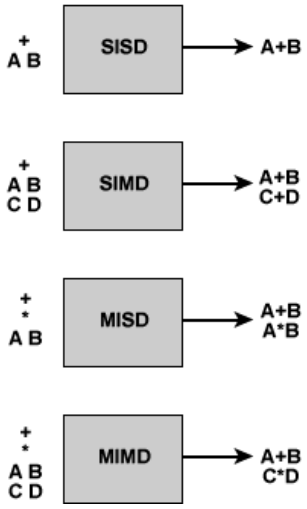
Pot sa sterg tot?
NU

Picam toti.-> Sa speram

LEBAR

Teorie:
SIMD vs MIMD
9) Clasificarea Flynn a sistemelor de calcul
--Taxonomia lui Flynn
• Impartirea sistemelor de calcul in functie de:
– Fluxul de Instructiuni – secventa de instructiuni executate de procesor
– Fluxul de Date – secventa de operanzi manipulata de procesor
• Bazate pe acest criteriu se desprind:
• I – SISD = Single Instruction Single Data Stream (structura
Von Neumann)
• II – SIMD = Single Instruction Multiple Data Stream
• III – MISD = Multiple Instruction Single Data Stream
• IV – MIMD = Multiple Instruction Multiple Data Stream

Flynn.
Capabilitatile celor 4 clase de sisteme de calcul.



1) Implementarea salturilor conditionate in SIMD si MIMD - comparatie

La MIMD e ca si cum ai rula mai multe thread-uri. Ideea e ca fiecare core are propriul program counter, flag-uri, etc. (o parte de registre proprii) asa ca salturi se petrec ca si cum ar fi un procesor separat fiecare core.

La SIMD, ai o inst, de genu' cum erau spu_add(), numai ca atunci cand "faci if" se activeaza un flag daca conditia a fost sau nu adevarat pt fiecare data. De exemplu poate exista inst m = spu_greater_or_equal(v, 0). Ei in urma executarii in m o sa existe pentru fiecare element din v, rezultatul operatiei, adica pt fiecare element, in m o sa fie o valoare diferita de zero pentru true, si 0 pentru false. Acum poti sa folosesti alta instructiune SIMD care il primeste pe m si executa ceva pe un core sau nu, daca elem specific din m e diferit de 0.

Modelul abstractizat la curs, e ca lebarul in ploaie niste flag-uri intr-un registru, si fiecare core primeste bit-ul sau, in loc de bytes intregi de date

CatalinV

Bonus: CLSS A, i; CEQL A, i; CMP i; LDMSK i; STMSK i;

8) Comparatia generala intre SIMD si MIMD

SIMD e de obicei mai ieftin (+ less hardware)

MIMD e mai flexibil, se preteaza la o clasa mai larga de probleme

SIMD nu merge bine pe cod cu multe branchuri

(ziceti si de definitie)

11) Reprezentarea datelor pe SIMD si MIMD

MIMD -> memorii separate , UMA, NUMA,<CC-NUMA>, COMA

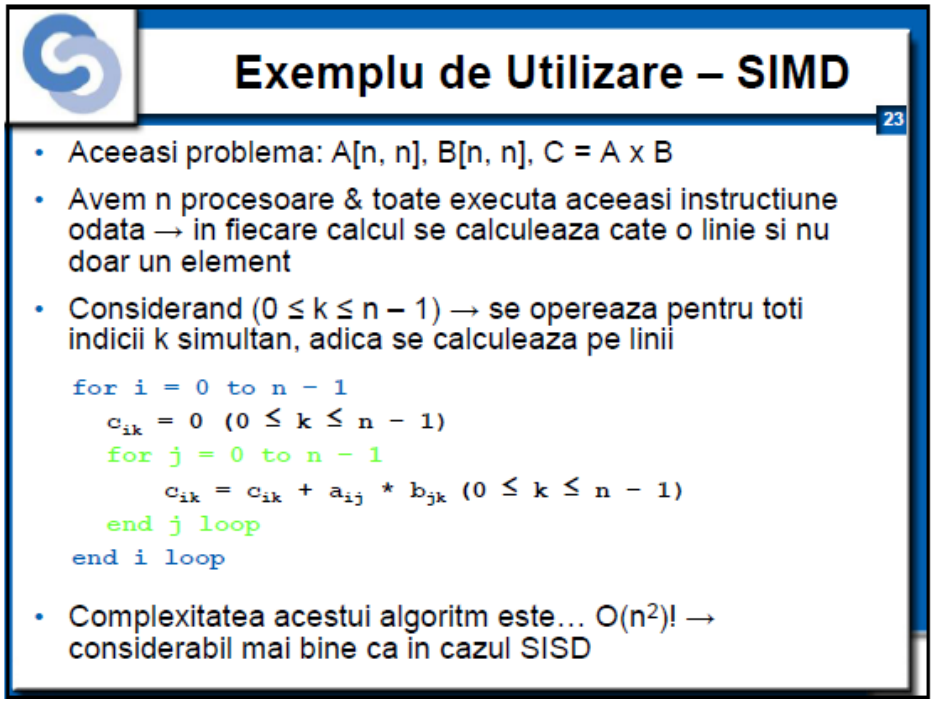
SIMD -> memorie comuna, procesare batch, < bus larg >

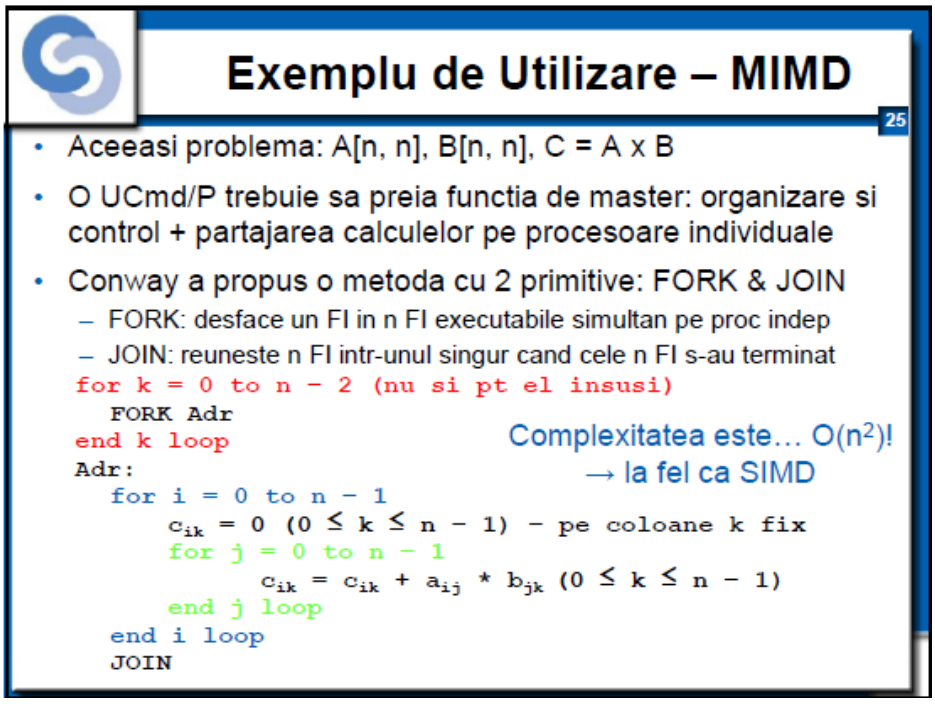
79) Comparatie SIMD si MIMD dpdv al structurii si organizarii datelor (nu garantez ca imi amintesc bine)
-- ca 11

28.Organizarea datelor in SIMD + Realizarea inmultirii unei matrici cu ea insasi
Organizarea datelor SIMD: cand consideram matricea ca fiind pe linii/coloane. Limbajele de programare care folosesc reprezentarea pe linii sunt C/C++, Java. Limbajele care folosesc reprezentarea pe coloane sunt: Fortran, Matlab (TODO: de completat daca mai gasesc altele)
Realizarea inmultirii unei matrici cu ea insasi : pseudocod instr simd

40. Comparatie SIMD MIMD la algoritmi

aici merge ex din curs cu inmultirea matricilor.. nimic fancy





Exemplu de Utilizare – MIMD

- Aceeasi problema: A[n, n], B[n, n], C = A x B
- O UCmd/P trebuie sa preia functia de master: organizare si control + partajarea calculelor pe procesoare individuale
- Conway a propus o metoda cu 2 primitive: FORK & JOIN
  - FORK: desface un FI in n FI executabile simultan pe proc indep
  - JOIN: reuneste n FI intr-unul singur cand cele n FI s-au terminat

```
for k = 0 to n - 2 (nu si pt el insusi)
   FORK Adr
end k loop
Adr:
   for i = 0 to n - 1
       c_ik = 0 (0 ≤ k ≤ n - 1) - pe coloane k fix
       for j = 0 to n - 1
             c_ik = c_ik + a_ij * b_jk (0 ≤ k ≤ n - 1)
       end j loop
   end i loop
   JOIN
```

Complexitatea este… $O(n^2)$!
→ la fel ca SIMD

Sa se scrie program SIMD si MIMD

47. program impartire vectori SIMD + MIMD, comparatii intre ele

52. Comparatie SIMD vs SISD pe exemplu de cod.

Inmultirea a doua matrice: O(N^3) -> SISD vs O(n^2) -> SIMD

66) MIMD, Exemplu pe progr cu dependinte de date

Aflarea diametrului unui graf.

Calculul mediei aritmetice a elementelor unui vector.

78) Adunarea elementelor a doi vectori in SIMD si MIMD (cod + cum retii datele)

6) Sa se scrie programul SIMD si MIMD care aduna 2 vectori (pe elemente). Sa se analizeze performantele. Specificati organizarea datelor

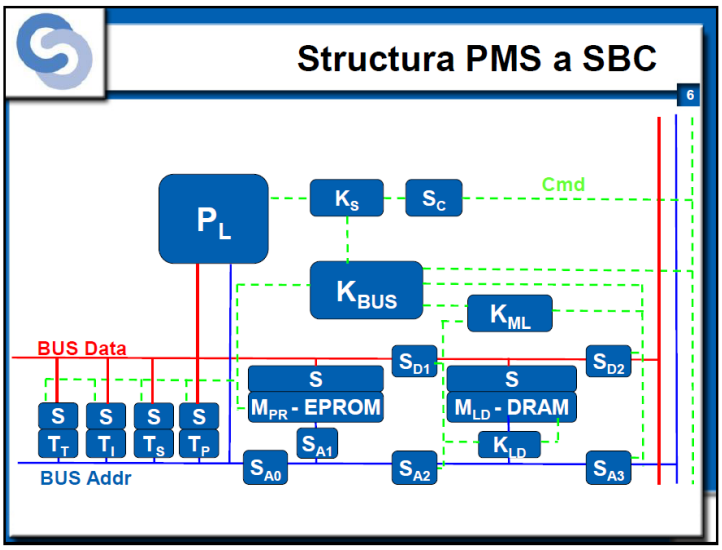
20)Adunare elemente dintr-o matrice cod SIMD MIMD plus organizarea datelor si concluzii

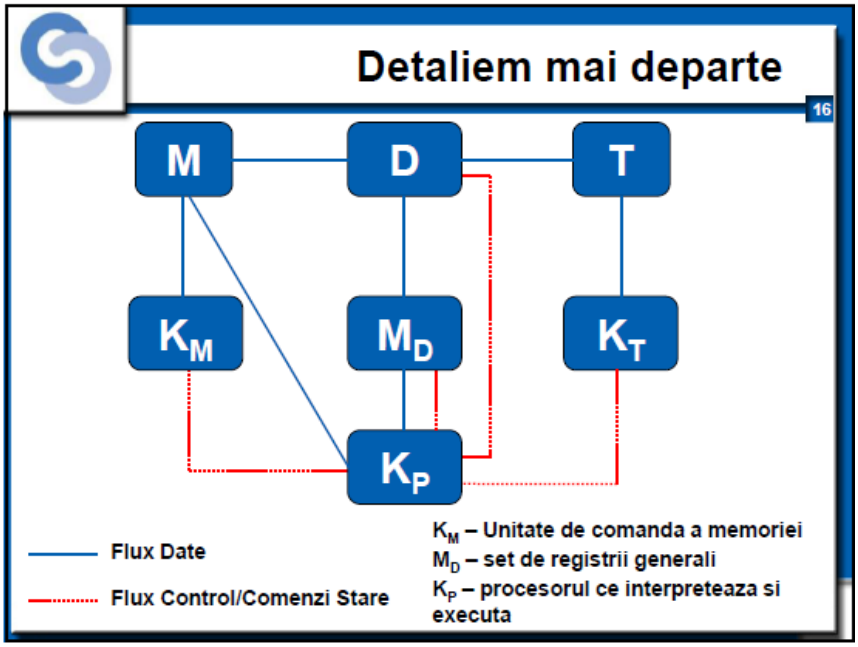
12) Sa se scrie programul ce determina produsul elementelor unei matrice pe SIMD si MIMD. Sa se analizeze performanta. Reprezentarea datelor.

PMS + primitive

2) Structura PMS

Ceva general despre primitive + (una dintre scheme)





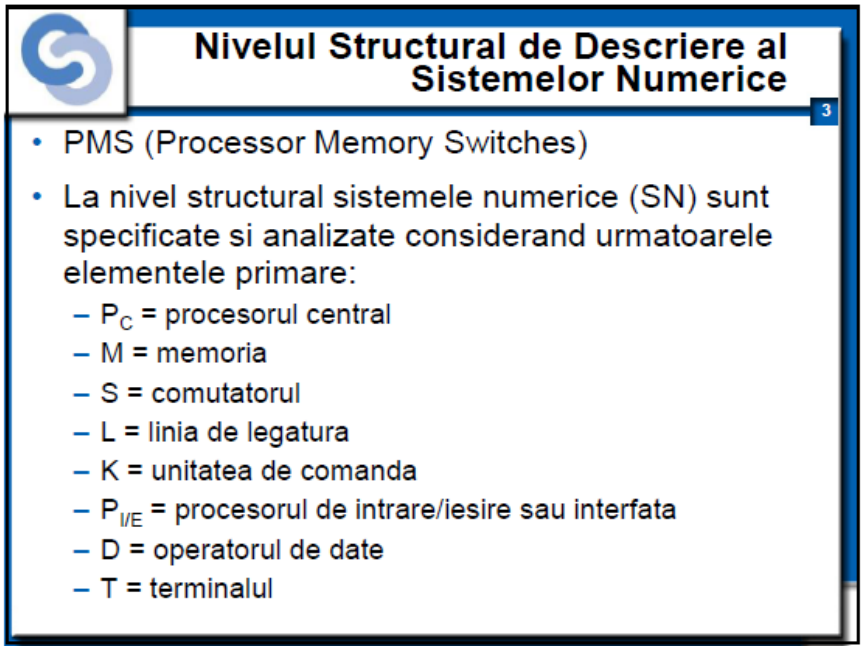
F interco

077) Descrieti in metoda PMS resursele de la laborator.

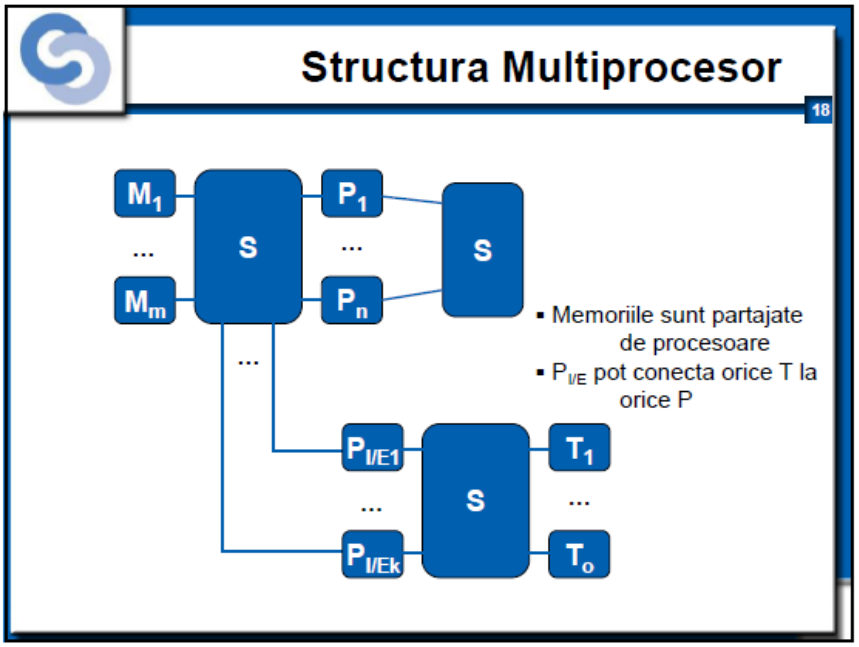
--- vezi 25 (dedesub), ca singurele resurse din laborator sunt calculatoarele

25) Descrie calculatorul tau folosind primitive PMS

-- cacat si smegma



27. Nivelul structural de organizare a sistemelor numerice. Exemplu de modelare PMS a unui sistem multiprocesor.

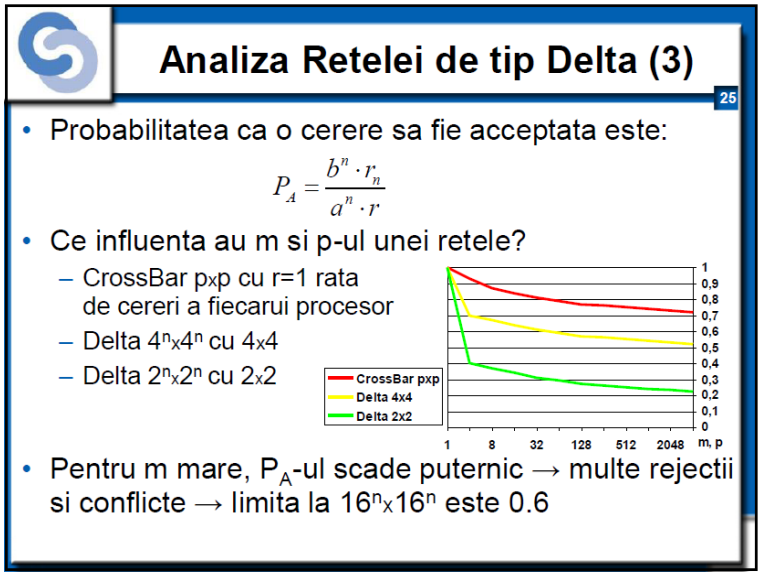


15) Exemplificati primitivele P si M de pe calculatorul propriu
--- [presupun] P = CPU si M = Cache + RAM + HDD?
de retinut ca RAM-ul e memorie principala, though

Comutatoare (Switch) - permutari

13) Comparatie RC Cross Bar si RC Delta

--- CB are mai multe basic switch-uri decat Delta, dar scaleaza mai bine dpdv al request-urilor servite cu succes, la numar mare de procesoare (e un grafic in pdf-ul 12)



16) Comparatie intre intercalare perfecta si butterfly

21)Comparatie intercalare perfecta si permutarea de baza

34. Comparatie Base Line Permutation - Bit Reversal Permutation

bit reversal -> a3 a2 a1 a0 → a0 a1 a2 a3 (reverse de la ai la aj (caz pentru 3 si 0))

BLP -> a3 a2 a1 a0 → a0 a3 a2 a1 (muta ai la sfarsit (cazul simplu pentru i = 0))

43. Comparatie permutare baza, permutare perfecta

38. Subiect cu permutari (parca era descrierea permutarilor de tip perfect shuffle)

68. comparatie intre permutarea de baza si perfect shuffle

63. Permutarea Butterfly

--- a3 a2 a1 a0 -> a0 a2 a1 a3

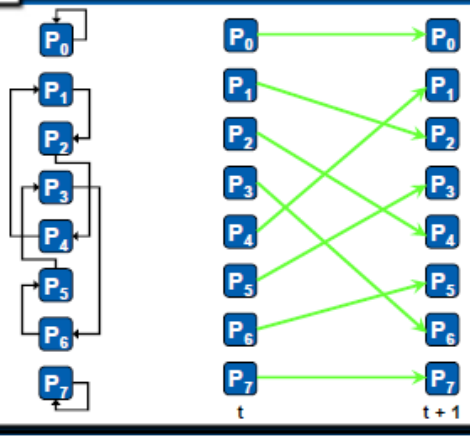
se aleg a0 si an-1 si se interschimba

//diferit de bit reversal permutation :)

3) Retea de comutare de tip interclasare perfecta

Shuffle – 8 Procesoare

10



1 – Base Line Permutation

22

- Bitul cel mai nesemnificativ devine cel mai semnificativ:
  P_base ($a_n\ a_{n-1} \dots a_1\ a_0$) = ($a_0\ a_n\ a_{n-1} \dots a_1$)
- Permutarea de baza este intercalarea perfecta inversa!
- Pentru 8 resurse avem P_base ($a_2\ a_1\ a_0$) = ($a_0\ a_2\ a_1$)
- Variante posibile sunt
  - Permutarea de baza k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_b_kinf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0\ a_{k-1} \dots a_1$)
    - Asigura conexiunea in submultimi de resurse
  - Permutarea de baza k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_b_ksup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_0$) = ($a_{n-k+1}\ a_n \dots a_{n-k+2}\ a_{n-k} \dots a_0$)
    - Asigura conexiunea intre submultimi de resurse
- TA: Variantele p_base pentru 8 resurse

2 – Shuffle Permutation

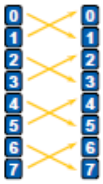
23

- Bitul cel mai semnificativ devine cel mai nesemnificativ:
  P_shuffle ($a_n\ a_{n-1} \dots a_1\ a_0$) = ($a_{n-1} \dots a_1\ a_0\ a_n$)
- Pentru 8 resurse avem P_shuffle ($a_2\ a_1\ a_0$) = ($a_1\ a_0\ a_2$)
- Variante posibile sunt
  - Permutarea shuffle k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_shuffle_kinf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k \dots a_1\ a_0\ a_{k-1}$)
    - Utilizare posibila: conexiunea memoriilor – adresa de baza a unui modul + adresa in cadrul acelui modul
  - Permutarea shuffle k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_shuffle_ksup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_0$) = ($a_{n-1} \dots a_{n-k+1}\ a_n\ a_{n-k} \dots a_0$)
  - Permutarea shuffle q # c, intre q·c resurse (nu putere a lui 2):
    - P_shuffle_q#c (ADR) = $q \cdot V_{ADR}$ mod (q·c -1) pentru $0 \le V_{ADR} < q \cdot c-1$
    - P_shuffle_q#c (ADR) = V_{ADR} pentru $V_{ADR} = q \cdot c-1$
    - V_{ADR} = 0, 1, 2, etc, n-1 succesiv

3 – Negate Bit Permutation

24

- Bitul cel mai nesemnificativ este negat
  $P_neg_bit\ (a_n\ a_{n-1} \dots a_1\ a_0) = (a_n\ a_{n-1} \dots a_1\ \overline{a}_0)$:
- $P_neg_bit\ (a_2\ a_1\ a_0) = (a_2\ a_1\ \overline{a}_0)$

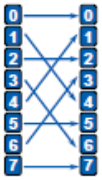


- Varianta posibila este negarea bitului k:
  $P_neg_bit_k\ (a_n\ a_{n-1} \dots a_{k-1} \dots a_1\ a_0) =$
  $(a_n\ a_{n-1} \dots \overline{a}_{k-1} \dots a_1\ a_0)$
- $P_neg_bit\ (a_2\ a_1\ a_0) = (a_2\ \overline{a}_1\ a_0)$

4 – Butterfly Permutation

25

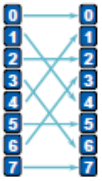
- Se interschimba bitul cel mai semnificativ cu cel mai nesemnificativ:
  $P_butterfly\ (a_n \dots a_1\ a_0) = (a_0\ a_{n-1} \dots a_1\ a_n)$
- $P_butterfly\ (a_2\ a_1\ a_0) = (a_0\ a_1\ a_2)$
- Butterfly este o permutare simetrica
- Variante posibile sunt
  - Permutarea butterfly k inferioara:
    - $P_butterfly_kinf\ (a_n \dots a_k\ a_{k-1} \dots a_1\ a_0) = (a_n \dots a_k\ a_0 \dots a_1\ a_{k-1})$
    - Ofera posibilitatea de conectare intre resursele clasei
  - Permutarea butterfly k superioara:
    - $P_butterfly_ksup\ (a_n \dots a_{n-k+1}\ a_{n-k} \dots a_1\ a_0) = (a_{n-k+1}\ a_{n-1} \dots a_n\ a_{n-k} \dots a_0)$
    - Ofera posibilitatea de conectare intre clase

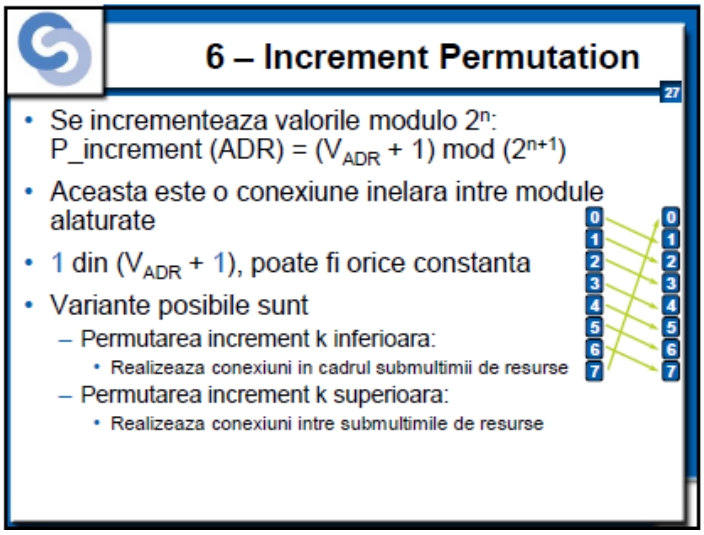


5 – Bit Reversal Permutation

26

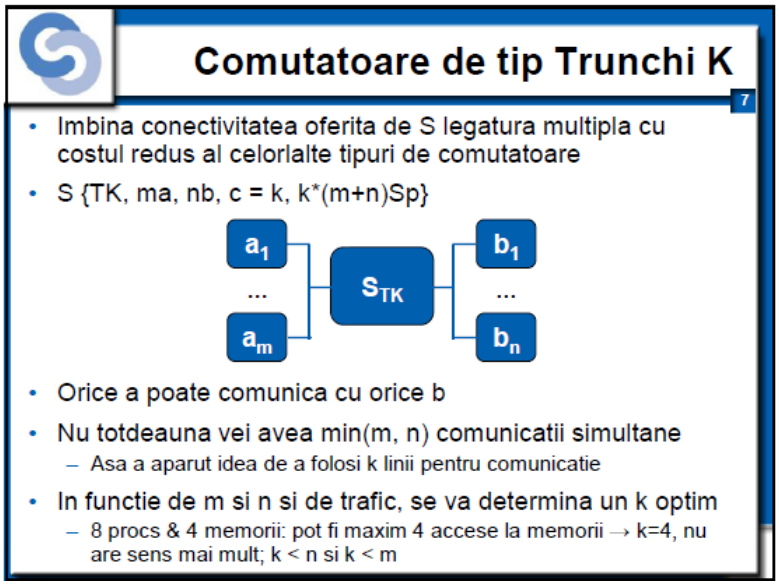
- Se interschimba complet ordinea bitilor:
  $P_reversal\ (a_n \dots a_1\ a_0) = (a_0\ a_1 \dots a_n)$
- $P_reversal\ (a_2\ a_1\ a_0) = (a_0\ a_1\ a_2)$
- Cu aceasta permutare se pot obtine echivalente
- Variante posibile sunt
  - Permutarea reversal k inferioara:
    - $P_reversal_kinf\ (a_n \dots a_k\ a_{k-1} \dots a_1\ a_0) = (a_n \dots a_k\ a_0\ a_1 \dots a_{k-1})$
  - Permutarea reversal k superioara:
    - $P_reversal_ksup\ (a_n \dots a_{n-k+1}\ a_{n-k} \dots a_1\ a_0) = (a_{n-k+1} \dots a_n\ a_{n-k} \dots a_0)$

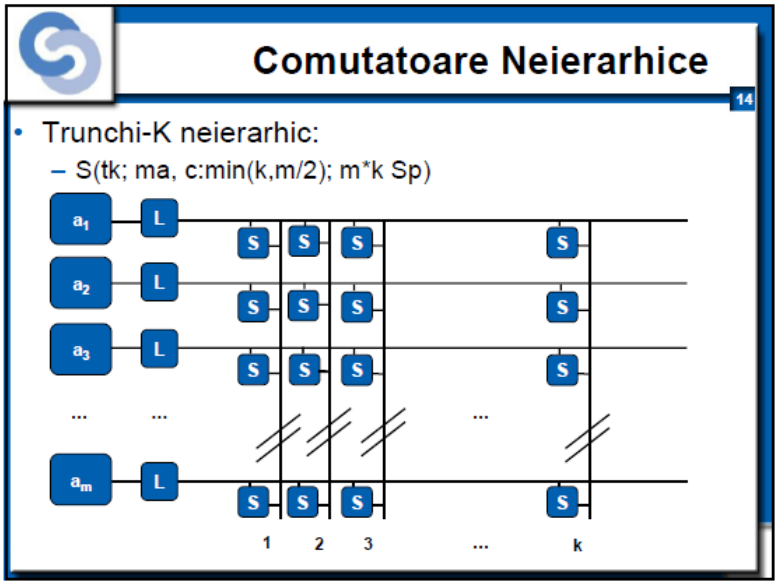




72) Comutatoare in cuplarea memorie procesor
simplex, duplex, dual duplex, cross-bar-switch, trunchi K

4) Comutator de trunchi K ierarhic si neierarhic
Comutator de tip k -> MAXIM k conexiuni, e mai simplu de implementat dar e posibil sa nu se paralelizeze pe toata lumea
ierarhic -> conexiuni diferite (ex: procesor cu memorie)
neierarhic -> conecteaza acelasi chestii (PP, MM, etc)
Bonus: faceti desenele :)





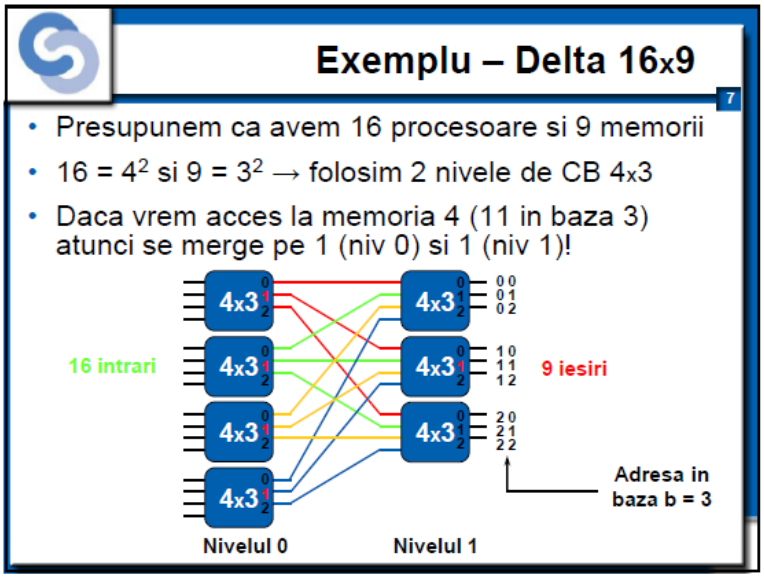
41. Probabilitatea ca o cerere sa fie acceptata la un comutator CB (formula)
--- fuck :(((. much fuck. so formula. WOW
read 214 - 216 din PDFul merged :)
read ca read, dar ce write?

7. Sa se proiecteze o Retea de Comutare de tip Delta intre P procesoare si M module de memorie.
a) 4P x 9M

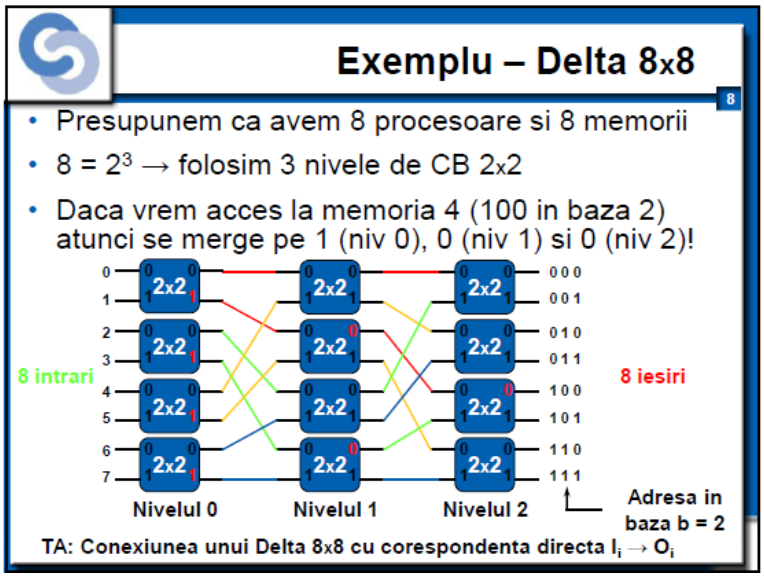
- tot 2 nivele: ia la lebar

b) 16P x 9M -> exemplu dat
c) 16P x 16M, default i , memory i

- 4 nivele de cate 2? sau
- 2 nivele de cate 4



d) 8P x 8M

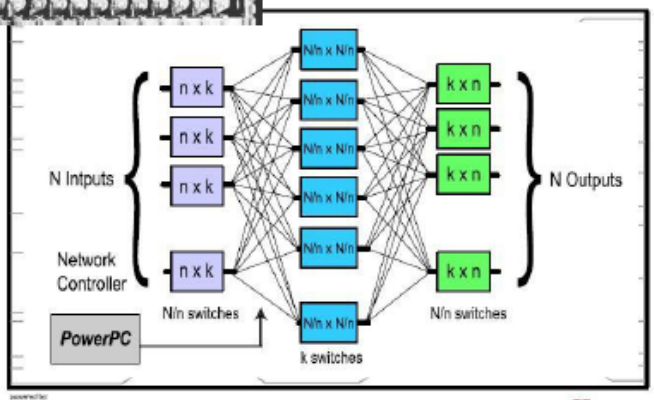


51. Comutator legatura multipla (CrossBar) ierarhic

Exemple de CrossBar Switch

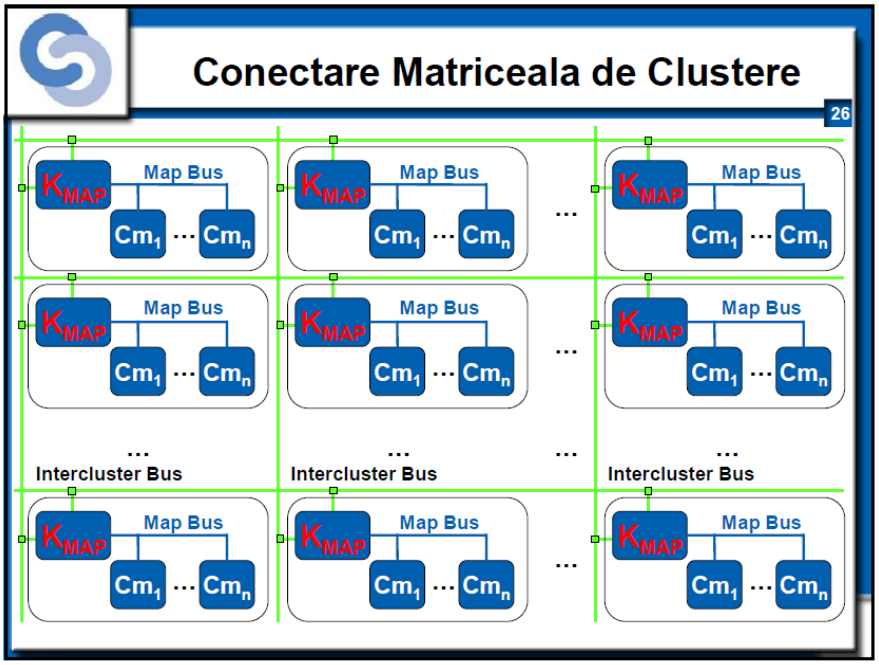
- Cross bar de la Bell System din 1960 cu 20 de verticale si 10 nivele

- Un crossbar performant, eficient si non-blocant in FPGA-urile Virtex-II si Virtex-II Pro/ProX

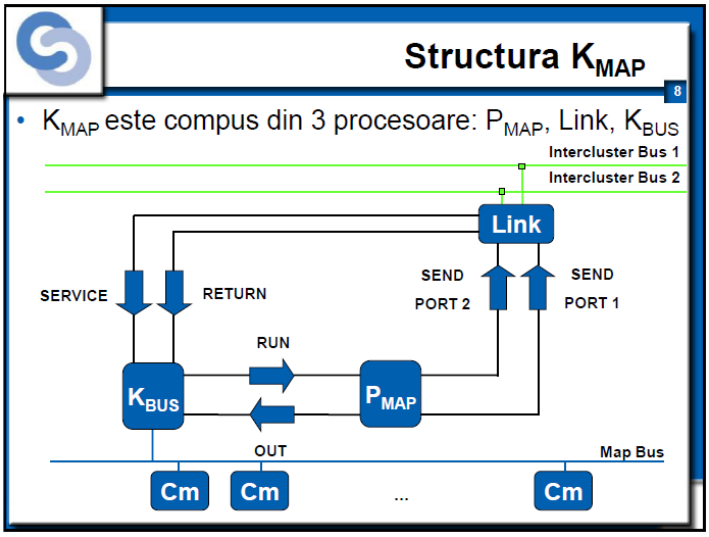


Clusters

Un cluster e un lebar mai gros.



Kmap gestioneaza magistrala Map Bus si *magistralele verzi*
-- accesul pe magistrala verde e exclusiv, nu pot sa fie doua mesaje in acelasi timp
(?) citez din curs: “Sursa de bottleneck intr-un cluster este Map Bus ↔ posibila doar o singura tranzactie la un moment dat” + “KMAP asigura: Controlul in cadrul unui cluster – prin excluderea mutuala a accesului la MapBus (arbitrare intre Cm-urile din acelasi cluster)”
Inteleg ca accesul exclusiv este pe Map Bus, si nu pe Intercluster Bus, altfel ar fi chiar a pain in the ass xD



•KBUS: unitate de comanda a Map Bus si va controla toate tranzactiile de pe aceasta magistrala

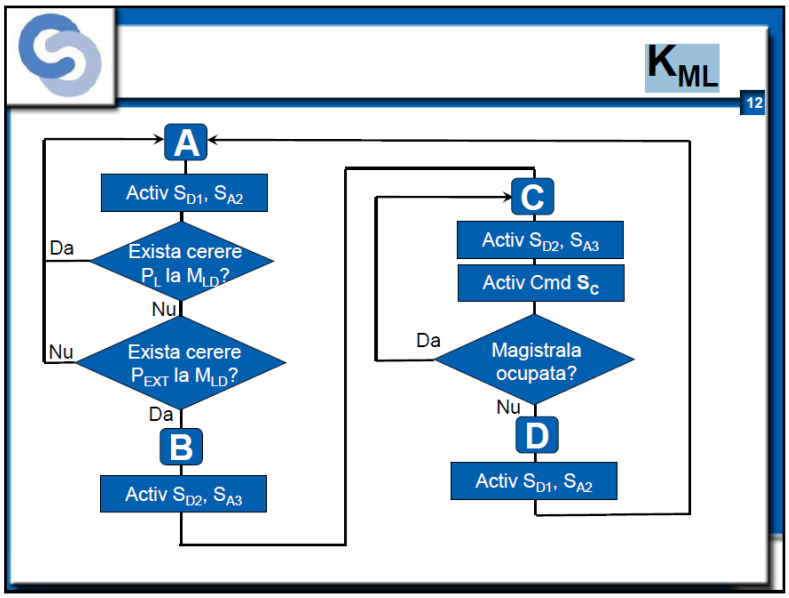
•PMAP: procesor de mapare care face toate translatarile de adresa (maxim 8 procese simultan)

•Link: este cel care gestioneaza si supravegheaza transmiterea mesajelor intercluster (send/recv)

Cm - nod intern de cluster (Computer module)

75) Schema logica Kml

10) Organigrama KML de la SBC-uri.



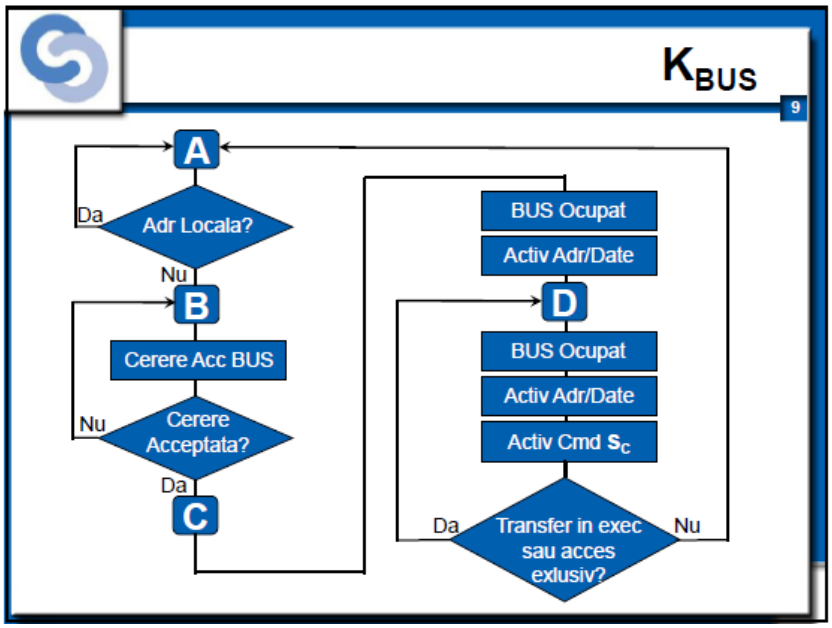
46. Organigrama Kbus

hop, sari peste astea

59. Descrierea schemei logice Kbus

29. Logica si arhitectura Kbus(SBC)

22)Diagrama KBUS



Cache

30. Care sunt cele 4 elemente de care trebuie tinut cont cand se realizeaza designul unui cache.

block placement, block identification, block replacement, write strategy

71) optimizari programe (trebuia vb de cache)

Row major vs Column major

36. Cum se masoara dimensiunea liniei unui cache? Exemplu de program si grafic.

--- Se face cu un vector la care crestem pasul pana cand performanta nu se mai degradeaza =>

--- => pasul == lungimea liniei de cache

61. Determinarea dimensiunii cache-ului

exploatam scaderea performantei la cache miss-uri

pentru linii : iteram prin vector marind stride-ul mereu, la un moment dat performanta scade f mult pentru ca ‘sarim’ din dimensiunea cache line si fortacem un cache miss la fiecare apel

pentru tot cache-ul : iteram de mai multe ori cu strideul de CACHE_LINE_SIZE, incrementand dimensiunea vectorului ce il tinem in cache. La un moment dat incepem sa facem trashing si stim ca am depasit dimensiunea cacheului

48. Block placement in cache.

direct, set asociativ (2, 4, 8, etc), complet asociativ

60. Block replacement

Random - cel mai simplu, surprinzator de eficient

LRU - greu de implementat

FIFO - usor de implementat

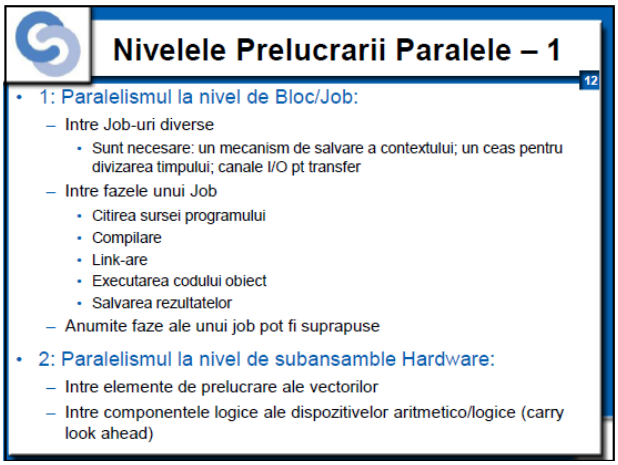
Others

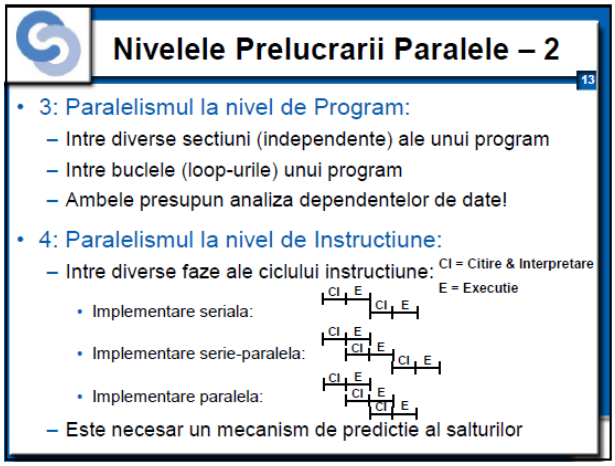
31. De ce nu se poate realiza un procesor de 10TFlops(minim 3 motive) ?
--- a) viteza memoriei e cu 10^20 mai mica decat ar trebui
--- b) pentru a permite dateleor sa ajunga avem nevoie de fiecare cuvant de memorie sa ocupe 3.7 Angstrom^3 (~ dim unei molecule mici)
--- c) no quantum yet

50. Arhitectura procesorului PMAP.

54. Mituri despre Cell BE.
--- nu stie double precision => fals, dar e semnificativ mai lent decat SP
--- se comporta ca un FPU

56. Tipuri de paralelizare la sistemele cu prelucrare paralela.





57. Structuri SN. P i/e structura
100. Nivelul structural de organizare a sist numerice

- Sisteme Matriceale (Processor Array):
    - Unitatea de baza a informatiei este vectorul
    - Dispun de instructiuni similare SC Von Neumann – operatiile asupra vectorilor sunt efectuate in aceeasi instructiune
- Sisteme Multiprocesor (Multiprocessor Systems):
    - Formate din N unitati de prelucrare interconectate printr-o retea de comutare (Strans/Slab cuplata)
    - Sistemele lucreaza independent la realizarea aceluiasi Job
- Sisteme Pipeline:

- Dispun de mai multe unitati de prelucrare asezate in banda de asamblare (RISC):
- Fiecare UC executa o prelucrare specifica si transfera rezultatul subansamblului adiacent

62. Comunicatia Kbus â Pmap

67) Cell BE bariere si solutii.
Memory Wall,
-more slower threads
-asynchronous loads
Efficiency Wall ,
-more slower threads
-specialized functions
Power Wall
-reduce transistor power
-reduce switching function

69) Memorii distribuite
70) comunicare in cadrul unui cluster

Top 500

MA DOARE LA LEBAR
18) Sa se aleaga un calculator din TOP 500 si sa vorbim despre el.
-- http://www.top500.org/system/177975
33. Caracteristicile arhitecturale pentru arhitectura cea mai des intalnita in top 500
-- cluster
65. Cel mai utilizat SO din top500
--- Linux
37. Arhitectura unui calculator din top 500 (arhitectura, numar procesoare, so, ...)
-- https://www.olcf.ornl.gov/wp-content/themes/olcf/titan/images/titan-infographic.png + Cray Linux
53. Cel mai popular mod de conectare in top500. Cel mai popular SO. Cea mai populara arhitectura.
-- InfiniBand, Linux, Cluster

Benchmark

64. Benchmark-uri pentru sisteme seriale

SPEC, TPC, BAPCo, SANDRA

--- cursul de benchmarking, slide-urile 7 si 8

32. Caracteristici HPC Challenge Benchmark.

--- compara sistemele in mai mult de o categorie

55. Benchmark-uri, probleme si rezolvari legate de acestea (ceva de genu). (nu trebuia sa zici de HPCC si LINPACK)

73) Pe ce nivele se face benchmark la HPC. De ce e bine ca sunt mai multe metrici?

el

Probleme:

Problema clasica anul trecut:

Ai N bile punctiforme pentru care ti se dau :

x

y coordinate initiale (coordonatele extremitatii initiala a
segmentului pe care se deplaseaza bila)

v viteza de deplasare

x_ext

y_ext coordonateme extremitatii finale a segmentului pe care se
deplaseaza bila

Daca doua bile se ciocnesc atunci dispar din sistem.

Se cere sa se afiseze coordonatele la care o bila dispare din system
(daca s-a ciocnit cu a olta bila) sau coordonatele extremitatii finale
pentru bilele care au ajuns in extremitati in perioada T de timp
(data).

a) utilizand instr SIMD calculate distanta parcursa de fiecare bila
(se dadea formula distantei dintre 2 pct)

b) sa se modeleze sistemul cu threaduri (pthreads/python threads)

c) sa se modeleze sistemul in Cell

din ce inteleg, trebuie sa se verifice daca 2 cate 2 segmentele se intersecteaza
daca 2 segmente se intersecteaza trebuie sa determinam la ce distanta
apoi sa vedem daca distanta respectiva este suficient de aproape sa fie ajunsa la timp de
ambele bile (portiunea de segment e parcursa cu viteza v in timp <= T)
in caz, afirmativ, retinem coordonatele coliziunii